


# GumNaaM_Helpers

Virtual University

## Mega File for FinaL TerM

## All Files in ONE FILE

**(Handouts_waqar_moaz_junaid_vu toper_orange monkey)**

الحمدللہ گمنام ہیلپرز ان تمام کتابوں میں مدد کرتے ہیں جن کے لنکس نیچے دیئے گئے ہیں
لیٰذا ہم سے صرف انہی کتابوں کی مدد کا سوال کیا جائے جن کے لنکس نیچے موجود ہیں

**Groups Join Krty hi Description Read krna na bhoolyn**

**جزاک اللہ خیرا و احسن الجزاء**

### GumNaaM_Helpers All What's App Group Links

| | |
|---|---|
| Cs101 https://chat.whatsapp.com/KCQ3jK6FcoYEvDYTF1gnBS | Eng101 https://chat.whatsapp.com/Cwp4Ag9TG6ZKxyRqThqM6S |
| Cs201 https://chat.whatsapp.com/DrYV1FAXmppImVZFwI6V2R | Eng201 https://chat.whatsapp.com/EvvxcPtmrM22VTZhz5NCTw |
| Cs301 https://chat.whatsapp.com/EPLvHztf8EB7aH8e1AtidV | Eco401 https://chat.whatsapp.com/K3NLvyue6BdK0sVMqDFoU2 |
| Cs302 https://chat.whatsapp.com/BaYfW0P67vNIsl2n36otZM | Mcm301 https://chat.whatsapp.com/GUyzfdHzu8r43TW16vhZWF |
| Cs304 https://chat.whatsapp.com/C0mk0wdo0BU46dlhcd67IO | Mgt101 https://chat.whatsapp.com/BmrYaQYuIBOKguHOiT6t2Y |
| Cs401 https://chat.whatsapp.com/Jz9WDdHX84lKgkrMPAV0Mf | Mgt201 https://chat.whatsapp.com/DZwztlUyxV50D6AshnvWCD |
| Cs402 https://chat.whatsapp.com/DW9AZLb4i1P9la4XmcCsRz | Mgt211 https://chat.whatsapp.com/EmX9xloZnbI12pX9m2DZwa |
| Cs403 https://chat.whatsapp.com/HXV25Lc6MnLKZgrA7xOqjF | Mgt301 https://chat.whatsapp.com/G62QTipycoXFrpOZxfDIBK |
| Cs408 https://chat.whatsapp.com/EFy2mrYJodx5dB5W1jtII7 | Mgt501 https://chat.whatsapp.com/H0dTUINbesw0GS4LLSt0W4 |
| Cs411 https://chat.whatsapp.com/IrOaF8Q3tkJKzeVTHapHR9 | Mgt502 https://chat.whatsapp.com/L3GSMFgwDl7DAsu9r0vAxI |
| Cs501 https://chat.whatsapp.com/Eggxx3X0SH2LhOysM57SII | Mgt503 https://chat.whatsapp.com/G7BNxqyLYwP48jfPTofkbJ |
| Cs502 https://chat.whatsapp.com/FbLU96Xhmr2FI99fnbz5Yw | Mgt602 https://chat.whatsapp.com/HXRpOrEL18vD7lb91nLHTB |
| Cs504 https://chat.whatsapp.com/Gn7w0UzNCQ2FzRK7ZzY4Io | Mgt610 https://chat.whatsapp.com/BxMcR1ouiao0wuqLlLj5dB |
| Cs506 https://chat.whatsapp.com/BL4FFmDKf2S3WIoLsAD2JK | Mth101 https://chat.whatsapp.com/LHsbl65FbwGH2SNFF9ipSb |
| Cs507 https://chat.whatsapp.com/EINDZkV4e4zGLGqL8imozR | Mth202 https://chat.whatsapp.com/GevRebusRaxFh6dgnzRJuM |
| Cs508 https://chat.whatsapp.com/HTMAwnJ4Bic2ky8MbT3PuW | Mth301 https://chat.whatsapp.com/EFm472nc2woDPrOEqnY1lj |
| Cs601 https://chat.whatsapp.com/KYoaAutdXRlDNS0Mm5XfzP | Mth401 https://chat.whatsapp.com/EP7tlcc17TzLtNaOgD4AcA |
| Cs602 https://chat.whatsapp.com/FpJkLAFNQpZHRyi6vhl5n8 | Mth501 https://chat.whatsapp.com/Dy7XUVFqqD5LLw4i5X4Xzi |
| Cs604 https://chat.whatsapp.com/KjDhHjxawtU43Veo9Eawa4 | Mth601 https://chat.whatsapp.com/BoD3akuXGK25MFB4UEZcRr |
| Cs605 https://chat.whatsapp.com/BoF34LtMlYK1l4nQaEO2Ce | isL201 https://chat.whatsapp.com/BtoqaMbbELW99MLLtQMGwC |
| Cs610 https://chat.whatsapp.com/FcNcztKyYfd9szsJEJino2 | Pak301 https://chat.whatsapp.com/KuR9Y7YWeA32enRwKeZJaR |
| Cs615 https://chat.whatsapp.com/HLVjBNcorFN9bwZhGEkFl3 | Phy101 https://chat.whatsapp.com/GjC6e47vF1zHwRs5iOc13J |
| Cs619 https://chat.whatsapp.com/FEklYr700uj0ihxqnk55A6 | Phy301 https://chat.whatsapp.com/IBrOdznEHDi92fGa76Dqmr |
| | Sta301 https://chat.whatsapp.com/BchodAnjbydKXLuxFfKGxr |

**...Paid Service Available for LMS Activities...**

**Only for those students who are so busy in office or work or job that they can't watch their LMS due to their Busyness**

FOR MORE INFO. CONTACT US ONLY WHAT'S APP 0302-1400084

# Software Engineering – 1

# (CS504)

## Lecture Notes

Delivered by

**Dr. Fakhar Lodhi**

© Copyright Virtual University of Pakistan

# TABLE OF CONTENTS

**Lecture 01:** Introduction to Software Engineering……...…….. 1

**Lecture 02:** Introduction to Software Development …… 11

**Lecture 03:** Requirement Engineering-1 …… 16

**Lecture 04:** Requirement Engineering-2 …… 20

**Lecture 05:** Relation of Several components of Software Requirements …….. 28

**Lecture 06:** Use Case Diagram for a Library System ……... 33

**Lecture 07:** Source and Sink Analysis …… 40

**Lecture 08:** State Transition Diagrams ……... 44

**Lecture 09:** Typical Processes …… 53

**Lecture 10:** Prototyping and GUI Design …… 62

**Lecture 11:** Software Design …… 69

**Lecture 12:** Coupling and Cohesion …….. 72

**Lecture 13:** Object Oriented Analysis and Design ……... 83

**Lecture 14:** Object Oriented Analysis and Design-2 …… 89

**Lecture 15:** UML Object Model Notations …… 92

**Lecture 16:** Derivation of Object Model-Coad Methodology ……. 93

**Lecture 17:** Derivation of Object Model-Coad Methodology -2 ……. 95

**Lecture 18:** CASE STUDY: Connie's Convenience Store ……. 97

**Lecture 19:** Identify Structure …… 100

**Lecture 20:** Interaction Diagrams …….. 106

**Lecture 21:** Sequence Diagrams (Message Types) …….. 108

**Lecture 22:** Software and System Architecture …….. 115

© Copyright Virtual University of Pakistan

**Lecture 23:** Architectural Views …… 122

**Lecture 24:** Architectural Models-I ……... 126

**Lecture 25:** Architectural Models-II …….. 130

**Lecture 26:** Introduction to Design Patterns …… 137

**Lecture 27:** Observer Pattern ……. 140

**Lecture 28:** Good Programming Practices and Guidelines…… 146

**Lecture 29:** File Handling Tips for C++ and Java ……... 155

**Lecture 30:** Layouts and Comments in Java and C++…… 162

**Lecture 31:** Coding Style Guidelines Continued... ……..167

**Lecture 32:** Clarity Trough Modularity ……... 170

**Lecture 33:** Common Coding Mistakes …….. 176

**Lecture 34:** Portability …… 179

**Lecture 35:** Exception Handling …… 184

**Lecture 36:** Software Verification and Validation …….. 192

**Lecture 37:** Testing vs. Development ……... 195

**Lecture 38:** Equivalence Classes or Equivalence Partitioning …….. 199

**Lecture 39:** White Box Testing …….. 202

**Lecture 40:** Unit Testing ……. 207

**Lecture 41:** Inspections vs. Testing …….. 210

**Lecture 42:** Debugging ……... 213

**Lecture 43:** Bug Classes …….. 216

**Lecture 44:** The Holistic Approach ……... 224

**Lecture 45:** Summary ……. 227

© Copyright Virtual University of Pakistan

# Lecture No. 1

Introduction to Software Engineering

An Introduction to Software Construction Techniques for Industrial Strength Software

## 1.1 Introduction

Software engineering is an interesting subject. In order to understand this subject we will need to look at a number of examples and case studies. And we will need to see how we can develop good software and how it could be improved in different scenarios? Before we move on to software engineering we need to understand what software actually is.

### ➢ What is Software?

When we write a program for computer we named it as software. But software is not just a program; many things other than the program are also included in software.

Some of the constituted items of software are described below.

- ***Program*:** The program or code itself is definitely included in the software.
- ***Data*:** The data on which the program operates is also considered as part of the software.
- ***Documentation*:** Another very important thing that most of us forget is documentation. All the documents related to the software are also considered as part of the software.

So the software is not just the code written in Cobol, Java, Fortran or C++. It also includes the data and all the documentation related to the program.

### ➢ Why is it important?

Undoubtedly software is playing a vital role in all the field of life these days. We can see many software applications being operated around us in our daily routine.

Some of the major areas in which software has played an important role are identified as under.

- *Business decision-making*: Software systems have played a major role in businesses where you have to analyze your data and on the basis of that analysis you have to make business decisions. This process of data analysis and decision-making has become very accurate and easy by the use of software.
- *Modern scientific investigation and engineering problem solving*: Scientific investigations and engineering problem solving require an intensive amount of



© Copyright Virtual University of Pakistan

calculations and data analysis. The accuracy of these analyses is also very important in scientific applications. This process has become very easy and accurate by the use of software. For example software systems are becoming more involved in bioinformatics and the process of DNA decoding is only possible by the use of software systems. Similarly many astronomical observations are being recorded and analyzed by the software systems these days.

- *Games*: We see many computer games these days that interests people of all ages. All these games are drive through software systems.
- *Embedded systems*: We see many kinds of gadgets being employed in our daily used things, like small microcontrollers used in our cars, televisions, microwave ovens etc. All these systems are controlled through the software.

Similarly in many other fields like education, office automation, Internet applications etc, software is being used. Due to its central importance and massive use in many fields it is contributing a lot in terms of economic activity started by the software products. Billions and trillions of dollars are being invested in this field throughout the world every year.

### ➢ Engineering

Before moving on to software engineering lets first discuss something about engineering itself. If you survey some of the dictionaries then you will find the following definition of engineering.

*"The process of productive use of scientific knowledge is called engineering."*

## 1.2 Difference between Computer Science and Software Engineering

The science concerned with putting scientific knowledge to practical use.
Webster's Dictionary

There are many engineering fields like electrical, mechanical and civil engineering. All these branches of engineering are based on physics. Physics itself is not engineering but the use of physics in making buildings, electronic devices and machines is engineering. When we use physics in constructing buildings then it is called civil engineering. When we use physics in making machines like engines or cars then it is called mechanical engineering. And when we apply the knowledge of physics in developing electronic devices then the process is called electrical engineering. The relation of computer science with software engineering is similar as the relation of physics with the electrical, mechanical or civil engineering or for that matter the relation of any basic science with any engineering field. So in this context we can define software engineering as:
*"This is the process of utilizing our knowledge of computer science in effective production of software systems."*

### ➢ Difference between Software and Other Systems

Now lets talk something about how a software system is different from any other systems. For example, how software is different from a car, a TV or the similar systems or what is the difference between software engineering and other engineering like


© Copyright Virtual University of Pakistan

# Lecture No. 23

## 8.6 Architectural Views

Software architecture defines the high level structure of the software by putting together a number of architectural elements in an organized fashion. These elements are chosen to satisfy the functional as well as non-functional requirements of the system. Perry and Wolfe proposed the following formula for software architecture:

Software architecture = {Elements, Forms, Rationale}

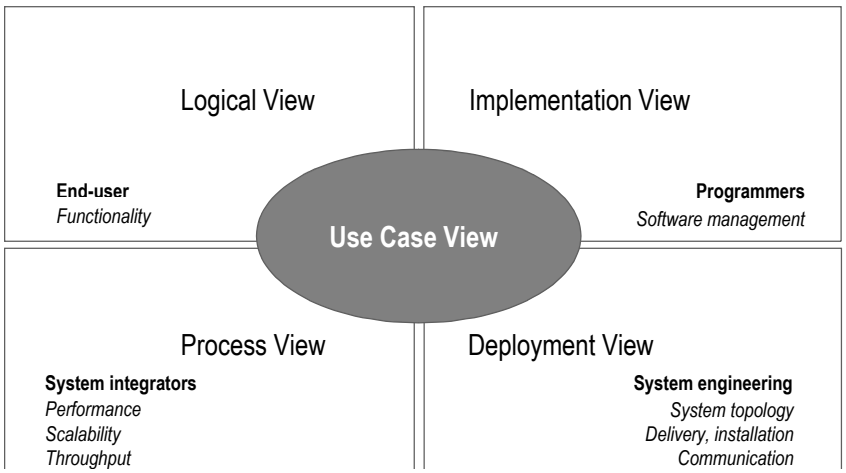
That is, a software architecture is a set of elements which have a certain form. These elements are further divided into three categories. These are: data elements, processing elements, and connecting elements. The data elements contain the information that is used and transformed in the system; the processing elements process the data elements and specify the transformation functions, and connecting elements connect different pieces of architecture together. These can themselves be data or processing elements.

This formula was modified by Boehm as follows:

Software architecture = {Elements, Forms, Rationale/Constraints}

Krutchen proposed that the software architecture model should be composed of multiple views as a software architecture deals with abstraction, with decomposition and composition, with style and esthetics as well as requirements from different stake holders. His architectural model is known as Krutchen's 4+1 architectural view model. As evident, this model proposes the development of 5 main views namely the logical view, the process view, the physical view, the development view, and the use case view. The logical view is the object model of the design, the *process* view captures the concurrency and synchronization aspects of the design, the *physical* view documents the mapping(s) of the software onto the hardware and reflects its distributed aspect, the *development* view describes the static organization of the software in its development environment, and the use case view uses selected use cases or scenarios to validate that the architecture supports the required functionality.

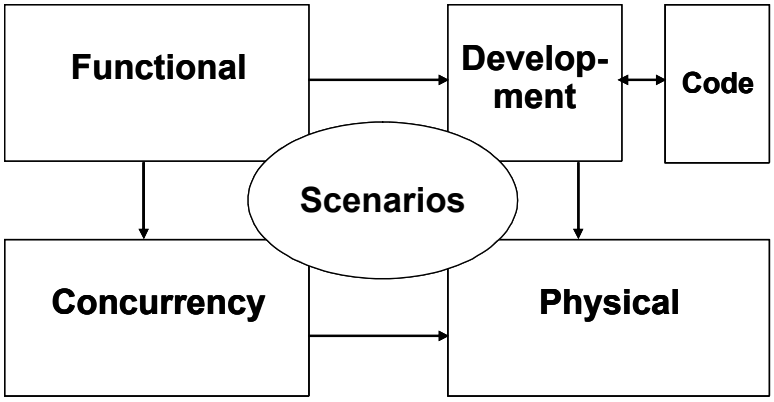
This model is shown in the following diagram.






© Copyright Virtual University of Pakistan

This model has been slightly modified by Clements et. al. and is shown in the following diagram.




Clement's modified version of Krutchen's 4+1 Architectural View Model

In this model, the architecture is again prepared and analyzed from 5 different perspectives. The 4 main views are Functional View, the Concurrency View, the Physical View, and the Development View. Code view is not present in the original Krutchen model and is basically an extension of the development view.

The Functional View comprises of various different functions provided by the system, key system abstraction, and the domain elements. It connects different dependencies and data flows into a single view. This view can be used by the domain engineers, product-line engineers, as well as the end users of the system. It can be used for understanding the functionality provided by the system, modifiability of the system, reusability, tool support, and allocation of work.

The Development View documents the different files and directories in the system and users of this view include the development and the configuration management staff as well as the project managers. The major uses of this view include maintenance, testing, configuration management, and version control.

The components of the Code View include classes, objects, procedures, functions, subsystems, layers, and modules. It documents the calling as well as the containing hierarchy of different components of the system. Its primary users are the programmers and designers of the system. The primary intent of developing this view is to use it for maintenance and portability.


© Copyright Virtual University of Pakistan

Concurrency View intends to document different parallel processes and threads in the system. Its main focus is on event synchronization and parallel data flows. It is used primarily by integrators, performance engineers, and testers. The main purpose of building this view is to identify ways and means to improve performance by highlighting possible opportunities for parallelism.

The Physical View depicts the physical organization of this system and how this system will be physically deployed. This includes different processors, sensors, and storage devices used by the system. This view connects various network elements and communication devices. The primary users of this system include hardware and system engineers. The view is developed with an intention to document and analyze system delivery and installation mechanism. The view is also used in understanding and analyzing issues pertaining to system performance, availability, scalability, and security.

Finally, there are Scenarios. Scenarios are basically use cases which describe the sequences of responsibilities and change cases which are changes to the system. As stated earlier, the first objective of developing a system is to fulfill the functional requirements setout for the system. These functional requirements are written in the form of use cases. Therefore, scenarios are used to understand and validate the system. Their secondary purpose is to communicate the design to the other users of the system. Scenarios tie all the view together into an integrated system. They are also used to understand the dynamic behavior of the system and understand the limits of design.

This is summarized in the following table:

| View | Components | Users | Rationale |
|---|---|---|---|
| **Functional View** | functions, key system abstractions, domain elements | domain engineers, product-line designers, end users | functionality, modifiability, product lines/reusability, tool support, work allocation |
| **Code View** | classes, objects, procedures, functions, subsystems, layers, modules | programmers, designers, reusers | modifiability/maintainability, portability, subsetability |
| **Development View** | files, directories | managers, programmers, configuration managers | managers, programmers, configuration managers |
| **Physical View** | CPUs, sensors, storage | hardware engineers, system engineers | system delivery and installation, performance, availability, scalability, security |
| **Concurrency View** | processes, threads | performance engineers, integrators, testers | performance, availability |


© Copyright Virtual University of Pakistan

### What Are Views Used For?

Views are an engineering tool to help achieve desired system qualities. Each view provides an engineering handle on certain quality attributes. In some systems, distinct views collapse into one (e.g., the concurrency and physical views may be the same for small systems.). Views are also used as documentation vehicle for current development and future development. Users of views include both managers and customers. For these reasons views must be *annotated* to support analysis. This annotation can be achieved with the help of scenarios and design *rationale*.

Structures can also be used to document how the current system was developed and how future development should occur. This information is needed by managers and customers.

Although one often thinks about a system's structure in terms of functionality, there are other system properties in addition to functionality (such as physical distribution, process communication, and synchronization) that must be reasoned about at an architectural level. Each structure provides a method for reasoning about some of the relevant quality attributes. The uses structure, for instance, must be engineered (not merely recorded) to build a system that can easily be extended or contracted. The calls structure is engineered to reduce bottlenecks. The module structure is engineered to produce modifiable systems, and so on. Each structure provides a different view into the system and a different leverage point for design to achieve desired qualities in the system.

### Hierarchical Views

Every view is potentially hierarchical, e.g. *functional view could contain* sub-functions. Similarly *development view* contains directories which contain files. C*ode view* would have modules and systems that contain sub-modules and sub-systems respectively. C*oncurrency view* contains processes that are further subdivided into threads. Finally, *physical view* clusters contain computers which contain processors.

Architectural views are related to each other in complicated ways. One has to choose the views that are useful to the system being built and to the achievement of qualities that are important for that particular application. The architectural views should be hierarchical (where needed) and should contain enough annotated information to support desired analyses.

### Architectural styles

The architectural model of a system may conform to a generic architectural model or style. An awareness of these styles can simplify the problem of defining system architectures. However, most large systems are heterogeneous and do not follow a single architectural style.


© Copyright Virtual University of Pakistan

Each style describes a system category that encompasses:

(1) a set of components (e.g., a database, computational modules) that perform a function required by a system,
(2) a set of connectors that enable "communication, coordination and cooperation" among components,
(3) constraints that define how components can be integrated to form the system, and

semantic models that enable a designer to understand the overall properties of a system by analyzing the known properties of its constituent parts.

# Lecture No. 24

## 8.7 Architectural models

Like analysis models, many different kinds of architectural models are developed during the architectural design process. Static structural model shows the major system components while a dynamic process model shows the process structure of the system. Interface models are developed to define sub-system interfaces.

### 8.8 Architectural Styles

Architectural design may be based upon a certain pattern of model. These different patterns are also called architectural styles. These styles have different characteristics and attributes and can be useful to solve problems related to a particular situation of requirement. Among the many styles, the most commonly practiced are the following:

- Data-centered architectures
- Client Server Architecture and its variations
- Layered architectures
- Reference Architecture

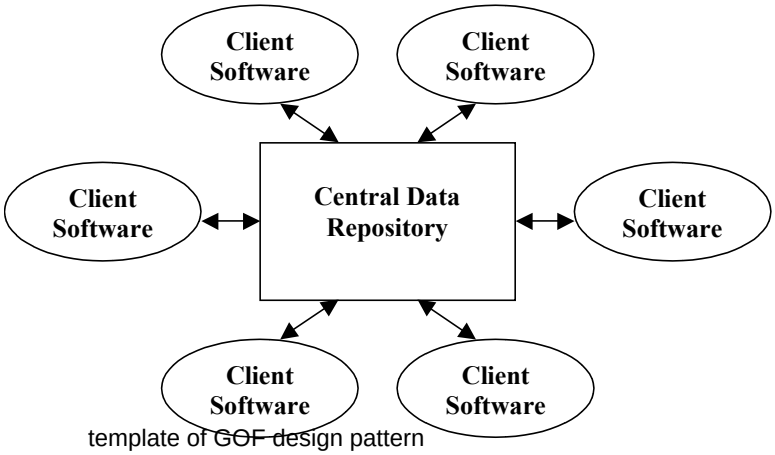
#### Data-Centered or the repository model

In any system, sub-systems need to exchange information and data. This may be done in two ways:

1. Shared data is held in a central database or repository and may be accessed by all sub-systems
2. Each sub-system maintains its own database and passes data explicitly to other sub-systems


© Copyright Virtual University of Pakistan

When large amounts of data are to be shared, the repository model of sharing is most commonly used. This model has been extensively used in the main-frame based




template of GOF design pattern includes

application. The model is depicted in the following diagram:

**Repository model characteristics**

- Advantages

  Repository model is an efficient way to share large amounts of data. In this case sub-systems need not be concerned with how data is produced Centralised management e.g. backup, security, etc. This model also provides a global view of the system and the sharing model is published as the repository schema.

- Disadvantages

  Repository model suffers from a number of disadvantages. First of all, sub-systems must agree on a repository data model. This inevitably leads to a compromise. The major problem however is that data evolution is difficult and expensive. There is also little scope for implementing specific management policies. It is also difficult to distribute efficiently

**8.9 Client-server model**

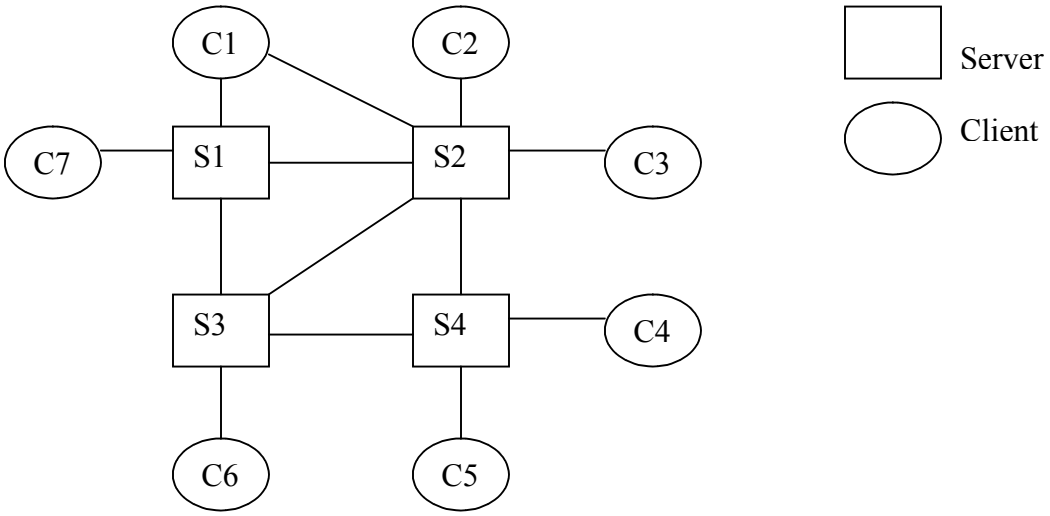
Client server model tries to distribute data and processing. This is a shift from main-frame based applications where both the data management and the processing of data used to be typically carried out by the same main-frame computer. In those applications, the user interface used to be a provided through a "dumb" terminal which did not have much processing power. With the availability of the cheaper but power machines, it was possible to shift some load from the back-end computer to other smaller machines.

The client-server model is a distributed system model which shows how data and processing is distributed across a range of components. In this model, the application is modeled as a set of services that are provided by servers and a set of clients that use these services. The system is organized as a set of stand-alone servers which provide specific services such as printing, data management, etc. and a set of clients which call on these


© Copyright Virtual University of Pakistan

services. These clients and servers are connected through a network which allows clients to access servers. Clients and servers are logical processes (not always physical machines). Clients know the servers but the servers do not need to know all the clients and the mapping of processes to processors is not always 1:1.

The following diagram depicts a general client-server organization.





© Copyright Virtual University of Pakistan

### 8.10Client/Server Software Components

A typical client-server architecture based system is composed of a number of different components. These include user interaction/presentation subsystem, application subsystem, database management subsystem, and middleware. The application subsystem implements requirements defined by the application within the context of the operating environment. In this case the application components may reside on either client or server side. Middleware provides the mechanism and protocols to connect clients with the servers.

#### Representative Client/Server Systems

Following are some of the representative server types in a client-server systems.

- File servers
  In this case, client requests selected records from a file and the server transmits records to client over the network.
- Database servers
  In this case, client sends requests, such as SQL queries, to the database server, the server processes the request and returns the results to the client over the network.
- Transaction servers
  In this configuration, client sends requests that invokes remote procedures on the server side, server executes procedures invoked and returns the results to the client.
- Groupware servers
  Groupware servers provide set of applications that enable communication among clients using text, images, bulletin boards, video, etc.

#### Client-server characteristics

- Advantages
  The main advantage of the client-server architecture is that it makes effective use of networked systems. Instead of building the system with very expensive large computers and hardware devices, cheaper hardware may be used to gain performance and scalability. In addition, adding new servers or upgrading existing servers becomes easier. Finally, distribution of data is straightforward in this case.

- Disadvantages
  The main disadvantage of this model is that there is no standard way of sharing data, so sub-systems may use different data organisation. Hence data interchange may be inefficient. From a management point of view, each server needs attention and hence there is redundant management in each server. Finally there is no central register of names and services - it may be hard to find out what servers and services are available.


© Copyright Virtual University of Pakistan

### 8.11 Representative Client/Server Configurations

The client-server model can have many different configurations. In the following sections, we look at some of these configurations.

#### Thin Client Model

This model was initially used to migrate legacy systems to client server architectures. In this case the legacy system may act as a server in its own right and the GUI may be implemented on a client. It chief disadvantage is that it places a heavy processing load on both the server and the network.

#### Fat Client Model

With advent of cheaper and more powerful hardware, people thought of using the processing power of client side machines. So the fat client model came into being. In this model, more processing is delegated to the client as the application processing is locally extended. It is suitable for new client/server systems when the client system capabilities are known in advance. It however is more complex than thin client model with respect to management issues. Since the client machine now also has a significant part of the application resident on it, new versions of each application need to be installed on every client.

# Lecture No. 25

## Zero Install

As discussed earlier, fat-client architecture posed major challenges in terms of installation and maintenance of the client side of the application, especially when there are large number of client machines. So the idea behind zero install architecture is to develop a system where no installation on the client side is needed. This can only be done when there is no or little processing done at the client side. This is basically a trade-off between using the computing power available at the client machine versus maintenance overhead. This in essence takes us back to an architecture which is similar to thin-client architecture. There is little difference though. In the classical thin-client architecture, the entire processing is carried-out by a single server, in the case of zero-install, the network environment is used to distribute server side processing by adding a number of servers which share processing load. Web-based application where the web-pages are put on a web-server is an example of this type of architecture. In this case, whenever there is a change, the web page is updated accordingly. Now when the user logs in, he gets to use the modified version of the application without any changes or updates at the client side.


© Copyright Virtual University of Pakistan

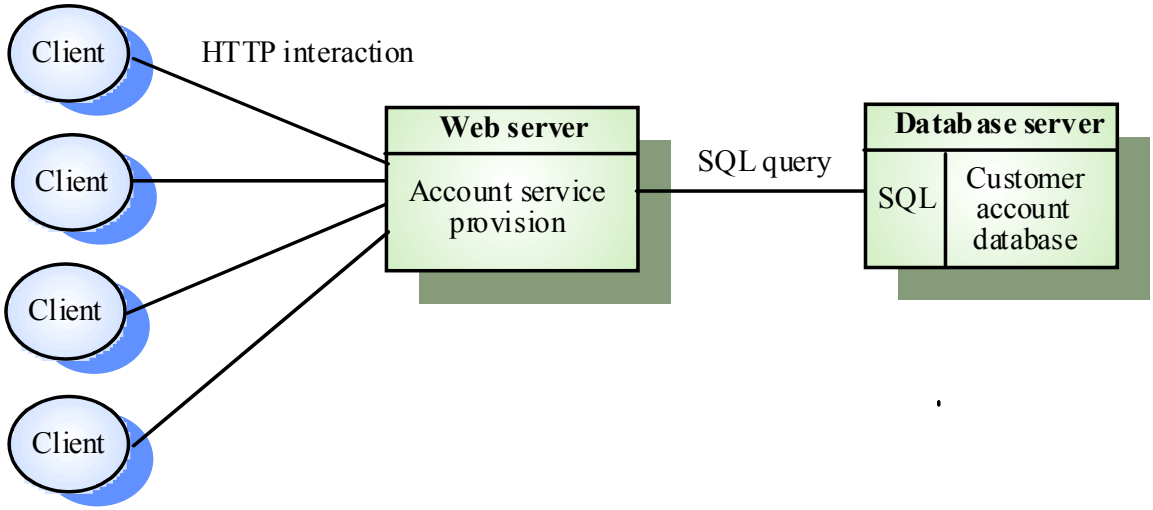
**N-Tier architecture**

N-tier architecture stems from the struggle to find a middle ground between the fat-client architecture and the thin-client architecture. In this case the idea is to enhance scalability and performance by distributing both the data and the application using multiple server machines. This could involve different types of servers such as application server, web server, and DB server. Three-tier architecture which is explained below is a specialized form of this architecture.

**Three-tier Architecture**

In this architecture, each application architecture layers (presentation, application, database) may run on separate processors. It therefore allows for better performance than a thin-client approach. It is simpler to manage than fat client approach and highly scalable (as demands increase add more servers).

A typical 3-tier architecture is depicted in the following diagram.



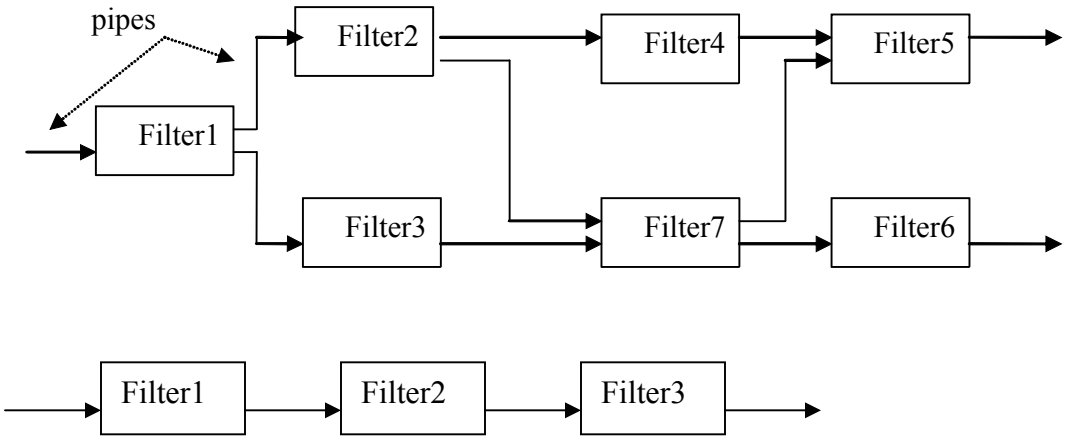

N-tier architecture generalizes the concepts of 3-tier architecture. In this case the system architecture may have more than 3 layers. That is, in n-tier architecture, in order to increase performance, we may distribute the application over different servers by putting different subsystems on different servers.


© Copyright Virtual University of Pakistan

### 8.12 Data Flow or Pipes and Filters Architecture

This architecture is very similar to data flow diagrams. This is used when the input data is processed through a series of transformations to yield the desired output. It is also known as pipes and filters architecture where each processing step is called a filter and the connecting link from one process to the other is called the pipe through which the information flows from one process to the other. An important aspect of this model is that each filter works independently of others and does not require knowledge of the working of any of the other filters including its neighbours.

If the dataflow has only a single sequence of processes with no alternative or parallel paths, then it is called batch sequential. These models are depicted in the following diagrams.

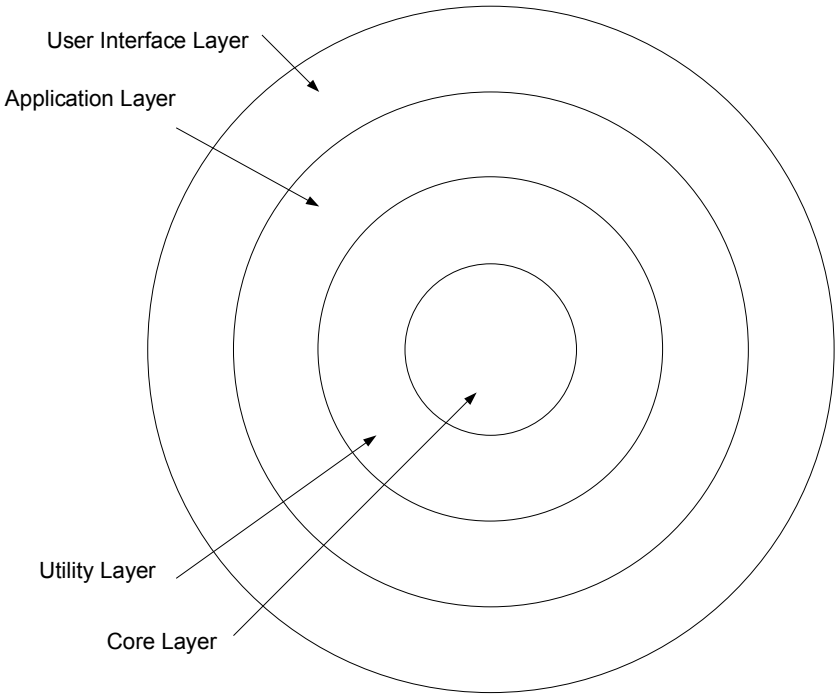



Batch Sequential


© Copyright Virtual University of Pakistan

**Layered Architecture**

As the name suggests, a layered architecture has many different layers. One typical example of a layered architecture is an operating system where different layers are used to provide services and functionality and the inner layers are closer to the machine hardware than the outer layers. In this way, each layer isolates the outer layer from inner complexities. In order to work properly, the outer layer only needs to know the interface provided by the inner layer. If there are any changes in the inner layer, as long as the interface does not change, the outer layer is not affected. This scheme tremendously portability of the system. The basic layered architecture is depicted in the following diagram.

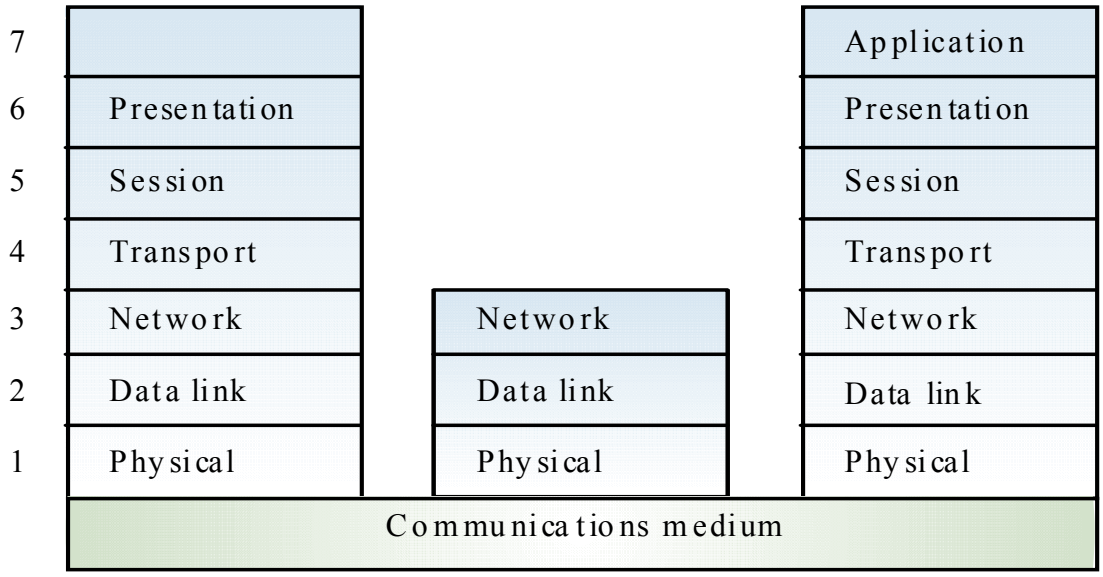




© Copyright Virtual University of Pakistan

### 8.13 Reference architectures

Reference architecture is not a physical architecture. It is only a reference for defining protocols and designing and implementing systems developed by different parties. Reference models are derived from a study of the application domain rather than from existing systems. It may be used as a basis for system implementation or to compare different systems. It acts as a standard against which systems can be evaluated. One very common example of such a reference model is the OSI model which is a layered model for communication systems. The success of this kind model is evident from the success of the Internet where all kinds of heterogeneous systems can talk to each other only because all of them use the same reference architecture.




OSI reference model



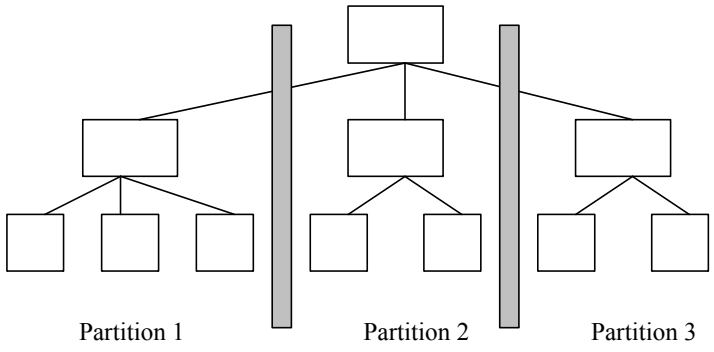
© Copyright Virtual University of Pakistan

### 8.14 Partitioning the Architecture

Partitioning of architecture is an important concept. What we basically want to do is distribute the responsibilities to different subsystems so that we get a software system which is easy to maintain. Partitioning results in a system that suffers from fewer side effects. This ultimately means that we get a system that is easier to test and extend and hence is easier to maintain.

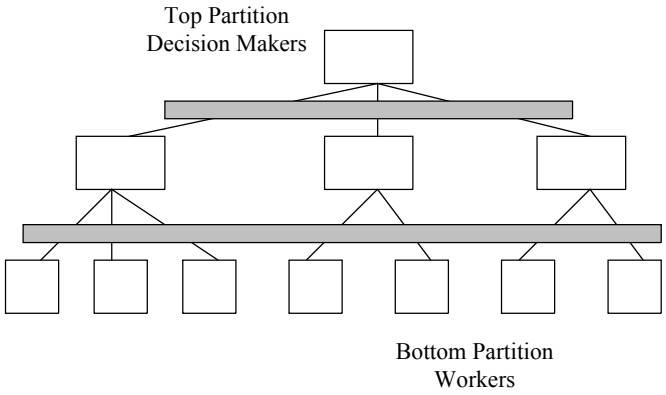
Partitioning of an architecture may be "horizontal" and/or "vertical".

In the horizontal partitioning we define separate branches of the module hierarchy for each major function and control modules are used to coordinate communication between functions. This concept is depicted in the following diagram.




Vertical partitioning divides the application from a decision making perspective. The architecture is partitioned in horizontal layers so that decision making and work are stratified with the decision making modules residing at the top of the hierarchy and worker coming at the bottom. This partitioning is also known as factoring and the general model is depicted in the following diagram.


© Copyright Virtual University of Pakistan




### 8.15 Analyzing Architecture design

In a given system, the required characteristics may conflict. Trade-offs seek optimal combinations of properties based on cost/benefit analysis. So the analysis requires an understanding of what is required and the relative priority of the attributes has to be established. The following sequence of steps provides a guideline for performing architectural analysis.

1. Collect scenarios.
2. Elicit requirements, constraints, and environment description.
3. Describe the architectural styles/patterns that have been chosen to address the scenarios and requirements. These views include module view, process view, and data flow view.
4. Evaluate quality attributes by considered each attribute in isolation.
5. Identify the sensitivity of quality attributes to various architectural attributes for a specific architectural style.

Critique candidate architectures (developed in step 3) using the sensitivity analysis conducted in step 5.


© Copyright Virtual University of Pakistan

# Lecture No. 26

Introduction to Design Patterns

## Design Patterns

Christopher Alexander says, "Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice." A Pattern Language: Towns/Buildings/Construction, 1977

Even though Alexander was talking about patterns in buildings and towns, what he says is true about object-oriented design patterns. Our solutions are expressed in terms of objects and interfaces instead of walls and doors, but the core of both kinds of patterns is a solution to a problem in a context.

## Design Patterns defined

"Description of communicating objects and classes that are customized to solve a general design in a particular context."

Patterns are devices that allow programs to share knowledge about their design. In our daily programming, we encounter many problems that have occurred, and will occur again. The question we must ask our self is how we are going to solve it *this* time. Documenting patterns is one way that you can reuse and possibly share the information that you have learned about how it is best to solve a specific program design problem.

Essay writing is usually done in a fairly well defined form, and so is documenting design patterns. The general form for documenting patterns is to define items such as:

- The motivation or context that this pattern applies to.
- Prerequisites that should be satisfied before deciding to use a pattern.
- A description of the program structure that the pattern will define.
- A list of the participants needed to complete a pattern.
- Consequences of using the pattern...both positive and negative.
- Examples!

## Historical perspective of design patterns

The origin of design patterns lies in work done by an architect named *Christopher Alexander* during the late 1970s. He began by writing two books, *A Pattern*


© Copyright Virtual University of Pakistan

*Language*[Alex77] and *A Timeless Way of Building* [Alex79] which, in addition to giving examples, described his rationalle for documenting patterns.

The pattern movement became very quiet until 1987 when patterns appeared again at an OOPSLA conference. Since then, many papers and presentations have appeared, authored by people such as Grady Booch, Richard Helm, and Erich Gamma, and Kent Beck. From then until 1995, many periodicals, featured articles directly or indirectly relating to patterns. In 1995, Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides published Design Patterns: Elements of Reusable Object-Oriented Software [Gamma95], which has been followed by more articles in trade journals.

The concept of design patterns is not new as we can find a number of similar pursuits in the history of program designing and writing. For instance, Standard Template Library (STL) is a library of reusable components provided by C++ compilers. Likewise, we use algorithms in data structures that implement typical operations of manipulating data in data structures. Another, similar effort was from Peter Coad whose patterns are known for object-oriented analysis and design.
Anti-patterns is another concept that corresponds to common mistakes in analysis and design. These are identified in order to prevent potential design and analysis defects from entering into the design. Another, similar concept is object-oriented framework that is a set of cooperative classes that make up reusable design of a system. Framework dictates the architecture of the software and describes the limitations and boundaries of architecture.
With this introduction, we now describe the format that has been adopted in GoF book for describing various design patterns.

# GOF Design Pattern Format

The basic template includes ten things as described below

## Name

- Works as idiom
- Name has to be meaningful

## Problem

- A statement of the problem which describes its intent
- The goals and objectives it wants to reach within the given context

## Context

- Preconditions under which the problem and its solutions seem to occur
- Result or consequence
- State or configuration after the pattern has been applied

## Forces

- Relevant forces and constraints and their interactions and conflicts.
- motivational scenario for the pattern.

## Solution

- Static and dynamic relationships describing how to realize the pattern.
- Instructions on how to construct the work products.


© Copyright Virtual University of Pakistan

- Pictures, diagrams, prose which highlight the pattern's structure, participants, and collaborations.

### Examples

- One or more sample applications to illustrate
  - a specific context
  - how the pattern is applied
-

### Resulting context

- the state or configuration after the pattern has been applied
- consequences (good and bad) of applying the pattern

### Rationale

- justification of the steps or rules in the pattern
- how and why it resolves the forces to achieve the desired goals, principles, and philosophies
- how are the forces orchestrated to achieve harmony
- how does the pattern actually work

### Related patterns

- the static and dynamic relationships between this pattern and other patterns

### Known uses

- to demonstrate that this is a proven solution to a recurring problem

## Classifications of deerns

### Creational patterns

- How to create and instantiate
- Abstract the instantiation process and make the system independent of its creational process.
- Class creational rules
- Object creational rules
- Abstract factory and factory method
- Abstract the instantiation process
- Make a system independent to its realization
- Class Creational use inheritance to vary the instantiated classes
- Object Creational delegate instantiation to an another object

### Structural patterns

- Deals with object's structure
- Class structural patterns concern the aggregation of classes to form the largest classes.
- Object structural patterns concerns the aggregation of objects to form the largest classes
- Class Structural patterns concern the aggregation of classes to form largest structures
- Object Structural pattern concern the aggregation of objects to form largest structures

### Behavioral patterns



© Copyright Virtual University of Pakistan

- Describe the patterns of communication between classes and objects
- How objects are communicating with each other
- Behavioral class patterns
- Behavioral object patterns
- Use object composition to distribute behavior between classes
- Help in distributing object's intelligence
- Concern with algorithms and assignment of responsibilities between objects
- Describe the patterns of communication between classes or objects
- Behavioral class pattern use inheritance to distribute behavior between classes
- Behavioral object pattern use object composition to distribute behavior between classes

In the following, one pattern from each of the above mentioned categories of design patterns is explained on GoF format.

# Lecture No. 27

## Observer Pattern

### Name

- Observer

### Basic intent

- It is intended to define a many to many relationship between objects so that when one object changes state all its dependants are notified and updated automatically.
- Dependence/publish-subscribe mechanism in programming language
    - Smalltalk being the first pure Object Oriented language in which observer pattern was used in implementing its Model View Controller (MVC) pattern. It was a publish-subscribe mechanism in which views (GUIs) were linked with their models (containers of information) through controller objects. Therefore, whenever underlying data changes in the model objects, the controller would notify the view objects to refresh themselves and vice versa.
    - MVC pattern was based on the observer pattern.

### Motivation

- It provides a common side effect of partitioning a system into a collection of cooperating classes that are in the need to maintain consistency between related objects.

### Description

- This can be used when multiple displays of state are needed.

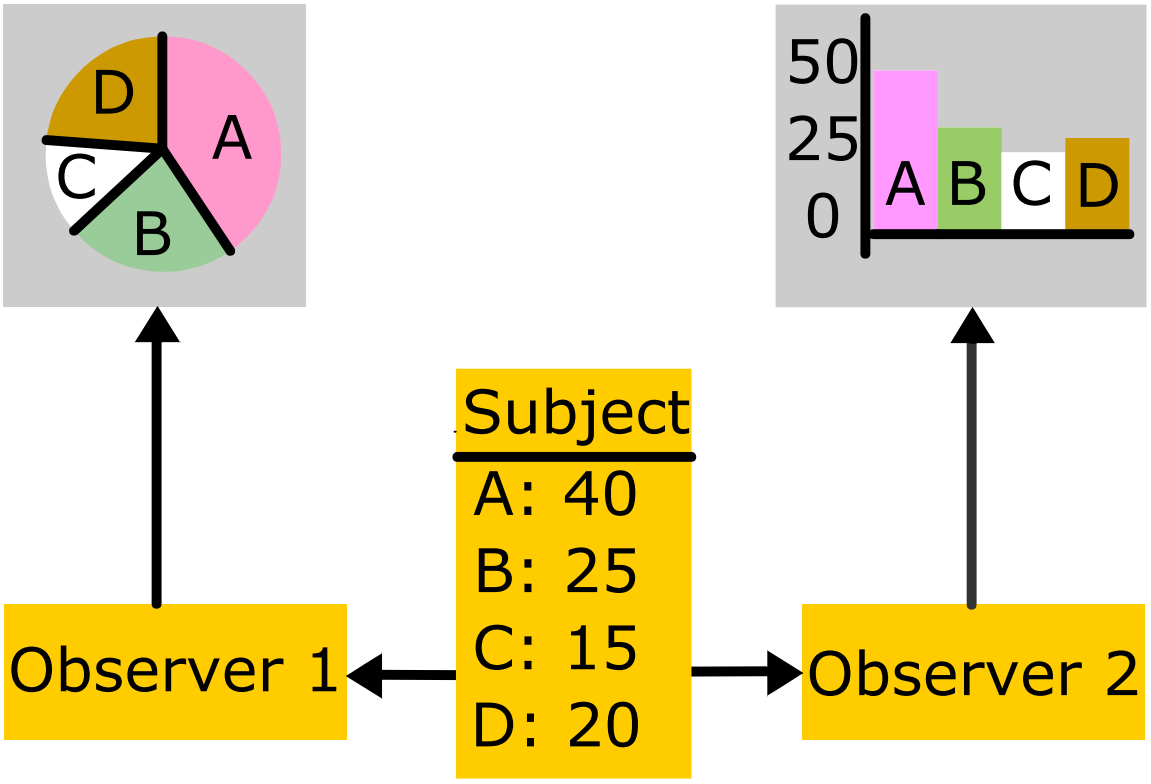
### Consequences

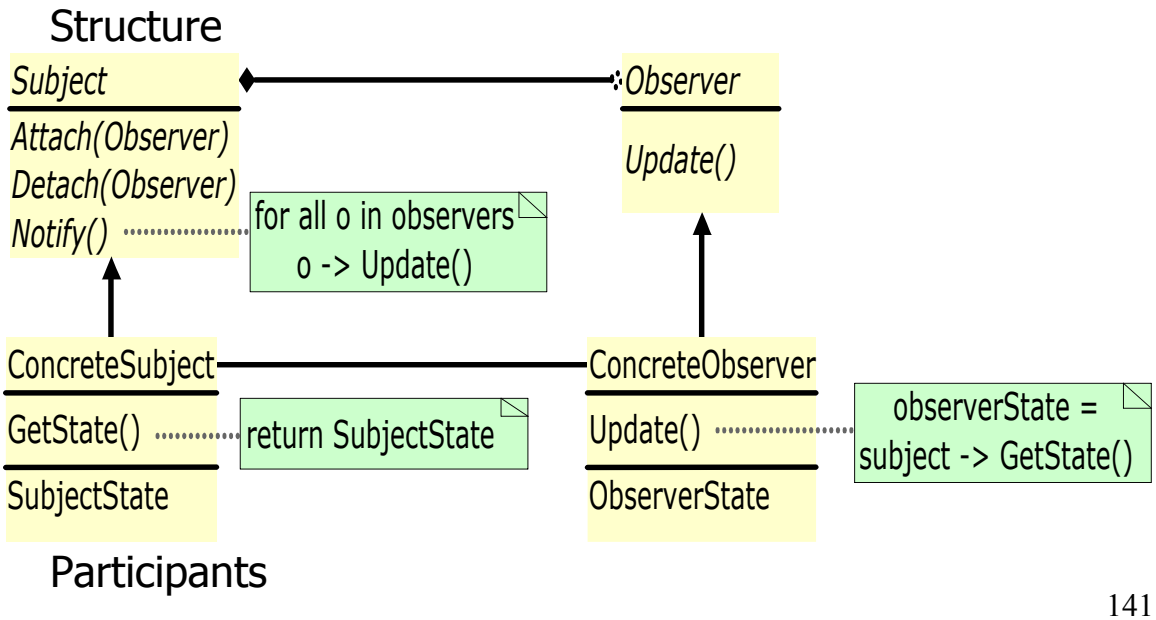
- **Optimizations to enhance display performance are impractical.**


© Copyright Virtual University of Pakistan

## Example implementation of Observer PatternObject Model

Many graphical user interface toolkits separate the presentational aspects of the user interface from the underlying application data. Classes defining application data and presentations can be reused independently. They can work together, too. Both a spreadsheet object and bar chart object can depict information in the same application data object using different presentations. The spreadsheet and the bar chart don't know about each other, thereby letting you reuse only the one you need. But they behave as though they do. When the user changes the information in the spreadsheet, the bar chart reflects the changes immediately, and vice versa.






© Copyright Virtual University of Pakistan

Subject

- Knows its observers. Any number of Observer objects may observe a subject.
- Provides an interface for attaching and detaching Observer objects.

Observer

- Defines an updating interface for objects that should be notified of changes in a subject.

ConcreteSubject

- Stores state of interest to concreteObserver objects.
- Sends a notification to its observers when its state changes.

ConcreteObserver

- Maintains a reference to a ConcreteSubject object.
- Stores state state that should stay consistent with the subject's.
- Implements the Observer updating interface to keep its state consistent with the subject's.

# Singleton Pattern

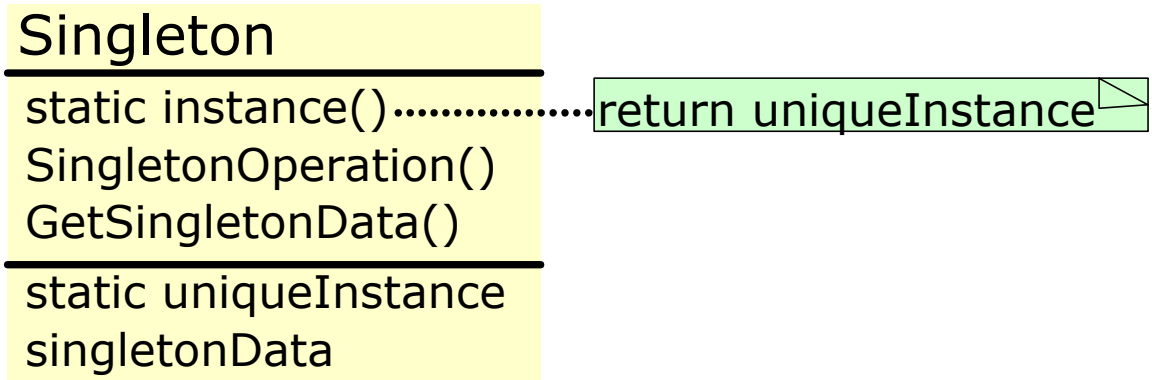
## Intent

- It ensures that a class only has one instance and provides a global point of access to it.

## Applicability

- Singleton pattern should be used when there must be exactly one instance of a class and it must be accessible to clients from a well-known access point.
- Singleton pattern should be used when controlling the total number of instances that would be created for a particular class.
- Singleton pattern should be used when the sole instance should be extensible by sub classing and clients should be able to use an extended instance without modifying their code.

## Structure




## Participants

Singleton



© Copyright Virtual University of Pakistan

- Defines an instance operation that lets clients access its unique instance. Instance is a class operation (that is, a class method in Smalltalk and a static member function in C++).
- May be responsible for creating its own unique instance.

### Singleton Pattern Example

The Singleton class is declared as

```
class Singleton {
public:
    static Singleton* Instance();
protected:
    Singleton();
private:
    static Singleton* _instance;
};
```

The corresponding implementation is

```
Singleton* Singleton::_instance = 0;

Singleton* Singleton::Instance(){
     if (_instance == 0) {
        _instance = new Singleton;
     }
     return _instance;
}
```

Clients access the singleton exclusively through the Instance member function. The variable _instance is initialized to 0, and the static member function Instance returns its value, initializing it with the unique instance if it is 0.

## Façade Pattern

### Intent

- It provides a unified interface to a set of interfaces in a sub-system.
- Façade defines a higher level interface that makes a subsystem easier to use

### Applicability

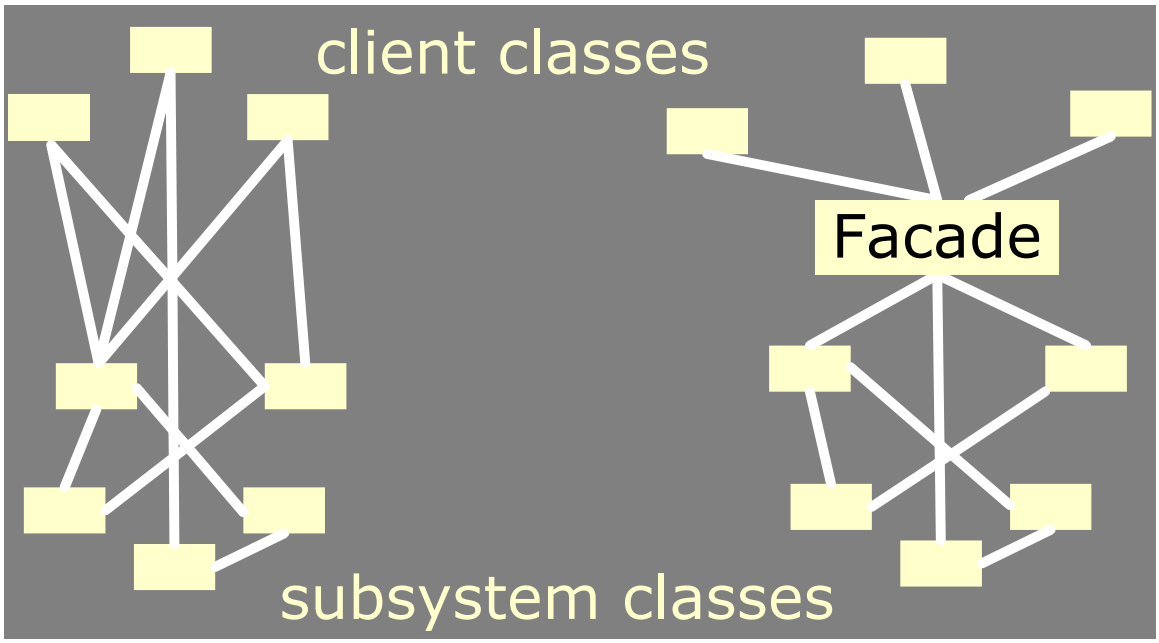
- You would use façade when you want to provide a simple interface to a complex sub-system.
- You would use façade pattern when there are many dependencies between clients and the implementation classes of an abstraction.
- You should introduce a façade to decouple the system from clients and other subsystems.
- You want to layer your subsystem.
- You would use façade when you want to provide a simple interface to a complex sub-system


© Copyright Virtual University of Pakistan

- You would use façade pattern when there are many dependencies between clients and the implementation classes of an abstraction
- You should introduce a façade to decouple the system from clients and other subsystems
- You want to layer the subsystem

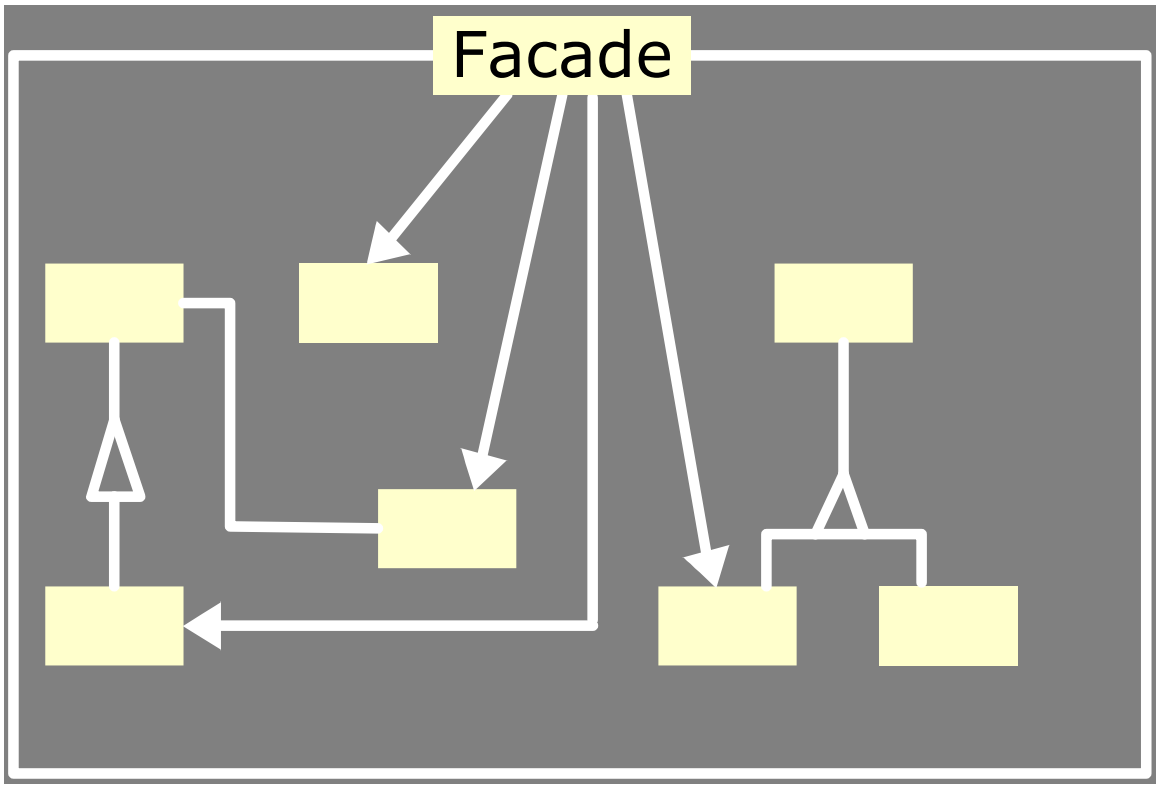
## Abstract example of façade

Structuring a system into subsystems helps reduce complexity. A common design goal is to minimize the communication and dependencies between subsystems. One way to achieve this goal is to introduce a façade object that provides a single, simplified interface to the more general facilities of a subsystem.




Structure


© Copyright Virtual University of Pakistan




## Participants

Façade

- Knows which subsystem classes are responsible for a request.
- Delegates client requests to appropriate subsystem objects.

Subsystem classes

- Implement subsystem functionality.
- Handle work assigned by the Façade object.
- Have no knowledge of the façade, that is, they keep no references to it.



© Copyright Virtual University of Pakistan

# Lecture No. 28

Good Programming Practices and Guidelines

## 10.1 Maintainable Code

As we have discussed earlier, in most cases, maintainability is the most desirable quality of a software artifact. Code is no exception. Good software ought to have code that is easy to maintain. Fowler says, "*Any fool can write code that computers can understand, good programmers write code that humans can understand."* That is, it is not important to write code that works, it is important to write code that works and is easy to understand so that it can be maintained. The three basic principles that guide maintainability are: simplicity, clarity, and generality or flexibility. The software will be easy to maintain if it is easy to understand and easy to enhance. Simplicity and clarity help in making the code easier to understand while flexibility facilitates easy enhancement to the software.



© Copyright Virtual University of Pakistan

### Self Documenting Code

From a maintenance perspective, what we need is what is called ***self documenting code***. A *self documenting code* is a code that explains itself without the need of comments and extraneous documentation, like flowcharts, UML diagrams, process-flow state diagrams, etc. That is, the meaning of the code should be evident just by reading the code without having to refer to information present outside this code.

The question is: how can we write code that is self-documenting?

There are a number of attributes that contributes towards making the program self documented. These include, the size of each function, choice of variable and other identifier names, style of writing expressions, structure of programming statements, comments, modularity, and issues relating to performance and portability.

The following discussion tries to elaborate on these points.

## Function Size

The size of individual functions plays a significant role in making the program easy or difficult to understand. In general, as the function becomes longer in size, it becomes more difficult to understand. Ideally speaking, a function should not be larger than 20 lines of code and in any case should not exceed one page in length. From where did I get this number 20 and one page? The number 20 is approximately the lines of code that fit on a computer screen and one page of course refers to one printed page. The idea behind these heuristics is that when one is reading a function, one should not need to go back and forth from one screen to the other or from one page to the other and the entire context should be present on one page or on one screen.


© Copyright Virtual University of Pakistan

## Identifier Names

Identifier names also play a significant role in enhancing the readability of a program. The names should be chosen in order to make them meaningful to the reader. In order to understand the concept, let us look at the following statement.

```
if (x==0)   // this is the case when we are allocating a new number
```

In this particular case, the meanings of the condition in the if-statement are not clear and we had to write a comment to explain it. This can be improved if, instead of using x, we use a more meaningful name. Our new code becomes:

```
if (AllocFlag == 0)
```

The situation has improved a little bit but the semantics of the condition are still not very clear as the meaning of 0 is not very clear. Now consider the following statement:

```
If (AllocFlag == NEW_NUMBER)
```

We have improved the quality of the code by replacing the number 0 with a named constant NEW_NUMBER. Now, the semantics are clear and do not need any extra comments, hence this piece of code is self-documenting.



© Copyright Virtual University of Pakistan

## 10.2 Coding Style Guide

Consistency plays a very important role in making it self-documenting. A consistently written code is easier to understand and follow. A coding style guide is aimed at improving the coding process and to implement the concept of standardized and relatively uniform code throughout the application or project. As a number of programmers participate in developing a large piece of code, it is important that a consistent style is adopted and used by all. Therefore, each organization should develop a style guide to be adopted by its entire team.

This coding style guide emphasizes on C++ and Java but the concepts are applicable to other languages as well.

## 10.3 Naming Conventions

Hungarian Notation was first discussed by Charles Simonyi of Microsoft. It is a variable naming convention that includes information about the variable in its name (such as data type, whether it is a reference variable or a constant variable, etc). Every company and programmer seems to have their own flavor of Hungarian Notation. The advantage of Hungarian notation is that by just looking at the variable name, one gets all the information needed about that variable.

*Bicapitalization* or *camel case* (frequently written *CamelCase*) is the practice of writing compound words or phrases where the terms are joined without spaces, and every term is capitalized. The name comes from a supposed resemblance between the bumpy outline of the compound word and the humps of a camel. CamelCase is now the official convention for file names and identifiers in the Java Programming Language.

In our style guide, we will be using a naming convention where Hungarian Notation is mixed with CamelCase.



© Copyright Virtual University of Pakistan

## General Naming Conventions

**General naming conventions for Java and C++**

1. Names representing types must be nouns and written in mixed case starting with upper case.

```
Line, FilePrefix
```

2. Variable names must be in mixed case starting with lower case.

```
line, filePrefix
```

This makes variables easy to distinguish from types, and effectively resolves potential naming collision as in the declaration `Line line;`

3. Names representing constants must be all uppercase using underscore to separate words.

```
MAX_ITERATIONS, COLOR_RED
```

In general, the use of such constants should be minimized. In many cases implementing the value as a method is a better choice. This form is both easier to read, and it ensures a uniform interface towards class values.

```
int    getMaxIterations()//    NOT:    MAX_ITERATIONS    =    25
{
                              return                          25;
}
```

4. Names representing methods and functions should be verbs and written in mixed case starting with lower case.

```
getName(), computeTotalWidth()
```

5. Names representing template types in C++ should be a single uppercase letter.

```
template<class                        T>                      ...
template<class C, class D> ...
```

6. Global variables in C++ should always be referred to by using the :: operator.
**::mainWindow.open() , ::applicationContext.getName()**

7. Private class variables should have _ suffix.

```
class                                                    SomeClass
{
                private              int                 length_;
                                                              ...
}
```

Apart from its name and its type, the *scope* of a variable is its most important feature. Indicating class scope by using _ makes it easy to distinguish class variables from local scratch variables.


© Copyright Virtual University of Pakistan

8. Abbreviations and acronyms should not be uppercase when used as name.

```
exportHtmlSource();          //    NOT:    xportHTMLSource();
openDvdPlayer();       // NOT: openDVDPlayer();
```

Using all uppercase for the base name will give conflicts with the naming conventions given above. A variable of this type would have to be named dVD, hTML etc. which obviously is not very readable.

9. Generic variables should have the same name as their type.

```
void setTopic (Topic topic)        // NOT: void setTopic (Topic value)
                                   // NOT: void setTopic (Topic aTopic)
                                       // NOT: void setTopic (Topic x)

void connect (Database database)  // NOT: void connect (Database db)
                                 // NOT: void connect (Database oracleDB)
```

Non-generic variables have a *role*. These variables can often be named by combining role and type:

```
Point              startingPoint,              centerPoint;
Name  loginName;
```

10. All names should be written in English.

```
fileName;    // NOT:   filNavn
```

11. Variables with a large scope should have long names, variables with a small scope can have short names. Scratch variables used for temporary storage or indices are best kept short. A programmer reading such variables should be able to assume that its value is not used outside a few lines of code. Common scratch variables for integers are *i*, *j*, *k*, *m*, *n* and for characters *c* and *d*.

12. The name of the object is implicit, and should be avoided in a method name.

```
line.getLength();    // NOT:  line.getLineLength();
```

The latter seems natural in the class declaration, but proves superfluous in use.



© Copyright Virtual University of Pakistan

**Specific Naming Conventions for Java and C++**

1. The terms *get/set* must be used where an attribute is accessed directly.

```
employee.getName();
matrix.getElement (2, 4);
employee.setName (name);
matrix.setElement (2, 4, value);
```

*2. is* prefix should be used for boolean variables and methods.

```
isSet, isVisible, isFinished, isFound, isOpen
```

Using the *is* prefix solves a common problem of choosing bad boolean names like *status* or *flag*. *isStatus* or *isFlag* simply doesn't fit, and the programmer is forced to chose more meaningful names.

There are a few alternatives to the *is* prefix that fits better in some situations. These are *has*, *can* and *should* prefixes:

```
boolean hasLicense();
boolean canEvaluate();
boolean shouldAbort = false;
```

3. The term *compute* can be used in methods where something is computed.

```
valueSet.computeAverage();  matrix.computeInverse()
```

Using this term will give the reader the immediate clue that this is a potential time consuming operation, and if used repeatedly, he might consider caching the result.

4. The term *find* can be used in methods where something is looked up.

```
vertex.findNearestVertex();   matrix.findMinElement();
```

This gives the reader the immediate clue that this is a simple look up method with a minimum of computations involved.

5. The term *initialize* can be used where an object or a concept is established.

```
printer.initializeFontSet();
```

*6. List* suffix can be used on names representing a list of objects.

`vertex` (one vertex), `vertexList` (a list of vertices)

Simply using the plural form of the base class name for a list (`matrixElement` (one matrix element), `matrixElements` (list of matrix elements)) should be avoided since the two only differ in a single character and are thereby difficult to distinguish.
A *list* in this context is the compound data type that can be traversed backwards, forwards, etc. (typically a `Vector`). A plain array is simpler. The suffix *Array* can be used to denote an array of objects.

*7. n* prefix should be used for variables representing a number of objects.

```
nPoints, nLines
```


© Copyright Virtual University of Pakistan

The notation is taken from mathematics where it is an established convention for indicating a number of objects.

*8. No* suffix should be used for variables representing an entity number.

```
tableNo, employeeNo
```

The notation is taken from mathematics where it is an established convention for indicating an entity number. An elegant alternative is to prefix such variables with an *i*: `iTable, iEmployee`. This effectively makes them *named* iterators.

9. Iterator variables should be called ***i***, ***j***, ***k*** etc.

```
while (Iterator i = pointList.iterator(); i.hasNext(); ) {
  :
}

for (int i = 0; i < nTables; i++) {
  :
}
```

The notation is taken from mathematics where it is an established convention for indicating iterators. Variables named *j*, *k* etc. should be used for nested loops only.

10. Complement names must be used for complement entities.

get/set, add/remove, create/destroy, start/stop, insert/delete, increment/decrement, old/new, begin/end, first/last, up/down, min/max, next/previous, old/new, open/close, show/hide
Reduce complexity by symmetry.

11. Abbreviations in names should be avoided.

```
computeAverage();      // NOT:  compAvg();
```

There are two types of words to consider. First are the common words listed in a language dictionary. These must never be abbreviated. Never write:

| | | |
|---|---|---|
| `cmd` | instead of | `command` |
| `cp` | instead of | `copy` |
| `pt` | instead of | `point` |
| `comp` | instead of | `compute` |
| `init` | instead of | `initialize` |

etc.

Then there are domain specific phrases that are more naturally known through their acronym or abbreviations. These phrases should be kept abbreviated. Never write:

| | | |
|---|---|---|
| `HypertextMarkupLanguage` | instead of | `html` |
| `CentralProcessingUnit` | instead of | `cpu` |
| `PriceEarningRatio` | instead of | `pe` |

etc.

12. Negated boolean variable names must be avoided.


© Copyright Virtual University of Pakistan

```
boolean isError;     // NOT:   isNotError
boolean isFound;     // NOT:   isNotFound
```

The problem arise when the logical **NOT** operator is used and double negative arises. It is not immediately apparent what `!isNotError` means.

13. Functions (methods returning an object) should be named after what they return and procedures (*void* methods) after what they do. This increases readability. Makes it clear what the unit should do and especially all the things it is *not* supposed to do. This again makes it easier to keep the code clean of side effects. Naming pointers in C++ specifically should be clear and should represent the pointer type distinctly.

```
Line *line          //NOT    Line *pLine; or Line *lineptr; etc
```



© Copyright Virtual University of Pakistan

# Lecture No. 29

## 10.4 File handling tips for Java and C++

1. C++ header files should have the extension *.h*. Source files can have the extension *.c++* (recommended), *.C*, *.cc* or *.cpp*.

```
MyClass.c++, MyClass.h
```

These are all accepted C++ standards for file extension.

2. Classes should be declared in individual header files with the file name matching the class name. Secondary private classes can be declared as inner classes and reside in the file of the class they belong to. All definitions should reside in source files.

```
class MyClass
{
  public:
    int getValue () {return value_;} // NO!
    ...
  private:
    int value_;
}
```

The header files should declare an interface, the source file should implement it. When looking for an implementation, the programmer should always know that it is found in the source file. The obvious exception to this rule is of course inline functions that must be defined in the header file.

3. Special characters like TAB and page break must be avoided. These characters are bound to cause problem for editors, printers, terminal emulators or debuggers when used in a multi-programmer, multi-platform environment.



© Copyright Virtual University of Pakistan

4. The incompleteness of split lines must be made obvious.

```
totalSum = a + b + c +
                  d + e);
    function (param1, param2,
                param3);
     setText ("Long line split"
                      "into two parts.");
    for (tableNo = 0; tableNo < maxTable;
                     tableNo += tableStep)
```

Split lines occurs when a statement exceed the 80 column limit given above. It is difficult to give rigid rules for how lines should be split, but the examples above should give a general hint.

In general:

- Break after a comma.
- Break after an operator.
- Align the new line with the beginning of the expression on the previous line.



© Copyright Virtual University of Pakistan

## Include Files and Include Statements for Java and C++

1. Header files must include a construction that prevents multiple inclusion. The convention is an all uppercase construction of the module name, the file name and the h suffix.

```
#ifndef MOD_FILENAME_H
#define MOD_FILENAME_H
 :
#endif
```

The construction is to avoid compilation errors. The construction should appear in the top of the file (before the file header) so file parsing is aborted immediately and compilation time is reduced.

## Classes and Interfaces

Class and Interface declarations should be organized in the following manner:

1. Class/Interface documentation.
2. `class` or `interface` statement.
3. Class (**`static`**) variables in the order **`public`**, **`protected`**, package (no access modifier), **`private`**.
4. Instance variables in the order **`public`**, **`protected`**, package (no access modifier), `private`.
5. Constructors.
6. Methods (no specific order).


© Copyright Virtual University of Pakistan

## 10.5 Statements in Java and C++

### Types

1. Type conversions must always be done explicitly. Never rely on implicit type conversion.

```
floatValue = (float) intValue;     // NOT:    floatValue = intValue;
```

By this, the programmer indicates that he is aware of the different types involved and that the mix is intentional.

2. Types that are local to one file only can be declared inside that file.

3. The parts of a class must be sorted *public*, *protected* and *private*. All sections must be identified explicitly. Not applicable sections should be left out. The ordering is *"most public first"* so people who only wish to use the class can stop reading when they reach the `protected/private` sections.

### Variables

1. Variables should be initialized where they are declared and they should be declared in the smallest scope possible.

2. Variables must never have dual meaning. This enhances readability by ensuring all concepts are represented uniquely. Reduce chance of error by side effects.

3. Class variables should never be declared public. The concept of information hiding and encapsulation is violated by public variables. Use private variables and access functions instead. One exception to this rule is when the class is essentially a data structure, with no behavior (equivalent to a C++ `struct`). In this case it is appropriate to make the class' instance variables public.

4. Related variables of the same type can be declared in a common statement. Unrelated variables should not be declared in the same statement.

```
float                 x,                 y,                 z;
float  revenueJanuary, revenueFebrury, revenueMarch;
```

The common requirement of having declarations on separate lines is not useful in the situations like the ones above. It enhances readability to group variables.

5. Variables should be kept alive for as short a time as possible. Keeping the operations on a variable within a small scope, it is easier to control the effects and side effects of the variable.


© Copyright Virtual University of Pakistan

6. Global variables should not be used. Variables should be declared only within scope of their use. Same is recommended for global functions or file scope variables. It is easier to control the effects and side effects of the variables if used in limited scope.

7. Implicit test for *0* should not be used other than for boolean variables and pointers.

```
if (nLines != 0)     // NOT:   if (nLines)
if (value != 0.0)    // NOT:   if (value)
```

It is not necessarily defined by the compiler that ints and floats 0 are implemented as binary 0. Also, by using explicit test the statement give immediate clue of the type being tested. It is common also to suggest that pointers shouldn't test implicit for 0 either, i.e. `if (line == 0)` instead of `if (line)`. The latter is regarded as such a common practice in C/C++ however that it can be used.

**Loop structures**

1. Only loop control statements must be included in the *for()* construction.

```
sum = 0;                    // NOT:     for (i=0, sum=0; i<100; i++)
for (i=0; i<100; i++)       //             sum += value[i];
  sum += value[i];
```

2. Loop variables should be initialized immediately before the loop.

```
boolean done = false;  // NOT:   boolean  done = false;
while (!done) {        //          :
  :                    //        while (!done) {
}                      //          :
                                  }
```

3. The use of *do .... while* loops should be avoided. There are two reasons for this. First is that the construct is superflous; Any statement that can be written as a *do .... while* loop can equally well be written as a *while* lopp or a *for* loop. Complexity is reduced by minimizing the number of constructs being used. The other reason is of readability. A loop with the conditional part at the end is more difficult to read than one with the conditional at the top.

4. The use of *break* and *continue* in loops should be avoided. These statements should only be used if they prove to give higher readability than their structured counterparts. In general *break* should only be used in *case* statements and *continue* should be avoided alltogether.

5. The form *for (;;)* should be used for empty loops.

```
for (;;) {             // NOT:   while (true) {
  :                    //          :
}                      //        }
```


© Copyright Virtual University of Pakistan

This form is better than the functionally equivalent *while (true)* since this implies a test against *true*, which is neither necessary nor meaningful. The form while(true) should be used for infinite loops.

**Conditionals**

1. Complex conditional expressions must be avoided. Introduce temporary boolean variables instead.

```
if ((elementNo < 0) || (elementNo > maxElement)||
    elementNo == lastElement) {
  :
}
```

should be replaced by:

```
boolean isFinished      = (elementNo < 0) || (elementNo > maxElement);
boolean isRepeatedEntry = elementNo == lastElement;
if (isFinished || isRepeatedEntry) {
  :
}
```

2. The nominal case should be put in the *if*-part and the exception in the *else*-part of an if statement.

```
boolean isError = readFile (fileName);
if (!isError) {
  :
}
else {
  :
}
```

3. The conditional should be put on a separate line.

```
if (isDone)               // NOT:  if (isDone) doCleanup();
  doCleanup();
```

4. Executable statements in conditionals must be avoided.

```
file = openFile (fileName, "w");  // NOT:   if ((file = openFile
(fileName, "w")) != null) {
if (file != null) {               //          :
  :                               //        }
}
```


© Copyright Virtual University of Pakistan

**Miscellaneous**

1. The use of magic numbers in the code should be avoided. Numbers other than *0* and *1* should be considered declared as named constants instead.

2. Floating point constants should always be written with decimal point and at least one decimal.

```
double total = 0.0;    // NOT: double total = 0;
double speed = 3.0e8; // NOT: double speed = 3e8;

double sum;
:
sum = (a + b) * 10.0;
```

This emphasizes the different nature of integer and floating point numbers even if their values might happen to be the same in a specific case. Also, as in the last example above, it emphasize the type of the assigned variable (`sum`) at a point in the code where this might not be evident.

3. Floating point constants should always be written with a digit before the decimal point.

```
double total = 0.5;    // NOT: double total = .5;
```

The number and expression system in Java is borrowed from mathematics and one should adhere to mathematical conventions for syntax wherever possible. Also, 0.5 is a lot more readable than .5; There is no way it can be mixed with the integer 5.

4. Functions in C++ must always have the return value explicitly listed.

```
int getValue()      // NOT: getValue()
{
  :
}
```

If not explicitly listed, C++ implies `int` return value for functions.

5. `goto` in C++ should not be used. Goto statements violates the idea of structured code. Only in some very few cases (for instance breaking out of deeply nested structures) should goto be considered, and only if the alternative structured counterpart is proven to be less readable.


© Copyright Virtual University of Pakistan

# Lecture No. 30

## 10.6 Layout and Comments in Java and C++

### Comments

The problem with comments is that they lie. Comments are not syntax checked, there is nothing forcing them to be accurate. And so, as the code undergoes change during schedule crunches, the comments become less and less accurate.

As Fowler puts it, comments should not be used as deodorants. Tricky code should not be commented but rewritten. In general, the use of comments should be minimized by making the code self-documenting by appropriate name choices and an explicit logical structure.

If, however, there is a need to write comments for whatever reason, the following guidelines should be observed.

1. All comments should be written in English. In an international environment English is the preferred language.

2. Use // for all comments, including multi-line comments.

```
// Comment spanning
// more than one line
```

Since multilevel commenting is not supported in C++ and Java, using // comments ensure that it is always possible to comment out entire sections of a file using /* */ for debugging purposes etc.

3. Comments should be indented relative to their position in the code.

```
while (true) {          // NOT:    while (true) {
  // Do something       //         // Do something
  something();          //             something();
}                       //         }
```



© Copyright Virtual University of Pakistan

# 10.7 Expressions and Statements

## Layout

1. Basic indentation should be 2.

```
for (i = 0; i < nElements; i++)
  a[i] = 0;
```

Indentation of 1 is to small to emphasize the logical layout of the code. Indentation larger than 4 makes deeply nested code difficult to read and increase the chance that the lines must be split. Choosing between indentation of 2, 3 and 4, 2 and 4 are the more common, and 2 chosen to reduce the chance of splitting code lines.

#### Natural form for expression

Expression should be written as if they are written as comments or spoken out aloud. Conditional expression with negation are always difficult to understand. As an example consider the following code:

if (! (block < activeBlock) || !(blockId >= unblocks))

The logic becomes much easier to follow if the code is written in the natural form as shown below:

if ((block >= activeBlock) || (blockId < unblocks))

## Parenthesize to remove ambiguity

Parentheses should always be used as they reduce complexity and clarify things by specifying grouping. It is especially important to use parentheses when different unrelated operators are used in the same expression as the precedence rules are often assumed by the programmers, resulting in logical errors that are very difficult to spot. As an example consider the following statement:

if (x & MASK == BITS)

This causes problems because == operator has higher precedence than & operator. Hence, MASK and BITS are first compared for equality and then the result, which is 0 or 1, is andded with x. This kind of error will be extremely hard to catch. If, however, parentheses are used, there will be no ambiguity as shown below.

if ((x & MASK) == BITS)



© Copyright Virtual University of Pakistan

Following is another example of the use of parentheses which makes the code easier to understand and hence easier to maintain.

```
leapYear = year % 4 == 0  && year % 100 != 0 || year % 400 == 0 ;
```

In this case parentheses have not been used and therefore the definition of a leap year is not very clear for the code. The code becomes self explanatory with the help of proper use of parentheses as shown below:

```
leapYear = ((year % 4 == 0)  && (year % 100 != 0))  ||
                    (year % 400 == 0);
```

## Breakup complex expressions

Complex expressions should be broken down into multiple statements. An expression is considered to be complex if it uses many operators in a single statement. As an example consider the following statement:

```
*x += (*xp=(2*k < (n-m) ? c[k+1] : d[k--]));
```

This statement liberally uses a number of operators and hence is very difficult to follow and understand. If it is broken down into simple set of statements, the logic becomes easier to follow as shown below:

```
if (2*k < n-m)
        *xp = c[k+1];
else
        *xp = d[k--];
*x = *x + *xp;
```

## 10.8 Shortcuts and cryptic code

Sometimes the programmers, in their creative excitement, try to write very concise code by using shortcuts and playing certain kinds of tricks. This results in a code which is cryptic in nature and hence is difficult to follow. Maintenance of such code therefore becomes a nightmare. Following are some examples of such code.

1. Let us start with a very simple shortcut, often used by programmers. Assume that we have the following statement.

   ```
   x *= a;
   ```

   For some reason the code was later modified to:



© Copyright Virtual University of Pakistan

```
x *= a + b;
```

This seemingly harmless change is actually a little cryptic and causes confusion. The problem lies with the semantics of this statement. Does it mean x = x*a+b or x = x*(a+b)? The second one is the right answer but is not obvious from the syntax and hence causes problems.

2. Let us now look at a more complex example. What is the following code doing?

```
subkey = subkey >> (bitoff – (bitoff >> 3) << 3));
```

As can be seen, it is pretty hard to understand and therefore difficult to debug in case there is any problem. What this code is actually doing is masking bitoff with octal 7 and then use the result to shift subkey those many time. This can be written as follows:

```
subkey = subkey >> (bitoff & 0x7);
```

It is quite evident that the second piece of code is much follow to read than the first one.

3. The following piece of code is taken from a commercial software:

```
a = a >> 2;
```

It is easy to see that a is shifted right two times. However, the real semantics of this code are hidden – the real intent here is to divide a by 4. No doubt that the code above achieves this objective but it is hard for the reader to understand the intent as to why a is being shifted right twice. It would have been much better had the code been written as follows:

```
a = a/4;
```

4. A piece of code similar to the following can be found in many data structures books and is part of circular implementation of queues using arrays.

```
bool Queue::add(int n)
{
        int k = (rear+1) % MAX_SIZE;
        if (front == k)
                return false;
        else {
                rear = k;
                queue[rear] = n;
                return true;
        }
}
```



© Copyright Virtual University of Pakistan

```
bool Queue::isFull()
{
        if (front == (rear+1 % MAX_SIZE))
                return true;
        else
                return false;
}
```

This code uses the % operator to set the rear pointer to 0 once it has reached MAX_SIZE. This is not obvious immediately. Similarly, the check for queue full is also not trivial.

In an experiment, the students were asked to implement double-ended queue in which pointer could move in both directions. Almost everyone made the mistake of writing something like rear = (rear-1) % MAX_SIZE. This is because the semantics of % operation are not obvious.

It is always much better to state the logic explicitly. Also, counting is also much easier to understand and code as compared to some tricky comparisons (e.g. check for isFull in this case). Application of both these principles resulted in the following code. It is easy to see that this code is easier to understand. It is interesting to note that when another group of students were asked to do implement double-ended queue after showing them this code, almost everyone did it without any problems.

```
bool Queue::add()
{
        if (! isFull() ) {
                rear++;
                if (rear == MAX_SIZE) rear = 0;
                QueueArray[rear] = n;
                size++;
                return true;
        }
        else return false;
}

bool Queue::isFull(int n)
{
        if (size == MAX_SIZE)
                return true;
        else
                return false;
}
```



© Copyright Virtual University of Pakistan

# Lecture No. 31

## Coding Style Guidelines (Continued)

### Switch Statement

In the switch statement, cases should always end with a break. A tricky sequence of fall-through code like the one below causes more trouble than being helpful.

```
switch(c) {
case '-'  : sign = -1;
case '+'  : c = getchar();
case '.'  : break;
default   : if (! isdigit(c))
                        return 0;
}
```

This code is cryptic and difficult to read. It is much better to explicitly write what is happening, even at the cost of duplication.

```
switch(c) {
case '-':   sign = -1;
                c = getchar();
                break;
case '+':   c = getchar();
                break;
case '.':   break;
default:    if (! isdigit(c))
                    return 0;
                break;
}
```

It would even be better if such code is written using the if statement as shown below.

```
if (c == '-') {
        sign = -1;
        c = getchar();
}
else if (c == '+') {
        c = getchar();
}
else if (c != '.' && !isdigit(c)) {
        return 0;
}
```



© Copyright Virtual University of Pakistan

## Magic Numbers

Consider the following code segment:

```
fac = lim / 20;
if (fac < 1)
        fac = 1;
for (i =0, col = 0; i < 27; i++, j++) {
        col += 3;
        k = 21 – (let[i] /fac);
        star = (let[i] == 0) ? ‘ ’ : ‘*’;
        for (j = k; j < 22; j++)
                draw(j, col, star);
}
draw(23, 1, ‘ ’);
for (i=‘A’; i <= ‘Z’; i++)
        cout << i;
```

Can you tell by reading the code what is meant by the numbers 20, 27, 3, 21, 22, and 23. These are constant that mean something but they do not give any indication of their importance or derivation, making the program hard to understand and modify. To a reader they work like magic and hence are called magic numbers. Any number (even 0 or 1) used in the code is a magic number. It should rather have a name of its own that can be used in the program instead of the number.

The difference would be evident if we look at the code segment below that achieves the same purpose as the code above.

```
enum {
        MINROW = 1,
        MINCOL = 1,
        MAXROW = 24,
        MAXCOL = 80,
        LABELROW = 1,
        NLET = 26,
        HEIGHT = MAXROW –4,
        WIDTH = (MAXCOL-1) / NLET
};

fac = (lim+HEIGHT-1) /HEIGHT;
if (fac < 1)
        fac = 1;
for (i =0; i < NLET; i++) {
        if (let[i] == 0)
                continue;
        for (j = HEIGHT – let[i] / fac; j < HEIGHT; j++)
                draw(j-1 + LABELROW,
                        (i+1)*WIDTH, ‘*’);
}

draw(MAXROW-1, MINCOL+1, ‘ ’);
```



© Copyright Virtual University of Pakistan

```
for (i='A'; i <= 'Z'; i++)
        cout << i;
```

## Use (or abuse) of Zero

The number 0 is the most abused symbol in programs written in C or C++. One can easily find code segment that 0 in a fashion similar to the examples below in almost every C/C++ program.

```
flag = 0;          // flag is boolean
str = 0;           // str is string
name[i] = 0;       // name is char array
x = 0;             // x is floating pt
i = 0;             // i is integer
```

This is a legacy of old style C programming. It is much better to use symbols to explicitly indicate the intent of the statement. It is easy to see that the following code is more in line with the self-documentation philosophy than the code above.

```
flag = false;
str = NULL;
name[i] = '\0';
x = 0.0;
i = 0;
```



© Copyright Virtual University of Pakistan

# Lecture No. 32

## 10.9 Clarity through modularity

As mentioned earlier, abstraction and encapsulation are two important tools that can help in managing and mastering the complexity of a program. We also discussed that the size of individual functions plays a significant role in making the program easy or difficult to understand. In general, as the function becomes longer in size, it becomes more difficult to understand. Modularity is a tool that can help us in reducing the size of individual functions, making them more readable. As an example, consider the following selection sort function.

```
void selectionSort(int a[], int size)
{
        int i, j;
        int temp;
        int min;

        for (i = 0; i < size-1; i++){
                min = i;
                for (j = i+1; j < size; j++){
                        if (a[j] < a[min])
                                min = j;
                }
                temp = a[i];
                a[i] = a[min];
                a[min] = temp;
        }
}
```

Although it is not very long but we can still improve its readability by breaking it into small functions to perform the logical steps. The modified code is written below:



© Copyright Virtual University of Pakistan

```
void swap(int &x, int &y)
{
        int temp;
        temp = x;
        x = y;
        y = temp;
}

int minimum(int a[], int from, int to)
{
        int i;
        int min;
        min = a[from];
        for (i = from; i <= to; i++){
                if (a[i] < a[min])
                        min = i;
        }
        return min;
}

void selectionSort(int a[], int size)
{
        int min;
        int i;
        for (i = 0; i < size; i++){
                min = minimum(a, i, size –1);
                swap(a[i], a[min]);
        }
}
```

It is easy to see that the new selectionSort function is much more readable. The logical steps have been abstracted out into the two functions namely, minimum and swap. This code is not only shorter but also as a by product we now have two functions (minimum and swap) that can be reused.

Reusability is one of the prime reasons to make functions but is not the only reason. Modularity is of equal concern (if not more) and a function should be broken into smaller pieces, even if those pieces are not reused. As an example, let us consider the quickSort algorithm below.


© Copyright Virtual University of Pakistan

```
void quickSort(int a[], int left, int right)
{
        int i, j;
        int pivot;
        int temp;
        int mid = (left + right)/2;
        if (left < right){
                i = left - 1;
                j = right + 1;
                pivot = a[mid];
                do {
                        do i++; while (a[i] < pivot);
                        do j--; while (a[i] < pivot);
                        if (i<j){
                                temp = a[i];
                                a[i] = a[j];
                                a[j] = temp;
                        }
                } while (i < j);
                temp = a[left];
                a[left] = a[j];
                a[j] = temp;

                quickSort(a, left, j);
                quickSort(a, j+1, right);
}
```

This is actually a very simple algorithm but students find it very difficult to remember. If is broken in logical steps as shown below, it becomes trivial.

```
void quickSort(int a[], int left, int right)
{
        int p;
        if (left < right){
                p = partition(a, left, right);
                quickSort(a, left, p-1);
                quickSort(a, p+1, right);
        }
}


int partition(int a[], int left, int right)
{
        int i; j;
        int pivot;
        i = left + 1;
        j = right;
        pivot = a[left];
        while(i < right && a[i] < pivot) i++;
        while(j > left && a[j] >= pivot) j++;
        if(i < j)
                swap(a[i], a[j]);
        swap(a[left], a[j]);
        return j;
}
```



© Copyright Virtual University of Pakistan

## 10.10 **Short circuiting || and &&**

The logical and operator, &&, and logical or operators, ||, are special due to the C/C++ short circuiting rule, i.e. a || b and a && b are short circuit evaluated. That is, logical expressions are evaluated left to right and evaluation stops as soon as the final truth value can be determined.

Short-circuiting is a very useful tool. It can be used where one boolean expression can be placed first to "guard" a potentially unsafe operation in a second boolean expression. Also, time is saved in evaluation of complex expressions using operators || and &&. However, a number of issues arise if proper attention is not paid.

Let us look at the following code segment taken from a commercially developed software for a large international bank:

```
struct Node {
        int data;
        Node * next;
};


Node *ptr;
...

while (ptr->data < myData && ptr != NULL){
        // do something here
}
```

What's wrong with this code?

The second part of condition, ptr != NULL, is supposed to be the guard. That is, if the value of the pointer is NULL, then the control should not enter the body of the while loop otherwise, it should check whether ptr->data < myData or not and then proceed accordingly. When the guard is misplaced, if the pointer is NULL then the program will crash because it is illegal to access a component of a non-existent object. This code is rewritten as follows. This time the short-circuiting helps in achieving the desired objective which would have been a little difficult to code without such help.

```
while (ptr != NULL && ptr->data < myData){
        // do something here
}
```



© Copyright Virtual University of Pakistan

## 10.11 Operand Evaluation Order and Side Effects

A side effect of a function occurs when the function, besides returning a value, changes either one of its parameters or a variable declared outside the function but is accessible to it. That is, a side effect is caused by an operation that may return an explicit result but it may also modify the values stored in other data objects. Side effects are a major source of programming errors and they make things difficult during maintenance or debugging activities. Many languages do not specify the function evaluation order in a single statement. This combined with side effects causes major problems. As an example, consider the following statement:

```
c = f1(a) + f2(b);
```

The question is, which function (f1 or f2) will be evaluated first as the C/C++ language does not specify the evaluation order and the implementer (compiler writer) is free to choose one order or the other. The question is: does it matter?

To understand this, let's look at the definition of f1 and f2.

```
int f1(int &x)
{
        x = x * 2;
        return x + 1;
}

int f2(int &y)
{
        y = y / 2;
        return y - 1;
}
```

In this case both f1 and f2 have side effects as they both are doing two things - changing the value of the parameter and changing the value at the caller side. Now if we have the following code segment,

```
a = 3;
b = 4;

c = f1(a) + f2(b);
```

then the value of a, b, and c would be as follows:

```
a = 6
b = 2
c = 8
```



© Copyright Virtual University of Pakistan

So far there seem to be any problem. But let us now consider the following statement:

```
c = f1(a) + f2(a);
```

What will be the value of a and c after this statement?

If f1 is evaluated before f2 then we have the following values:

```
a = 3
c = 9 // 7 + 2
```

On the other hand, if f2 is evaluated before f1 then, we get totally different results.

```
a = 2
c = 3 // 3 + 0
```



© Copyright Virtual University of Pakistan

# Lecture No. 33

**Common Coding mistakes**

Following is short list of common mistakes made due to side-effects.

1.
```
array[i++] = i;
```
If i is initially 3, array[3] might be set to 3 or 4.

2.
```
array[i++] = array[i++] = x;
```
Due to side effects, multiple assignments become very dangerous. In this example, a whole depends upon when i is incremented.

3.
“,” is very dangerous as it causes side effects. Let’s look at the following statement:

```
int i, j = 0;
```

Because of the syntax, many people would assume that i is also being initialized to 0, while it is not. Combination of , and = -- is fatal. Look at the following statement:

```
a = b, c = 0;
```

A majority of the programmers would assume that all a, b, and c are being initialized to 0 while only c is initialized and a and b have garbage values in them. This kind of overlook causes major programming errors which are not caught easily and are caused only because there are side effects.

**Guidelines**

If the following guidelines are observed, one can avoid hazards caused by side effects.

1. never use “,” except for declaration
2. if you are initializing a variable at the time of declaration, do not declare another variable in the same statement
3. never use multiple assignments in the same statement
4. Be very careful when you use functions with side effects – functions that change the values of the parameters.
5. Try to avoid functions that change the value of some parameters and return some value at the same time.


© Copyright Virtual University of Pakistan

## 10.12 Performance

In many cases, performance and maintainability are at odds with one another. When planning for performance, one should always remember the 80/20 rule - you spend 80 percent of your time in 20 percent of the code. That is, we should not try to optimize everything. The proper approach is to profile the program and then identify bottlenecks to be optimized. This is similar to what we do in databases – we usually normalize the database to remove redundancies but then partially de-normalize if there are performance issues.

As an example, consider the following. In this example a function isspam is profiled by calling 10000 times. The results are shown in the following table:

| sec | % | Cum% | cycles | instructions | calls | function |
|---|---|---|---|---|---|---|
| | | | 990000 | 10000 | 00 | strchr |
| | | | 80000 | 6000 | 00 | strncmp |
| | | | 0000 | 0000 | 00 | strstr |
| | | | 5559 | 2213 | 35 | strlen |
| | | | 000 | 000 | | isspam |
| 11 other functions with insignificant performance overhead | | | | | | |

The profiling revealed that most of time was spent in strchr and strncmp and both of these were called from strstr.

When a small set (a couple of functions) of functions which use each other is so overwhelmingly the bottleneck, there are two alternatives:

1. use a better algorithm
2. rewrite the whole set

In this particular case strstr was rewritten and profiled again. It was and found out that although it was much faster but now 99.8% of the time was spent in strstr.

The algorithm was rewritten and restructured again by eliminating strstr, strchr, and strncmp and used memcmp. Now memcmp was much more complex than strstr but it gained efficiency by eliminating a number of loops and the new results are as shown:

| sec | % | Cum% | cycles | instructions | calls | function |
|---|---|---|---|---|---|---|
| | | | 0000 | 90000 | 000 | memcmp |
| | | | 0000 | 0000 | | isspam |
| | | | 4 | 3 | | strlen |

strlen now went from over two million calls to 652.



© Copyright Virtual University of Pakistan

Many details of the execution can be discovered by examining the numbers. The trick is to concentrate on hot spots by first identifying them and then cooling them. As mentioned earlier, most of the time is spent in loops. Therefore we need to concentrate on loops.

As an example, consider the following:

```
for (j = i; j < MAX_FIELD; j++)
  clear(j);
```

This loop clears field before each new input is read. It was observed that it was taking almost 50% of the total time. On further investigation it was found out that MAX_FIELD was 200 but the actual fields that needed to be cleared were 2 or 3 in most cases. The code was subsequently modified as shown below:

```
for (j = i; j < maxField; j++)
  clear(j);
```

This reduced the overall execution time by half.



© Copyright Virtual University of Pakistan

# Lecture No. 34

## 10.13 Portability

Many applications need to be ported on to many different platforms. As we have seen, it is pretty hard to write error free, efficient, and maintainable software. So, if a major rework is required to port a program written for one environment to another, it will be probably not come at a low cost. So, we ought to find ways and means by which we can port applications to other platforms with minimum effort. The key to this lies in how we write our program. If we are careful during writing code, we can make it portable. On the other hand if we write code without portability in mind, we may end-up with a code that is extremely hard to port to other environment. Following is brief guideline that can help you in writing portable code.

### Stick to the standard

1. Use ANSI/ISO standard C++
2. Instead of using vendor specific language extensions, use STL as much as possible

### Program in the mainstream

Although C++ standard does not require function prototypes, one should always write them.

```
double sqrt();          // old style acceptable by ANSI C

double sqrt(double);    // ANSI – the right approach
```

### Size of data types

Sizes of data types cause major portability issues as they vary from one machine to the other so one should be careful with them.

```
int i, j, k;

…
j = 20000;
k = 30000;

i = j + k;
```



© Copyright Virtual University of Pakistan

```
                    // works if int is 4 bytes
// what will happen if int is 2 bytes?
```



© Copyright Virtual University of Pakistan

**Order of Evaluation**

As mentioned earlier during the discussion of side effects, order of evaluation varies from one implementation to other. This therefore also causes portability issues. We should therefore follow guidelines mentioned in the side effect discussion.

**Signedness of char**

The language does not specify whether char is signed or unsigned.

```
char c;
        // between 0 and 255 if unsigned
        // -128 to 127 if signed

c = getchar();
if (c == EOF)  ??

        // will fail if it is unsigned
```

It should therefore be written as follows:

```
int c;
c = getchar();
if (c == EOF)
```

**Arithmetic or Logical Shift**

The C/C++ language has not specified whether right shift >> is arithmetic or logical. In the arithmetic shift sign bit is copied while the logical shift fills the vacated bits with 0. This obviously reduces portability.

Interestingly, Java has introduced a new operator to handle this issue. >> is used for for arithmetic shift and >>> for logical shift.

**Byte Order and Data Exchange**

The order in which bytes of one word are stored is hardware dependent. For example in Intel architecture the lowest byte is the most significant byte while in Motorola architecture the highest byte of a word is the most significant one. This causes problem when dealing with binary data and we need to be careful while exchanging data between to heterogeneous machines. One should therefore only use text for data exchange. One should also be aware of the internationalization issues and hence should not assume ASCII as well as English.



© Copyright Virtual University of Pakistan

**Alignment**

The C/C++ language does not define the alignment of items within structures, classes, an unions. data may be aligned on word or byte boundaries. For example:

```
struct X {
        char c;
        int i;
};
```

address of i could be 2, 4, or 8 from the beginning of the structure. Therefore, using pointers and then typecasting them to access individual components will cause all sorts of problems.



© Copyright Virtual University of Pakistan

**Bit Fields**

Bit fields allow the packing of data in a structure. This is especially useful when memory or data storage is at a premium. Typical examples:

- Packing several objects into a machine word. *e.g.* 1 bit flags can be compacted -- Symbol tables in compilers.
- Reading external file formats -- non-standard file formats could be read in. *E.g.* 9 bit integers.

C lets us do this in a structure definition by putting :*bit length* after the variable. *i.e.*

```
struct packed_struct {
        unsigned int f1:1;
        unsigned int f2:1;
        unsigned int f3:1;
        unsigned int f4:1;
        unsigned int type:4;
        unsigned int funny_int:9;
} pack;
```

Here the `packed_struct contains 6 members: Four 1 bit` *flags* `f1..f3, a 4 bit type and a 9 bit funny_int.`

C automatically packs the above bit fields as compactly as possible, provided that the maximum length of the field is less than or equal to the integer word length of the computer. If this is not the case then some compilers may allow memory overlap for the fields whilst other would store the next field in the next word.

Bit fields are a convenient way to express many difficult operations. However, bit fields do suffer from a lack of portability between platforms:

- integers may be signed or unsigned
- Many compilers limit the maximum number of bits in the bit field to the size of an `integer` which may be either 16-bit or 32-bit varieties.
- Some bit field members are stored left to right others are stored right to left in memory.
- If bit fields too large, next bit field may be stored consecutively in memory (overlapping the boundary between memory locations) or in the next word of memory.

Bit fields therefore should not be used.


© Copyright Virtual University of Pakistan

# Lecture No. 35

## 10.14 Exception handling

Exception handling is a powerful technique that separates error-handling code from normal code. It also provides a consistent error handling mechanism. The greatest advantage of exception handling is its ability to handle asynchronous errors.

The idea is to raise some error flag every time something goes wrong. There is a system that is always on the lookout for this error flag. Third, the previous system calls the error handling code if the error flag has been spotted. The raising of the imaginary error flag is simply called *raising* or *throwing* an error. When an error is thrown the overall system responds by *catching* the error. Surrounding a block of error-sensitive code with exception handling is called *trying* to execute a block. The following code segment illustrates the general exception handling mechanism.

```
try {
___...
___...
___throw Exception()
___...
___...
} catch( Exception e )
{
___...
___...
}
```

One of the most powerful features of exception handling is that an error can be thrown over function boundaries. This allows programmers to put the error handling code in one place, such as the *main*-function of your program.



© Copyright Virtual University of Pakistan

## Exceptions and code complexity

A number of invisible execution paths can exist in simple code in a language that allows exceptions. The complexity of a program may increase significantly if there are exceptional paths in it. Consider the following code:

```
String EvaluateSalaryAnadReturnName( Employee e)
{
  if (e.Title() == "CEO" || e.Salary() > 10000)
  {
        cout << e.First() << " " << e.Last() << " is overpaid" << endl;
  }
  return e.First() + " " + e.Last();
}
```

Before moving any further, let's take the following assumptions:

1. Different order of evaluating function parameters are ignored.
2. Failed destructors are ignored
3. Called functions are considered atomic
    a. for example: e.Title() could throw for several reasons but all that matters for this function is whether e.Title() results in an exception or not.
4. To count as different execution paths, an execution path must be made-up of a unique sequence of function calls performed and exited in the same way.

**Question**: How many more execution paths are there?

**Ans**: 23. There are 3 non-exceptional paths and 20 exceptional paths.

**The non-exceptional paths:**

The three non-exceptional paths are enumerated as below:

1. if (e.Title() == "CEO" || e.Salary() > 10000)
    - if e.Title() == "CEO" is true then the second part is not evaluated and e.Salary() will not be called.
    - cout will be performed

2. if e.Title() != "CEO" and e.Salary() > 10000
    - both parts of the condition will be evaluated
    - cout will be performed.

3. if e.Title() != "CEO" and e.Salary() <= 10000
    - both parts of the condition will be evaluated
    - cout will not be performed.


© Copyright Virtual University of Pakistan

**Exceptional Code Paths**

The 20 exceptional code path are listed below.

1. String EvaluateSalaryAnadReturnName( Employee e)

   The argument is passed by value, which invokes the copy constructor. This copy operation might throw an exception.

2. if (e.Title() == “CEO” || e.Salary() > 10000)

   e.Title() might itself throw, or it might return an object of class type by value, and that copy operation might throw.

3. if (e.Title() == “CEO” || e.Salary() > 10000)

   Same as above.

4. if (e.Title() == “CEO” || e.Salary() > 10000)

   To match a valid ==() operator, the string literal may need to be converted to a temporary object of class type and that construction of the temporary might throw.

5. if (e.Title() == “CEO” || e.Salary() > 10000)

   Same as above.

6. if (e.Title() == “CEO” || e.Salary() > 10000)

   operator ==() might throw.

7. if (e.Title() == “CEO” || e.Salary() > 10000)

   Same as above.



© Copyright Virtual University of Pakistan

8. if (e.Title() == “CEO” || e.Salary() > 10000)

Same as above.

9-13 cout << e.First() << “ “ << e.Last() << “ is overpaid” << endl;

As per C++ standard, any of the five calls to << operator might throw.

14-15 cout << e.First() << “ “ << e.Last() << “ is overpaid” << endl

similar to 2 and 3.

16-17 return e.First() + “ “ + e.Last();

similar to 14-15.

18-19 return e.First() + “ “ + e.Last();

similar to 6,7, and 8.

20. return e.First() + “ “ + e.Last();

similar to 4.

**Summary:**

- A number of invisible execution paths can exist in simple code in a language that allows exceptions.
- Always be exception-aware. Know what code might emit exceptions.



© Copyright Virtual University of Pakistan

**The Challenge:**
Can we make this code exception safe and exception neutral? That is, rewrite it (if needed) so that it works properly in the presence of an exception and propagates all exceptions to the caller?

**Exception-Safety:**
A function is exception safe if it might throw but do not have any side effects if it does throw and any objects being used, including temporaries, are exception safe and clean-up there resources when destroyed.

**Exception Neutral:**
A function is said to be exception neutral if it propagates all exceptions to the caller.

**Levels of Exception Safety**
- Basic Guarantee: Ensures that temporaries are destroyed properly and there are no memory leaks.
- Strong Guarantee: Ensures basic guarantee as well as there is full-commit or roll-back.
- No-throw Guarantee: Ensure that a function will not throw.

Exception-safety requires either no-throw guarantee or basic and strong guarantee.

**Does the function satisfy basic guarantee?**
Yes. Since the function does not create any objects, in the presence of an exception, it does not leak any resources.

**Does the function satisfy strong guarantee?**
No. The strong guarantee says that if the function fails because of an exception, the program state must not be change.

This function has two distinct side-effects:
- an overpaid message is emitted to cout.
- A name strings is returned.

As far as the second side-effect is concerned, the function meets the strong guarantee because if an exception occurs the value will never be returned.

As far as the first side-effect is concerned, the function is not exception safe for two reasons:
- if exception is thrown after the first part of the message has been emitted to cout but before the message has been completed (for example if the fourth << operator throws), then a partial message was emitted to cout.
- If the message emitted successfully but an exception occurs in later in the function (for example during the assembly of the return value), then a


© Copyright Virtual University of Pakistan

message was emitted to cout even though the function failed because of an exception. It should be complete commit or complete roll-back.

**Does the function satisfy no-throw guarantee?**

No. This is clearly not true as lots of operations in the function might throw.

**Strong Guarantee**

To meet strong guarantee, either both side-effects are completed or an exception is thrown and neither effect is performed.

```
// First attempt:

String EvaluateSalaryAnadReturnName( Employee e)
{
        String result = e.First() + " " + e.Last();

        if (e.Title() == "CEO" || e.Salary() > 10000)
        {

                String message = result + " is overpaid\n";
                cout << message;

        }
        return result;
}
```

What happens if the function is called as follows:

```
String theName;
theName = evaluateSalarayAndReturnName(someEmplyee);
```

1. string copy constructor is invoked because the result is returned by value.
2. The copy assignment operator is invoked to copy the result into theName
3. If either copy fails then the function has completed its side-effects (since the message was completely emitted and the return value was completely constructed) but the result has been irretrievable lost.



© Copyright Virtual University of Pakistan

Can we do better and perhaps avoid the problem by avoiding the copy?

```
// Second attempt:

String EvaluateSalaryAnadReturnName( Employee e, String &r)
{
        String result = e.First() + " " + e.Last();

        if (e.Title() == "CEO" || e.Salary() > 10000)
        {

                String message = result + " is overpaid\n";
                cout << message;

        }
        r = result;
}
```

Looks better but assignment to r might still fail which leaves us with one side-effect completed and other incomplete.

```
// Third attempt:

auto_ptr<String> EvaluateSalaryAnadReturnName( Employee e)
{
        auto_ptr<String> result = new String(e.First() + " " + e.Last());

        if (e.Title() == "CEO" || e.Salary() > 10000)
        {

                String message = (*result) + " is overpaid\n";
                cout << message;

        }
        return result;();       // rely on transfer of ownership
                                // this can't throw
}
```

We have effectively hidden all the work to construct the second side-effect (the return value), while we ensured that it can be safely returned to the caller using only non-throwing operation after the first side-effect has completed the printing of the message. In this case we know that once the function is complete, the return value will make successfully into the hands of the caller and be correctly cleaned-up in all cases. This is because the aut_ptr semantics guarantee that If the caller accepts the returned value, the act of accepting a copy of the auto_ptr causes the caller to take the ownership and if the caller does not accept the returned value, say by ignoring the



© Copyright Virtual University of Pakistan

return value, the allocated string will automatically be destroyed with proper clean-up.

**Exception Safety and Multiple Side-effects**

It is difficult and some-times impossible to provide strong exception safety when there are two or more side-effects in one function and these side-effects are not related with each other. For example we would not have been able to provide exception safety if there are two output messages, one to cout and the other one to cerr. This is because the two cannot be combined.

When such a situation comes with two or more unrelated side-effects which cannot be combined then the best way to handle such a situation is break it into two separate functions. That way, at least, the caller would know that these are two separate atomic steps.

## Summary

1. Providing the strong exception-safety guarantee often requires you to trade-off performance.
2. If a function has multiple un-related side-effects, it cannot always be made strongly exception safe. If not, it can be done only by splitting the function into several functions, each of whose side-effects can be performed atomically.
3. Not all functions need to be strongly exception-safe. Both the original code and attempt#1 satisfy the basic guarantee. For many clients, attempt # 1 is sufficient and minimizes the opportunity for side-effects to occur in the exceptional situation, without requiring the performance trade-off of attempt #3.


© Copyright Virtual University of Pakistan

# Lecture No. 36

Software Verification and Validation

## 11.1 Software Testing

To understand the concept of software testing correctly, we need to understand a few related concepts.

### Software verification and validation

Verification and validation are the processes in which we check a product against its specifications and the expectations of the users who will be using it. According to a known software engineering expert Berry Boehm, verification and validation are

**Verification**

- Does the product meet system specifications?
- Have you built the product right?

**Validation**

- Does the product meet user expectations?
- Have you built the right product?

It is possible that a software application may fulfill its specifications but it may deviate from users expectations or their desired behavior. That means, software is verified but not validated. How is it possible? It is possible because during the requirements engineering phase, user needs might not have been captured precisely or the analyst might have missed a major stakeholder in the analysis. Therefore, it is important to verify as well as validate the software product.

## 11.2 Defect

The second major and a very important concept is Defect. A defect is a variance from a desired product attribute. These attributes may involve system specifications well as user expectation. Anything that may cause customer dissatisfaction, is a defect. Whether these defects are in system specifications or in the software products, it is essential to point these out and fix.

Therefore software defect is that phenomenon in which software deviates from its expected behavior. This is non-compliance from the expected behavior with respect to written specifications or the stakeholder needs.

### Software and Defect

Software and defects go side by side in the software development life cycle. According to a famous saying by Haliburton, Death and taxes are inevitable. According to Kernighan: Death, taxes, and bugs are the only certainties in the life of a programmer. Software and defects cannot be separated, however, it is important to learn how discovering defects at an appropriate stage improves the software quality. Therefore, software application needs to be verified as well as validated for a successful deployment.

### Software Testing

With these concepts, we are in a position to define software testing. Software testing is the process of examining the software product against its requirements. Thus it is a process that involves verification of product with respect to its written requirements and conformance of requirements with user needs. From another perspective, software testing



© Copyright Virtual University of Pakistan

is the process of executing software product on test data and examining its output vis-à-vis the documented behavior.

### Software testing objective

- The correct approach to testing a scientific theory is not to try to verify it, but to seek to refute the theory. That is to prove that it has errors. (Popper 1965)
- The goal of testing is to expose latent defects in a software system before it is put to use.
- A software tester tries to break the system. The objective is to show the presence of a defect not the absence of it.
- Testing cannot show the absence of a defect. It only increases your confidence in the software.
- This is because exhaustive testing of software is not possible – it is simply too expansive and needs virtually infinite resources.

### Successful Test

From the following sayings, a successful test can be defined
"If you think your task is to find problems then you will look harder for them than if you think your task is to verify that the program has none" – Myers 1979.

"A test is said to be successful if it discovers an error" – doctor's analogy.

The success of a test depends upon the ability to discover a bug not in the ability to prove that the software does not have one. As, it is impossible to check all the different scenarios of a software application, however, we can apply techniques that can discover potential bugs from the application. Thus a test that helps in discovering a bug is a successful test. In software testing phase, our emphasis is on discovering all the major bugs that can be identified by running certain test scenarios. However it is important to keep in mind that testing activity has certain limitations.

### Limitations of testing

With the help of the following example, we shall see how difficult it may become to discover a defect from a software application.

## Example (adapted from Backhouse)

This is a function that compares two strings of characters stored in an array for equality. The tester who has to test this function has devised the following test cases in which different combinations of inputs are tried and their expected behavior is documented. From the look of this table, it seems that almost all major combinations of inputs have been documented.

Inputs and Expected Outputs

| A | b | Expected result |
|---|---|---|
| "cat" | "dog" | False |
| "" | "" | True |
| "hen" | "hen" | True |
| "hen" | "heN" | False |
| " " | "" | False |



© Copyright Virtual University of Pakistan

| | | |
|---|---|---|
| “” | “ball” | False |
| “cat” | “” | False |
| “HEN” | “hen” | False |
| “rat” | “door” | False |
| “ ” | “ ” | True |

# Results of testing

The tester runs all the above-mentioned test cases and function returns the same results as expected. But it is still not correct. What can be a problem To analyze the problem, lets look at the code of this string equal routine

# Code of the Function

```
bool isStringsEqual(char a[], char b[])
{
   bool result;
   if (strlen(a) != strlen(b))
      result = false;
   else {
      for (int i =0; i < strlen(a); i++)
         if (a[i] == b[i])
            result = true;
         else result = false;
   }
   return result;
}
```

# Analysis of the code

**It passes all the designated tests but fails for two different strings of same length ending with the same character. For example, “cut” and “rat” would results in true which is not correct.**

**The above-mentioned defect signifies a clear limitation of the testing process in discovering a defect which is not very frequent. However, it should be noted from this example that a tester cannot generate all possible combinations of test cases to test an application as the number of scenarios may become exhaustive.**

## Testing limitations

- In order to prove that a formula or hypothesis is incorrect all you have to do to show only one example in which you prove that the formula or theorem is not working.
- On the other hand, million of examples can be developed to support the hypothesis but this will not prove that it is correct.
- These examples only help you in coming up with a hypothesis but they are not proves by themselves and they only enhance your comfort level in that particular hypothesis or in this particular case, in your piece of software.
- You cannot test a program completely because:
  - the domain of the possible inputs is too large to test.
  - there are too many possible paths through the program to test.
- According to Discrete Mathematics



© Copyright Virtual University of Pakistan

- To prove that a formula or hypothesis is incorrect you have to show only one example.
- To prove that it is correct any numbers of examples are insufficient. You have to give a formal proof of its correctness.

### 11.3 Test Cases and Test Data

In order to test a software application, it is necessary to generate test cases and test data which is used in the application. Test cases correspond to application functionality such that the tester writes down steps which should be followed to achieve certain functionality. Thus a test case involves

- Input and output specification plus a statement of the function under test.
- Steps to perform the function
- Expected results that the software application produces

However, test data includes inputs that have been devised to test the system.

# Lecture No. 37

### 11.3 Testing vs. development

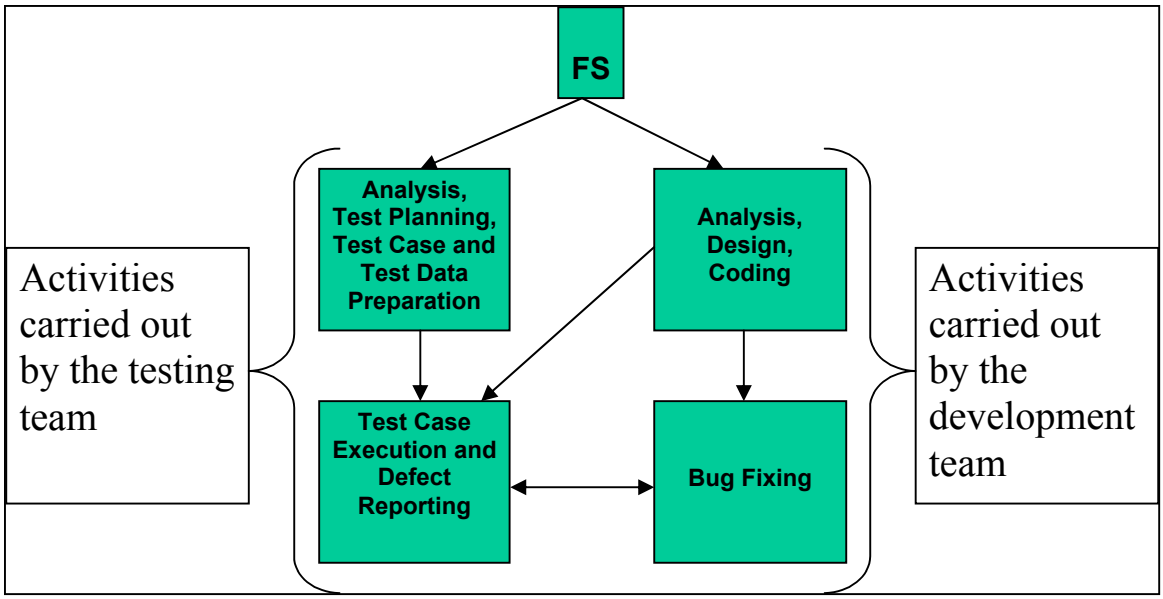
Testing is an intellectually demanding activity and has a lifecycle parallel to software development. A common misperception about testing is that it is not a challenging activity. It should be noted here that the testing demands grip over the domain and application functionality as is required from an analyst or a designer who has to develop the application from requirements. As without having an in-depth knowledge about the system and the requirements from users, a tester cannot write test cases that can verify and validate software application with respect to documented specifications and user needs. Writing test cases and generating test data are processes that demand scenario-building capabilities. These activities essentially require destructive instincts in a tester for the purpose of breaking system to discover loopholes into its functionality.

At the time when these two activities are being performed, merely initial design of the application is completed. Therefore, tester uses his/her imagination to come up with use patterns of the application that can help him/her in describing exact steps that should be executed in order to test a particular functionality. Moreover, tester needs to figure out loose points in the system from where he/she can discover defects. All these activities are highly imaginative and a tester is supposed to possess above average (if not excellent) analytical skills.

We shall explain the testing activities parallel to development activities with the help of the following diagram



© Copyright Virtual University of Pakistan




Description

- Functional specification document is the starting point, base document for both testing and the development
- Right side boxes describe the development, whereas, left side boxes explain the testing process
- Development team is involved into the analysis, design and coding activities.
- Whereas, testing team too is busy in analysis of requirements, for test planning, test cases and test data generation.
- System comes into testing after development is completed.
- Test cases are executed with test data and actual results (application behavior) are compared with the expected results,
- Upon discovering defects, tester generates the bug report and sends it to the development team for fixing.
- Development team runs the scenario as described in the bug report and try to reproduce the defect.
- If the defect is reproduced in the development environment, the development team identifies the root cause, fixes it and sends the patch to the testing team along with a bug resolution report.
- Testing team incorporates the fix (checking in), runs the same test case/scenario again and verifies the fix.
- If problem does not appear again testing team closes down the defect, otherwise, it is reported again.

## 11.5 The Developer and Tester

| Development | Testing |
|---|---|
| Development is a creative activity | Testing is a destructive activity |
| Objective of development is to show that the program works | Objective of testing is to show that the program does not work |


© Copyright Virtual University of Pakistan

Scenarios missed or misunderstood during development analysis would never be tested correctly because the corresponding test cases would either be missing or would be incorrect.

The left side column is related to the development, and the right side describes the testing. Development is a creative process as developers have to build the system, whereas, testing is a destructive activity as the goal of a tester is to break the system to discover the defects. Objective of development is to show that the program works, objective of testing is to show that program does not work. However, FS is the base document for both of these activities.

Tester analyzes FS with respect to testing the system whereas; developer analyzes FS with respect to designing and coding the system. If developer does not understand the FS correctly then he cannot implement and test it right. Thus if the same person who has developed a system, tests it, chances of carrying the same misunderstanding in testing will be very high. Therefore, an independent testing can only prove his understanding wrong. Therefore, it is highly recommended that developer should not try to test his/her own work.

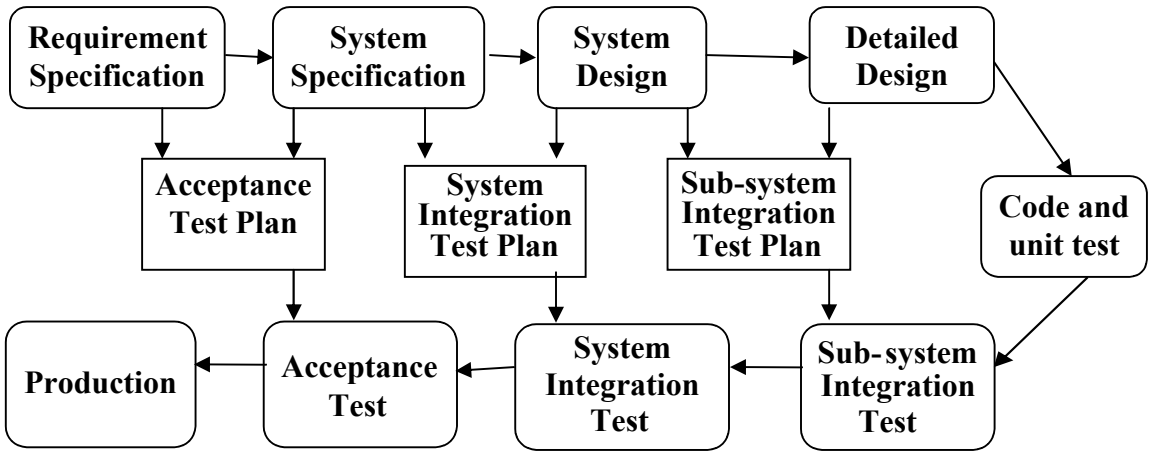
## 11.6 Usefulness of testing

Objective of testing is to discover and fix as many errors as possible before the software is put to use. That is before it is shipped to the client and the client runs it for acceptance. In software development organizations, a rift exists between the development and the testing teams. Often developers are found questioning about the significance or even need to have the testing resources in the project teams. Whoever doubts on the usefulness of the testing team should understand what could happen if the application is delivered to client without testing? At the best, the client may ask to fix all the defects (free of cost) he would discover during the acceptance testing. At the worst, probably he would sue the development firm for damages. However, in practice, clients are often seen complaining about the deliverables and a couple of defected deliverables are sufficient for breaking the relations next to the cancellation of contract.

Therefore, it should be well preserved among the community of developers that testers are essential rather inevitable. A good tester has a knack of smelling errors – just like auditors and it is for the good of the organization not to harm it.

## 11.7 Testing and software phases

With the help of the following diagram we shall explain different phases of testing






© Copyright Virtual University of Pakistan

Software development process diagram

# Description of testing phases

- Unit testing – testing individual components independent of other components.
- Module testing – testing a collection of dependent components – a module encapsulates related components so it can be tested independently.
- Subsystem testing – testing of collection of modules to discover interfacing problems among interacting modules.
- System testing – after integrating subsystems into a system – testing this system as a whole.
- Acceptance test – validation against user expectations. Usually it is done at the client premises.
- Alpha testing – acceptance testing for customized projects, in-house testing for products.
- Beta testing – field testing of product with potential customers who agree to use it and report problem before system is released for general use

In the following two types of testing activities are discussed.

# 11.8 Black box testing

In this type of testing, a component or system is treated as a black box and it is tested for the required behavior. This type of testing is not concerned with how the inputs are transformed into outputs. As the system's internal implementation details are not visible to the tester. He gives inputs using an interface that the system provides and tests the output. If the outputs match with the expected results, system is fine otherwise a defect is found.

# 11.9 Structural testing (white box)

As opposed to black box testing, in structural or white box testing we look inside the system and evaluate what it consists of and how is it implemented. The inner of a system consists of design, structure of code and its documentation etc. Therefore, in white box testing we analyze these internal structures of the program and devise test cases that can test these structures.

**Effective testing**

The objective of testing is to discover the maximum number of defects with a minimum number of resources before the system is delivered to the next stage. Now the question arises here how to increase the probability of finding a defect?

As, good testing involves much more than just running the program a few times to see whether it works or not. A good tester carries out a thorough analysis of the program to devise test cases that can be used to test the system systematically and effectively. Problem here is how to develop a representative set of test cases that could test a complete program. That is, selection of a few test cases from a huge set of possibilities. What should be the sets of inputs that should be used to test the system effectively and efficiently?

## String Equal Example

- For how many equal strings do I have to test to be in the comfortable zone?
- For how many unequal strings do I have to test to be in the comfortable zone?



© Copyright Virtual University of Pakistan

- When should I say that further testing is unlikely to discover another error? Testing types

To answer these questions, we divide a problem domain in different classes. These are called Equivalence Classes.

# Lecture No. 38

## Equivalence Classes or Equivalence Partitioning

Two tests are considered to be equivalent if it is believed that:

- if one discovers a defect, the other probably will too, and
- if one does not discover a defect, the other probably won't either.

Equivalence classes help you in designing test cases to test the system effectively and efficiently. One should have reasons to believe that the test cases are equivalent. As for this purpose, one would need to understand the system and see in how many partitions it can be divided. These partitions should be devised such that a clear distinction should be marked. Test cases written for one partition should not yield the same results when run for the second partition as otherwise these two partitions should become one. However, finding equivalence classes is a subjective process, as two people analyzing a program will probably come up with different sets of equivalence classes .

### Equivalence partitioning guidelines

- Organize your equivalence classes. Write them in some order, use some template, sequence, or group them based on their similarities or distinctions. These partitions can be hierarchical or organized in any other manner.
- Boundary conditions: determine boundary conditions. For example, adding in an empty linked list, adding after the last element, adding before the first element, etc.
- You should not forget invalid inputs that a user can give to a system. For example, widgets on a GUI, numeric instead of alphabets, etc.

## Equivalence partitioning example

In the following example, we shall see how equivalence partitions can be developed for a string matching function.

### String matching

#### Organization

For equivalence partitions, we divide the problem in two obvious categories of equal strings and the other one for unequal strings. Within these equivalent partitions, further partitioning is done. Following is the description of the equivalence partitions and their test cases

#### Test cases for equivalence partitions

#### Equal

- Two equal strings of arbitrary length
  - All lower case "cat" "cat"
  - All upper case "CAT" "CAT"



© Copyright Virtual University of Pakistan

- Mixed case "Cat" "Cat"
- Numeric values "123" "123"
- Two strings with blanks only " " " "
- Numeric and character mixed "Cat1" "Cat1"
- Strings with special characters "Cat#1" "Cat#1"

- Two NULL strings "" ""

Unequal Strings

- Two different equal strings of arbitrary length
  - Two strings with different length "cat" "mouse"
  - Two strings of same length "cat" "dog"
- Check for case sensitivity
  - Same strings with different characters capitalized "Cat" "caT"
- One string is empty
  - First is NULL "" "cat"
  - Second is NULL "cat" ""

## 11.10 Basis Code Structures

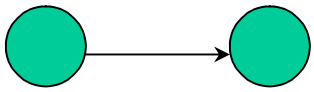
For structural testing it is important to know about basic coding structures. There are four basic coding structures sequence, if statement, case statement, and while loop. These four basic structures can be used to express any type of code.

## Flow graph notation

In analysis and design, you have already seen the flow graph notation. This is used to describe flow of data or control in an application. However, we do not use flow graphs to describe decisions. That is, how a branch is taken is not shown in flow graphs.
In the following, we are using flow graph notation to describe different coding structures.

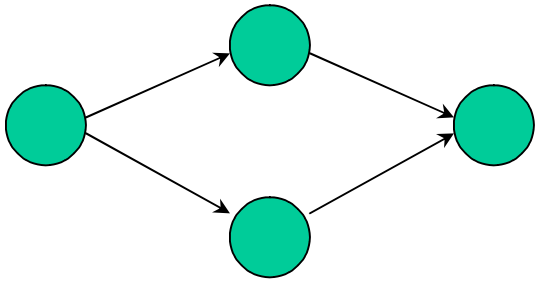
## Sequence

Sequence depicts programming instructions that do not have branching or any control information. So we lump together several sequential instructions in one node of the graph.



## If

Second structural form is the If statement. In the following graph, the first node at the left depicts the if statement and the two nodes next to the first node correspond to the successful case (if condition is true) and unsuccessful case (if condition is false) consecutively. The control comes to the same instruction from either of these intermediate instructions.

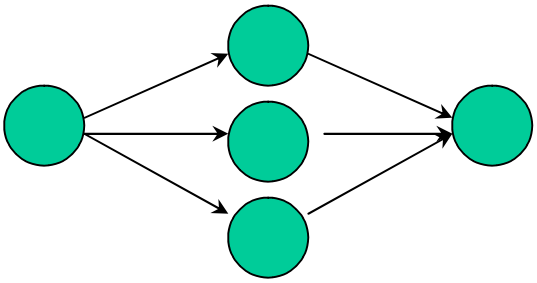




© Copyright Virtual University of Pakistan

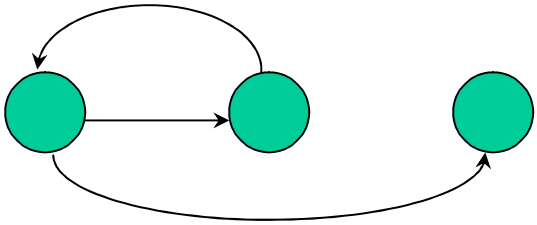
## Case

In Case statement, control can take either of several branches (as opposed to only two in If statement.) First node represents the switch statement (C/C++) and nodes in middle correspond to all different cases. Program can take one branch and result into the same instruction.



## While

A while loop structure consists of a loop guard instruction through which the iteration in the loop is controlled. The control keeps iterating in the loop as long as the loop guard condition is true. It branches to the last instruction when it becomes false.



## Flow graph for bubble sort

In the following example, code is given for a bubble sort function. The diagram opposite to the code is the corresponding flow graph model. It consists of six nodes. Node one is the while loop instruction that contains loop guard and node six is the ending instruction of the while loop. Node two corresponds to the for loop instructions. Node three corresponds to the swapping instruction. Nodes five and six correspond to the last instructions of the if statement and the for loop consecutively.

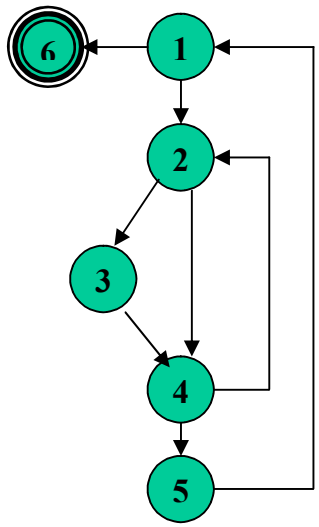
Point to note here is the assignment of node numbers to program instructions. As, the corresponding flow graph model would consist of nodes that correspond to instructions which are major decision points in the code. For example, when this function will be invoked, control will certainly come to the while loop instruction and at the minimum it will traverse from node one to node six. However, if control enters into the while loop even for a single iteration, it will traverse through nodes two, four and five for certain and node three too if it enters for loop and check in the if condition is true.

So all these combinations are to be tested during white box testing.



© Copyright Virtual University of Pakistan

```
sorted = false;
while (!sorted) {                  //1
    sorted = true;
    for (i=0; i < N-1; i++) {      //2
        if a[i] > a[i+1] {
            swap(a[i], a[i+1]);    //3
            sorted = false;
        }                          //4
    }                              //5
}                                  //6
```




## Paths

Following are possible paths from starting to the end of this code.
Path1: 1-6
Path2: 1-2-3-4-5-1-6
Path3: 1-2-4-5-1-6
Path4: 1-2-4-2-3-4-5-6-1

# Lecture No. 39

### White box testing

As described in the section above, in white box testing we test the structure of the program. In this technique the test cases are written in a manner to cover different possibilities in code. Below are described three coverage schemes.

## Coverage

- **Statement Coverage**: In this scheme, statements of the code are tested for a successful test that checks all the statements lying on the path of a successful scenario.
- **Branch Coverage**: In this scheme, all the possible branches of decision structures are tested. Therefore, sequences of statements following a decision are tested.
- **Path Coverage**: In path coverage, all possible paths of a program from input instruction to the output instruction are tested. An exhaustive list of test cases is generated and tested against the code.

### White Box Testing Example

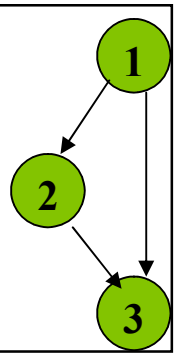
With the help of the following example, we shall see how many test cases are generated for each type of coverage schemes.


© Copyright Virtual University of Pakistan

```
if (a = = b)          //1

    c = d;            //2

a++;                  //3
```




### Statement coverage:

- a=1,b=1
- If a==b then statement 2, statement 3

### Branch coverage

- Statement 1: two braches 1-2, 1-3
- Test case 1: if a =1, b=1 then statement 2
- Test case 2: if a=1,b=2: then statement 3

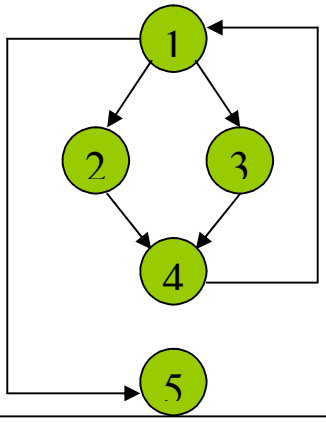
### Path coverage

Same as branch testing

## Paths in a program containing loops

Now we shall apply the path coverage scheme on a piece of code that contains a loop statement and see how many test cases can possibly be developed.

```
for (i = 0; i < N; i++) {          //1
  if (condition1)
        // do something here       //2
  else
        // do something here       //3
  // something here                //4
}
                                   //5
```




### Paths

The following is an analysis of the above-mentioned code and the flow diagram. It determines the number of paths against different iterations of the loop.

- N = 0: If the control does not enter into the loop then only one path will be traversed. It is 1-5.
- N=1: Two different paths can possibly be traversed (depending on condition).
  - 1-2-4-1-5
  - 1-3-4-1-5
- N=2: Four possible paths can be traversed.
  - 1-2-4-1-2-4-1-5
  - 1-2-4-1-3-4-1-5



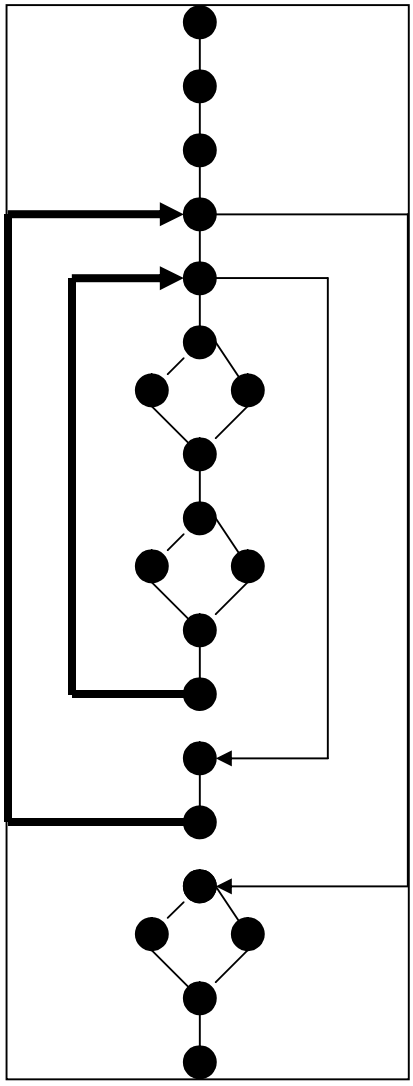
© Copyright Virtual University of Pakistan

- 1-3-4-1-2-4-1-5
- 1-3-4-1-3-4-1-5

- Generalizing the relation between loop variable N and the number of possible paths, for the value of N, $2^N$ paths are possible
  - Thus if N = 20 it means more then 1 million paths are possible.

## Flow graph of a hypothetical program

Following is a flow graph of a hypothetical program whose structure is such that it consists of two loops, one embedded in the other. Left side of the diagram signifies the loops where inner loop seem to be containing two if statements. Second loop has a branch and then program finishes.
Simple graph contains 1852 paths with each loop not iterated more than twice



Thus, the number of paths in a program that contains loops tends to infinity. It is impossible to conduct exhaustive testing of a program that may consist of infinite number of test cases. The question arises, how many test cases need to be executed in order to test all the major scenarios in the code at least once? The answer is, calculate the cyclomatic


© Copyright Virtual University of Pakistan

complexity of the code. This will give us a number that corresponds to the total number of test cases that need to be generated in order to test all the statements and branches in the code at least once.

## Cyclomatic complexity

The concept of cyclomatic complexity is extremely useful in white box testing when analyzing the relative complexity of the program to be tested. It revolves around independent paths in a program which is any path through the program (from start to end) that introduces at least one new set of processing statements or a new condition. An independent path covers statements and branches of the code.

Cyclomatic complexity is a quantitative measure of the logical complexity of a program. It defines number of independent paths in the basis set of a program. It provides an upper bound for the number of tests that must be conducted to ensure that all statements and branches have been executed at least once.

Cyclomatic Complexity, V(G), for a flow graph G is defined as:

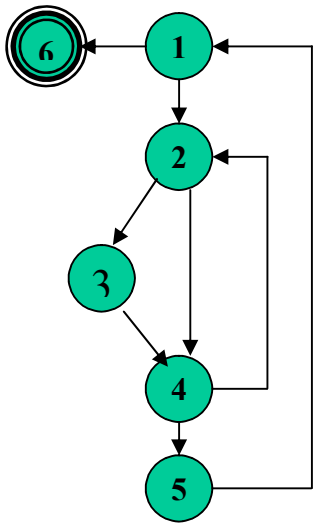
$$V(G) = E - N + 2$$

Where E is the number of edges and N is the number of nodes in the flow graph G. Cyclomatic complexity provides us with an upper bound for the number of independent paths that comprise the basis set.

## Cyclomatic Complexity of a Sort Procedure

Following is the same bubble sort program that we discussed above. This time we shall calculate its cyclomatic complexity and see how many test cases are needed to test this function.

```
Sorted = FALSE;
while (!sorted) {                  //1
   sorted = TRUE;
   for (i=0; i < N-1; i++) {       //2
      if a[i] > a[i+1] {
         swap(a[i], a[i+1]);       //3
         sorted = FALSE;
      }                            //4
   }                               //5
}                                  //6
```




Cyclomatic complexity

- Number of edges = 8
- Number of nodes = 6
- C(G) = 8-6+2 = 4

Paths to be tested

- Path1: 1-6



© Copyright Virtual University of Pakistan

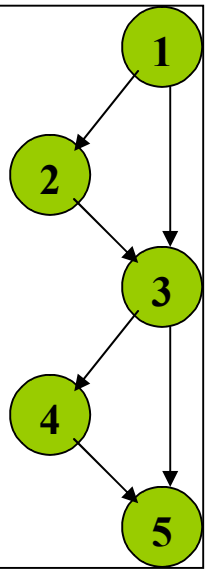
- Path2: 1-2-3-4-5-1-6
- Path3: 1-2-4-5-1-6
- Path4: 1-2-4-2-3-4-5-6-1

**Infeasible paths**

Infeasible path is a path through a program which is never traversed for any input data.

## Example

```
if (a == b)          //1
    c = c-1;         //2
if (a != b)          //3
    c = c+1;         //4
                     //5
```




In the above-mentioned example, there are two infeasible paths that will never be traversed.

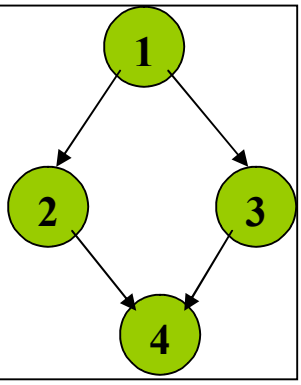
- Path1: 1-2-3-4-5
- Path2: 1-3-5

A good programming practice is such that minimize infeasible paths to zero. It will reduce the number of test cases that need to be generated in order to test the application. How can we minimize infeasible paths, by simply using else part with the if statement and avoid program statements as given above.

## Modified code segment

Infeasible paths can be analyzed and fixed.

```
if (a == b)          //1
      c = c-1;       //2
else
      c = c+1;       //3
                     //4
```




There are no infeasible paths now!



© Copyright Virtual University of Pakistan

# Lecture No. 40

## 11.11 Unit testing

A software program is made up of units that include procedures, functions, classes etc. The unit testing process involves the developer in testing of these units. Unit testing is roughly equivalent to chip-level testing for hardware in which each chip is tested thoroughly after manufacturing. Similarly, unit testing is done to each module, in isolation, to verify its behaviour. Typically the unit test will establish some sort of artificial environment and then invoke routines in the module being tested. It then checks the results returned against either some known value or against the results from previous runs of the same test (regression testing). When the modules are assembled we can use the same tests to test the system as a whole.

Software should be tested more like hardware, with

- Built-in self testing: such that each unit can be tested independently
- Internal diagnostics: diagnostics for program units should be defined.
- Test harness

The emphasis is on built in testability of the program units from the very beginning where each piece should be tested thoroughly before trying to wire them together.

## Unit Testing Principles

- In unit testing, developers test their own code units (modules, classes, etc.) during implementation.
- Normal and boundary inputs against expected results are tested.
- Thus unit testing is a great way to test an API.

## Quantitative Benefits

- **Repeatable**: Unit test cases can be repeated to verify that no unintended side effects have occurred due to some modification in the code.
- **Bounded**: Narrow focus simplifies finding and fixing defects.
- **Cheaper**: Find and fix defects early

## Qualitative Benefits

- **Assessment-oriented**: Writing the unit test forces us to deal with design issues - cohesion, coupling.
- **Confidence-building**: We know what works at an early stage. Also easier to change when it's easy to retest.

## Testing against the contract (Example)

When we write unit tests we want to write test cases that ensure a given unit honors its contract. This will tell us whether the code meets the contract and whether the contract means what we think the unit is supposed to do in the program.

### Contract for square root routine

```
result = squareRoot(argument);
assert (abs (result * result – argument) < epsilon);
```



© Copyright Virtual University of Pakistan

The above contract tells us what to test:

- Pass in a negative argument and ensure that it is rejected
- Pass in an argument of zero to ensure that it is accepted (this is a boundary value)
- Pass in values between zero and the maximum expressible argument and verify that the difference between the square of the result and the original argument is less than some value epsilon.

When you design a module or even a single routine, you should design both its contract and the code to test that contract. By designing code to pass a test that fulfill its contract, you might consider boundary conditions and other issues that you wouldn't consider otherwise. The best way to fix errors is to avoid them in the first place. By building the tests *before* you implement the code you get to try out the interface before you commit to it.

### Unit Testing Tips

Unit test should be conveniently located

- For small projects you can imbed the unit test for a module in the module itself
- For larger projects you should keep the tests in the package directory or a /test subdirectory of the package

By making the code accessible to developers you provide them with:

- Examples of how to use all the functionality of your module
- A means to build regression tests to validate any future changes to the code

You can use the main routine with conditional compilation to run your unit tests.

## 11.12 Defect removal efficiency

Is the ability to remove defects from the application. We shall further elaborate this idea with the help of the above-mentioned table.

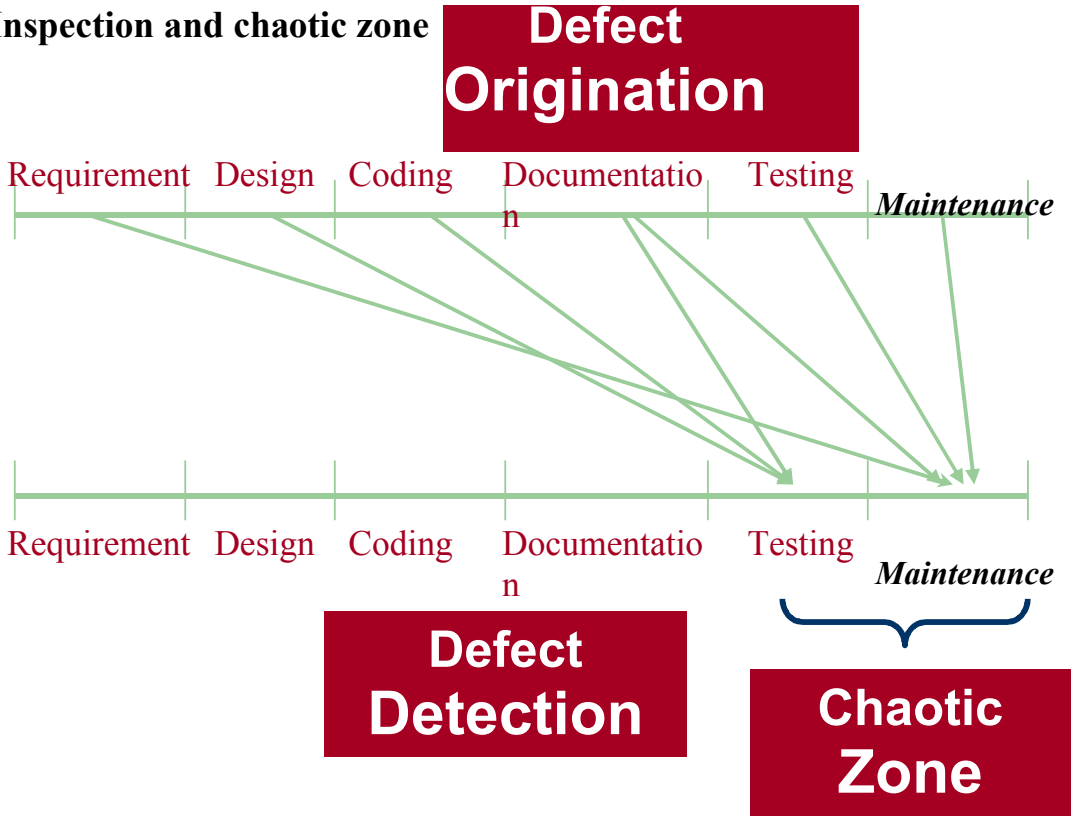
| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| **Design Inspection** | | | | | ● | | ● | ● | ● |
| **Code Inspection** | | | | ● | | ● | | ● | ● |
| **Quality Assurance** | | | ● | | | ● | ● | | ● |
| **Testing** | | ● | ● | ● | ● | ● | ● | ● | ● |
| **Worst** | **30%** | **37%** | **50%** | **55%** | **65%** | **75%** | **77%** | **85%** | **95%** |
| **Median** | **40%** | **53%** | **65%** | **70%** | **80%** | **87%** | **90%** | **97%** | **99%** |
| **Best** | **50%** | **60%** | **75%** | **80%** | **87%** | **93%** | **95%** | **99%** | **99.9%** |

The above table depicts data that was published after analyzing 1500 projects. In these projects, four different types of quality assurance mechanisms were employed. It is evident from this table that testing alone can only remove 53% of defects. However, testing and quality assurance mechanisms combine yield up to 65% efficiency. Whereas, if we combine code inspection and testing together, results are up to 75%. Similarly, design inspections and testing yield up to 80%. Moreover, combining design inspections, quality assurance and testing results are up to 95%. If all four techniques are combined, results are up to 99.9%.



© Copyright Virtual University of Pakistan

**Inspection and chaotic zone**




In this diagram, a chaotic zone has been defined. In fact, if defects are not discovered and fixed at the appropriate stage, then at the testing and maintenance phases, these defects are piled up. Therefore, a chaotic zone is formed for the testing and the development teams as the number of defects which are piled up, destabilize the application and it becomes extremely hard to fix all these defects as some of these bugs may involve changes in requirements and design. At testing or the maintenance phases, fixing a defect in requirements or design becomes extremely expensive, as underlying code will have to be changed as well.

If we combine the results of the above two diagrams, it is evident that testing alone does not suffice. We need to employ inspection techniques and combine them with testing to increase the effectiveness of defect removal efficiency.

## 11.13 Defect origination

In inspections the emphasis is on early detection and fixing of defects from the program. Following are the points in a development life cycle where defects enter into the program.

- Requirements
- Design
- Coding
- User documentation
- Testing itself can cause defects due to bad fixes
- Change requests at the maintenance or initial usage time



© Copyright Virtual University of Pakistan

It is important to identify defects and fix them as near to their point of origination as possible.

# Lecture No. 41

## 11.14 Inspection versus Testing

Inspections and testing are complementary and not opposing verification techniques. Both should be used during the verification and validation process. Inspections can check conformance with a specification but not conformance with the customer's real requirements. Inspections cannot check non-functional characteristics such as performance, usability, etc. Inspection does not require execution of program and they maybe used before implementation. Many different defects may be discovered in a single inspection. In testing, one defect may mask another so several executions are required. For inspections, checklists are prepared that contain information regarding defects. Reuse domain and programming knowledge of the viewers likely to help in preparing these checklists. Inspections involve people examining the source representation with the aim of discovering anomalies and defects. Inspections may be applied to any representation of the system (requirements, design, test data, etc.) Thus inspections are a very effective technique for discovering errors in a software program.

## Inspection pre-conditions

A precise specification must be available before inspections. Team members must be familiar with the organization standards. In addition to it, syntactically correct code must be available to the inspectors. Inspectors should prepare a checklist that can help them during the inspection process.

## Inspection checklists.

Checklist of common errors in a program should be developed and used to drive the inspection process. These error checklists are programming language dependent such that the inspector has to analyze major constructs of the programming language and develop checklists to verify code that is written using these checklists. For example, in a language of weak type checking, one can expect a number of peculiarities in code that should be verified. So the corresponding checklist can be larger. Other example of programming language dependant defects are defects in variable initialization, constant naming, loop termination, array bounds, etc.

### Inspection Checklist

Following is an example of an inspection checklist.

| | |
|---|---|
| Exception management faults | • Have all possible error conditions been taken into account? |
| Fault Class | • Inspection Check |



© Copyright Virtual University of Pakistan

| | |
|---|---|
| Data faults | • Are all program variables initialized before their values are used?<br>• Have all constants been named?<br>• Should the lower bound of arrays be 0, 1, or something else?<br>• Should the upper bound of arrays be size or size -1?<br>• If character strings are used, is a delimiter explicitly assigned? |
| Control faults | • For each conditional statement, is the condition correct?<br>• Is each loop certain to terminate?<br>• Are compound statements correctly bracketed?<br>• In case statements, are all possible cases accounted for? |
| Input/Output faults | • Are all input variables used?<br>• Are all output variables assigned a value before they are output? |
| Interface faults | • Do all function and procedure calls have correct number of parameters?<br>• Do formal and actual parameters types match?<br>• Are the parameters in right order?<br>• If components access shared memory, do they have the same model of shared memory structure? |
| Storage management faults | • If a linked structure is modified, have all links been correctly assigned?<br>• If dynamic storage is used, has space been allocated correctly?<br>• Is space explicitly de-allocated after it is no longer required? |

In the checklist mentioned above, a number of fault classes have been specified and their corresponding inspection checks are described in the column at the right side. This type of checklist helps an inspector to look for specific defects in the program. These inspection checks are the outcomes of experience that the inspector has gained out of developing or testing similar programs.

## 11.15 Static analyzers

Static analyzers are software tools for source text processing. They parse the program text and try to discover potentially erroneous conditions and bring these to the attention of the verification and validation team. These tools are very effective as an aid to inspections. But these are supplement to but not a replacement for inspections.



© Copyright Virtual University of Pakistan

# Checklist for static analysis

| | |
|---|---|
| Data faults | • variable used before initialization<br>• variable declared but never used<br>• variables assigned twice but never used between assignments<br>• possible array bound violations<br>• undeclared variables |
| Control faults | • unreachable code<br>• unconditional branches into loops |
| Input/Output faults | • variable output twice with no intervening assignment |
| Storage Management fault | • unassigned pointers<br>• pointer arithmetic |


© Copyright Virtual University of Pakistan

# Lecture No. 42

## Debugging

### 12.1 Debugging

As Benjamin Franklin said, “in this world, nothing is certain but death and taxes.” If you are in the software development business, however, you can amend that statement. Nothing in life is certain except death, taxes, and software bugs. If you cryogenically freeze yourself, you can delay death indefinitely. If you move to a country with no income tax, you can avoid paying taxes by not buying anything. If you develop software, however, no remedy known to mankind can save you from the horror of software bugs.

## What is a Bug?

We call them by many names: software defects, software bugs, software problems, and even software “features.” Whatever you want to call them, they are things the software does that it is not supposed to do (or, alternatively, something the software doesn’t do that it is supposed to). Software bugs range from program crashes to returning incorrect information to having garbled displays.

## 12.2 A Brief History of Debugging

It would appear that as long as there have been computers, there have been computer bugs. However, this is not exactly true. Even though the earliest known computer programs contained errors, they were not, at that time, referred to as “bugs.” It took a lady named Admiral Grace Hopper to actually coin the term “bug.”

After graduating from Vassar in 1928, she went to Yale to receive her master’s degree in mathematics. After graduating from Yale, she worked at the university as a mathematics professor. Leaving Yale in 1943, with the onset of World War II, Mrs. Hopper decided to work for the Navy. Mrs. Hopper’s first assignment was under Commander Howard Aiken at Howard University, working at the Bureau of Ordinance Computation. She was a programmer on the Mark II, the world’s first automatically sequenced digital computer. The Mark II was used to determine shooting angles for the big guns in varying weather conditions during wartime.

It was during her term with the Mark II that Hopper was credited with coining the term “bug” for a computer problem. The first “bug” was actually a moth, which flew through an open window and into one of the Mark II’s relays. At that time, physical relays were used in computers, unlike digital components we use today. The moth shorted out across two contacts, temporarily shutting down the system. The moth would later be removed (de-bugged?) and pasted into the logbook of the project. From that point on, if her team was not producing numbers or working on the code, they claimed to be “debugging” the system.

From that auspicious beginning, computer debugging developed into something of a hit-or-miss procedure for quite a few years. Early debugging efforts mostly centered around either data dumps of the system or used output devices, such as printers and display lights, to indicate when an error occurred. Programmers would then step through the code line by line until they could determine the location of the problem. The next step in the



© Copyright Virtual University of Pakistan

evolution of debugging came with the advent of command-line debuggers. These simple programs were an amazing step forward for the programmers. Although difficult to use, even at the time, these programs represented the first real attempt to turn debugging from a hit-or-miss proposition into a reproducible process.
Once the debugger became a part of the programmer's arsenal, the software world began to consider other ways that programs could be more easily debugged. Software projects starting getting bigger, and the same techniques that worked well for small projects no longer worked when the program reached a certain size.

### Importance of Debugging

As we mentioned earlier in this course, one of the prime objectives of software engineering is to develop cost effective software applications. According to a survey, when a software application is in the maintenance phase, 20% of its lifecycle cost is attributed towards the defects which are found in the software application after installation. Please bear in mind that the maintenance is the phase in which 2/3rd of the overall software cost incurs. Therefore, 20% of the 2/3rd cost is again a huge cost and we need to understand why this much cost and effort is incurred. In fact, when a software application is installed and being used, any peculiarity in it can cost a lot of direct and indirect damages to the organization. A system downtime is the period in which tremendous pressure is on developers end to fix the problem and make the system running again. In these moments, every second costs hugs losses to the organization and it becomes vital to find out the bug in the software application and fix it. Debugging techniques are the only mechanism to reach at the code that is malfunctioning. In the following subsection, we shall discuss an incident that took place in 1990 and see how much loss the company had to suffer due to a mere bug in the software application.

## 12.4 Problem at AT&T

In the telecommunications industry, loss of service is known as an outage. For most of us, when as outage occurs, we lose our telephone service; we cannot make calls and we cannot receive calls. Outages are accepted and expected hazards in the industry.
On January 15. 1990, AT&T had a US wide telephone system outage that lasted for nine hours. The cause was due to a program error in the software that was meant to make the system more efficient. Eight years later, on April 13, 1998, AT&T suffered another massive failure in its frame-relay network, which affected ATM machines, credit card transactions and other business data services. This failure lasted 26 hours. Again, the bug was introduced during a software upgrade.

### Description of the Problem

The code snippet that caused the outage is illustrated as follows

```
1. do {
2.     . . .
3.     switch (expression) {
4.         case 0: {
5.             if (some_condition) {
```


© Copyright Virtual University of Pakistan

```
6.                  . . .
7.                  break;
8.              } else {
9.                  . . .
10.             }
11.           . . .
12.           break;
13.       }
14.     . . .
15.   }
16.   . . .
17. } while (some_other_condition);
```

In this case the break statement at line 7 was the culprit. As implemented, if the logical_test on line 5 was successful, the program should have proceeded to line 6 to execute those statements. When the program stepped to line 7, the break statement caused the program to exit the "switch" block between line 3 and line 15, and proceeded to execute the codes in line 16. However, this path of execution was not the intention of the programmer. The programmer intended the break statement in line 7 to break the if-then clause; so, after the program executed line 7, it was supposed to continue execution in line 11.

### AT&T statement about the Problem

"We believe that the software design, development, and testing processes we use are based on solid, quality foundations. All future releases of software will continue to be religiously tested. We will use the experience we've gained through this problem to further improve our procedures."

We do not believe we can fault AT&T's software development process for the 1990 outage, and we have no reason to believe that AT&T did not rigorously test its software update. In hindsight, it is easy to say that if the developers had only tested the software, they would have seen the bug. Or that if they had performed a code inspection, they would have found the defect. Code inspection might have helped in this case. However, the only way the code inspection could have uncovered this bug is if another engineer saw this particular line of code and asked the original programmer if that was his or her intension. The only reason that the code reviewers might have asked this question is if they were familiar with the specifications for this particular code block.

## 12.5Art and Science of Debugging

Debugging is taken as an art but in fact it is a scientific process. As people learn about different defect types and come across situations in which they have to debug the code, they develop certain heuristics. Next time they come across a similar situation, they apply those heuristics and solve the problem in lesser time and with a lesser effort. While discussing the debugging process we discuss the phenomenon of "you miss the obvious". When a person writes a code, he develops certain impression about that code. One can term this impression as a personal bias that the developer builds towards his creation the


© Copyright Virtual University of Pakistan

"code" and when he has to check this code, he can potentially miss out obvious mistakes due to this impression or bias. Therefore, it is strongly recommended that in order to reach to a defect in the code, one needs "another pair of eyes". That is, start discovering the defect by applying your own heuristics and if you could reach to the problem, fine, otherwise ask a companion to help you in this process. We shall further elaborate this idea based on the following example.

## Program at Bulletin Board Example

Following piece of code was once placed on a bulletin board making an invitation to people to discover the problem in this code. Please look at this code and discover the problem

```
1. while (i =0; i < 10; i++) {
2.     cout << i << newl;
3. }
```

Well if you could not guess it by now, the problem lies with the syntax of the while loop. This is so obvious that almost everyone forgot to ponder upon when placed on the bulletin board of a university. The loop is "while" but the syntax used is a "for" loop.

In order to reach such defects, one needs a scientific approach to check and verify the code methodically. Based on this discussion, we are now in a position to introduce a few classes of bugs to the reader. This is not an exhaustive list since there could be a number of other classes of bugs as well but the following classes will help the reader know about some well known bugs that we usually find in the code.

# Lecture No. 43

### 12.6 Bug Classes

## Memory and resource leak

A memory leak bug is one in which memory is somehow allocated from either the operating system or an internal memory "pool", but never deallocated when the memory is finished being used.

### Symptoms

- System slowdowns
- Crashes that occur "randomly" over a long period of time

### Example 1

Let's take a look at a simple memory leak error that can occur trivially in a C program. This type of code is found in hundreds of programs across the programming spectrum. It illustrates the simplest possible case of a memory leak, which is when memory is allocated and not deallocated in all cases. This particular form of the bug is the most frustrating because the memory is usually deallocated, but not always



© Copyright Virtual University of Pakistan

```
char *buffer = new char[kMaxBufferSize+1];
memset(buffer,0,kMaxBufferSize+1);

// Do some stuff to fill and work with the character buffer

if (IsError(nCondition)) // Did we get an error in the processing piece?
{
  Message(("An error occured.
            Skipping final stage"));
  return FailureCode;
}

// Final stage of code

// Free up all allocated memory

delete buffer;
return okCode;
```

Note that in many cases, this code works perfectly. If no error occurs in the processing stage (which is the norm) the memory is freed up properly. If, however, the processing stage encounters an error, the memory is not freed up and a leak occurs.

## Example 2

The following code snippet from some old C++ code contains a slightly more insidious bug. It illustrates the point well.

```
Class Foo
{
  private:
    int nStringLength;
    char *sString;
  public:
    Foo()
    {
      nStringLength = 20; //Default
      sString = new
      char[nStringLength + 1];
    }
    ~Foo()
    {
      delete sString;
    }
    void SetString(const char
                   *inString)
    {
```



© Copyright Virtual University of Pakistan

```
        sString = new char
                  [strlen(inString+1)];
        if(inString == NULL)
          return;
        strncpy(sString, inString,
                  strlen(inString));
        nStringLength =
                strlen(inString)+1;
    }
};
```

Let's discuss what happens. In most cases, the Foo object is created and nothing bad happens. If, however, you call the SetString method, all bets are off. The previously allocated string is overwritten by the new allocation, and the old allocated memory goes nowhere. We have an instant memory leak. Worse, we can do this multiple times if we accidentally call the method with a NULL string because it first allocates the block, and then checks to see if the input string was correct. Don't do things like this. If you see an allocation in a class, check to see if the string can be allocated before it gets to that point.

# Logical Errors

A logical error occurs when the code is syntactically correct but does not do what you expect it to do.

# Symptoms

- The code is misbehaving in a way that isn't easily explained.
- The program doesn't crash, but the flow of the program takes odd branches through the code.
- Results are the opposite of what is expected.
- Output looks strange, but has no obvious symptoms of corruption.

# Example

```
// Make sure that the input is valid. For this value, valid ranges are 1-10 and 15-20

if((input >= 1 && input <= 10) &&
  (input >= 15 && input <= 20))
  {
    // Do something for valid case
  }
else
  {
    // Do something for invalid case
  }
```

In order for the code to enter the valid case, the number must be between 1 and 10 and be between 15 and 20. If a number is between 1 and 10, how can it possibly be between 15 and 20 as well? Seems unlikely, doesn't it? This is a typical logical error.

# Coding errors



© Copyright Virtual University of Pakistan

A coding error is a simple problem in writing the code. IT can be a failure to check error returns, a failure to check for certain valid conditions, or a failure to take into account other parts of the system. Yet another form of a coding error is incorrect parameter passing and invalid return type coercion.

# Symptoms

- Unexpected errors in black box testing.
- The errors that unexpectedly occur are usually caused by coding errors.
- Compiler warnings.
- Coding errors are usually caused by lack of attention to details.

# Example

In the following example, a function accepts an input integer and converts it into a string that contains that integer in its word representation.

```
void convertToString(int InInteger,
      char* OutString, int* OutLength)
{
  switch(InInteger){
     case 1: OutString = "One";OutLength = 3;
               break;
     case 2: OutString = "Two";OutLength = 3;
               break;
     case 3: OutString = "Three";OutLength = 5;
               break;
     case 4: OutString = "Four";OutLength = 4;
               break;
     case 5: OutString = "Five";OutLength = 4;
               break;
     case 6: OutString = "Six";OutLength = 3;
               break;
     case 7: OutString = "Seven";OutLength = 5;
               break;
     case 8: OutString = "Eight";OutLength = 5;
               break;
     case 9: OutString = "Nine";OutLength = 4;
               break;

     }
}
```

There are a few things to notice in the preceding code. The ConvertToString function does not handle all cases of inputs, which becomes very obvious you pass a zero value. Worse, the function does not initialize the output variables, leaving them at whatever they happened to be when they came into the function. This isn't a problem for the output string, necessarily, but it will become a serious issue for the output length.

# Memory over-runs



© Copyright Virtual University of Pakistan

a memory overrun occurs when you use memory that does not belong to you. This can be caused by overstepping an array boundary or by copying a string that is too big for the block of memory it is defined to hold. Memory overruns were once extremely common in the programming world because of the inability to tell what the actual size of something really was.

# Symptoms

- Program crashes quite regularly after a given routine is called, that routine should be examined for a possible overrun condition.
- If the routine in question does not appear to have any such problem the most likely cause is that another routine, called in the prior sequence, has already trashed variables or memory blocks.
- Checking the trace log of the called routines leading up to one with the problem will often show up the error.

# Example

This particular example shows not only a memory overrun, but also how most programmers "fix" problems in an application. The ZeroArray function steps all over the array boundaries by initializing 100 separate entries. The problem is that that particular array only has 50 slots available in its allocation. What happens at that point is that the function goes past the end of the array and starts to walk on things beyond its control.

```
const kMaxEntries = 50;
int gArray[kMaxEntries];
char szDummyBuffer[256];
int nState = 10;

int ZeroArray (int *pArray)
{
  for (inti=0;i<100;++i)
    pArray[i] = 0;
}
```

# Loop Errors

- Loop errors break down into several different subtypes.
- They occur around a loop construct in a program.
- Infinite loops, off-by-one loops, and improperly exited loops.

# Symptoms

- If your program simply locks up, repeatedly displays the same data over and over, or infinitely displays the same message box, you should immediately suspect an infinite loop error.
- Off-by-one loop errors are quite often seen in processes that perform calculations.
- If a hand calculation shows that the total or final sum is incorrect by the last data point, you can quickly surmise that an off-by-one loop error is to blame.
- Likewise, if you were using graphics software and saw all of the points on the screen, but the last two were unconnected, you would suspect an off-by-one error.


© Copyright Virtual University of Pakistan

- Watching for a process that terminates unexpectedly when it should have continued.

# Example

```
bool doneFlag;
doneFlag = false;
while(!doneFlag){
  ...
  if( impossibleCondition )
    doneFlag = true;
}
```

The preceding code fragment contains an indirect looping error such that the loop will continue until the impossibleCondition becomes true which is impossible to happen.

```
int anArray[50];
int i;
i = 50;
while(i >= 0){
  anArray[i] = 0;
  i = i - 1;
}
```

In this example, the programmer was trying to be smart. He worked his way backward though the array, assuming that this way there would be no chance of an error. Of course, if you examine the loop, you will find that it performed 51 times. There are only fifty elements in the array. This will certainly lead to a problem later on in the program, if it doesn't first trigger an immediate error from the system.

```
int nIndex = 0;
for(int i=0; i<kMaxIterations; ++i)
{
  while (nIndex < 20)
  {
    ComputeSomething(i*20 + nIndex);
    nIndex ++;
  }
}
```

The final situation is an improper exit condition for a loop. This is most easily illustrated using a set of nested loops. In the above example, we are trying to compute something within the inner loop for some number of iterations. The code appears to do what we said it should do. It computes whatever it is we are computing in the inner loop for each iteration of the outer loop, 20 times. The problem, however, is that the exit condition simply says that nIndex should be less than 20. The first time through the outer loop, this variable will become 21, and the inner loop will exit. This is correct, and exactly the way


© Copyright Virtual University of Pakistan

you would expect this to happen. The problem, however, is that the next time through the loop, the inner loop will not be executed at all.

# Pointer errors

- A pointer error is any case where something is being used as an indirect pointer to another item.
- Uninitialized pointers: These are pointers that are used to point at something, but we fail to ever assign them something to point at.
- Deleted pointers, which continue to be used.
- An Invalid pointer is something that is pointing to a valid block of memory, but that memory does not contain the data you expect it to.

# Symptoms

- The program usually crashes or behaves in an unpredictable and baffling way.
- You will generally observe stack corruptions, failure to allocate memory, and odd changing of variable values.
- Changing a single line of code can change where the problem occurs.
- If the problem "goes away" when you place a print statement or new variable into the code that you suspect contains the problem.

# Example

Let's take a look at one of the most common forms of the pointer error, using a pointer after it has been deleted. As you can see by the following example, it is not always clear when you are doing this incorrectly:

```
void cleanup_function(char *ptr)
{
  SaveToDisk(ptr);
  delete ptr;
}

int func()
{
  char *s = new char[80];
  cleanup_function(s);
  delete s;
}
```

In this example, the programmer meant to be neat and tidy. He probably wrote the code originally to allocate the pointer at the top of this function, and then by habit put in a deallocation at the bottom of the function. This is, after all, good programming practice to avoid memory leaks. The problem is introduced with th3e cleanup_function routine. This function was probably written at a later time by another developer and might have been used to ensure that all allocated pointers were stored to the permanent drive and then freed up to avoid any possible leaks. In fact, the code in question comes from a set of functions that was reused from a memory pool system that saved its data to disk before destroying the pointer, so that another pointer could be retrieved from the persistent pool.



© Copyright Virtual University of Pakistan

The problem is, the reuse did not take into account the existing system. This problem is common when reusing only parts of a system.
When the second delete occurs at the bottom of func(), the results are unpredictable. At best, the delete function recognizes that this pointer has been deleted already and doesn't do anything. At worst, memory is corrupted and you are left with a memory crash somewhat later in the program, which will be very difficult to find and fix.

# Boolean bugs

Boolean bugs occur because the mathematical precision of Boolean algebra has virtually nothing to do with equivalent English words.
When we say "and", we really mean the boolean "or" and vice versa.

# Symptoms

- When the program does exactly the opposite of what you expect it to. For example, you might have thought you needed to select only one entry from a list in order to proceed. Instead, the program will not continue until you select more than one. Worse, it keeps telling you to select only one value.
- For true/false problems, you will usually see some sort of debug output indicating an error in a function, only to see the calling function proceed as though the problem had not occurred.

# Examples

The following code contains a simple example of a function that returns a counter-intuitive Boolean value and shows how that value leads to problems in the rest of the application code. This case performs an action and indicates to the user whether or not the action succeeded.

```
int DoSomeAction(int InputNum)
{
  if(InputNum < 1 || InputNum  > 10)
       DoSomeAction = 0;
  else
  {
    PerformTheAction(InputNum);
    DoSomeAction = NumberOfAction + 1;
    NumberOfAction = NumberOfAction + 1;
  }
  return DoSomeAction;
}
```

After looking at the function, ask yourself this – What is the return value in the case of success? What would it be in the case of failure? How would you know this by simply glancing at the function declaration?

```
int Value;
int Ret;

Value = 11;
```


© Copyright Virtual University of Pakistan

```
Ret = DoSomeAction(Value);
if(Ret)
  cout <<Error in DoSomeAction<<;
```

If you were debugging this application, you might start out by wondering why it didn't work. Further investigation would show that the error statement is never triggered in spite of the fact that the routine did, in fact, encounter an error.. Why did this happen? It happened because the routine didn't return nonzero value for failure, it returned nonzero for success. This Boolean value was not intuitive given the condition of the function and led to a poor assumption by the programmer who wrote the code that used it.

# Lecture No. 44

## 12.7 Holistic approach

Holistic means

- Emphasizing the importance of the whole and the interdependence of its parts.
- Concerned with wholes rather than analysis or separation into parts

What this meant is that a holistic approach focuses on the entire system rather than whatever piece appears to be broken. Holistic medicine, for example, concerns itself with the state of the body as a whole, not the disease that is currently attacking it. Similarly, programmers and debuggers must understand that you cannot treat the symptoms of a problem, you must focus on the application system as a whole.

Now that you understand what holistic means, how do you apply holistic concepts to debugging the software programs and procedures? Let's start by thinking about what emphasizing the whole over the individual pieces means in debugging. When you are debugging an application, you often simply look at the problem reported by the user and how you can make that problem go away.

As a simple example, consider the case where the program mysteriously crashes at a particular point in the code each and every time the program is run. The point at which the code crashes makes no sense at all. For example, in C++, you might have a member function of a class that reads:

```
void SetX(int X)
{
  mX = X;
}
```

The program always crashes on the line that assigns X to the member variable mX(mX = X). Looking at this code snippet, we see very few valid ways in which the program could be encountering a problem at this point. Oddly, we might discover that inserting a message statement makes the problem stop occurring when the indicated steps are followed for reproducing the problem. Is the problem fixed?



© Copyright Virtual University of Pakistan

Most experienced programmers, managers, or debuggers would say that the problem is not fixed, although few could tell you exactly why that is the case. The fact is, in this case, we are directly treating the symptoms (the program crashing) rather than looking into the actual problem and ignoring the overall problem is endemic in our industry.

## 12.8 The debugging process

In normal circumstances, you will have a user description of the problem. This description might have been given to you directly by the user, or it might have been gathered by a customer support person or other no technical person, In any event, this data has to be considered suspiciously until you can get a first hand description of what really happened. First hand accounts of the problem are always useful, so be sure to write down exactly what you are told. That way, you can compare several accounts of the same problem and look for similarities. Consider, for example, the following accounts of a reported bug in a system:

1. "I started by trying to setup a Favorites list. I first went to the home page. Then, I selected Favorites from the menu on the right. I scrolled down to the third entry and pressed Enter on the keyboard. Then I moved to the fourth entry on the submenu and clicked it with the mouse. Finally, I entered my name as Irving, clicked on OK, and the program crashed."

2. "I went to the programming menu and selected the New menu option. I then clicked on the Create Object menu entry and entered the name HouseObject for the object name field. Then I clicked OK, and the program crashed."

3. "I selected the New Project menu option, and then clicked on the icon that looks like a Gear. I entered the name Rudolph for the project name, and then clicked OK. The program crashed."

    - After all, they appear to relate to three different sections of the program.
      Most people, when reporting bugs, focus on what they think was the important step in the process.
    - Filter out the unimportant information and see what each case really has in common.
      First, each user clicked on the OK button to finalize the process, and the program crashed.
    - It might seem likely, therefore, that the program OK handler contains a fatal flaw.
    - A bit of experimentation will show whether this is the case or not.
    - When we examine the statements of the witnesses, we notice that each was working with a menu.
    - We can see that each person used both the keyboard and the mouse to select from the menus before clicking the OK button.

"I selected Favorites from the menu on the right. I scrolled down to the third entry and pressed Enter on the keyboard. Then I moved to the fourth entry on the submenu and


© Copyright Virtual University of Pakistan

clicked it with the mouse."

"... selected the New menu option. I then clicked on the Create Object menu entry ..."

"I selected the New Project menu option, and then clicked on the icon that looks ..."

- We can infer, therefore, that the user in this case used both the keyboard and mouse.
  The next logical place to look is in the menu handler to see whether it deals with mouse and keyboard entries differently.
- The final clue is in the third entry, where the user did not select anything from the menu with the mouse, but instead clicked on an icon.
- While looking at the code, I would try to see what happens when the program deals with a combination of keyboard and menu entry.
- It is likely, given the user discussion, that you will find the root of the problem in this area.
- After finding such a common bug, it is likely that the fix will repair a whole lot of problems at once
- This is a debugger's dream.

### Good clues, Easy Bugs

#### Get A Stack Trace

In the debugging process a stack trace is a very useful tool.
Following stack trace information may help in debugging process.

- Source line numbers in stack trace is the single, most useful piece of debugging information.
- After that, values of arguments are important
  - Are the values improbable (zero, very large, negative, character strings with non-alphabetic characters?
- Debuggers can be used to display values of local or global variables.
  - These give additional information about what went wrong.

## Non-reproducible bugs

- Bugs that won't "stand still" (almost random) are the most difficult to deal with.
- Randomness itself, however, is information.
- Are all variables initialized? (random data in variables could affect output).
- Does bug disappeared when debugging code is inserted? Memory allocation (malloc) problems are probably a culprit.
- Is the crash site far away from anything that could be wrong?
- Check for dangling pointers.

### Example

```
char *msg(int n, char *s)
{
  char buf[100];
```



© Copyright Virtual University of Pakistan

```
   sprintf(buf, "error %d: %s\n",
            n, s);
   return buf;
}
...
p = msg(20, "Output values");
...
q = msg(30, "Input values");
...
printf("%s\n",p);
```

# Lecture No. 45

## Summary



© Copyright Virtual University of Pakistan

**FINALTERM EXAMINATION**
**Spring 2010**
**CS504- Software Engineering - I (Session - 2)**

**Question No: 1 ( Marks: 1 ) - Please choose one**
**The first step in any OOA process model is to_____**

- ► build an object-relationship model
- ► define collaborations between objects
- ► **elicit customer requirements** **(A PRACTITIONER'S APPROACH Page603)**
- ► select a representation language

**Question No: 2 ( Marks: 1 ) - Please choose one**
**A necessary supplement to transform or transaction mapping needed to create a complete architectural design is _____.**

- ► entity relationship diagrams
- ► the data dictionary
- ► **processing narratives for each module** **(A PRACTITIONER'S APPROACH Page422)**
- ► test cases for each module

**Question No: 3 ( Marks: 1 ) - Please choose one**
**What types of errors are missed by black-box testing and can be uncovered by white-box testing?**

- ► behavioral errors
- ► **subtle logic errors** **Click here for detail**
- ► performance errors
- ► input error

**Question No: 4 ( Marks: 1 ) - Please choose one**
**Which test criteria should be applied in a phase of testing?**

- ► functional validity
- ► interface integrity
- ► correctness
- ► **all of the given** **(A PRACTITIONER'S APPROACH Page522)**

**Question No: 5 ( Marks: 1 ) - Please choose one**
**The hardest single part of building a software system is deciding precisely ---------- to build.**

- ► **what (Page 17)**
- ► How
- ► When
- ► Why

**Question No: 6 ( Marks: 1 ) - Please choose one**
**In sequence Diagrams objects are organized in a ----------line**

- ► **horizontal (Page 106)**
- ► vertical
- ► horizontal and vertical
- ► non of the above

**Question No: 7 ( Marks: 1 ) - Please choose one**
**Every view is potentially Hierarchical in this regard which of the view are Hierarchical ?**

- ► Functioanl View
- ► Development View
- ► *Concurrency view*
- ► **All of given (Page 125)**

**Question No: 8 ( Marks: 1 ) - Please choose one**
**Like analysis models, many different kinds of ----------models are developed**

- ► **Architectural (Page 126)**
- ► System
- ► Design
- ► Logical

**Question No: 9 ( Marks: 1 ) - Please choose one**
**Idea behind zero install architecture is to develop a system where no installation on the ---------- is needed.**

- ► **Client side (Page 130)**
- ► Server Side
- ► Client & Server Side
- ► Non of the all

**Question No: 10 ( Marks: 1 ) - Please choose one**
**The construction is used to avoid ---------- errors.**

- ► **compilation (Page 157)**
- ► runtime
- ► design time
- ► non of them

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 11 ( Marks: 1 ) - Please choose one**
**All comments should be written in----------**

- **English (Page 162)**
- French
- C++
- JAVA

**Question No: 12 ( Marks: 1 ) - Please choose one**
**One of the Guidelines to avoid common mistakes is to never use --------- except for declaration**

- **, (Page 176)**
- ;
- =
- ||

**Question No: 13 ( Marks: 1 ) - Please choose one**
**The order in which bytes of one word are stored is ---------- dependent**

- **Hardware (Page 181)**
- Software
- language
- system

**Question No: 14 ( Marks: 1 ) - Please choose one**
**Verification and validation are the processes in which we check a product against its --------- and the ---------- of the users who will be using it**

- **specifications, expectations (Page 192)**
- statements, expectations
- specifications , arguments
- statements, arguments

**Question No: 15 ( Marks: 1 ) - Please choose one**
**Two tests are considered to be equivalent if it is believed that: if one discovers a defect, the other probably will too, and if one does not discover a defect, ----------.**

- the other probably may
- non of them
- **the other probably won't either (Page 199)**
- the other probably will

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 16 ( Marks: 1 ) - Please choose one**
**It was lady named ---------- who actually coin the term “bug" for the fiest time.**

- ► Elisay Chistopher
- ► **Admiral Grace Hopper (Page 213)**
- ► Ana Nicholson
- ► Jane Hopper

**Question No: 17 ( Marks: 1 ) - Please choose one**
**The first “bug” was actually a moth, which flew through an open window and into one of the Mark ----------’s relays.**

- ► **II (Page 213)**
- ► I
- ► III
- ► IV

**Question No: 18 ( Marks: 1 ) - Please choose one**
**Symptoms of logical errors are ----------**

- ► code is misbehaving
- ► The program doesn't crash, but the flow of program takes odd branches through the code.
- ► Results are the opposite
- ► Output looks strange,
- ► **all of the given (Page 218)**

**Question No: 19 ( Marks: 1 ) - Please choose one**
**First hand accounts of the problem are always useful in Debugging process.**

- ► **True (Page 225)**
- ► False

**Question No: 20 ( Marks: 1 ) - Please choose one**
**Software architecture elements are further divided into categories which in total are ----------**

- ► 2
- ► **3 (Page 122)**
- ► 4
- ► 5

**Question No: 21 ( Marks: 1 ) - Please choose one**
**Establishing responsibilities for objects includes**

- ► Generalization Relationships
- ► Specialization Relationships
- ► all of the above
- ► **identifying Association relationships (Page 101)**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 22 ( Marks: 1 ) - Please choose one**
**Source files can have the extension ----------**

- ► *c++* .
- ► .*C*
- ► .cpp
- ► **all of the given (Page 155)**

**Question No: 23 ( Marks: 1 ) - Please choose one**
**Be very careful when you use functions with side effects – functions that change the values of the ----------**

- ► **parameters (Page 176)**
- ► variables
- ► constants
- ► None of given

**Question No: 24 ( Marks: 1 ) - Please choose one**
**Which of the following is not a characteristic of software?**

- ► **Software is tangible**
- ► A change to a piece of code may implicitly affect the functions of the rest of the code.
- ► Software is configurable.
- ► Software does not wear and tear.

**Question No: 25 ( Marks: 1 ) - Please choose one**
**Which of the following is a fact finding method?**

- ► Site visits
- ► **Prototyping**
- ► Study of similar systems
- ► All of given

Seven common fact-finding methods (Sampling, Research, Observation, Questionnaires, Interviews, Prototyping, Joint Requirements Planning) are introduced as a means to discover requirements.

**Question No: 26 ( Marks: 1 ) - Please choose one**
**The statements given below are associated with system development, testing and maintenance. Identify the correct statement from among them:**

► Some of the activities in the implementation phase are building individual system components, writing of programs and development of user interfaces.
► The post implementation review is performed to see that the computer system is working.
► None of Given
► System development phase consists of a development phase and an implementation phase.

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**FINALTERM EXAMINATION**
**Spring 2010**

**Question No: 1 ( Marks: 1 ) - Please choose one**
**Quantitative methods for assessing the quality of proposed architectural designs are readily available.**

- ► True
- ► **False** http://highered.mcgraw-hill.com/sites/0072853182/student_view0/chapter10/multiple_choice_quiz.html

**Question No: 2 ( Marks: 1 ) - Please choose one**
**A decision table should be used_____**

- ► to document all conditional statements
- ► to guide the development of the project management plan
- ► only when building an expert system
- ► **when a complex set of conditions and actions appears in a component** **Click here for detail**

**Question No: 3 ( Marks: 1 ) - Please choose one**
**Test cases should be designed long before testing begins.**

- ► **True** **(A PRACTITIONER'S APPROACH Page 467)**
- ► False

**Question No: 4 ( Marks: 1 ) - Please choose one**
**Which of the following are characteristics of testable software?**

- ► observability
- ► simplicity
- ► stability
- ► **all of the given** **(A PRACTITIONER'S APPROACH Page 469)**

**Question No: 5 ( Marks: 1 ) - Please choose one**
**Comparison testing is typically done to test two competing products as part of customer market analysis prior to product release.**

- ► True
- ► **False** **Click here for detail**

**Question No: 6 ( Marks: 1 ) - Please choose one**
**By collecting software metrics and making use of existing software reliability models it is possible to develop meaningful guidelines for determining when software testing is done.**

- ► **True** **Click here for Detail**
- ► False

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 7 ( Marks: 1 ) - Please choose one**
**Configuration reviews are not needed if regression testing has been rigorously applied during software integration.**

- ► True
- ► **False** **Click here for detail**

**Question No: 8 ( Marks: 1 ) - Please choose one**
**A change becomes ---------- because of close presence of data and fucntions**

- ► **Localized (Page 81)**
- ► Private
- ► Global
- ► Accessible

**Question No: 9 ( Marks: 1 ) - Please choose one**
**Requirement engineering mainly deals with the ---------- of the system**

- ► **definition phase (Page 16)**
- ► development phase
- ► maintenance
- ► non of the above

**Question No: 10 ( Marks: 1 ) - Please choose one**
**40-60% of all defects found in software projects can be traced back to poor ----------**

- ► **Requirements (Page 17)**
- ► Design
- ► Coding
- ► Testing

**Question No: 11 ( Marks: 1 ) - Please choose one**
**In the N-Tire Architecture the idea is to enhance scalability and ---------- by distributing both the data and the application using multiple server machines.**

- ► **Performance (Page 131)**
- ► Efficiency
- ► Usability
- ► non of the all

**Question No: 12 ( Marks: 1 ) - Please choose one**
**MVC pattern was based on the ---------- pattern.**

- ► **Observer (Page 140)**
- ► Structural
- ► Behavioral
- ► Non of them

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 13 ( Marks: 1 ) - Please choose one**
**Classes should be declared in individual header files with the file name matching the ---------- name.**

► **class (Page 155)**
► method
► object
► non of the all

**Question No: 14 ( Marks: 1 ) - Please choose one**
**Unit testing is roughly equivalent to ---------- testing for hardware in which each chip is tested thoroughly after manufacturing**

► Circuit level
► **Chip Level (Page 207)**
► component level
► system level

**Question No: 15 ( Marks: 1 ) - Please choose one**
**Software should be tested more like hardware, with Built-in self testing: such that each unit can be tested ----------**

► freely
► **independently (Page 207)**
► dependtly
► completely

**Question No: 16 ( Marks: 1 ) - Please choose one**
**A system ---------- is the period in which tremendous pressure is on developers end to fix the problem and make the system running again**

► Uptime
► **Downtime (Page 214)**
► Currentime
► futuretimr

**Question No: 17 ( Marks: 1 ) - Please choose one**
**Which one of the given below is not a symptom of memory overrun?**

► Program crashes quite regularly after a given routine is called, that routine should be examined for a possible overrun condition.
► If the routine in question does not appear to have any such problem the most likely cause is that another routine, called in the prior sequence, has already
trashed variables or memory blocks.
► Checking the trace log of the called routines leading up to one with the problem will often show up the error.
► **Compiler warnings. (Page 220)**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 18 ( Marks: 1 ) - Please choose one**
**Software architecture elements are further divided into categories which in total are ----------**

- ► 2
- ► **3 (Page 122) rep**
- ► 4
- ► 5

**Question No: 19 ( Marks: 1 ) - Please choose one**
**Three tier architecture contains ---------- layers**

- ► Presentation
- ► Application
- ► Database
- ► **All of the above (Page 131)**

**Question No: 20 ( Marks: 1 ) - Please choose one**
**1. Variables should be initialized where they are declared and they should be declared in the ---------- scope possible.**

- ► **Smallest (Page 158)**
- ► largest
- ► medium
- ► None of the given

**Question No: 21 ( Marks: 1 ) - Please choose one**
**---------- are two important tools that can help in managing and mastering the complexity of a program.**

- ► **abstraction and encapsulation (Page 170)**
- ► abstraction and Inheritence
- ► abstraction and Polymarhphism
- ► None of given

**Question No: 22 ( Marks: 1 ) - Please choose one**
**Which of the following is a/are tool used in requirement analysis?**

- ► Flow Graphs
- ► **Data Flow Diagrams (not sure)**
- ► Activity Networks
- ► Module Dependency Diagrams

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 23 ( Marks: 1 ) - Please choose one**
**Consider the following comment. "The software which I bought won't run on windows and when it runs I can't use WORD at the same time". Which of the following do you think are violated by the newly bought software?**

- ► dependability, interchangeability
- ► **platform independence, interoperability**
- ► reliability, dependency
- ► interoperability, reliability

**Question No: 24 ( Marks: 1 ) - Please choose one**
**…… is a diagramming technique used to identify the types of objects in the system and the static relationships that exist among them?**

- ► Class Diagram
- ► Document flow diagrams
- ► Data flow diagrams
- ► **Flow charts**

**Question No: 25 ( Marks: 1 ) - Please choose one**
**Which of the following interaction style best suit the design of an interface for visually impaired users?**

- ► **direct manipulation** **(A PRACTITIONER'S APPROACH Page 469)**
- ► direct manipulation
- ► natural language
- ► command line

**Question No: 26 ( Marks: 1 ) - Please choose one**
**Which of the following testing involve purely black box testing?**

- ► unit testing, beta testing
- ► acceptance testing, interface testing
- ► beta testing, acceptance testing
- ► **integration testing, interface testing**

**FINALTERM EXAMINATION**
**Spring 2010**

**Question No: 1 ( Marks: 1 ) - Please choose one**
**What are the three generic phases of software engineering?**

- ► **definition, development, support** **(A PRACTITIONER'S APPROACH Page 96)**
- ► what, how, where
- ► programming, debugging, maintenance
- ► analysis, design, testing

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 2 ( Marks: 1 ) - Please choose one**
**In the context of requirements analysis, partitioning results in the elaboration of data, function, or behavior.**

- **True** **Click here for detail**
- False

**Question No: 3 ( Marks: 1 ) - Please choose one**
**Quantitative methods for assessing the quality of proposed architectural designs are readily available.**

- True
- **False** **Click here for detail**

**Question No: 4 ( Marks: 1 ) - Please choose one**
**In refining the DFD during transaction mapping it is unnecessary to create a PSPEC since only the CSPEC is relevant to this type of architectural style.**

- True
- **False** **click here for detail**

**Question No: 5 ( Marks: 1 ) - Please choose one**
**In transaction mapping the first level factoring results in the_____**

- creation of a CFD
- **derivation of the control hierarchy** **Click here for detail**
- distribution of worker modules
- refinement of the module view

**Question No: 6 ( Marks: 1 ) - Please choose one**
**Variable names must be in mixed case starting with upper case.**

- True
- **False (Page 150)**

**Question No: 7 ( Marks: 1 ) - Please choose one**
**Class variables should be declared public. This concept violates which of the following.**

- Information hiding
- Encapsulation
- **Information hiding and Encapsulation (Page 158)**
- None of given

**Question No: 8 ( Marks: 1 ) - Please choose one**
**Real-time applications add a new and potentially difficult element to the testing mix**

- performance
- reliability
- security
- **time** **(Page 498)**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 9 ( Marks: 1 ) - Please choose one**
**Top-down integration testing has as it's major advantage(s) that**

- ► low level modules never need testing
- ► **major decision points are tested early** **Click here for detail**
- ► no stubs need to be written
- ► none of the given

**Question No: 10 ( Marks: 1 ) - Please choose one**
**Configuration reviews are not needed if regression testing has been rigorously applied during software integration.**

- ► True
- ► **False** **Click here for detail**

**Question No: 11 ( Marks: 1 ) - Please choose one**
**isAuthorized, assess Performance,get item for UPC, get cashier for number, are the examples of**

- ► **Services (Page 102)**
- ► Attributes
- ► Links
- ► Non of the above

**Question No: 12 ( Marks: 1 ) - Please choose one**
**the *development* view describes the ----------organization of the software in its development environment,**

- ► **Static (Page 122)**
- ► Dynamic
- ► Still
- ► Static & Dynamic

**Question No: 13 ( Marks: 1 ) - Please choose one**
**---------- being the first pure Object Oriented language in which observer pattern was used in implementing its Model View Controller**

- ► **Smalltalk (Page 140)**
- ► PASCAL
- ► JAVA
- ► C++

**Question No: 14 ( Marks: 1 ) - Please choose one**
**Hungarian Notation was first discussed by Charles Simonyi of----------**

- ► **Microsoft. (Page 149)**
- ► Oracle
- ► Apple Macintosh
- ► Non of the all

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 15 ( Marks: 1 ) - Please choose one**
**The language does not specify whether ---------- is signed or unsigned.**

- ► **char (Page 181)**
- ► integer
- ► double
- ► constant

**Question No: 16 ( Marks: 1 ) - Please choose one**
**A number of invisible execution paths can exist in simple code in a language that allows ----------**

- ► **exceptions (Page 185)**
- ► defects
- ► errors
- ► all of them

**Question No: 17 ( Marks: 1 ) - Please choose one**
**Bugs Fixing is done by which of the teams in Software Development lifecycle?**

- ► **Development Team (Page 196)**
- ► Testing Team
- ► Analysis & Design Team
- ► Process Team

**Question No: 18 ( Marks: 1 ) - Please choose one**
**Software Bugs have multiple names, Which one of the below is not the name of Software Bugs ----------**
**-**

- ► Bugs
- ► Defects
- ► Errors
- ► **Mistakes (Page 213)**

**Question No: 19 ( Marks: 1 ) - Please choose one**
**Symptoms of logical errors are ----------**

- ► code is misbehaving
- ► The program doesn't crash, but akes odd branches through the code.
- ► Results are the opposite
- ► Output looks strange,
- ► **all of the given (Page 218)**

**Question No: 20 ( Marks: 1 ) - Please choose one**
**First hand accounts of the problem are always useful in Debugging process.**

- ► **True (Page 225)**
- ► False

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 21 ( Marks: 1 ) - Please choose one**
**In the debugging process a stack trace is a very useful tool.**

- **True (Page 226)**
- False

**Question No: 22 ( Marks: 1 ) - Please choose one**
**Establishing responsibilities for objects includes**

- Generalization Relationships
- Specialization Relationships
- all of the above
- **identifying Association relationships (Page 101)**

**Question No: 23 ( Marks: 1 ) - Please choose one**
**Stakeholders are different people who would be interested in the ----------**

- **Software (Page 24)**
- System
- Product
- All of the given

**Question No: 24 ( Marks: 1 ) - Please choose one**
**A ---------- is a code that explains itself without the need of comments and extraneous documentation**

- **Self documenting code (Page 147)**
- Self telling Code
- Self Documenting Design
- None of the given

**Question No: 25 ( Marks: 1 ) - Please choose one**
**1. Variables should be initialized where they are declared and they should be declared in the ---------- scope possible.**

- **Smallest (Page 158)**
- largest
- medium
- None of the given

**Question No: 26 ( Marks: 1 ) - Please choose one**
**Which of the following is an/are advantage(s) of object oriented analysis & design over structured system analysis & design?**

- Ease of modeling real world events
- Reusability
- Maintainability
- **All of given** **Click here for detail**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

## FINAL TERM EXAMINATION SPRING 2010

**Question No: 1 ( Marks: 1 ) - Please choose one**
**The two main documents produced during this phase are Requirement Statement and Requirement Specification. They are also called Requirement Definition and ----------**

- ► **Functional Specification (Page 25)**
- ► Mathematical specification
- ► System Specification
- ► None of the given

**Question No: 2 ( Marks: 1 ) - Please choose one**
**A decision table should be used_____**

- ► To document all conditional statements
- ► To guide the development of the project management plan
- ► Only when building an expert system
- ► **when a complex set of conditions and actions appears in a component** **Click here for detail**

**Question No: 3 ( Marks: 1 ) - Please choose one**
**Which of the following is a/are tool used in requirement analysis?**

- ► Flow Graphs
- ► **Data Flow Diagrams (not sure)**
- ► Activity Networks
- ► Module Dependency Diagrams

**Question No: 4 ( Marks: 1 ) - Please choose one**
**MVC stands for ----------**

- ►**Model View Controller (Page 140)**
- ►Modern View Center
- ►Model View Center
- ►Modern View Controller

**Question No: 5 ( Marks: 1 ) - Please choose one**
**A ---------- is a code that explains itself without the need of comments and extraneous documentation**

- ►**Self documenting code (Page 147)**
- ►Self telling Code
- ►Self Documenting Design
- ►Non of the above

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 6 ( Marks: 1 ) - Please choose one**

**Bugs that won't "stand still" (almost random) are the ---------- to deal with.**

►least difficult

►**most difficult (Page 226)**

►very easy

►easy

**Question No: 7 ( Marks: 1 ) - Please choose one**

**In refining the DFD during transaction mapping it is unnecessary to create a PSPEC since only the CSPEC is relevant to this type of architectural style.**

► True

► **False**

**Question No: 8 ( Marks: 1 ) - Please choose one**

**Association is a type _____of relation**

►**Weak (OOP , 49)**

►Stronger

►Normal

►None of above

**Question No: 9 ( Marks: 1 ) - Please choose one**

**Variable names must be in mixed case starting with upper case.**

► True

► **False (Page 150)**

**Question No: 10 ( Marks: 1 ) - Please choose one**

**Bugs that won't (almost random) are the ---------- to deal with.**

►**stand still (Page 226)**

►Dynamic

►Static

►None of above

**Question No: 11 ( Marks: 1 ) - Please choose one**

**CRUD include following:-**

►Creat

►Update

►Read

►**All of above (Page 53)**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 12 ( Marks: 1 ) - Please choose one**
**Bugs Finding is done by which of the teams in Software Development lifecycle?**

- ► Development Team
- ► **Testing Team (Page 196)**
- ► Analysis & Design Team
- ► Process Team

**Question No: 13 ( Marks: 1 ) - Please choose one**
**Software Bugs have multiple names, which one of the below is not the name of Software Bugs ----------**

- ► Bugs
- ► Defects
- ► Errors
- ► **Mistakes (Page 213)**

**Question No: 14 ( Marks: 1 ) - Please choose one**
**In the debugging process a stack trace is a very useful tool.**

- ► **True (Page 226)**
- ► False

**Question No: 15 ( Marks: 1 ) - Please choose one**
**A ---------- is a code that explains itself without the need of comments and extraneous documentation**

- ► **Self documenting code (Page 147)**
- ► Self telling Code
- ► Self Documenting Design
- ► None of the given

**Question No: 16 ( Marks: 1 ) - Please choose one**
**Project ---------- defines the concept and range of the proposed solution, and limitations identify certain capabilities that the product will not include**

- ► **Scope (Page 30)**
- ► Agreement
- ► Plan
- ► None of the given

**Question No: 17 ( Marks: 1 ) - Please choose one**
**In the N-Tire Architecture the idea is to enhance scalability and ---------- by distributing both the data and the application using multiple server machines.**

- ► **Performance (Page 131)**
- ► Efficiency
- ► Usability
- ► non of the all

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 18 ( Marks: 1 )**

**The cyclomatic complexity metric provides the designer with information regarding the number of**

- ► Cycles in the program
- ► Errors in the program
- ► **Independent logic paths in the program** **Click here for detail**
- ► Statements in the program

**Question No: 19 ( Marks: 1 )**

**Which of the followings is not a testing type?**

- ► Subsystem testing
- ► Alpha testing
- ► Beta Testing
- ► **Gamma Testing (Page 198)**

**Question No: 20 ( Marks: 1 )**

**Which of the items listed below is not one of the software engineering layers?**

- ► Process
- ► **Manufacturing** **Click here for detail**
- ► Methods
- ► Tools

**Question No: 21 ( Marks: 1 )**

**Which piece of code is self-documented?**

- ►if (x==0) // this is the case when we are allocating a new number
- ►if (AllocFlag == 0)
- ►**If (AllocFlag == NEW_NUMBER) (Page 148)**
- ►None of the given

**FINALTERM EXAMINATION**
**Spring 2010**

**Question No: 1 ( Marks: 1 ) - Please choose one**

**UML (unified modeling language) analysis modeling focuses on the _____ .**

- ► behavioral model and environment model.
- ► behavioral model and implementation model.
- ► user model and environmental model
- ► **user model and structural model** **(A PRACTITIONER'S APPROACH Page 604)**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 2 ( Marks: 1 ) - Please choose one**
**What makes requirements elicitation difficult?**

- ► bounding scope
- ► understanding user needs
- ► requirements volatility
- ► **all of the above (A PRACTITIONER'S APPROACH Page 285)**

**Question No: 3 ( Marks: 1 ) - Please choose one**
**In the architecture trade-off analysis method the architectural style should be described using the**

- ► module view
- ► process view
- ► **all of the given (A PRACTITIONER'S APPROACH Page 403)**
- ► data flow view

**Question No: 4 ( Marks: 1 ) - Please choose one**
**In refining the DFD during transaction mapping it is unnecessary to create a PSPEC since only the CSPEC is relevant to this type of architectural style.**

- ► True
- ► **False** click here for detail

**Question No: 5 ( Marks: 1 ) - Please choose one**
**A decision table should be used_____**

- ► to document all conditional statements
- ► to guide the development of the project management plan
- ► only when building an expert system
- ► **when a complex set of conditions and actions appears in a component** Click here for detail

**Question No: 6 ( Marks: 1 ) - Please choose one**
**Variable names must be in mixed case starting with upper case.**

- ► True
- ► **False (Page 150)**

**Question No: 7 ( Marks: 1 ) - Please choose one**
**Which of these are objectives for software testing?**

- ► determine the productivity of programmers
- ► eliminate the need for future program maintenance
- ► eliminate every error prior to release
- ► **uncover software errors (A PRACTITIONER'S APPROACH Page 467)**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 8 ( Marks: 1 ) - Please choose one**
**Comparison testing is typically done to test two competing products as part of customer market analysis prior to product release.**

- ► True
- ► **False** **Click here for detail**

**Question No: 9 ( Marks: 1 ) - Please choose one**
**In software quality assurance work there is no difference between software verification and software validation**

- ► True
- ► **False** **Click here for detail**

**Question No: 10 ( Marks: 1 ) - Please choose one**
**Top-down integration testing has as it's major advantage(s) that**

- ► low level modules never need testing
- ► major decision points are tested early
- ► **no stubs need to be written** **Click here for detail**
- ► none of the given

**Question No: 11 ( Marks: 1 ) - Please choose one**
**Rooko mut jane do depicts one of risks associated with inadequate requriements gathering, the name of that risk is ----------**

- ► **Ambiguous requirements (Page 20)**
- ► Creeping Requirements
- ► Minimal Specification
- ► Non of the above

**Question No: 12 ( Marks: 1 ) - Please choose one**
**Interaction Diagrams depict the ---------- Behavior of the system**

- ► Static
- ► **Dynamic (Page 106)**
- ► Active
- ► Non of the above

**Question No: 13 ( Marks: 1 ) - Please choose one**
**In Sequence Diagrams events are organized in a----------time line**

- ► Vertical
- ► **Horizontal (Page 106)**
- ► Vertical & Horizontal
- ► Non of the above

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 14 ( Marks: 1 ) - Please choose one**
**the development view describes the ----------organization of the software in its development environment,**

- ► **Static (Page 122)**
- ► Dynamic
- ► Still
- ► Static & Dynamic

**Question No: 15 ( Marks: 1 ) - Please choose one**
**Every view is potentially Hierarchical in this regard which of the view are Hierarchical ?**

- ► Functioanl View
- ► Development View
- ► C*oncurrency view*
- ► **All of given (Page 125)**

**Question No: 16 ( Marks: 1 ) - Please choose one**
**object-oriented framework that is a set of cooperative classes that make up reusable ---------- of a system**

- ► Code
- ► **Design (Page 138)**
- ► Analysis
- ► non of the all

**Question No: 17 ( Marks: 1 ) - Please choose one**
**When planning for performance, one should always remember the ---------- rule**

- ► **80/20 (Page 177)**
- ► 20/80
- ► 80/22
- ► 80/90

**Question No: 18 ( Marks: 1 ) - Please choose one**
**In order to make a code more portable, Instead of using vendor specific language extensions, use ---------- as much as possible**

- ► **STL (Page 179)**
- ► ANSI
- ► ISO
- ► CMMI

**Question No: 19 ( Marks: 1 ) - Please choose one**
**Performance & Usability are examples of ---------- requirements**

- ► Business

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

► Functioanl
► **Non-Functioanl (Page 39)**
► User

**Question No: 20 ( Marks: 1 ) - Please choose one**
**Static analyzers are software tools for ---------- processing.**

► analysis text
► **source text (Page 211)**
► design text
► non of the given

**Question No: 21 ( Marks: 1 ) - Please choose one**
**Software Bugs have multiple names, Which one of the belw is not the name of Software Bugs ----------**
► Bugs
► Defects
► Errors
► **Mistakes (Page 213)**

**Question No: 22 ( Marks: 1 ) - Please choose one**
**According to a survey, when a software application is in the maintenance phase, --------- of its lifecycle cost is attributed towards the defects which are found in the software application after installation**

► **20 % (Page 214)**
► 25 %
► 30%
► 35%

**Question No: 23 ( Marks: 1 ) - Please choose one**
**Bugs that won't "----------" (almost random) are the most difficult to deal with.**

► **stand still (Page 226)**
► dynamic
► running
► stand tall

**Question No: 24 ( Marks: 1 ) - Please choose one**
**1. Variables should be initialized where they are declared and they should be declared in the ---------- scope possible.**

► **Smallest (Page 158)**
► largest
► medium
► None of the given

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 25 ( Marks: 1 ) - Please choose one**
**If you are initializing a variable at the time of declaration, do not declare another ---------- in the same statement.**

- ► **variable (Page 176)**
- ► object
- ► class
- ► None of given

**Question No: 26 ( Marks: 1 ) - Please choose one**
**During Requirement Engineering process which of the following is/are document(s) used for fact finding?**

- ► Company's employee list
- ► Samples of the company databases
- ► The company's mission statement and plan
- ► **All of given**

**FINALTERM EXAMINATION**
**Fall 2008**

**Question No: 1 ( Marks: 1 ) - Please choose one**
**Object-oriented domain analysis is concerned with the identification and specification of reusable capabilities within an application domain.**

- ➢ **True** **Click here for detail**
- ➢ False

**Question No: 2 ( Marks: 1 ) - Please choose one**
**Fourth generation techniques**

- ➢ Allow software to be developed without any testing.
- ➢ Eliminate the need for costly requirements gathering activities.
- ➢ **Can reduce the time required to develop software.**

**(A PRACTITIONER'S APPROACH Page 73)**

- ➢ Are best used by non-programmers to build small systems.

**Question No: 3 ( Marks: 1 ) - Please choose one**
**Data design actually begins during the creation of the analysis model, not the architectural model.**

- ➢ **True** **(A PRACTITIONER'S APPROACH Page 397)**
- ➢ False

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 4 ( Marks: 1 ) - Please choose one**
**The states shown in a state transition diagram do not necessarily correspond to the processes shown in a control flow diagram for the same system.**

- **True (A PRACTITIONER'S APPROACH Page 397)**
- False

**Question No: 5 ( Marks: 1 ) - Please choose one**
**The criteria used to assess the quality of an architectural design should be based on system**

- **accessibility and reliability (A PRACTITIONER'S APPROACH Page 404)**
- data and control
- functionality
- implementation details

**Question No: 6 ( Marks: 1 ) - Please choose one**
**A useful technique for evaluating the overall complexity of a proposed architecture is to look at the component**

- number and size of components
- **flow dependencies and sharing dependencies (A PRACTITIONER'S APPROACH Page 406)**
- size and cost
- none of the given

**Question No: 7 ( Marks: 1 ) - Please choose one**
**The three basic principles that guide maintainability are: simplicity, clarity, and_____ .**

- **Generality (Page 146)**
- Reliability
- All of the given choices
- None of the Given choices

**Question No: 8 ( Marks: 1 ) - Please choose one**
**In order to make a program self documented a number of attributes required. Which one is the attribute/s of self documented program**

- **All of the given choices (Page 147)**
- Size of each function
- Choice of variable
- Modularity

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 9 ( Marks: 1 ) - Please choose one**
**Floating point constants should always be written with decimal point and at least**

- **one decimal (Page 161)**
- two decimal
- three decimal
- none of the given

**Question No: 10 ( Marks: 1 ) - Please choose one**
**The code becomes self explanatory with the help of proper use of parentheses. Select the right one.**

- leapYear = year % 4 == 0 && year % 100 != 0 || year % 400 == 0 ;
- leapYear = ((year % 4 == 0) && (year % 100 != 0) || ((year % 400 == 0));
- **leapYear = ((year % 4 == 0) && (year % 100 != 0)) || (year % 400 == 0); (Page 164)**
- leapYear = (year % 4 == 0) && (year % 100 != 0) || (year % 400 == 0);
- none of the given options

**Question No: 11 ( Marks: 1 ) - Please choose one**
**Graph-based testing methods can only be used for object-oriented systems**

- True
- **False** **Click here for detail**

**Question No: 12 ( Marks: 1 ) - Please choose one**
**Configuration reviews are not needed if regression testing has been rigorously applied during software integration.**

- True
- **False** **Click he re for Detail**

**Question No: 13 ( Marks: 1 ) - Please choose one**
**A class is a ---------- of objects**

- **Template (Page 85)**
- Stereotype
- Collection
- Non of the above

**Question No: 14 ( Marks: 1 ) - Please choose one**
**A public Interface provides a way for with other Classes.**

- **Communication**
- Accessibility
- Reaching
- All of the above

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 15 ( Marks: 1 ) - Please choose one**
**Flow Charts represents**

- **sequence Activity (Page 50)**
- random activity
- parallel activity
- non of the above

**Question No: 16 ( Marks: 1 ) - Please choose one**
**Software architecture defines the high level structure of the software by putting together a number of architectural ---------- in an organized fashion.**

- **Elements (Page 122)**
- parts
- components
- non of the all

**Question No: 17 ( Marks: 1 ) - Please choose one**
**Patterns are devices that allow programs to share knowledge about their ----------.**

- **Design (Page 137)**
- Code
- Analysis
- Non of the all

**Question No: 18 ( Marks: 1 ) - Please choose one**
**MVC stands for ----------**

- **Model View Controller (Page 140)**
- Modern View Center
- Model View Center
- Modern View Controller

**Question No: 19 ( Marks: 1 ) - Please choose one**
**A ---------- is a code that explains itself without the need of comments and extraneous documentation**

- **Self documenting code (Page 147)**
- Self telling Code
- Self Documenting Design
- Non of the them

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 20 ( Marks: 1 ) - Please choose one**
**Type conversions must always be done ----------**

- **Explicitly (Page 158)**
- implicitly
- simultaneously
- non of them

**Question No: 21 ( Marks: 1 ) - Please choose one**
**Types that are ----------to one file only can be declared inside that file.**

- **Local (Page 158)**
- Global
- Private
- General

**Question No: 22 ( Marks: 1 ) - Please choose one**
**The use of do while loops should be ----------**

- **Avoided (Page 159)**
- encouraged
- practiced
- non of them

**Question No: 23 ( Marks: 1 ) - Please choose one**
**One of the causes of the portability issues is the order of ---------- varies from one implementation to other.**

- **Evaluation (Page 181)**
- numbers
- variables
- symbols

**Question No: 24 ( Marks: 1 ) - Please choose one**
**---------- is a powerful technique that separates error-handling code from normal code.**

- **Exception handling (Page 184)**
- Code handling
- variable handling
- pointer handling

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 25 ( Marks: 1 ) - Please choose one**
**The goal of testing is to expose ---------- defects in a software system before it is put to use.**

- **Latent (Page 193)**
- already present
- current
- runtime

**Question No: 26 ( Marks: 1 ) - Please choose one**
**Static analyzers are software tools for ---------- processing.**

- analysis text
- **source text (Page 211)**
- design text
- non of the given

**Question No: 27 ( Marks: 1 ) - Please choose one**
**The first bug was actually a moth, which flew through an open window and into one of the Mark --------- 's relays.**

- **II (Page 213)**
- I
- III
- IV

**Question No: 28 ( Marks: 1 ) - Please choose one**
**Which one of the given below is not a symptom of memory overrun?**

- Program crashes quite regularly after a given routine is called, that routine should be examined for a possible overrun condition.
- If the routine in question does not appear to have any such problem the most likely cause is that another routine, called in the prior sequence, has already trashed variables or memory blocks.
- Checking the trace log of the called routines leading up to one with the problem will often show up the error.
- **Compiler warnings. (Page 220)**

**Question No: 29 ( Marks: 1 ) - Please choose one**
**Holistic medicine, concerns itself with the state of the body as a whole, not the ---------- that is currently attacking it.**

- reason
- **disease (Page 224)**
- source
- non of the given

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 30 ( Marks: 1 ) - Please choose one**
**Bugs that won't "stand still" (almost random) are the ---------- to deal with.**

- least difficult
- **most difficult (Page 226)**
- very easy
- easy

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

# CS504 Short Notes
# Lecture 22

### What is Software architecture?

It provides an over all view of a high level system.

In it, we define how a system is decomposed into sub systems, components modules and relationship among those components and subsystems etc.

## What is the importance of software architecture?

It serves as a mean of communication between the stack holders.

Helps in making early design decisions.

Reusability is achieved.

Attributes of software architecture address both functional and non functional requirements of a software system. These attributes are as under:

Performance

by localizing the operations to minimize the communication between subsystems.

Security

by adopting a layered architecture.

Safety
by isolating a specific/safety critical components.

Availability
by building redundancy i.e. by providing suitable alternatives in case of an expected or unexpected system failure.

Maintainability

by using self contained components i.e. change in a component should not bring a change in another component.

vustudents.pk

# Lecture 23

A software Architecture usually consists of the following elements which are presented in a model below :
Logical view/Functionl view
comprises of various different functions provided by the system abstraction and the domain elements.

Development View

Comprises of the different files and directories in the system.

Code view
Comprises of classes, objects, procedures function, subsystems, layers and modules.

Concurrency View

Documents different parallel processes and threads in the system having its main focus on even synchronization and parallel data flow.

Physical view

Depicts how a system would physically be deployed.

Architectural styles may describe

i) A set of components that perform a function required by a system.
ii) A set of connectors that enable communication among components
iii) Integrating components from the system and semantic models that enable a designer to understand the over properties of a system.

## what is control modelling?

Architectural design process consists of three activities; System structure, modular decomposition and control modeling. These three activities are performed to design architecture of a system.
In system structuring, we divide system into major blocks (sub systems). In modular decomposition, these sub-systems or blocks are further decomposed into modules. These modules should be connected in term of control flow to make system executable. This is done through control modeling. In control modeling, we model flow of control among different modules of the system to enable different sub-systems to communicate and for working as a single system.

In client server model why server does not need to know about clients? please explain
Normally the total no. of servers in a organization remain fix whereas the clients go on increasing and decreasing. If we implement the system in a way, where server doesn't know about the clients, then this gives us the flexibility to add/delete clients easily. In this case adding a client mean just connect the client with the server and start using the services.
Also in most of the cases server don't need to know about the clients, so keeping record of clients may be burden especially in case when the clients increase significantly.



vustudents.pk

# Lecture 24

Architectural Styles:

Data centered
shared data is held in a centeral data base or each subsystem maintains its own database.

Client server model

a distributed system model which describes how data and processing is distributed across a range of components. Application is modeled as set of services provided by the servers and a set of clients that use these services. Clients know the servers but the servers do not need to know all the clients. Mapping between processors is also not always 1:1.

Server types, File servers, Data base servers, Transaction servers, Groupware servers.

Server Configuration:

Thing client Model
Client machine has no functionality on it. It just provides the user interface.

Fat client Model

more processing is delegated to the client as the application processing is locally extended.
concept of architectural views, specialy logical view please sir elaborate it in simple words and it is also request that define it with the help of any real life example.???

Please note that an architectural views represent the viewpoints of different stakeholders. That is a view presents system from one angel, other view presents system from another angel and so on. eg, just as a building architect might create wiring diagrams, floor plans, and elevations to describe different facets of a building to its different stakeholders (electricians, owners, planning officials), so an IT architect might create logical view, implementation view and so on.

In logical view, architect presents the system from end user point of view that what functionality and services the system provides. It describes how the system is structured in terms of units of implementation. Normally this view includes class diagram, communication diagram, sequence diagrams etc.
An Architecture style is a set of principles which you can use to build a system, normally these principals are re-used in applications, e.g. client/server architectural style, component based, layered etc etc. each of which provides a way to implement software structure.

Non functional requirements are qualities and characteristics of a system. They define the criteria to judge the operation of the system. e.g. system must be maintainable, system must be secure, system must be user friendly etc.

vustudents.pk

# Lecture 25

Zero Install

Where there is no or little processing done at the client side. It differes from Thin client model in a way that in case of thin client model, the entire processing is done by a single server, whearas in case of zero install, the network envoirnment is used to distribute server side processing by adding a number of servers which share processing load.

N-Tier

Here, the idea is to enhance the scalability and performance by distributing both the data and the application using multiple server machines.

3-Tier

each application architencture layers may run on separate processors. These layers include presentation, application and data base layers. In case of N-tire architecture, the system may have more than 3 layers.

Data Flow/Pipe architecture

Used when the input data is processed through aseries of transformations to yield the desired output. Each filter works independently others.

Layard Architecture

Each layer isolates the outer layer from the inner complexities. The outer layer only needs to know the interface provided by the inner layer.

Reference Architecture

It is only a reference for defining protocols and designing and implementing systems developed by different parties.

Partitioning the Architencture

To distribute the responsibilities to different subsystems so that we get a software system which is easy to maintain. Partitioning may be horizontal or vertical.

Horizontal

separate branches of the module hierarchy for each major function and control modules are used to co-ordinate communication between functions.

Vertical

The architecture is partitioned in horizontal layers having decision making modules residing at the top and worker at the bottom.



vustudents.pk

The standard and mostly recognized pattern to be followed for SRS, is that specified by IEEE. So why should anyone follow some other pattern dictated by some other GANG...GOF? Yes. Standard is defined by IEEE. where GOF is collection of design pattrens which can be followed. But these too, does not restrict the design pattren choice. Choice of design pattren is dependent upon requirements of a system. Even if IEEE standard is not followed, it does not make harm. Design pattren can be chosen from GOF and even design pattren can be hybrid or customized depending upon need.

### What is meant by Frame Work? and what is an integrated Frame Work? is it same as famous Microsoft .Net Frame Work? how do design patterns apply to frame work?

Framework is a a real or conceptual structure intended to serve as a support or guide for the building of something that expands the structure into something useful. For example, .NET framework is a framework used to develop, deploy and to run applications by using provided services, libraries and modules. Design pattrens represent solutions to problems that arise when developing software within a particular context. Patterns can be characterized as more abstract descriptions of frameworks, which are implemented in a particular language.

### importance of documatation?? also good documentations techniques.?????

Documentation is very important. Work done at each phase is required in the next phase. For example, requirements are mapped in design so work done at requirement phase is required in design phase. Design is required to convert into implementation. Best way to convey previous work done is through documentation. So at one hand, documentation is required for well managed development. On the other, it is required for later maintainenace. When system is in use, some problem occurs, maintenance team requires documentation to identify exact place of problem as well as for measuring the impact of change on other parts.
This is the reason that why documentation is needed to be focused. There is no specific technique for documentation. Documentation is done in parallel with development. At every stage, everything is documented. For example, in requirement phase, SRS and FRS are developed. In design phase, design document is developed containing every model. In testing phase, test-cases are written. Document of a project is collection of all these documents. They are merged into a single document to provide complete information about system.

## Lecture 26

Design Patterns

are the devices that allow programs to share knowledge about their design, such knowledge/ solutions can be used repeatedly over time.

GOF (gang of four) Design pattern format includes :

Name, problem, context, forces, solution , result, rationality , related patterns and known uses of the format being applied .

Patterns can be classified as under:

According to their creation



vustudents.pk

here, we look into how the patterns are created.

According to structure here we deal with the object's nature.

According to behavior here, we use inheritance to distribute behavior between classes.

Framework:

A framework simply dictates the architecture of the software. In other words, framework consists of re-useable patterns.

Best Regards from pp !

Can we say that the OSI reference model represents horizontal partitioning of a system?

OSI reference model is a standard reference model for communication between two end users in a network. Layers of OSI model works on different layers of network and enables user to communicate. Where horizontal partitioning is a way to organize modules based on functionality. Such partitioning eases in later extension with least effect on other modules. So we can not say that OSI model represents horizontal partitioning because horizontal partitioning is work being done at application layer where OSI model is meant for working on all layers of a network.

Let's suppose there is a system which comprises of following servers connected via network:-

1. Web Server
2. Email Server
3. Security Server
4. Database Server

A client which wants to access the web-server, can directly access the web-server by http protocol....

However, Will the client also access email server directly or will it be indirectly through web-sever? Similarly what will be the case for database server?

Database server, web server, email sever etc are always remotely located. When we talk about network that consists of a number of clients. There is always a main server (gate-way server). Every request of client is directed towards that server. Server will direct each client accordingly whether clients wants to connect with email server, web server or database server. Gate-way server is a sort of traffic controller in network. An other reason of directing data through server is locational transperacy and security servers as well as of network.

## Lecture 27

Observer pattern / Creational :

It basically relates to a many to many relationship between objects so that when on object changes state,all of its dependants are notified and updated accordingly.

It is used when multiple displays of state are needed or in other words, when data is to be presented in more than one way.



vustudents.pk

Singleton pattern / Behavioural :

It basically makes sure that a class only has one/few instance/s of a class and it must be accessible to clients from a well know access point.

Façade pattern / Structural :

It simply provides a unified interface/door-way to a set of interfaces in a subsystem. It defines a higher level interface that makes a subsystem easier to use.

Regards from Persian !

Software architecture just represent non-functional and functional requirements? Is it represent Business requirement and user requirements ? Kindly explain that which thing comes under software architecture...???

Software architecture defines the high level structure of the software by putting together a number of architectural elements in an organized fashion. These are: data elements, processing elements, and connecting elements. These elements are chosen to satisfy the functional as well as non-functional requirements of the system.
Software Architecture does not represent non-functional and functional requirements but a way to achieve/satisfy these requirements. A software system is built by focusing on all requirements. When we say non-functional and functional; we are covering every possible requirement of a system. If a system is performing all required functionalities, that means we have achieved business requirement and we have also achieved user requirements as functionalities are for users.
Software architecture covers all design and architectural decisions which are related to data, processes and connecting elements.
How can we differentiate main frame based application, thin client model and zero install? As all three have many similarities.??

As you have studied in your handouts that in any system, sub-systems may need to exchange information and data.

There are two ways in which it can be done:

1. Keeping the data into a central database or repository and may be accessed by all subsystems
2. Each sub-system maintains its own database and passes data explicitly to other subsystems

For large main frame based applications, we use the first approach for sharing the data among different subsystems. The Data-Centered architecture have the goal of achieving the quality of integrability of data. The term refers to systems in which the access and update of a widely accessed data store is their primary goal. Basically, it is nothing more than a centralized data store that communicates with a number of clients. There is a software installed on each of the clients connected in the system and using this installed software, the client may send, share or



vustudents.pk

retrieve the data with any other client in the system.

In case of Zero-Install, nothing needs to be installed or put into client. This makes system maintenance an easy task. Whole processing will be carried out by the server which may affect the performance of the system in case of a single server. To overcome this problem, different processing responsibilities are distributed among number of servers. The major difference between Data Repository Model and Zero-Install is the availability of number of servers required for the processing. Data Repository is comprised of such a single data server where as in case of Zero-Install; different processing responsibilities of the system are distributed among number of different servers such as database server, application server, web server etc.

What is difference between terminals and computers? Kindly explain the differences.?? Terminal is client computer. Consider a network in which there is a sever and 100 client computers. These client computers are terminals. When you are working on your computer at home, that is not terminal because it is not connected to any network. But when you are working in VU lab, your computer is terminal of network. So in client-server model based network, every client is a terminal node of network.

is uml include out documentations??

Unified Modelling Language (UML) is a design methodology that aids us in designing the software. It is part of design documentation which is ultimtely added in final documentation. UML is a part of documentation.

## what is main work of facade ?

Answer :
The facade pattern is a software design pattern commonly used with object-oriented programming. A facade is used when one wants an easier or simpler interface to work with.
A facade can:
1) Make a software library easier to use, understand and test, since the facade has convenient methods for common tasks;
2) Make the library more readable, for the same reason;
3) Reduce dependencies of outside code on the inner workings of a library, since most code uses the facade, thus allowing more flexibility in developing the system;
4) wrap a poorly-designed collection of APIs with a single well-designed API.

# Lecture 28

Maintainability of a code means that the code should be clear, simple and flexible.

Self documenting the code means that the code should explain itself without the need of comments and extraneous documentation.

Naming conventions are 12 whereas specific naming conventions are about13 which you can



vustudents.pk

read and get idea about in the handouts.

Summing it up, the code should be user friendly. And this can be achieved only through adopting simplicity, clearness, flexibility and conventions.

Regards from Persian !

explain the N-Tier architecture ?

Answer : In n-tier architecture, in order to increase performance, we may distribute the application over different servers by putting different subsystems on different servers. In n-tier architecture, the presentation, the application processing, and the data management are logically separate processes. It is also possible that different components of presentation or database may be put on separate servers. For example, in case of three-tier architecture, there are normally two servers that is, the database server and application server but in case of N-tier architecture, each responsibility of the system such as database or logic may be put on more than one servers. These kinds of architectures are used in the situation in large system where the main concern is to maintain the system efficiency.

**how we can differentiate with thin, flat, zero model ?**

Answer : In case of Thin Client Model, GUI are implemented on client side, whereas all the system processing is done by Server. It means that all the powers are assigned to server and in this case, the client only sends a request and server acknowledges it after doing some processing. Thin-client computing is also a way of easily maintaining computational services at a reduced total cost of ownership. Thin-client computing is also a way of easily maintaining computational services at a reduced total cost of ownership. Virtual Network Computing (VNC) is a graphical desktop sharing system which uses thin-client model.

In Fat Client Model, some of the processing responsibilities are also assignment to Client which reduces the burden over the server. For example, in this model, the application is installed on client side, making a distributed processing model where responsibilities are shared among client and server components of the system. But there is some disadvantage associated with this model. That is, if there is any need for application modification or updating, it is needed to be done on all the clients in the system which increases the maintenance effort. This client models are suitable for multiplayer video gaming and many other things. Most PCs (personal computers), for example, are also fat clients because they have their own hard drive DVD drives, software applications and so on.

Due to the maintenance overhead associated with Fat Client Model, it is difficult to implement such model and hence the solution it to adopt Zero Install Model. As oppose to Fat Client, nothing is installed on client side. Web-based application where web-pages are put on a web-server is an example of this type of architecture.

## Lecture 29



vustudents.pk

For More Visit VUStudents.pk

File handling tips for Java and C++

1. C++ header files should have the extension *.h*. Source files can have the extension *.c++* (recommended), *.C*, *.cc* or *.cpp*.
MyClass.c++, MyClass.h
2. Classes should be declared in individual header files with the file name matching the class name. Secondary private classes can be declared as inner classes and reside in the file of the class they belong to. All definitions should reside in source files.
3. Special characters like TAB and page break must be avoided.

4. The incompleteness of split lines must be made obvious.

Include Files and Include Statements for Java and C++

1. Header files must include a construction that prevents multiple inclusion.

Statements in Java and C++

1. Type conversions must always be done explicitly.

   floatValue = (float) intValue; // NOT: floatValue = intValue;

2. Types that are local to one file only can be declared inside that file.

3. The parts of a class must be sorted *public*, *protected* and *private*. All sections must be identified explicitly.

There are many more tips like that in the handouts so I suggest you should have a look at the handouts as well.

Regards from Persian !

what do you ment by application server.

An application server is a server that provides software applications with services such as security, data services, transaction support, load balancing, and management of large distributed systems.

## what do you mean by web server ?

Web server can refer to either the hardware (the computer) or the software (the computer application) that helps to deliver Web content that can be accessed through the Internet.

vustudents.pk

For More Visit VUStudents.pk

Comments : In general, the use of comments should be minimized by making the code self-documenting by appropriate name choices and an explicit logical structure.
Expressions and Statements

Basic indentation should be 2.
for (i = 0; i < nElements; i++)
a[i] = 0;
Lec 11 :
Expression should be written as if they are written as comments or spoken out aloud.

Parentheses should always be used as they reduce complexity and clarify things by specifying grouping.

Complex expressions should be broken down into multiple statements. An expression is considered to be complex if it uses many operators in a single statement.
Sometimes the programmers, in their creative excitement, try to write very concise code by using shortcuts and playing certain kinds of tricks. This results in a code which is cryptic in nature and hence is difficult to follow.

In Lec-19 we studied about how to make structure of a system. My question is what is the purpose of defining inheritance relationship? I mean in example given Cash Payment, Cheque and Charge are child classess of Payment. Why I wrote Payment class in the first place, in system I will be using objects of either Charge, Cash payment or Cheque; then why we exclusively wrote Payment class first and then inherits from it; although we are not using its object in the system. Isn't it just increasing the code writing process; kindly explain?
Ans:
Purpose of inheritance is enhancing code reusability. Payment class is made parent class and cheque, charges and cash payment are child classes in given example. We have done it for reuse purpose. Later when you need any of these functionalities, the method of class Payment will be inherited by sub classes allowing us not to repeat those methods in sub classes again. In this way, it is not increasing code writing rather it is promoting code reuse and hence it will save our time.

**what parameters are used to measure and analyze design quality?**
Ans
here are four parameters used to analyze the quality of a software design. These parameters include efficiency, compactness, reusability, and maintainability.

In the switch statement, cases should always end with a break. An if statement is a useful alternative of switch statement.

Magic Numbers

These are constant that mean something but they do not give any indication of their importance or derivation, making the program hard to understand and modify. To a reader they work like magic and hence are called magic numbers.

It is much better to use symbols to explicitly indicate the intent of the statement.



vustudents.pk

Code example of High Coupling
Ans
Consider a simple shopping cart application that uses a CartContents class to keep track of the items in the shopping cart and an Order class for processing a purchase. The Order needs to determine the total value of the contents in the cart, it might do that like so:



```
public class CartEntry
{
   public float Price;
   public int Quantity;
}

public class CartContents
{
   public CartEntry[] items;
}

public class Order
{
   private CartContents cart;
   private float salesTax;

   public Order(CartContents cart, float salesTax)
   {
      this.cart = cart;
      this.salesTax = salesTax;
   }
```

vustudents.pk

```
    public float OrderTotal()
    {
        float cartTotal = 0;
        for (int i = 0; i < cart.items.Length; i++)
        {
            cartTotal += cart.items[i].Price * cart.items[i].Quantity;
        }
        cartTotal += cartTotal*salesTax;
        return cartTotal;
    }
}
```

Notice how the OrderTotal method (and thus the Order class) depends on the implementation details of the CartContents and the CartEntry classes. If we were to try to change this logic to allow for discounts, we'd likely have to change all 3 classes. Also, if we change to using a List collection to keep track of the items we'd have to change the Order class as well. So the above examples represents the classis example of tight coupling.

Lec 32:

Abstraction and encapsulation are two important tools that can help in managing and mastering the complexity of a program.

The logical and operator, &&, and logical or operators, ||, are special due to the C/C++ short circuiting rule, i.e. a || b and a && b are short circuit evaluated. That is, logical expressions are evaluated left to right and evaluation stops as soon as the final truth value can be determined.

```
struct Node {
int data;
Node * next;
};
Node *ptr;
...
while (ptr->data < myData && ptr != NULL){
// do something here
}
```

What's wrong with this code?

ptr != NULL, is supposed to be the guard. That is, if the value of the pointer is NULL, then the control should not enter the body of the while loop otherwise, it should check whether ptr->data < myData or not and then proceed accordingly.

This would be the right choice !

```
while (ptr != NULL && ptr->data < myData){
// do something here
}
```

A side effect of a function occurs when the function, besides returning a value, changes either one of its parameters or a variable declared outside the function but is accessible to it.

As an example,
consider the following statement:

```
c = f1(a) + f2(b);
```

The question is, which function (f1 or f2) will be evaluated first as the C/C++ language does not specify the evaluation order and the implementer (compiler writer) is free to choose one order or the other. The question is: does it matter?
To understand this, let's look at the definition of f1 and f2.

```
int f1(int &x)
{
x = x * 2;
return x + 1;
}
int f2(int  &y)
{
y = y / 2;
return x - 1;
}
```

In this case both f1 and f2 have side effects as they both are doing two things – changing the value of the parameter and changing the value at the caller side. Now if we have the following code segment,

```
a = 3;
b = 4;
c = f1(a) + f2(b);
```

then the value of a, b, and c would be as follows:

```
a = 6
b = 2
c = 8
```

So far there seem to be any problem. But let us now consider the following statement:

```
c = f1(a) + f2(a);
```

What will be the value of a and c after this statement?
If f1 is evaluated before f2 then we have the following values:

```
a = 3
b = 9 // 7 + 2
```

On the other hand, if f2 is evaluated before f2 then, we get totally different results.
a = 2
b = 3 // 3 + 0

how the objects are identified in peter codd's technique?
Ans
Peter Coad methodology divides the objects into different categories to make it easier to find them and establish their attributes, services, and collaborations. This activity, consisting of 6 steps is well explained in lecture 16. I recommend you to watch video lecture 16 as well as conslut same lecture from handouts for more details.

How can we decrease coupling?
Ans
Software coupling is defined as the degree to which a software module relies or depends on other modules. High coupling means that related classes have to know internal details of each other, changes ripple through the system, and the system is potentially harder to understand. Coupling can be reduced if we focus the following steps:
1. Make the code easier to read.
2. Make classes easier to consume by other developers by hiding the inner workings of classes behind well-designed APIs.
3. Isolate potential changes to a small area of code.
4. Reuse classes in completely new contexts.
This query has aklready been replied in lesson 14 mdb. I recommend you to consult mdb of every lecture. It will not only enhance your knowledge but also restricts you to ask repeated questions.

Lec 33:

Performance :

In many cases, performance and maintainability are at odds with one another. When planning for performance, one should always remember the 80/20 rule - you spend 80 percent of your time in 20 percent of the code. That is, we should not try to optimize everything.

When a small set (a couple of functions) of functions which use each other is so overwhelmingly the bottleneck, there are two alternatives:

1. use a better algorithm
2. rewrite the whole set

```
for (j = i; j < MAX_FIELD; j++)
clear(j);
```

This loop clears field before each new input is read. It was observed that it was taking almost 50% of the total time.

code was subsequently modified as shown below:

```
for (j = i; j < maxField; j++)
clear(j);
```

This reduced the overall execution time by half.

IS it necessary to show the connection between both database servers or both database servers are independent. Thanks
Ans
Its up to you. You can show connection between both data servers. Even if you don't show connect between them. There is no issue.

DIFFRENT PHARASE OF TESTING AND DIAGRAM?
Ans
Here ar different phases.
Unit testing : For individual components independent of other components.
Module testing : It is for testing a collection of dependent components
Subsystem testing: testing of collection of modules to discover interfacing problems among interacting modules.
System testing: It is for after integrating subsystems into a system
Acceptance test: validation against user expectations.
Alpha testing: testing for customized projects, in-house testing for products.
Beta testing: field testing of product with potential customers who agree to use it and report problem before system is released for general use

Lec 34 :

If we are careful during writing code, we can make it portable. On the other hand if we write code without portability in mind, we may end-up with a code that is extremely hard to port to other environment.
Guidelines to achieve Portability :

1. Use ANSI/ISO standard C++

2. Instead of using vendor specific language extensions, use STL as much as possible Sizes of data types cause major portability issues as they vary from one machine to the other so one should be careful with them.

Order of evaluation varies from one implementation to other.
The language does not specify whether char is signed or unsigned.

```
char c;
// between 0 and 255 if unsigned
// -128 to 127 if signed
c = getchar();
if (c == EOF) ??
// will fail if it is unsigned
```

It should therefore be written as follows:

```
int c;
c = getchar();
if (c == EOF)
```

Bit Fields :
Bit fields allow the packing of data in a structure. This is especially useful when memory or data storage is at a premium.

However, bit fields do suffer from a lack of portability between platforms:
 integers may be signed or unsigned
 Many compilers limit the maximum number of bits in the bit field to the size of an integer which may be either 16-bit or 32-bit varieties.
 Some bit field members are stored left to right others are stored right to left in memory.
 If bit fields too large, next bit field may be stored consecutively in memory (overlapping the boundary between memory locations) or in the next word of memory. Bit fields therefore should not be used.

WHAT ARE STATIC ANALYSIS AND HOW WE CAN CHEACK THE CHECK LIST?

MULTIPLE EXPRESSSIONS EXPLAIN?

Ans

Analysis of computer software performed without actually executing, or running, that software. Looking at applications in a non-runtime environment.

CHECK LIST:
Data faults, Control faults,Input/Output faults, Storage Management fault are cheked in static analyzer application.

DIFFRENCE BETWEEN ARCHITUCTURE AND SYSTEM'S?
Ans
Modularity is a tool that can help us in reducing the size of individual functions, making them more readable.
It increases the extent to which software is composed of separate, interchangeable components called modules by breaking down program functions into modules, each of which accomplishes one function and contains everything necessary to accomplish this.

The architecture of a system is its 'skeleton'. It's the highest level of abstraction of a system. What kind of data storage is present, how do modules interact with eachother, what recovery systems are in place.
While
System architectures hold the key to success or failure of a system every bit as much as the software architecture does for the software.

Lec 35:

Exception handling is a powerful technique that separates error-handling code from normal code. It also provides a consistent error handling mechanism.

One of the most powerful features of exception handling is that an error can be thrown over function boundaries. This allows programmers to put the error handling code in one place, such as the *main*-function of your program.

general exception handling mechanism :
try {
___...
___...
___throw Exception()
___...
___...

```
} catch( Exception e )
{
___...
___...
}
```

A number of invisible execution paths can exist in simple code in a language that allows exceptions. The complexity of a program may increase significantly if there are exceptional paths in it.

Providing the strong exception-safety guarantee often requires you to trade-off performance.
If a function has multiple un-related side-effects, it cannot always be made strongly exception safe.
Not all functions need to be strongly exception-safe.

1. LL ME GENRAL NAMING CONVENTION FOR C++ AND JAVA?

2. WHAT ARE CODE STRUCTURES?

Ans

Code structures:A program is usually not limited to a linear sequence of instructions. During its process it may bifurcate, repeat code or take decisions. For that purpose, C++ provides control structures that serve to specify what has to be done by our program, when and under which circumstances.For structural testing it is important to know about basic coding structures. There are four basic coding structures sequence, if statement, case statement, and while loop. These four basic structures can be used to express any type of code.
Plz consult handouts for naming conventions as these are well explained there.

1. WHILE LOOP SHOULD BE AVOIDED?

2. VE EQUIVALENCE PARTITIONING GUIDELINES?

3. WHAT ARE SYMPTOMS AND GIVE AN EXAMPLE OF MEMORY OVERRUN BUG CLASS?

Ans

1-do-while loops are less readable than ordinary while loops and for loops since the conditional is at the bottom of the loop. The reader must scan the entire loop in order to understand the scope of the loop. In addition, do-while loops are not needed. Any do-while loop can easily be rewritten into a while loop or a for loop. Reducing the number of constructs used enhance readbility.

2-Equivalence partitioning guidelines:
I-Organize your equivalence classes. Write them in some order, use some template, sequence, or group them based on their similarities or distinctions. These partitions can be hierarchical or organized in any other manner.
II_Boundary conditions: determine boundary conditions. For example, adding in an empty linked list, adding after the last element, adding before the first element, etc.
III- You should not forget invalid inputs that a user can give to a system. For example, widgets on a GUI, numeric instead of alphabets, etc.

3-Memory overrun occurs when you use memory that does not belong to you. This can be caused by overstepping an array boundary or by copying a string that is too big for the block of memory it is defined to hold. Memory overruns were once extremely common in the programming world because of the inability to tell what the actual size of something really was.

Lec 36 : Software Testing -> Software verification and validation :

processes in which we check a product against its specifications and the expectations of the users who will be using it.
Defect : A defect is a variance from a desired product attribute. These attributes may involve system specifications well as user expectation.
Software and Defect : Software and defects go side by side in the software development life cycle.
Software Testing : the process of examining the software product against its requirements. Thus it is a process that involves verification of product with respect to its written requirements and conformance of requirements with user needs.
Software testing objective : The correct approach to testing a scientific theory is not to try to verify it, but to seek to refute the theory. That is to prove that it has errors.
Testing limitations :

example in which you prove that the formula or theorem is not working.
 On the other hand, million of examples can be developed to support the hypothesis but this will not prove that it is correct.
Test Cases and Test Data :
Thus a test case involves
 Input and output specification plus a statement of the function under test.
 Steps to perform the function
 Expected results that the software application produces

1,UNIT TESTING , SOFTWARE TESTING?

2,WHERE TERM COMPUTE CAN BE USED IN METHODS?

3, NON FUNCTIONAL REQUIREMENTS OF AN E-LEARNINGSYSTEM?

(A)USER FRIENDLESS,(B)TIME TAKEN TO DOWNLOAD (C)ONLINE ASSIGNMENT SUBMISSION FACILITY?

(DLINE CHATTING FACILITY?(E)REBUSTESS

Ans

1) Unit testing means, testing a single module. Normally this testing is performed by developer itself.

2) Please refer lecture no. 28 for this question.

3) There may be different non functional requirements e.g. availability, usability, security etc.

Please explain the Pattern and where and how we use it in software ? with examples regards Aorangzeb

Ans

In software development, there are certain problems (patterns) that are repeated again and again. The generic solution to these problems are called design pattern. E.g. for all the dynamic websites, the same style is followed i.e. there is a central server, which stores all the data, and clients access that data through browser. All the websites follow the same rule. So, we can say that the style (client/server) is a design pattern, as it is a common solution to a common occurring problem.

PLEASE TELL THAT SHOULD WE REDESIGN THE MODEL BY USING THE Façade Pattern OR IMPLEMENT IT IN THE C++….
Thanks.
Ans
You only have to re-design the model by implementing facade pattern. No need to provide any code.

Could you please explain little bit the main difference between Singleton Pattern & Observer Pattern.
Ans
In observer pattern, all the objects (views) continually synchronize with the central data object, i.e. whenever the data is modified all the views are also modified immediately (views in this way, always represent the latest data).

On the other hand the nature of singleton pattern is quite different, its implementation allows the instantiation of only a single object of any class on which singleton pattern is applied. I.e. you can not create more than one objects of this class.

Lec 37 : Testing vs. development :
 Functional specification document is the starting point, base document for both testing and the development
 Right side boxes describe the development, whereas, left side boxes explain the testing process
 Development team is involved into the analysis, design and coding activities.
 Whereas, testing team too is busy in analysis of requirements, for test planning,

test cases and test data generation.

System comes into testing after development is completed.

Test cases are executed with test data and actual results (application behavior) are compared with the expected results,

 Upon discovering defects, tester generates the bug report and sends it to the development team for fixing.

 Development team runs the scenario as described in the bug report and try to reproduce the defect.

 If the defect is reproduced in the development environment, the development team identifies the root cause, fixes it and sends the patch to the testing team along with a bug resolution report.

 Testing team incorporates the fix (checking in), runs the same test case/scenario again and verifies the fix.

 If problem does not appear again testing team closes down the defect, otherwise, it is reported again.

testing phases : Unit testing , Module testing , Subsystem testing, System testing, Acceptance test, Alpha testing , Beta testing

Black box testing : not concerned with how the inputs are transformed into outputs. If the outputs match with the expected results, system is fine otherwise a defect is found.

Structural testing (white box) : we look inside the system and evaluate what it consists of and how is it implemented.

plz tell me about asynchronous error and synchronous error?

and my 2nd question is what is throwing error?

Ans

There are some statements in program that always throw error i.e. the error is raised in these statements at every execution of program, e.g the statement.

int x = y / 0 ;

will always generate an exception. Such errors are called synchronous errors.

On the other hand, some errors may occur at anytime, May occur in one execution but not in other, e.g. at any stage the program could not get the sufficient memory (may be due to number of other programs are executing). Such errors that are not specific to a statement but can occur at anytime are called asynchronous errors, these errors are very difficult to handle, and Exception handling helps us catching these errors.

Throwing error means raising an error and delegating someone, the responsibility to handle that error.

Sorry so asking such basic question. Software engineering is based upon computer science. what is the difference between IT and Computer science.
Regards
Ans

Computer science is a discipline that involves the understanding and design of computers and computational processes. Computer Science is about learning and understanding the mathematical, scientific and engineering principles underlying every kind of computing system, from mobile phones and the internets, through systems that interpret natural language, to the supercomputers that forecast tomorrow's weather or simulate the effects of disease on the human heart.
Where, Information technology is the study, design, development, implementation, support or management of computer based information systems, particularly software applications and computer hardware. IT deals with the use of electronic computers and computer software to convert, store, protect, process, transmit and securely retrieve information.
Basic difference between these two disciplines is that computer science focuses on computers and its processing where IT is concerned with the use of technology for information.

Lec 38: Equivalence Classes or Equivalence Partitioning : Two tests are considered to be equivalent if it is believed that:
 if one discovers a defect, the other probably will too, and
 if one does not discover a defect, the other probably won't either.
Equivalence partitioning guidelines :
Organize your equivalence classes. Write them in some order, template etc.
Boundary conditions: determine boundary conditions. For example, adding in/after an empty linked list etc
don't forget invalid inputs that a user can give to a system.
String matching
Organization : we divide the problem in two obvious categories of equal
strings and the other one for unequal strings. Within these equivalent partitions, further partitioning is done. You can see the examples in the handouts page 200.
Basis Code Structures : Flow graph notation :
we do not use flow graphs to describe decisions. That is, how a branch is taken is not shown in flow graphs.
I'd sugget you to have a look at flow graphs in the handouts for further clarification.

How can I use profiler in my programs; I use both Dev C++ am MS Visual Studio environment to write and compile my C++ programs; Does any of them have a built-in profiler? if yes, how can I use to check statistics of my code and optimize my programs?
Ans
Yes Dev C++ has a built-in profiler. To use the profiler in Dev C++, first you need to enable the profiling info generation option in Compiler Options. To enable this option, follow the steps given below:

Tools -> Compiler Options -> Code Generation/Optimization -> Generate profiling information for analysis

Set the Generate profiling information for analysis option to YES

After that follow these steps:

Tools -> Compiler Options -> Linker -> Generate debugging information

Set the option Generate debugging information to YES

Now next step is to open your program and then follow these options and rebuild and run your application:

Execute -> Rebuild All
Execute -> Run

After this, follow the steps below to view the profiling statistics

Execute -> Profile analysis

Reference to lecture no. 33, I did not get the difference between Row Major and Column Major. Can you explain in detail with example...
Ans
Row major and Column major are data organizational techniques used in programming languages. More, specifically, these are methods for for storing multidimensional arrays in linear memory. In row-major storage, a multidimensional array in linear memory is accessed such that rows are stored one after the other. Where in Column major storage, a multidimensional array in linear memory is accessed such that columns are stored one after the other. Row major is used in C where Column major is technique of scientific languages like FORTRAN.

what is software and test ?

Ans

Software is a program/collection of programs that are used to operate computer. Test is examination of software to make sure that it is performing required functionality.

Lec 39 :
White box testing : Coverage :
 Statement Coverage: In this scheme, statements of the code are tested for a successful test that checks all the statements lying on the path of a successful scenario.
 Branch Coverage: In this scheme, all the possible branches of decision structures are tested. Therefore, sequences of statements following a decision are tested.
 Path Coverage: In path coverage, all possible paths of a program from input instruction to the output instruction are tested. An exhaustive list of test cases is generated and tested against the code.
Example :
if (a = = b) //1
c = d; //2
a++; //3
Statement coverage:
 a=1,b=1

1

2

If a==b then statement 2, statement 3

Branch coverage

- Statement 1: two braches 1-2, 1-3
- Test case 1: if a =1, b=1 then statement 2
- Test case 2: if a=1,b=2: then statement 3

Path coverage

Same as branch testing

Cyclomatic complexity : It defines number of independent paths in the basis set of a program.

Cyclomatic Complexity, V(G), for a flow graph G is defined as: V(G) = E - N + 2

Where E is the number of edges and N is the number of nodes in the flow graph G.

Bit fields allow the packing of data in a structure. This is especially useful when memory or data storage is at a premium

I read this line in handouts. Kindly explain what does premium mean in the above sentence? I mean what is the sense of this word in this sentence. Does premium mean that memory can have chances to be leaked?

Ans

Packing the data in bit fields means you use individual bits in variables to store the data rather than using separate variables. So, bit fields let you save your memory e.g. suppose you have a system which maintains the status of eight different hardware appliances, and you use a character variable to store the status of each appliance. Then storing the status of eight appliances means 8 character variables (each of 8 bits and thus total 64 bits consumption).

Now suppose you use a single character variable of 8 bits and use each of its bits to store the status of each appliance, this means that you need only one variable (of 8 bits) and not eight separate variables, so you have saved the memory of 7 variables.

Thus bit fields let you save your memory. This is very important when memory is very expensive or you are paying some amount to use memory. So, saving memory in this way

means, saving the money. Premium means when memory is purchased and using memory is an expensive task.

what is meant by Hungarian Notation:

Shall I write Medicine NAme as medicineName or MedicineName. Please kindly explain

Ans

Hungarian notation is a naming convention in computer programming, in which the name of a variable indicates its type or intended use.

Lec 40 : Unit testing : A software program is made up of units that include procedures, functions, classes etc. The unit testing process involves the developer in testing of these units.
Software should be tested more like hardware, with
 Built-in self testing: such that each unit can be tested independently
 Internal diagnostics: diagnostics for program units should be defined.
 Test harness
Unit Testing Principles
 In unit testing, developers test their own code units (modules, classes, etc.) during implementation.
 Normal and boundary inputs against expected results are tested.

 Thus unit testing is a great way to test an API (*Application programming interface*)

Unit Testing Tips :
 For small projects you can imbed the unit test for a module in the module itself
 For larger projects you should keep the tests in the package directory or a /test subdirectory of the package. By making the code accessible to developers you provide them with:
 Examples of how to use all the functionality of your module
 A means to build regression tests to validate any future changes to the code
Inspection and chaotic zone : if defects are not discovered and fixed at the appropriate stage, then at the testing and maintenance phases, these defects are piled up becoming chaotic zone for the testing and developing teams.

Please explain to me the exception handling. what is this throwing and catching. Explain the whole construct to me thanks

Ans

Exception Handling
Exception handling is a programming language construct or computer hardware mechanism designed to handle the occurrence of a condition that changes the normal flow of execution. For signaling conditions that are part of the normal flow of execution, see the concepts of signal and event handler. In general, the current state will be saved in a predefined location and the execution will switch to a predefined handler. Depending on the situation, the handler may later resume the execution at the original location, using the saved information to restore the original state. For example, an exception that will usually be resumed is a page fault, while a division by zero usually cannot be resolved transparently. From the processing point of view, hardware interrupts are similar to resumable exceptions, although they are usually not related to the current program flow. From the point of view of the author of a routine, raising an exception is a useful way to signal that the routine could not execute normally. For example, when an input argument is invalid (a zero denominator in division) or when a resource it relies on is unavailable (like a missing file, or a hard disk error). In systems without exceptions, routines would need to return some special error code. However, this is sometimes complicated by the semipredicate problem, in which users of the routine need to write extra code to distinguish normal return values from erroneous ones. In runtime engine environments such as Java or .NET, there exist tools that attach to the runtime engine and every time that an exception of interest occurs, they record debugging information that existed in memory at the time the exception was thrown (call stack and heap values). These tools are called Automated Exception Handling or Error Interception tools and provide 'root-cause' information for exceptions.

i wnat to become a software engineer so what courses i should specialize an kind of studey i will do

Ans

For the said purpose you need master the one specific field in Software Engineering. There are different areas in Software Field but you may choose one of the below.:-
Software Developer
Software Quality Assurance Engineer

Software Process Engineer
System Analyst
It depends upon you in which area you are interested in.

plz define that waht are classes and interfaces

Ans

objects define their interaction with the outside world through the methods that they expose. Methods form the object's interface with the outside world; the buttons on the front of your television set, for example, are the interface between you and the electrical wiring on the other side of its plastic casing. You press the "power" button to turn the television on and off. In its most common form, an interface is a group of related methods with empty bodies.
Now we talk about Classes
Classes
In object-oriented terms, we say that your bicycle is an instance of the class of objects known as bicycles. A class is the blueprint from which individual objects are created.

Lec 41:
Inspection versus Testing : Inspections and testing are complementary and not opposing verification techniques.
Both should be used during the verification and validation process.

Inspection checklists.
Checklist of common errors in a program should be developed and used to drive theinspection process. These error checklists are programming language dependent such thatthe inspector has to analyze major constructs of the programming language and develop checklists to verify code that is written using these checklists.

Static analyzers
Static analyzers are software tools for source text processing. They parse the program text and try to discover potentially erroneous conditions and bring these to the attention of the verification and validation team.

i want to know that keyword are the part of Coding Styles / Conventions or not. can i find differences of java and C# coding style on the basis of keywords.

Ans

Yes may find differences in this aspect. But you need to foucs on other areas as well.

what is meant by multiple increments?
Ans
multilple increments means" deliver multiple parts of the software system with the passage of time and with increment( additional features/functionality) to the customer.

Lec 42:
Importance of Debugging :
According to a survey,when a software application is in the maintenance phase, 20% of its lifecycle cost is attributed towards the defects which are found in the software application after installation. Please bear in mind that the maintenance is the phase in which 2/3rd of the overall software cost incurs.

Therefore, 20% of the 2/3rd cost is again a huge cost and we need to understand why this much cost and effort is incurred. In fact, when a software application is installed and being used, any peculiarity in it can cost a lot of direct and indirect damages to the organization.

and Science of Debugging:
Debugging is taken as an art but in fact it is a scientific process. As people learn about different defect types and come across situations in which they have to debug the code, they develop certain heuristics. Next time they come across a similar situation, they apply those heuristics and solve the problem in lesser time and with a lesser effort.

1) Black Whole, when data is not processed ?
2) Miracle, There is no any input but there is some out put
   What we mean by synch ?
   what we mean by gray whole?

Ans

These are processes containing some mistakes (DFD common mistakes):
Black hole: A process which takes input but does not generate any output.
Miracle: A process which does not take any input but produces an output.
Gray hole: A process in which output is produced without required and suffieicent inputs.

what does we mean by
1) #ifndef
2) #define
3) #endif
4) %
I saw such statements in many C codes, % is used many times with printf ()

Ans

These are preprocessing directives used as different conditional parameters to control the compiling of source code.

From lecture 37 i have come to conclusion that structural testing is more elaborate and better as compared to black box ... is it correct?
Ans
We can not comment that structural testing is more better than black box. There are two things; in large and complex embedded system, if we want to identify which module is problematic, we will do black box testing as only white box testing will be time consuming. Secondly, in complex and simple both systems, first black box would be used to identify the module/system containing issue and then structural testing would be used for further analysis. So both types are equally effective. And testing is incomplete if any one of them is not done.

Lec 43:
A memory leak bug is one in which memory is somehow allocated from either the operating system or an internal memory "pool", but never deallocated when the memory is finished being used.

Symptoms
 System slowdowns
 Crashes that occur "randomly" over a long period of time
Logical Errors
A logical error occurs when the code is syntactically correct but does not do what you expect it to do.
Symptoms
 The code is misbehaving in a way that isn't easily explained.
 The program doesn't crash, but the flow of the program takes odd branches through the code.
 Results are the opposite of what is expected.
 Output looks strange, but has no obvious symptoms of corruption.
Coding errors: A coding error is a simple problem in writing the code.
Symptoms
 Unexpected errors in black box testing.
 The errors that unexpectedly occur are usually caused by coding errors.
 Compiler warnings.
 Coding errors are usually caused by lack of attention to details.

Memory over-runs: a memory overrun occurs when you use memory that does not belong to you. This can be
caused by overstepping an array boundary or by copying a string that is too big for the block of memory it is defined to hold.

Symptoms
 Program crashes quite regularly after a given routine is called, that routine should be examined for a possible overrun condition.
 If the routine in question does not appear to have any such problem the most likely cause is that another routine, called in the prior sequence, has already trashed variables or memory blocks.
 Checking the trace log of the called routines leading up to one with the problem will often show up the error.
Loop Errors
 Loop errors break down into several different subtypes.
 They occur around a loop construct in a program.
 Infinite loops, off-by-one loops, and improperly exited loops.

Symptoms

 If your program simply locks up, repeatedly displays the same data over and over, or infinitely displays the same message box, you should immediately suspect an infinite loop error.

 Off-by-one loop errors are quite often seen in processes that perform calculations.

 If a hand calculation shows that the total or final sum is incorrect by the last data point, you can quickly surmise that an off-by-one loop error is to blame.

 Likewise, if you were using graphics software and saw all of the points on the screen, but the last two were unconnected, you would suspect an off-by-one error.

 Watching for a process that terminates unexpectedly when it should have continued.

Pointer errors:

 A pointer error is any case where something is being used as an indirect pointer to another item.

 Uninitialized pointers: These are pointers that are used to point at something, but we fail to ever assign them something to point at.

 Deleted pointers, which continue to be used.

 An Invalid pointer is something that is pointing to a valid block of memory, but that memory does not contain the data you expect it to.

Symptoms

 The program usually crashes or behaves in an unpredictable and baffling way.

 You will generally observe stack corruptions, failure to allocate memory, and odd changing of variable values.

 Changing a single line of code can change where the problem occurs.

 If the problem "goes away" when you place a print statement or new variable into the code that you suspect contains the problem.

Boolean bugs

Boolean bugs occur because the mathematical precision of Boolean algebra has virtually nothing to do with equivalent English words.

Symptoms

When the program does exactly the opposite of what you expect it to. For example, you might have thought you needed to select only one entry from a list in order to proceed. Instead, the program will not continue until you select more than one. Worse, it keeps telling you to select only one value.
 For true/false problems, you will usually see some sort of debug output indicating an error in a function, only to see the calling function proceed as though the problem had not occurred.

Test cases are part of ATP?
Ans
Test cases corresponds to application functionality which are required to user. Where ATP is complete procedure of testing a system. Yes, test cases are part of ATP.

Whose responsibility is it to write the Acceptance Test Procedure (ATP)?
Ans
Acceptance test procedure is responsibility of quality assurance team/department

How do a test Engineer carry out unit testing of the c++ code. As we have learned in OOP a class e.g. is dependent upon so many outer variables and other objects as well, so how will the test engineer test it independently? Is he suppose to write another piece of code to test each class separately?.... is there any off the shelf tool available for this purpose?
Ans
Unit testing is a two way work as developer is doing unit testing by himself when he is writing code. When quality assurance team or test engineer performs test, they mention error containing module/block to developer for testing because test engineer can't do unit testing of code. It is done by the develper. When developer is asked then he again checks the code for issues. Code analysis, code checkers and code optimizers are also used for this purpose.

Please tell me about the term beta release or beta testing in simple words. Regards
Ans
In beta testing, some customers/users are provided with developed product. They use it and provide feedback about product which is (if required) incorporated in the ptoduct. After that, it is delivered for general use.

What is test cases and what is test data?
Ans
A test case is a set of conditions or variables under which a tester will determine whether an application or software system is working correctly or not. Where test data is data which has

been specifically identified for use in tests. Some data may be used in a confirmatory way, typically to verify that a given set of input to a given function produces some expected result. Other data may be used in order to challenge the ability of the program to respond to unusual, extreme, exceptional, or unexpected input.

what is the basic difference between test cases and test data? Please explain this with an example. I shall be thankful to you. Regards,
Ans
A test case is a set of conditions or variables under which a tester will determine whether an application or software system is working correctly or not. Where test data is data which has been specifically identified for use in tests. For example, you want to develop a test-case for login functionality of LMS. Set of conditions defined (Pre-condition, Step, Action, Expected Response, Pass/Fail, Comments, Post-conditions) is test case. When you give a user name and pasword in LMS to check login functionality then this user name and password is test data.
Lec 44:
Holistic approach:
Holistic means
 Emphasizing the importance of the whole and the interdependence of its parts.
 Concerned with wholes rather than analysis or separation into parts
The debugging process:
First hand accounts of the problem are always useful, so be sure to write
down exactly what you are told. That way, you can compare several accounts of the same problem and look for similarities.
Good clues, Easy Bugs
Get A Stack Trace
Following stack trace information may help in debugging process.
 Source line numbers in stack trace is the single, most useful piece of debugging information.
 After that, values of arguments are important
o Are the values improbable (zero, very large, negative, character strings
with non-alphabetic characters?
 Debuggers can be used to display values of local or global variables.
o These give additional information about what went wrong.
Non-reproducible bugs
 Bugs that won't "stand still" (almost random) are the most difficult to deal with. 
Randomness itself, however, is information.
 Are all variables initialized? (random data in variables could affect output).
 Does bug disappeared when debugging code is inserted? Memory allocation
(malloc) problems are probably a culprit.
 Is the crash site far away from anything that could be wrong?
 Check for dangling pointers.

What is the effective testing?
Ans
he objective of testing is to discover the maximum number of defects with a minimum number of resources before the system is delivered to the next stage. If this objective is achieved then it is effective testing. That is, testing is effective enough to ensure the product rightness.

what is the goal of the tester person?

Ans

Responsibility of the tester is to achieve testing objective which is, to discover the maximum number of defects with a minimum number of resources before the system is delivered to the next stage. A good tester does not rely on few times testing of the program but carries out a thorough analysis of the program to devise test cases that can be used to test the system systematically and effectively from every perspective of testing each and every functionality.

Q:Describe the importance of software debugging? Q: Differentiate Hungarian notation,Bi capitalization with example?
Ans
Both Hungarian and Bicapitalization are two naming conventions used to name the variables or identifiers in the code.

Hungarian Notation is mainly confined to Microsoft Windows programming environments, such as Microsoft C, C++ and Visual Basic. Hungarian notation involves storing information about the variable in the very name of the variable.

With Hungarian notation, variable names are separated into two parts:

1. The lowercase prefix, which contains information about the variable type, and
2. The qualifier, which tells you what the variable contains. The qualifier usually begins with a capital letter.

For example: pstrError

BiCapitalization notation is a common name for the practice of writing compound words or phrases where the words are joined without spaces, and each word is capitalized within the compound.

For example: totalMarks
numOfStudents

How this profiling and performance tuning is affect on SE program?
Ans
It is hard to understand the behavior of the program statically therefore a thorough analysis based on runtime data is needed. A Performance profiling is a technique to investigate runtime behavior of the program. The purpose is to identify the bottlenecks in the program with the objective to increase optimization.

What we mean by profiler, is it some kind of software, please guide me.
Ans
Profilers are used in the performance engineering process. Profilers are software used to perform profiling. Profiling is a way to measure where a program spends time and helps to uncover performance problems in the code. Profilers use a wide variety of techniques to collect data, including hardware interrupts, code instrumentation, instruction set simulation, operating system hooks, and performance counters.

Is Unit Testing is white box testing or it is black box testing?
Ans

Unit testing is testing individual components independent of other components. It is performed to check an individual component. So it is both, black box testing as well as white box because we check component input/output along with complete program structure.

# CS504 QUIZ 3 WITH MOAZ,JUNAID FILES

# Final term preparation 2022

# ORANGE MONKEY TEAM

1. **Dynamic process model shows the process _____ of the system.**

**Structure….confirm**

dynamic process model shows the process structure of the system.

2. **Physical view in Krutchen's 4+1 architectural view model captures _____.**

**Mapping of the software onto the hardware and reflects is distributed aspect…..confirm**

and synchronization aspects of the design, the *physical* view documents the mapping(s) of the software onto the hardware and reflects its distributed aspect, the *development*

3. **A self documenting code is a code that explains itself without the need of comments and extraneous documentation like _____.**

**Flowcharts, UML diagrams, process-flow state diagrams…all options correct….confirm**

A *self documenting code* is a code that explains itself without the need of comments and extraneous documentation, like flowcharts, UML diagrams, process-flow state diagrams,

4. **Using proper paranthesis normally makes the code.**

**All options correct…..confirm**

using proper **paranthesis normally m**akes the code.

easy to read

easy to understand

less ambiguous

**All of the given options**

# Software Engineering – 1

# (CS504)

## Lecture Notes

Delivered by

**Dr. Fakhar Lodhi**

**Lecture 23:** Architectural Views ..... 122

**Lecture 24:** Architectural Models-I ..... 126

**Lecture 25:** Architectural Models-II ..... 130

**Lecture 26:** Introduction to Design Patterns ..... 137

**Lecture 27:** Observer Pattern ..... 140

**Lecture 28:** Good Programming Practices and Guidelines… ..... 146

**Lecture 29:** File Handling Tips for C++ and Java ..... 155

**Lecture 30:** Layouts and Comments in Java and C++… ..... 162

**Lecture 31:** Coding Style Guidelines Continued. ..... 167

**Lecture 32:** Clarity Trough Modularity ..... 170

**Lecture 33:** Common Coding Mistakes ..... 176

**Lecture 34:** Portability ..... 179

**Lecture 35:** Exception Handling ..... 184

**Lecture 36:** Software Verification and Validation ..... 192

**Lecture 37:** Testing vs. Development ..... 195

**Lecture 38:** Equivalence Classes or Equivalence Partitioning ..... 199

**Lecture 39:** White Box Testing ..... 202

**Lecture 40:** Unit Testing ..... 207

**Lecture 41:** Inspections vs. Testing ..... 210

**Lecture 42:** Debugging ..... 213

**Lecture 43:** Bug Classes ..... 216

**Lecture 44:** The Holistic Approach ..... 224

**Lecture 45:** Summary ..... 227



© Copyright Virtual University of Pakistan

# Lecture No. 23

## 8.6 Architectural Views

Software architecture defines the high level structure of the software by putting together a number of architectural elements in an organized fashion. These elements are chosen to satisfy the functional as well as non-functional requirements of the system. Perry and Wolfe proposed the following formula for software architecture:

Software architecture = {Elements, Forms, Rationale}

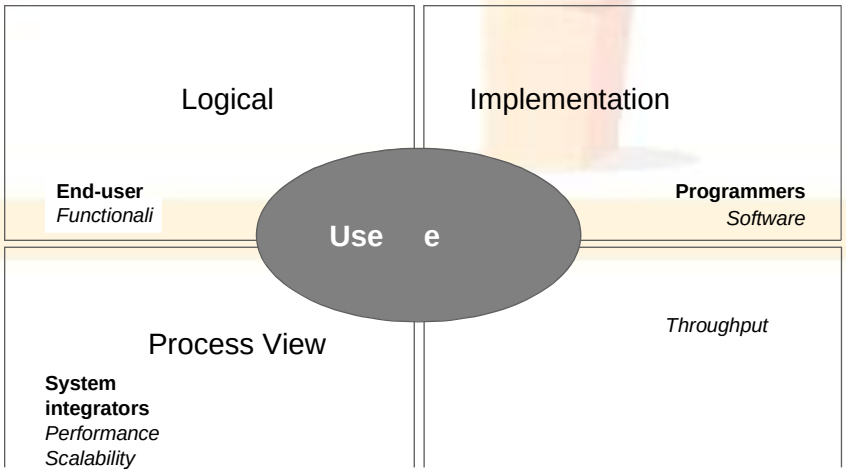
That is, a software architecture is a set of elements which have a certain form. These elements are further divided into three categories. These are: data elements, processing elements, and connecting elements. The data elements contain the information that is used and transformed in the system; the processing elements process the data elements and specify the transformation functions, and connecting elements connect different pieces of architecture together. These can themselves be data or processing elements.

This formula was modified by Boehm as follows:

Software architecture = {Elements, Forms, Rationale/Constraints}

Krutchen proposed that the software architecture model should be composed of multiple views as a software architecture deals with abstraction, with decomposition and composition, with style and esthetics as well as requirements from different stake holders. His architectural model is known as Krutchen's 4+1 architectural view model. As evident, this model proposes the development of 5 main views namely the logical view, the process view, the physical view, the development view, and the use case view. The logical view is the object model of the design, the *process* view captures the concurrency and synchronization aspects of the design, the *physical* view documents the mapping(s) of the software onto the hardware and reflects its distributed aspect, the *development* view describes the static organization of the software in its development environment, and the use case view uses selected use cases or scenarios to validate that the architecture supports the required functionality.

This model is shown in the following diagram.






© Copyright Virtual University of Pakistan

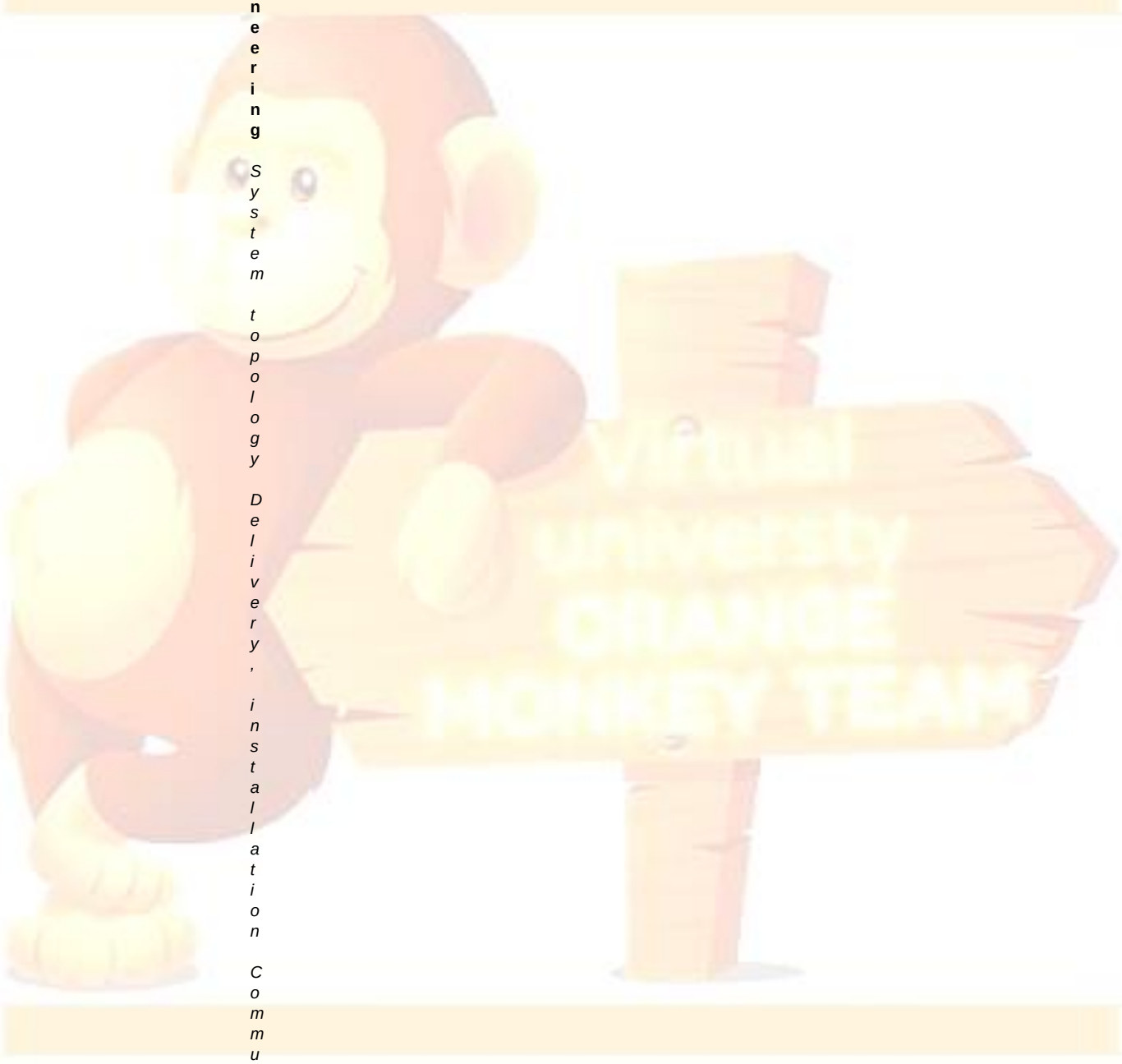
Deployment View

**System engineering**

*System topology*

*Delivery, installation*

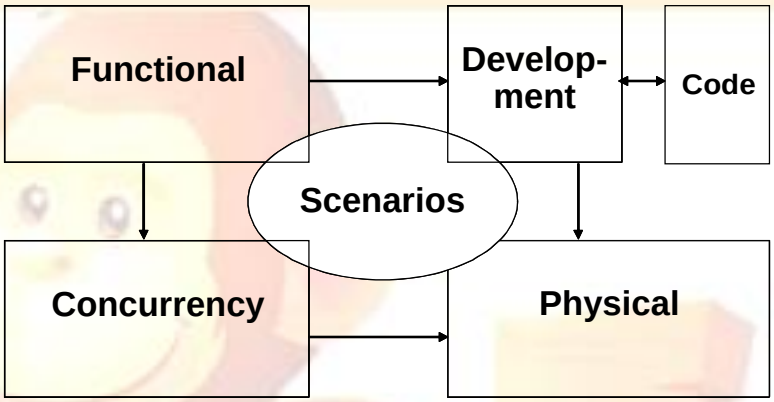
*Communication*





© Copyright Virtual University of Pakistan

This model has been slightly modified by Clements et. al. and is shown in the following diagram.




Clement's modified version of Krutchen's 4+1 Architectural View Model

In this model, the architecture is again prepared and analyzed from 5 different perspectives. The 4 main views are Functional View, the Concurrency View, the Physical View, and the Development View. Code view is not present in the original Krutchen model and is basically an extension of the development view.

The Functional View comprises of various different functions provided by the system, key system abstraction, and the domain elements. It connects different dependencies and data flows into a single view. This view can be used by the domain engineers, product-line engineers, as well as the end users of the system. It can be used for understanding the functionality provided by the system, modifiability of the system, reusability, tool support, and allocation of work.

The Development View documents the different files and directories in the system and users of this view include the development and the configuration management staff as well as the project managers. The major uses of this view include maintenance, testing, configuration management, and version control.

The components of the Code View include classes, objects, procedures, functions, subsystems, layers, and modules. It documents the calling as well as the containing hierarchy of different components of the system. Its primary users are the programmers and designers of the system. The primary intent of developing this view is to use it for maintenance and portability.


© Copyright Virtual University of Pakistan

Concurrency View intends to document different parallel processes and threads in the system. Its main focus is on event synchronization and parallel data flows. It is used primarily by integrators, performance engineers, and testers. The main purpose of building this view is to identify ways and means to improve performance by highlighting possible opportunities for parallelism.

The Physical View depicts the physical organization of this system and how this system will be physically deployed. This includes different processors, sensors, and storage devices used by the system. This view connects various network elements and communication devices. The primary users of this system include hardware and system engineers. The view is developed with an intention to document and analyze system delivery and installation mechanism. The view is also used in understanding and analyzing issues pertaining to system performance, availability, scalability, and security.

Finally, there are Scenarios. Scenarios are basically use cases which describe the sequences of responsibilities and change cases which are changes to the system. As stated earlier, the first objective of developing a system is to fulfill the functional requirements setout for the system. These functional requirements are written in the form of use cases. Therefore, scenarios are used to understand and validate the system. Their secondary purpose is to communicate the design to the other users of the system. Scenarios tie all the view together into an integrated system. They are also used to understand the dynamic behavior of the system and understand the limits of design.

This is summarized in the following table:

| View | Components | Users | Rationale |
|---|---|---|---|
| **Functional View** | functions, key system abstractions, domain elements | domain engineers, product-line designers, end users | functionality, modifiability, product lines/reusability, tool support, work allocation |
| **Code View** | classes, objects, procedures, functions, subsystems, layers, modules | programmers, designers, reusers | modifiability/maintainability, portability, subsetability |
| **Development View** | files, directories | managers, programmers, configuration managers | managers, programmers, configuration managers |
| **Physical View** | CPUs, sensors, storage | hardware engineers, system engineers | system delivery and installation, performance, availability, scalability, security |
| **Concurrency View** | processes, threads | performance engineers, integrators, testers | performance, availability |


© Copyright Virtual University of Pakistan

**What Are Views Used For?**

Views are an engineering tool to help achieve desired system qualities. Each view provides an engineering handle on certain quality attributes. In some systems, distinct views collapse into one (e.g., the concurrency and physical views may be the same for small systems.). Views are also used as documentation vehicle for current development and future development. Users of views include both managers and customers. For these reasons views must be *annotated* to support analysis. This annotation can be achieved with the help of scenarios and design *rationale*.

Structures can also be used to document how the current system was developed and how future development should occur. This information is needed by managers and customers.

Although one often thinks about a system's structure in terms of functionality, there are other system properties in addition to functionality (such as physical distribution, process communication, and synchronization) that must be reasoned about at an architectural level. Each structure provides a method for reasoning about some of the relevant quality attributes. The uses structure, for instance, must be engineered (not merely recorded) to build a system that can easily be extended or contracted. The calls structure is engineered to reduce bottlenecks. The module structure is engineered to produce modifiable systems, and so on. Each structure provides a different view into the system and a different leverage point for design to achieve desired qualities in the system.

**Hierarchical Views**

Every view is potentially hierarchical, e.g. *functional view could contain* sub-functions. Similarly *development view* contains directories which contain files. C*ode view* would have modules and systems that contain sub-modules and sub-systems respectively. C*oncurrency view* contains processes that are further subdivided into threads. Finally, *physical view* clusters contain computers which contain processors.

Architectural views are related to each other in complicated ways. One has to choose the views that are useful to the system being built and to the achievement of qualities that are important for that particular application. The architectural views should be hierarchical (where needed) and should contain enough annotated information to support desired analyses.

**Architectural styles**

The architectural model of a system may conform to a generic architectural model or style. An awareness of these styles can simplify the problem of defining system architectures. However, most large systems are heterogeneous and do not follow a single architectural style.


© Copyright Virtual University of Pakistan

Each style describes a system category that encompasses:

(1) a set of components (e.g., a database, computational modules) that perform a function required by a system,
(2) a set of connectors that enable "communication, coordination and cooperation" among components,
(3) constraints that define how components can be integrated to form the system, and

semantic models that enable a designer to understand the overall properties of a system by analyzing the known properties of its constituent parts.

# Lecture No. 24

## Architectural models

Like analysis models, many different kinds of architectural models are developed during the architectural design process. Static structural model shows the major system components while a dynamic process model shows the process structure of the system. Interface models are developed to define sub-system interfaces.

### Architectural Styles

Architectural design may be based upon a certain pattern of model. These different patterns are also called architectural styles. These styles have different characteristics and attributes and can be useful to solve problems related to a particular situation of requirement. Among the many styles, the most commonly practiced are the following:

- Data-centered architectures
- Client Server Architecture and its variations
- Layered architectures
- Reference Architecture

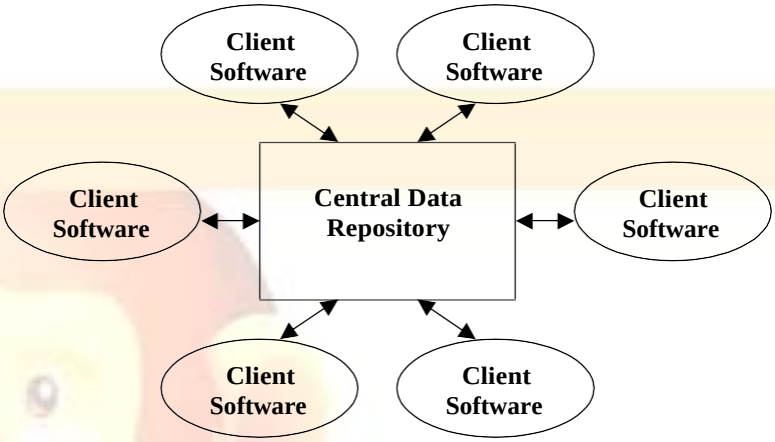
### Data-Centered or the repository model

In any system, sub-systems need to exchange information and data. This may be done in two ways:

1. Shared data is held in a central database or repository and may be accessed by all sub-systems
2. Each sub-system maintains its own database and passes data explicitly to other sub-systems


© Copyright Virtual University of Pakistan

When large amounts of data are to be shared, the repository model of sharing is most commonly used. This model has been extensively used in the main-frame based




application. The model is depicted in the following diagram:

**Repository model characteristics**

- Advantages

  Repository model is an efficient way to share large amounts of data. In this case sub-systems need not be concerned with how data is produced Centralised management e.g. backup, security, etc. This model also provides a global view of the system and the sharing model is published as the repository schema.

- Disadvantages

  Repository model suffers from a number of disadvantages. First of all, sub-systems must agree on a repository data model. This inevitably leads to a compromise. The major problem however is that data evolution is difficult and expensive. There is also little scope for implementing specific management policies. It is also difficult to distribute efficiently

**Client-server model**

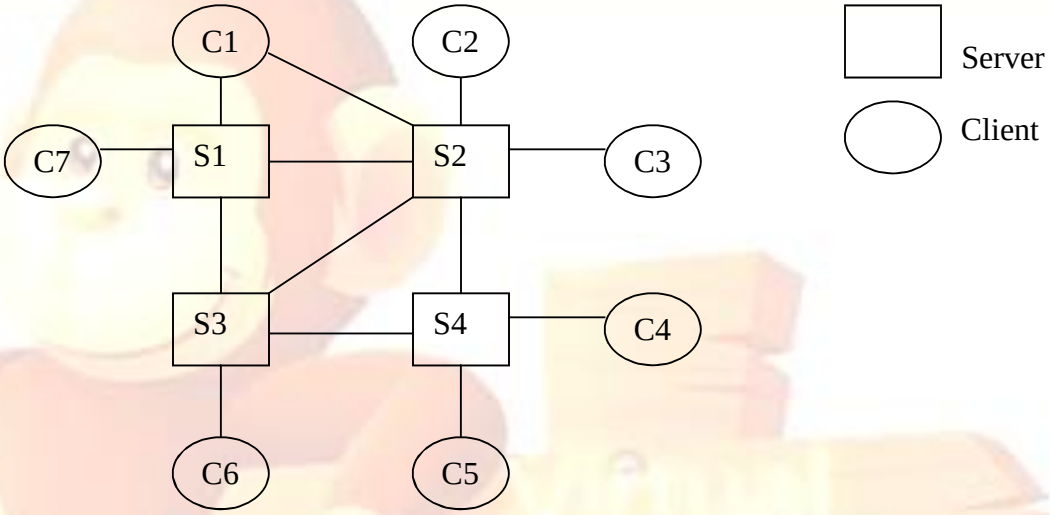
Client server model tries to distribute data and processing. This is a shift from main-frame based applications where both the data management and the processing of data used to be typically carried out by the same main-frame computer. In those applications, the user interface used to be a provided through a "dumb" terminal which did not have much processing power. With the availability of the cheaper but power machines, it was possible to shift some load from the back-end computer to other smaller machines.

The client-server model is a distributed system model which shows how data and processing is distributed across a range of components. In this model, the application is modeled as a set of services that are provided by servers and a set of clients that use these services. The system is organized as a set of stand-alone servers which provide specific services such as printing, data management, etc. and a set of clients which call on these


© Copyright Virtual University of Pakistan

services. These clients and servers are connected through a network which allows clients to access servers. Clients and servers are logical processes (not always physical machines). Clients know the servers but the servers do not need to know all the clients and the mapping of processes to processors is not always 1:1.

The following diagram depicts a general client-server organization.





© Copyright Virtual University of Pakistan

### Client/Server Software Components

A typical client-server architecture based system is composed of a number of different components. These include user interaction/presentation subsystem, application subsystem, database management subsystem, and middleware. The application subsystem implements requirements defined by the application within the context of the operating environment. In this case the application components may reside on either client or server side. Middleware provides the mechanism and protocols to connect clients with the servers.

### Representative Client/Server Systems

Following are some of the representative server types in a client-server systems.

- File servers

  In this case, client requests selected records from a file and the server transmits records to client over the network.
- Database servers

  In this case, client sends requests, such as SQL queries, to the database server, the server processes the request and returns the results to the client over the network.
- Transaction servers

  In this configuration, client sends requests that invokes remote procedures on the server side, server executes procedures invoked and returns the results to the client.
- Groupware servers

  Groupware servers provide set of applications that enable communication among clients using text, images, bulletin boards, video, etc.

### Client-server characteristics

- Advantages

  The main advantage of the client-server architecture is that it makes effective use of networked systems. Instead of building the system with very expensive large computers and hardware devices, cheaper hardware may be used to gain performance and scalability. In addition, adding new servers or upgrading existing servers becomes easier. Finally, distribution of data is straightforward in this case.

- Disadvantages

  The main disadvantage of this model is that there is no standard way of sharing data, so sub-systems may use different data organisation. Hence data interchange may be inefficient. From a management point of view, each server needs attention and hence there is redundant management in each server. Finally there is no central register of names and services - it may be hard to find out what servers and services are available.


© Copyright Virtual University of Pakistan

### Representative Client/Server Configurations

The client-server model can have many different configurations. In the following sections, we look at some of these configurations.

#### Thin Client Model

This model was initially used to migrate legacy systems to client server architectures. In this case the legacy system may act as a server in its own right and the GUI may be implemented on a client. It chief disadvantage is that it places a heavy processing load on both the server and the network.

#### Fat Client Model

With advent of cheaper and more powerful hardware, people thought of using the processing power of client side machines. So the fat client model came into being. In this model, more processing is delegated to the client as the application processing is locally extended. It is suitable for new client/server systems when the client system capabilities are known in advance. It however is more complex than thin client model with respect to management issues. Since the client machine now also has a significant part of the application resident on it, new versions of each application need to be installed on every client.

# Lecture No. 25

## Zero Install

As discussed earlier, fat-client architecture posed major challenges in terms of installation and maintenance of the client side of the application, especially when there are large number of client machines. So the idea behind zero install architecture is to develop a system where no installation on the client side is needed. This can only be done when there is no or little processing done at the client side. This is basically a trade-off between using the computing power available at the client machine versus maintenance overhead. This in essence takes us back to an architecture which is similar to thin-client architecture. There is little difference though. In the classical thin-client architecture, the entire processing is carried-out by a single server, in the case of zero-install, the network environment is used to distribute server side processing by adding a number of servers which share processing load. Web-based application where the web-pages are put on a web-server is an example of this type of architecture. In this case, whenever there is a change, the web page is updated accordingly. Now when the user logs in, he gets to use the modified version of the application without any changes or updates at the client side.


© Copyright Virtual University of Pakistan

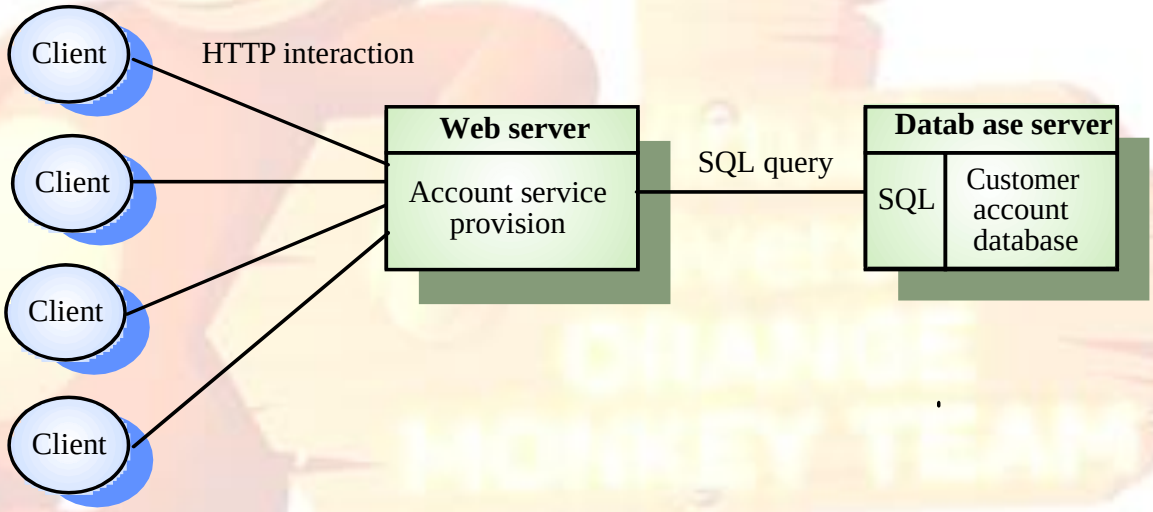
**N-Tier architecture**

N-tier architecture stems from the struggle to find a middle ground between the fat-client architecture and the thin-client architecture. In this case the idea is to enhance scalability and performance by distributing both the data and the application using multiple server machines. This could involve different types of servers such as application server, web server, and DB server. Three-tier architecture which is explained below is a specialized form of this architecture.

**Three-tier Architecture**

In this architecture, each application architecture layers (presentation, application, database) may run on separate processors. It therefore allows for better performance than a thin-client approach. It is simpler to manage than fat client approach and highly scalable (as demands increase add more servers).

A typical 3-tier architecture is depicted in the following diagram.



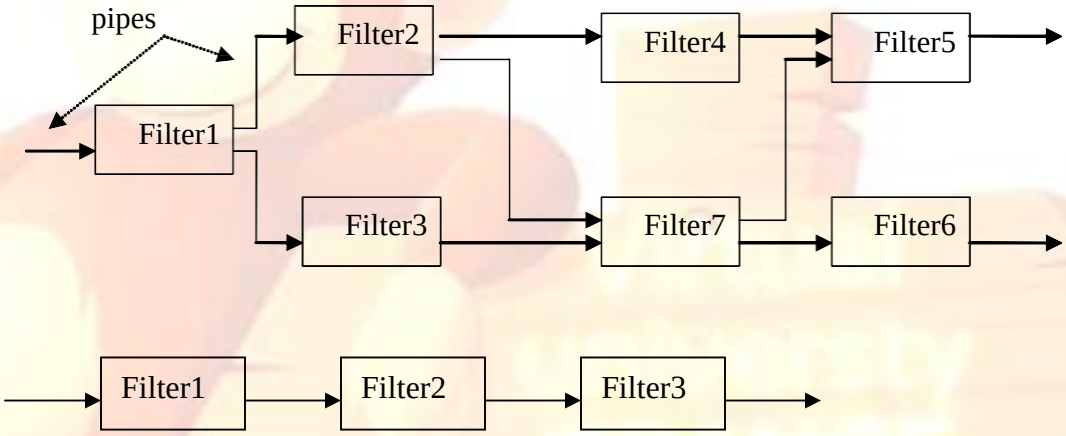

N-tier architecture generalizes the concepts of 3-tier architecture. In this case the system architecture may have more than 3 layers. That is, in n-tier architecture, in order to increase performance, we may distribute the application over different servers by putting different subsystems on different servers.


© Copyright Virtual University of Pakistan

**Data Flow or Pipes and Filters Architecture**

This architecture is very similar to data flow diagrams. This is used when the input data is processed through a series of transformations to yield the desired output. It is also known as pipes and filters architecture where each processing step is called a filter and the connecting link from one process to the other is called the pipe through which the information flows from one process to the other. An important aspect of this model is that each filter works independently of others and does not require knowledge of the working of any of the other filters including its neighbours.

If the dataflow has only a single sequence of processes with no alternative or parallel paths, then it is called batch sequential. These models are depicted in the following diagrams.

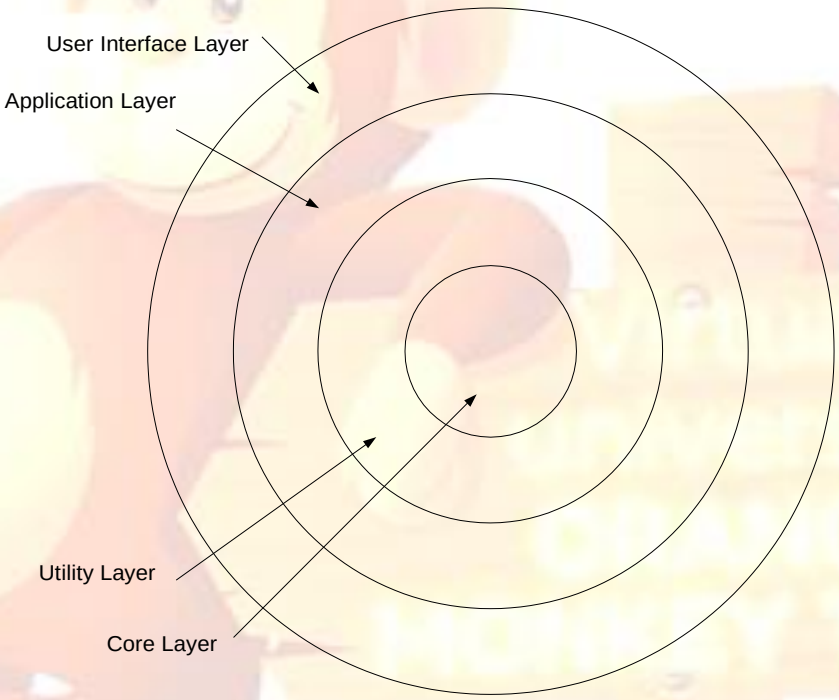



Batch Sequential


© Copyright Virtual University of Pakistan

**Layered Architecture**

As the name suggests, a layered architecture has many different layers. One typical example of a layered architecture is an operating system where different layers are used to provide services and functionality and the inner layers are closer to the machine hardware than the outer layers. In this way, each layer isolates the outer layer from inner complexities. In order to work properly, the outer layer only needs to know the interface provided by the inner layer. If there are any changes in the inner layer, as long as the interface does not change, the outer layer is not affected. This scheme tremendously portability of the system. The basic layered architecture is depicted in the following diagram.

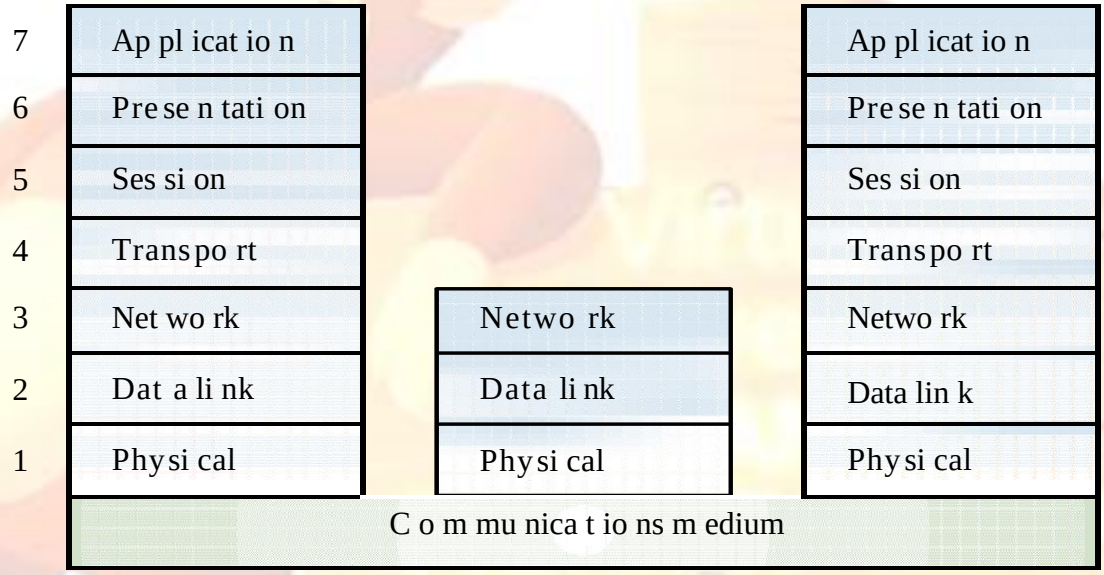




© Copyright Virtual University of Pakistan

**Reference architectures**

Reference architecture is not a physical architecture. It is only a reference for defining protocols and designing and implementing systems developed by different parties. Reference models are derived from a study of the application domain rather than from existing systems. It may be used as a basis for system implementation or to compare different systems. It acts as a standard against which systems can be evaluated. One very common example of such a reference model is the OSI model which is a layered model for communication systems. The success of this kind model is evident from the success of the Internet where all kinds of heterogeneous systems can talk to each other only because all of them use the same reference architecture.




OSI reference model



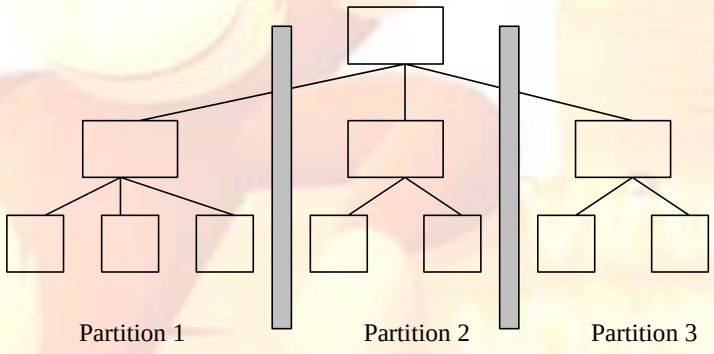
© Copyright Virtual University of Pakistan

**Partitioning the Architecture**

Partitioning of architecture is an important concept. What we basically want to do is distribute the responsibilities to different subsystems so that we get a software system which is easy to maintain. Partitioning results in a system that suffers from fewer side effects. This ultimately means that we get a system that is easier to test and extend and hence is easier to maintain.

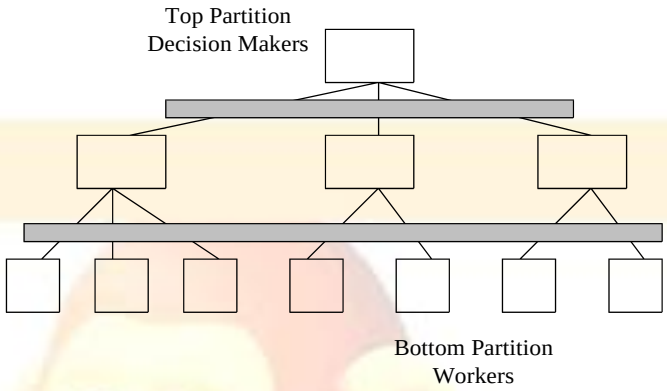
Partitioning of an architecture may be "horizontal" and/or "vertical".

In the horizontal partitioning we define separate branches of the module hierarchy for each major function and control modules are used to coordinate communication between functions. This concept is depicted in the following diagram.




Vertical partitioning divides the application from a decision making perspective. The architecture is partitioned in horizontal layers so that decision making and work are stratified with the decision making modules residing at the top of the hierarchy and worker coming at the bottom. This partitioning is also known as factoring and the general model is depicted in the following diagram.


© Copyright Virtual University of Pakistan




**Analyzing Architecture design**

In a given system, the required characteristics may conflict. Trade-offs seek optimal combinations of properties based on cost/benefit analysis. So the analysis requires an understanding of what is required and the relative priority of the attributes has to be established. The following sequence of steps provides a guideline for performing architectural analysis.

1. Collect scenarios.
2. Elicit requirements, constraints, and environment description.
3. Describe the architectural styles/patterns that have been chosen to address the scenarios and requirements. These views include module view, process view, and data flow view.
4. Evaluate quality attributes by considered each attribute in isolation.
5. Identify the sensitivity of quality attributes to various architectural attributes for a specific architectural style.

Critique candidate architectures (developed in step 3) using the sensitivity analysis conducted in step 5.


© Copyright Virtual University of Pakistan

# Lecture No. 26

Introduction to Design Patterns

## Design Patterns

Christopher Alexander says, "Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice." A Pattern Language: Towns/Buildings/Construction, 1977

Even though Alexander was talking about patterns in buildings and towns, what he says is true about object-oriented design patterns. Our solutions are expressed in terms of objects and interfaces instead of walls and doors, but the core of both kinds of patterns is a solution to a problem in a context.

## Design Patterns defined

"Description of communicating objects and classes that are customized to solve a general design in a particular context."

Patterns are devices that allow programs to share knowledge about their design. In our daily programming, we encounter many problems that have occurred, and will occur again. The question we must ask our self is how we are going to solve it *this* time. Documenting patterns is one way that you can reuse and possibly share the information that you have learned about how it is best to solve a specific program design problem.

Essay writing is usually done in a fairly well defined form, and so is documenting design patterns. The general form for documenting patterns is to define items such as:

- The motivation or context that this pattern applies to.
- Prerequisites that should be satisfied before deciding to use a pattern.
- A description of the program structure that the pattern will define.
- A list of the participants needed to complete a pattern.
- Consequences of using the pattern...both positive and negative.
- Examples!

## Historical perspective of design patterns

The origin of design patterns lies in work done by an architect named *Christopher Alexander* during the late 1970s. He began by writing two books, *A Pattern*


© Copyright Virtual University of Pakistan

*Language*[Alex77] and *A Timeless Way of Building* [Alex79] which, in addition to giving examples, described his rationalle for documenting patterns.

The pattern movement became very quiet until 1987 when patterns appeared again at an OOPSLA conference. Since then, many papers and presentations have appeared, authored by people such as Grady Booch, Richard Helm, and Erich Gamma, and Kent Beck. From then until 1995, many periodicals, featured articles directly or indirectly relating to patterns. In 1995, Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides published Design Patterns: Elements of Reusable Object-Oriented Software [Gamma95], which has been followed by more articles in trade journals.

The concept of design patterns is not new as we can find a number of similar pursuits in the history of program designing and writing. For instance, Standard Template Library (STL) is a library of reusable components provided by C++ compilers. Likewise, we use algorithms in data structures that implement typical operations of manipulating data in data structures. Another, similar effort was from Peter Coad whose patterns are known for object-oriented analysis and design.

Anti-patterns is another concept that corresponds to common mistakes in analysis and design. These are identified in order to prevent potential design and analysis defects from entering into the design. Another, similar concept is object-oriented framework that is a set of cooperative classes that make up reusable design of a system. Framework dictates the architecture of the software and describes the limitations and boundaries of architecture.

With this introduction, we now describe the format that has been adopted in GoF book for describing various design patterns.

# GOF Design Pattern Format

The basic template includes ten things as described below

## Name

- Works as idiom
- Name has to be meaningful

## Problem

- A statement of the problem which describes its intent
- The goals and objectives it wants to reach within the given context

## Context

- Preconditions under which the problem and its solutions seem to occur
- Result or consequence
- State or configuration after the pattern has been applied

## Forces

- Relevant forces and constraints and their interactions and conflicts.
- motivational scenario for the pattern.

## Solution

- Static and dynamic relationships describing how to realize the pattern.
- Instructions on how to construct the work products.


© Copyright Virtual University of Pakistan

- Pictures, diagrams, prose which highlight the pattern's structure, participants, and collaborations.

### Examples

- One or more sample applications to illustrate
  - a specific context
  - how the pattern is applied
-

### Resulting context

- the state or configuration after the pattern has been applied
- consequences (good and bad) of applying the pattern

### Rationale

- justification of the steps or rules in the pattern
- how and why it resolves the forces to achieve the desired goals, principles, and philosophies
- how are the forces orchestrated to achieve harmony
- how does the pattern actually work

### Related patterns

- the static and dynamic relationships between this pattern and other patterns

### Known uses

- to demonstrate that this is a proven solution to a recurring problem

## Classifications of deerns

### Creational patterns

- How to create and instantiate
- Abstract the instantiation process and make the system independent of its creational process.
- Class creational rules
- Object creational rules
- Abstract factory and factory method
- Abstract the instantiation process
- Make a system independent to its realization
- Class Creational use inheritance to vary the instantiated classes
- Object Creational delegate instantiation to an another object

### Structural patterns

- Deals with object's structure
- Class structural patterns concern the aggregation of classes to form the largest classes.
- Object structural patterns concerns the aggregation of objects to form the largest classes
- Class Structural patterns concern the aggregation of classes to form largest structures
- Object Structural pattern concern the aggregation of objects to form largest structures

### Behavioral patterns


© Copyright Virtual University of Pakistan

- Describe the patterns of communication between classes and objects
- How objects are communicating with each other
- Behavioral class patterns
- Behavioral object patterns
- Use object composition to distribute behavior between classes
- Help in distributing object's intelligence
- Concern with algorithms and assignment of responsibilities between objects
- Describe the patterns of communication between classes or objects
- Behavioral class pattern use inheritance to distribute behavior between classes
- Behavioral object pattern use object composition to distribute behavior between classes

In the following, one pattern from each of the above mentioned categories of design patterns is explained on GoF format.

# Lecture No. 27

## Observer Pattern

### Name

- Observer

### Basic intent

- It is intended to define a many to many relationship between objects so that when one object changes state all its dependants are notified and updated automatically.
- Dependence/publish-subscribe mechanism in programming language
    - Smalltalk being the first pure Object Oriented language in which observer pattern was used in implementing its Model View Controller (MVC) pattern. It was a publish-subscribe mechanism in which views (GUIs) were linked with their models (containers of information) through controller objects. Therefore, whenever underlying data changes in the model objects, the controller would notify the view objects to refresh themselves and vice versa.
    - MVC pattern was based on the observer pattern.

### Motivation

- It provides a common side effect of partitioning a system into a collection of cooperating classes that are in the need to maintain consistency between related objects.

### Description

- This can be used when multiple displays of state are needed.

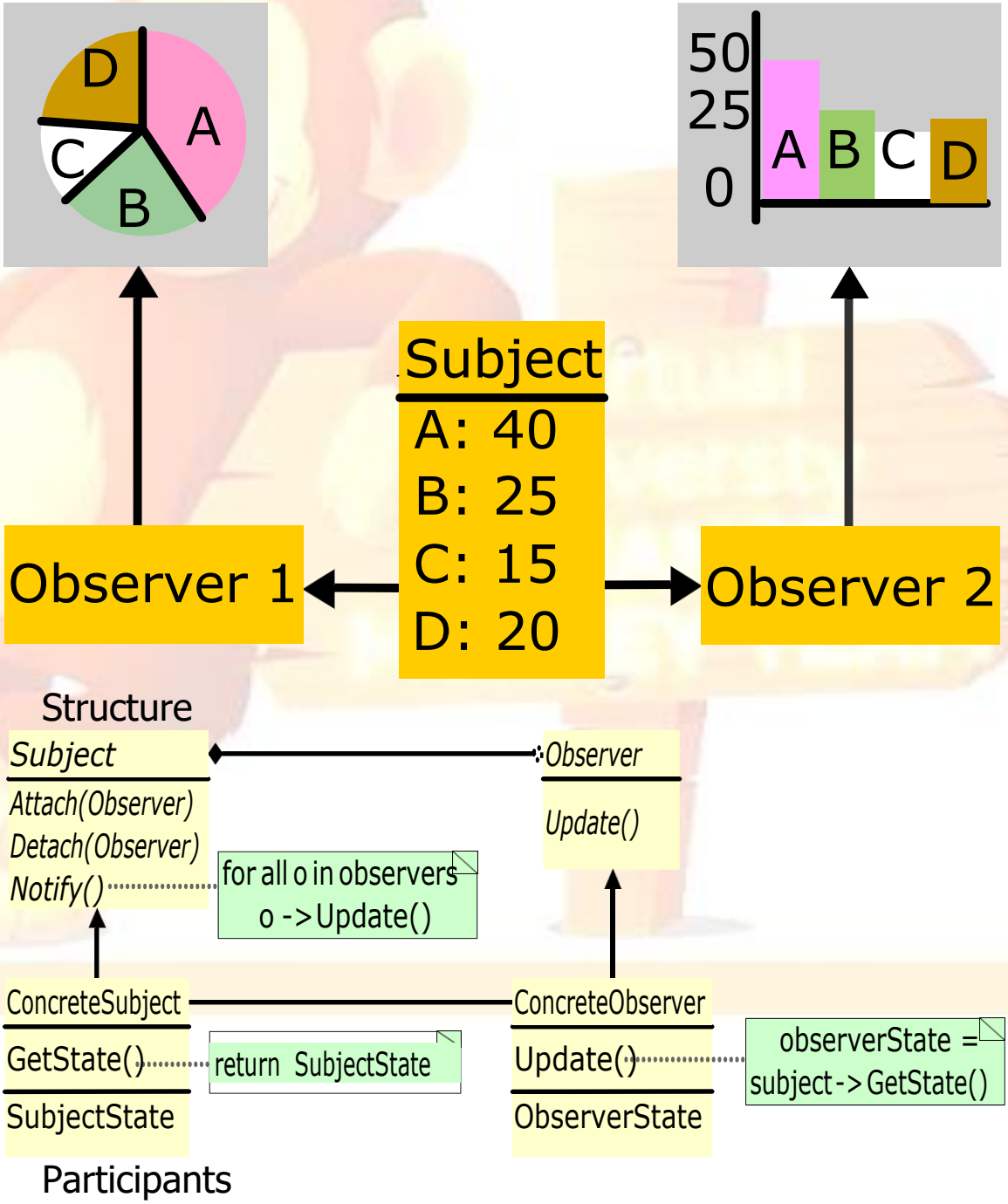
### Consequences

- **Optimizations to enhance display performance are impractical.**


© Copyright Virtual University of Pakistan

## Example implementation of Observer PatternObject Model

Many graphical user interface toolkits separate the presentational aspects of the user interface from the underlying application data. Classes defining application data and presentations can be reused independently. They can work together, too. Both a spreadsheet object and bar chart object can depict information in the same application data object using different presentations. The spreadsheet and the bar chart don't know about each other, thereby letting you reuse only the one you need. But they behave as though they do. When the user changes the information in the spreadsheet, the bar chart reflects the changes immediately, and vice versa.






© Copyright Virtual University of Pakistan

Subject

- Knows its observers. Any number of Observer objects may observe a subject.
- Provides an interface for attaching and detaching Observer objects.

Observer

- Defines an updating interface for objects that should be notified of changes in a subject.

ConcreteSubject

- Stores state of interest to concreteObserver objects.
- Sends a notification to its observers when its state changes.

ConcreteObserver

- Maintains a reference to a ConcreteSubject object.
- Stores state state that should stay consistent with the subject's.
- Implements the Observer updating interface to keep its state consistent with the subject's.

# Singleton Pattern

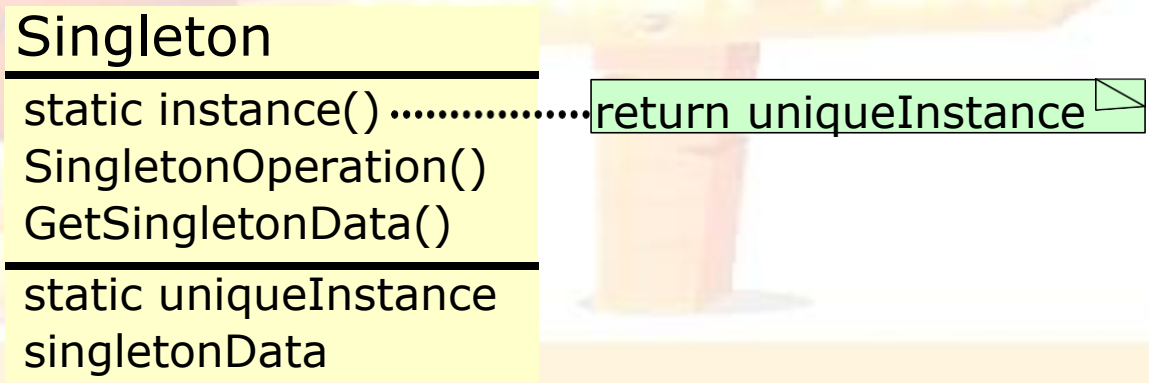
## Intent

- It ensures that a class only has one instance and provides a global point of access to it.

## Applicability

- Singleton pattern should be used when there must be exactly one instance of a class and it must be accessible to clients from a well-known access point.
- Singleton pattern should be used when controlling the total number of instances that would be created for a particular class.
- Singleton pattern should be used when the sole instance should be extensible by sub classing and clients should be able to use an extended instance without modifying their code.

## Structure




## Participants

Singleton


© Copyright Virtual University of Pakistan

- Defines an instance operation that lets clients access its unique instance. Instance is a class operation (that is, a class method in Smalltalk and a static member function in C++).
- May be responsible for creating its own unique instance.

### Singleton Pattern Example

The Singleton class is declared as

```
class Singleton {
public:
    static Singleton* Instance();
protected:
    Singleton();
private:
    static Singleton* _instance;
};
```

The corresponding implementation is

```
Singleton* Singleton::_instance = 0;

Singleton* Singleton::Instance(){
     if (_instance == 0) {
         _instance = new Singleton;
     }
     return _instance;
}
```

Clients access the singleton exclusively through the Instance member function. The variable _instance is initialized to 0, and the static member function Instance returns its value, initializing it with the unique instance if it is 0.

## Façade Pattern

### Intent

- It provides a unified interface to a set of interfaces in a sub-system.
- Façade defines a higher level interface that makes a subsystem easier to use

### Applicability

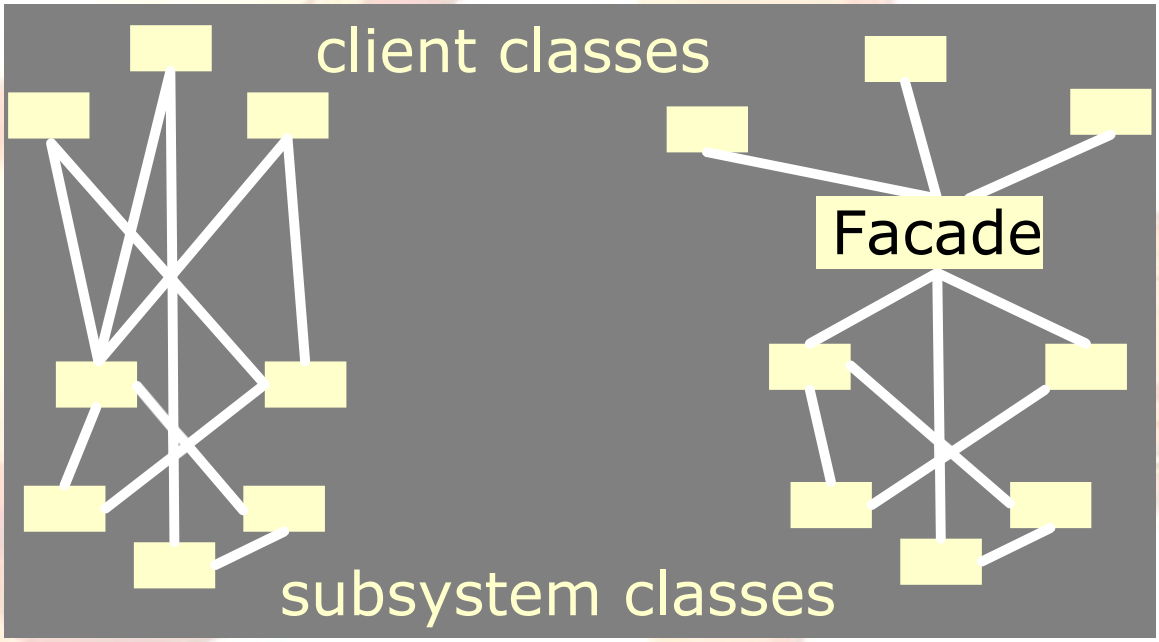
- You would use façade when you want to provide a simple interface to a complex sub-system.
- You would use façade pattern when there are many dependencies between clients and the implementation classes of an abstraction.
- You should introduce a façade to decouple the system from clients and other subsystems.
- You want to layer your subsystem.
- You would use façade when you want to provide a simple interface to a complex sub-system


© Copyright Virtual University of Pakistan

- You would use façade pattern when there are many dependencies between clients and the implementation classes of an abstraction
- You should introduce a façade to decouple the system from clients and other subsystems
- You want to layer the subsystem

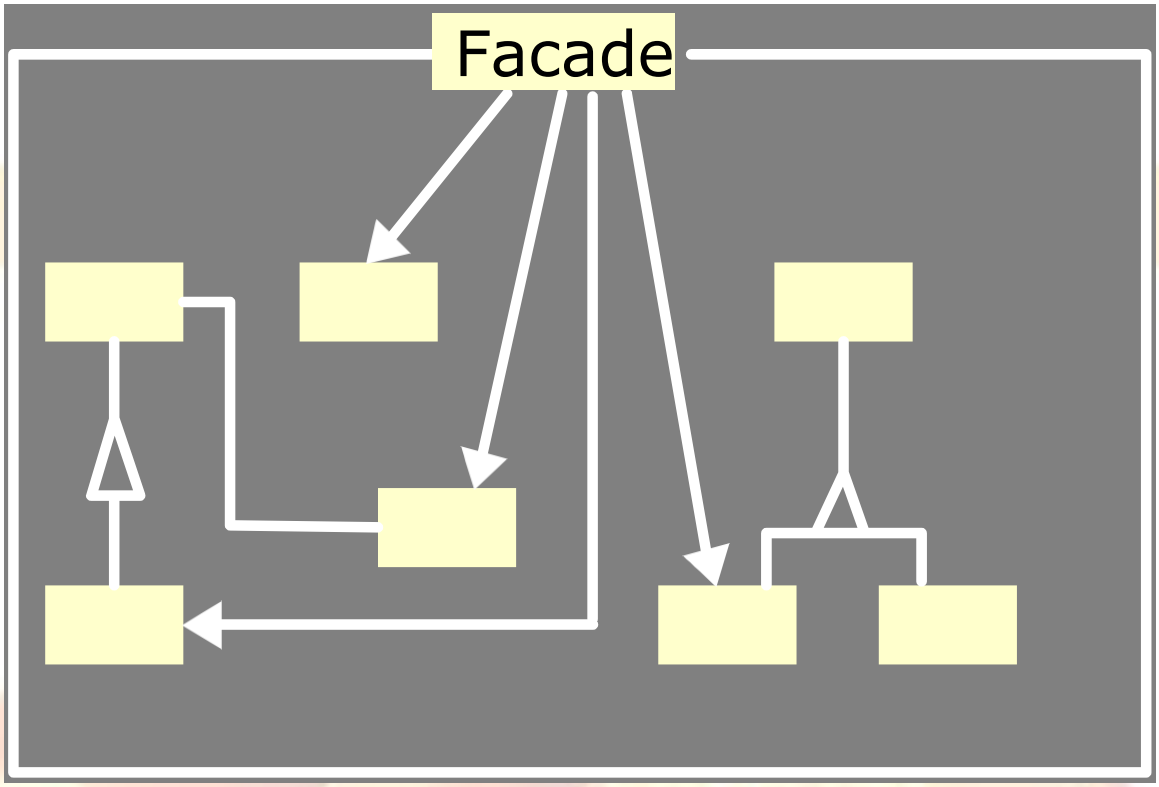
## Abstract example of façade

Structuring a system into subsystems helps reduce complexity. A common design goal is to minimize the communication and dependencies between subsystems. One way to achieve this goal is to introduce a façade object that provides a single, simplified interface to the more general facilities of a subsystem.




Structure


© Copyright Virtual University of Pakistan




## Participants

Façade

- Knows which subsystem classes are responsible for a request.
- Delegates client requests to appropriate subsystem objects.

Subsystem classes

- Implement subsystem functionality.
- Handle work assigned by the Façade object.
- Have no knowledge of the façade, that is, they keep no references to it.


© Copyright Virtual University of Pakistan

# Lecture No. 28

Good Programming Practices and Guidelines

## Maintainable Code

As we have discussed earlier, in most cases, maintainability is the most desirable quality of a software artifact. Code is no exception. Good software ought to have code that is easy to maintain. Fowler says, "*Any fool can write code that computers can understand, good programmers write code that humans can understand."* That is, it is not important to write code that works, it is important to write code that works and is easy to understand so that it can be maintained. The three basic principles that guide maintainability are: simplicity, clarity, and generality or flexibility. The software will be easy to maintain if it is easy to understand and easy to enhance. Simplicity and clarity help in making the code easier to understand while flexibility facilitates easy enhancement to the software.





© Copyright Virtual University of Pakistan

### Self Documenting Code

From a maintenance perspective, what we need is what is called ***self documenting code***. A *self documenting code* is a code that explains itself without the need of comments and extraneous documentation, like flowcharts, UML diagrams, process-flow state diagrams, etc. That is, the meaning of the code should be evident just by reading the code without having to refer to information present outside this code.

The question is: how can we write code that is self-documenting?

There are a number of attributes that contributes towards making the program self documented. These include, the size of each function, choice of variable and other identifier names, style of writing expressions, structure of programming statements, comments, modularity, and issues relating to performance and portability.

The following discussion tries to elaborate on these points.

## Function Size

The size of individual functions plays a significant role in making the program easy or difficult to understand. In general, as the function becomes longer in size, it becomes more difficult to understand. Ideally speaking, a function should not be larger than 20 lines of code and in any case should not exceed one page in length. From where did I get this number 20 and one page? The number 20 is approximately the lines of code that fit on a computer screen and one page of course refers to one printed page. The idea behind these heuristics is that when one is reading a function, one should not need to go back and forth from one screen to the other or from one page to the other and the entire context should be present on one page or on one screen.


© Copyright Virtual University of Pakistan

## Identifier Names

Identifier names also play a significant role in enhancing the readability of a program. The names should be chosen in order to make them meaningful to the reader. In order to understand the concept, let us look at the following statement.

```
if (x==0) // this is the case when we are allocating a new number
```

In this particular case, the meanings of the condition in the if-statement are not clear and we had to write a comment to explain it. This can be improved if, instead of using x, we use a more meaningful name. Our new code becomes:

```
if (AllocFlag == 0)
```

The situation has improved a little bit but the semantics of the condition are still not very clear as the meaning of 0 is not very clear. Now consider the following statement:

```
If (AllocFlag == NEW_NUMBER)
```

We have improved the quality of the code by replacing the number 0 with a named constant NEW_NUMBER. Now, the semantics are clear and do not need any extra comments, hence this piece of code is self-documenting.


© Copyright Virtual University of Pakistan

# Coding Style Guide

Consistency plays a very important role in making it self-documenting. A consistently written code is easier to understand and follow. A coding style guide is aimed at improving the coding process and to implement the concept of standardized and relatively uniform code throughout the application or project. As a number of programmers participate in developing a large piece of code, it is important that a consistent style is adopted and used by all. Therefore, each organization should develop a style guide to be adopted by its entire team.

This coding style guide emphasizes on C++ and Java but the concepts are applicable to other languages as well.

## Naming Conventions

Hungarian Notation was first discussed by Charles Simonyi of Microsoft. It is a variable naming convention that includes information about the variable in its name (such as data type, whether it is a reference variable or a constant variable, etc). Every company and programmer seems to have their own flavor of Hungarian Notation. The advantage of Hungarian notation is that by just looking at the variable name, one gets all the information needed about that variable.

*Bicapitalization* or *camel case* (frequently written *CamelCase*) is the practice of writing compound words or phrases where the terms are joined without spaces, and every term is capitalized. The name comes from a supposed resemblance between the bumpy outline of the compound word and the humps of a camel. CamelCase is now the official convention for file names and identifiers in the Java Programming Language.

In our style guide, we will be using a naming convention where Hungarian Notation is mixed with CamelCase.


© Copyright Virtual University of Pakistan

## General Naming Conventions

**General naming conventions for Java and C++**

1. Names representing types must be nouns and written in mixed case starting with upper case.

```
Line, FilePrefix
```

2. Variable names must be in mixed case starting with lower case.

```
line, filePrefix
```

This makes variables easy to distinguish from types, and effectively resolves potential naming collision as in the declaration `Line line;`

3. Names representing constants must be all uppercase using underscore to separate words.

```
MAX_ITERATIONS, COLOR_RED
```

In general, the use of such constants should be minimized. In many cases implementing the value as a method is a better choice. This form is both easier to read, and it ensures a uniform interface towards class values.

```
int    getMaxIterations()//    NOT:    MAX_ITERATIONS    =    25
{
                                return                          25;
}
```

4. Names representing methods and functions should be verbs and written in mixed case starting with lower case.

```
getName(), computeTotalWidth()
```

5. Names representing template types in C++ should be a single uppercase letter.

```
template<class                          T>                      ...
template<class C, class D> ...
```

6. Global variables in C++ should always be referred to by using the :: operator.

**::mainWindow.open() , ::applicationContext.getName()**

7. Private class variables should have _ suffix.

```
class                                                       SomeClass
{
                    private             int                  length_;
                                                                ...
}
```

Apart from its name and its type, the *scope* of a variable is its most important feature. Indicating class scope by using _ makes it easy to distinguish class variables from local scratch variables.



© Copyright Virtual University of Pakistan

8. Abbreviations and acronyms should not be uppercase when used as name.

```
exportHtmlSource();          //    NOT:    xportHTMLSource();
openDvdPlayer();       // NOT: openDVDPlayer();
```

Using all uppercase for the base name will give conflicts with the naming conventions given above. A variable of this type would have to be named dVD, hTML etc. which obviously is not very readable.

9. Generic variables should have the same name as their type.

```
void setTopic  (Topic  topic)        // NOT: void setTopic (Topic   value)
                                     // NOT: void setTopic (Topic  aTopic)
                                         // NOT: void setTopic  (Topic  x)

void  connect  (Database  database)  //  NOT:  void  connect  (Database  db)
                                  // NOT: void connect (Database oracleDB)
```

Non-generic variables have a *role*. These variables can often be named by combining role and type:

```
Point                startingPoint,                centerPoint;
Name  loginName;
```

10. All names should be written in English.

```
fileName;     // NOT:    filNavn
```

11. Variables with a large scope should have long names, variables with a small scope can have short names. Scratch variables used for temporary storage or indices are best kept short. A programmer reading such variables should be able to assume that its value is not used outside a few lines of code. Common scratch variables for integers are *i, j, k, m, n* and for characters *c* and *d*.

12. The name of the object is implicit, and should be avoided in a method name.

```
line.getLength();     // NOT:  line.getLineLength();
```

The latter seems natural in the class declaration, but proves superfluous in use.



© Copyright Virtual University of Pakistan

**Specific Naming Conventions for Java and C++**

1. The terms *get/set* must be used where an attribute is accessed directly.

```
employee.getName();
matrix.getElement (2, 4);
employee.setName (name);
matrix.setElement (2, 4, value);
```

2. *is* prefix should be used for boolean variables and methods.

```
isSet, isVisible, isFinished, isFound, isOpen
```

Using the *is* prefix solves a common problem of choosing bad boolean names like *status* or *flag*. *isStatus* or *isFlag* simply doesn't fit, and the programmer is forced to chose more meaningful names.

There are a few alternatives to the *is* prefix that fits better in some situations. These are *has*, *can* and *should* prefixes:

```
boolean hasLicense();
boolean canEvaluate();
boolean shouldAbort = false;
```

3. The term *compute* can be used in methods where something is computed.

```
valueSet.computeAverage();  matrix.computeInverse()
```

Using this term will give the reader the immediate clue that this is a potential time consuming operation, and if used repeatedly, he might consider caching the result.

4. The term *find* can be used in methods where something is looked up.

```
vertex.findNearestVertex();   matrix.findMinElement();
```

This gives the reader the immediate clue that this is a simple look up method with a minimum of computations involved.

5. The term *initialize* can be used where an object or a concept is established.

```
printer.initializeFontSet();
```

6. *List* suffix can be used on names representing a list of objects.

`vertex` (one vertex), `vertexList` (a list of vertices)

Simply using the plural form of the base class name for a list (`matrixElement` (one matrix element), `matrixElements` (list of matrix elements)) should be avoided since the two only differ in a single character and are thereby difficult to distinguish.
A *list* in this context is the compound data type that can be traversed backwards, forwards, etc. (typically a `Vector`). A plain array is simpler. The suffix *Array* can be used to denote an array of objects.

7. *n* prefix should be used for variables representing a number of objects.

```
nPoints, nLines
```


© Copyright Virtual University of Pakistan

The notation is taken from mathematics where it is an established convention for indicating a number of objects.

8. *No* suffix should be used for variables representing an entity number.

```
tableNo, employeeNo
```

The notation is taken from mathematics where it is an established convention for indicating an entity number. An elegant alternative is to prefix such variables with an *i*: `iTable, iEmployee`. This effectively makes them *named* iterators.

9. Iterator variables should be called ***i***, ***j***, ***k*** etc.

```
while (Iterator i = pointList.iterator(); i.hasNext(); ) {
  :
}

for (int i = 0; i < nTables; i++) {
  :
}
```

The notation is taken from mathematics where it is an established convention for indicating iterators. Variables named *j*, *k* etc. should be used for nested loops only.

10. Complement names must be used for complement entities.

get/set, add/remove, create/destroy, start/stop, insert/delete, increment/decrement, old/new, begin/end, first/last, up/down, min/max, next/previous, old/new, open/close, show/hide
Reduce complexity by symmetry.

11. Abbreviations in names should be avoided.

```
computeAverage();      // NOT:  compAvg();
```

There are two types of words to consider. First are the common words listed in a language dictionary. These must never be abbreviated. Never write:

| | | |
|---|---|---|
| `cmd` | instead of | `command` |
| `cp` | instead of | `copy` |
| `pt` | instead of | `point` |
| `comp` | instead of | `compute` |
| `init` | instead of | `initialize` |

etc.

Then there are domain specific phrases that are more naturally known through their acronym or abbreviations. These phrases should be kept abbreviated. Never write:

| | | |
|---|---|---|
| `HypertextMarkupLanguage` | instead of | `html` |
| `CentralProcessingUnit` | instead of | `cpu` |
| `PriceEarningRatio` | instead of | `pe` |

etc.

12. Negated boolean variable names must be avoided.

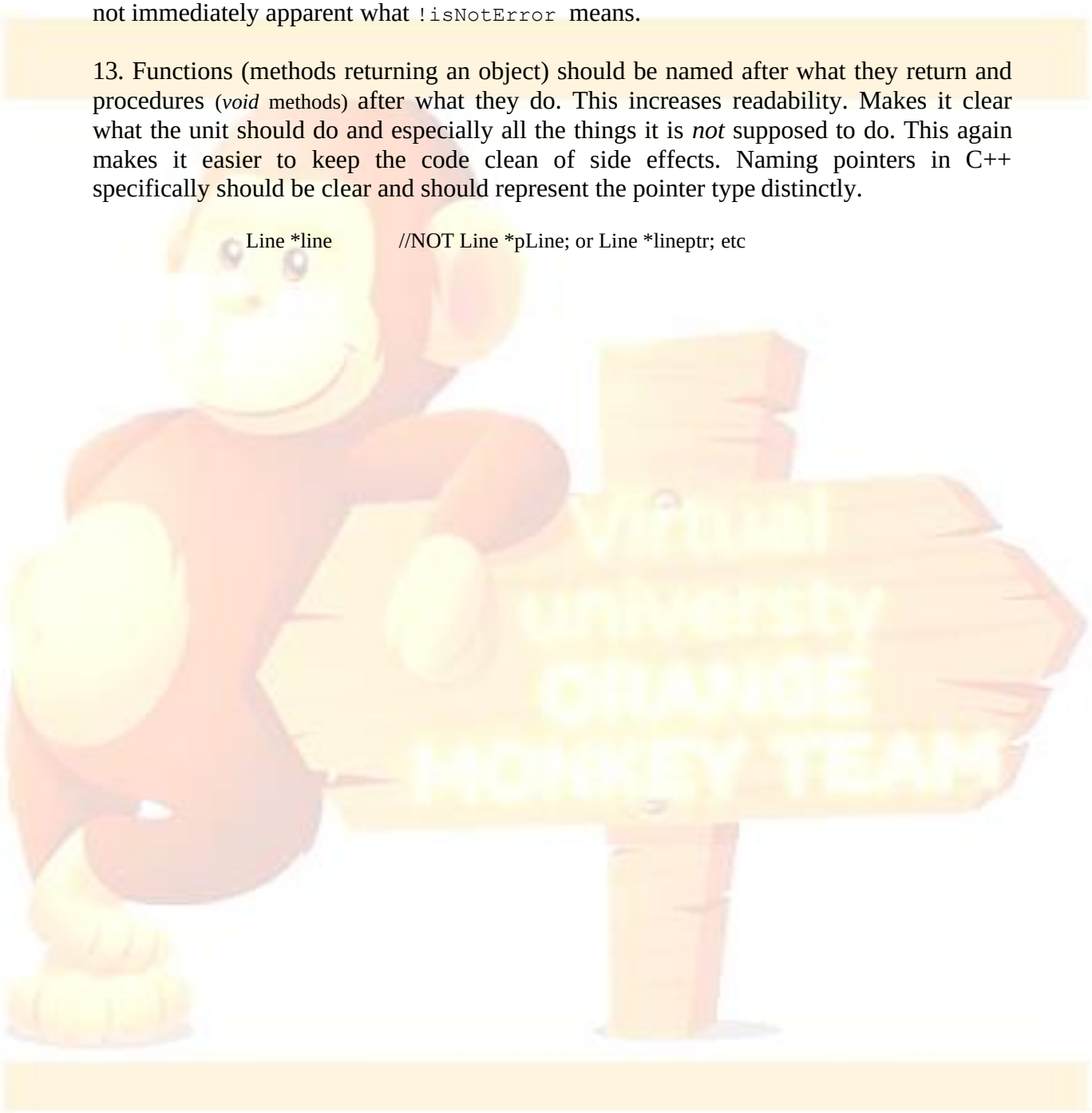

© Copyright Virtual University of Pakistan

```
boolean isError;     // NOT:   isNotError
boolean isFound;     // NOT:   isNotFound
```

The problem arise when the logical **NOT** operator is used and double negative arises. It is not immediately apparent what `!isNotError` means.

13. Functions (methods returning an object) should be named after what they return and procedures (*void* methods) after what they do. This increases readability. Makes it clear what the unit should do and especially all the things it is *not* supposed to do. This again makes it easier to keep the code clean of side effects. Naming pointers in C++ specifically should be clear and should represent the pointer type distinctly.

```
Line *line          //NOT Line *pLine; or Line *lineptr; etc
```





© Copyright Virtual University of Pakistan

# Lecture No. 29

## File handling tips for Java and C++

1. C++ header files should have the extension *.h*. Source files can have the extension *.c++* (recommended), *.C*, *.cc* or *.cpp*.

```
MyClass.c++, MyClass.h
```

These are all accepted C++ standards for file extension.

2. Classes should be declared in individual header files with the file name matching the class name. Secondary private classes can be declared as inner classes and reside in the file of the class they belong to. All definitions should reside in source files.

```
class MyClass
{
  public:
    int getValue () {return value_;} // NO!
    ...
  private:
    int value_;
}
```

The header files should declare an interface, the source file should implement it. When looking for an implementation, the programmer should always know that it is found in the source file. The obvious exception to this rule is of course inline functions that must be defined in the header file.

3. Special characters like TAB and page break must be avoided. These characters are bound to cause problem for editors, printers, terminal emulators or debuggers when used in a multi-programmer, multi-platform environment.


© Copyright Virtual University of Pakistan

4. The incompleteness of split lines must be made obvious.

```
totalSum = a + b + c +
                   d + e);
function (param1, param2,
                param3);
 setText ("Long line split"
                        "into two parts.");
for (tableNo = 0; tableNo < maxTable;
                    tableNo += tableStep)
```

Split lines occurs when a statement exceed the 80 column limit given above. It is difficult to give rigid rules for how lines should be split, but the examples above should give a general hint.

In general:

- Break after a comma.
- Break after an operator.
- Align the new line with the beginning of the expression on the previous line.





© Copyright Virtual University of Pakistan

## Include Files and Include Statements for Java and C++

1. Header files must include a construction that prevents multiple inclusion. The convention is an all uppercase construction of the module name, the file name and the h suffix.

```
#ifndef MOD_FILENAME_H
#define MOD_FILENAME_H
 :
#endif
```

The construction is to avoid compilation errors. The construction should appear in the top of the file (before the file header) so file parsing is aborted immediately and compilation time is reduced.

## Classes and Interfaces

Class and Interface declarations should be organized in the following manner:

1. Class/Interface documentation.
2. `class` or `interface` statement.
3. Class (**`static`**) variables in the order **`public`**, **`protected`**, package (no access modifier), **`private`**.
4. Instance variables in the order **`public`**, **`protected`**, package (no access modifier), `private`.
5. Constructors.
6. Methods (no specific order).


© Copyright Virtual University of Pakistan

## Statements in Java and C++

### Types

1. Type conversions must always be done explicitly. Never rely on implicit type conversion.

```
floatValue = (float) intValue;      // NOT:    floatValue = intValue;
```

By this, the programmer indicates that he is aware of the different types involved and that the mix is intentional.

2. Types that are local to one file only can be declared inside that file.

3. The parts of a class must be sorted *public*, *protected* and *private*. All sections must be identified explicitly. Not applicable sections should be left out. The ordering is *"most public first"* so people who only wish to use the class can stop reading when they reach the `protected/private` sections.

### Variables

1. Variables should be initialized where they are declared and they should be declared in the smallest scope possible.

2. Variables must never have dual meaning. This enhances readability by ensuring all concepts are represented uniquely. Reduce chance of error by side effects.

3. Class variables should never be declared public. The concept of information hiding and encapsulation is violated by public variables. Use private variables and access functions instead. One exception to this rule is when the class is essentially a data structure, with no behavior (equivalent to a C++ `struct`). In this case it is appropriate to make the class' instance variables public.

4. Related variables of the same type can be declared in a common statement. Unrelated variables should not be declared in the same statement.

```
float                 x,                 y,                 z;
float  revenueJanuary, revenueFebrury, revenueMarch;
```

The common requirement of having declarations on separate lines is not useful in the situations like the ones above. It enhances readability to group variables.

5. Variables should be kept alive for as short a time as possible. Keeping the operations on a variable within a small scope, it is easier to control the effects and side effects of the variable.


© Copyright Virtual University of Pakistan

6. Global variables should not be used. Variables should be declared only within scope of their use. Same is recommended for global functions or file scope variables. It is easier to control the effects and side effects of the variables if used in limited scope.

7. Implicit test for *0* should not be used other than for boolean variables and pointers.

```
if (nLines != 0)     // NOT:   if (nLines)
if (value != 0.0)    // NOT:   if (value)
```

It is not necessarily defined by the compiler that ints and floats 0 are implemented as binary 0. Also, by using explicit test the statement give immediate clue of the type being tested. It is common also to suggest that pointers shouldn't test implicit for 0 either, i.e. `if (line == 0)` instead of `if (line)`. The latter is regarded as such a common practice in C/C++ however that it can be used.

**Loop structures**

1. Only loop control statements must be included in the *for()* construction.

```
sum = 0;                      // NOT:     for (i=0, sum=0; i<100; i++)
for (i=0; i<100; i++)         //              sum += value[i];
  sum += value[i];
```

2. Loop variables should be initialized immediately before the loop.

```
boolean done = false;  // NOT:   boolean  done = false;
while (!done) {        //          :
  :                    //        while (!done) {
}                      //          :
                                  }
```

3. The use of *do while* loops should be avoided. There are two reasons for this. First is that the construct is superflous; Any statement that can be written as a *do while* loop can equally well be written as a *while* lopp or a *for* loop. Complexity is reduced by minimizing the number of constructs being used. The other reason is of readability. A loop with the conditional part at the end is more difficult to read than one with the conditional at the top.

4. The use of *break* and *continue* in loops should be avoided. These statements should only be used if they prove to give higher readability than their structured counterparts. In general *break* should only be used in *case* statements and *continue* should be avoided alltogether.

5. The form *for (;;)* should be used for empty loops.

```
for (;;) {             // NOT:   while (true) {
  :                    //          :
}                      //        }
```


© Copyright Virtual University of Pakistan

This form is better than the functionally equivalent *while (true)* since this implies a test against *true*, which is neither necessary nor meaningful. The form while(true) should be used for infinite loops.

**Conditionals**

1. Complex conditional expressions must be avoided. Introduce temporary boolean variables instead.

```
if ((elementNo < 0) || (elementNo > maxElement)||
    elementNo == lastElement) {
  :
}
```

should be replaced by:

```
boolean isFinished      = (elementNo < 0) || (elementNo > maxElement);
boolean isRepeatedEntry = elementNo == lastElement;
if (isFinished || isRepeatedEntry) {
  :
}
```

2. The nominal case should be put in the *if*-part and the exception in the *else*-part of an if statement.

```
boolean isError = readFile (fileName);
if (!isError) {
  :
}
else {
  :
}
```

3. The conditional should be put on a separate line.

```
if (isDone)               // NOT:  if (isDone) doCleanup();
  doCleanup();
```

4. Executable statements in conditionals must be avoided.

```
file = openFile (fileName, "w");   // NOT:    if ((file = openFile
(fileName, "w")) != null) {
if (file != null) {                //           :
  :                                //         }
}
```


© Copyright Virtual University of Pakistan

**Miscellaneous**

1. The use of magic numbers in the code should be avoided. Numbers other than *0* and *1* should be considered declared as named constants instead.

2. Floating point constants should always be written with decimal point and at least one decimal.

```
double total = 0.0;    // NOT: double total = 0;
double speed = 3.0e8; // NOT: double speed = 3e8;

double sum;
:
sum = (a + b) * 10.0;
```

This emphasizes the different nature of integer and floating point numbers even if their values might happen to be the same in a specific case. Also, as in the last example above, it emphasize the type of the assigned variable (`sum`) at a point in the code where this might not be evident.

3. Floating point constants should always be written with a digit before the decimal point.

```
double total = 0.5;    // NOT: double total = .5;
```

The number and expression system in Java is borrowed from mathematics and one should adhere to mathematical conventions for syntax wherever possible. Also, 0.5 is a lot more readable than .5; There is no way it can be mixed with the integer 5.

4. Functions in C++ must always have the return value explicitly listed.

```
int getValue()      // NOT: getValue()
{
  :
}
```

If not explicitly listed, C++ implies `int` return value for functions.

5. `goto` in C++ should not be used. Goto statements violates the idea of structured code. Only in some very few cases (for instance breaking out of deeply nested structures) should goto be considered, and only if the alternative structured counterpart is proven to be less readable.


© Copyright Virtual University of Pakistan

# Lecture No. 30

## Layout and Comments in Java and C++

### Comments

The problem with comments is that they lie. Comments are not syntax checked, there is nothing forcing them to be accurate. And so, as the code undergoes change during schedule crunches, the comments become less and less accurate.

As Fowler puts it, comments should not be used as deodorants. Tricky code should not be commented but rewritten. In general, the use of comments should be minimized by making the code self-documenting by appropriate name choices and an explicit logical structure.

If, however, there is a need to write comments for whatever reason, the following guidelines should be observed.

1. All comments should be written in English. In an international environment English is the preferred language.

2. Use // for all comments, including multi-line comments.

```
// Comment spanning
// more than one line
```

Since multilevel commenting is not supported in C++ and Java, using // comments ensure that it is always possible to comment out entire sections of a file using /* */ for debugging purposes etc.

3. Comments should be indented relative to their position in the code.

```
while (true) {          // NOT:    while (true) {
  // Do something       //         // Do something
  something();          //             something();
}                       //         }
```



© Copyright Virtual University of Pakistan

# Expressions and Statements

## Layout

1. Basic indentation should be 2.

```
for (i = 0; i < nElements; i++)
  a[i] = 0;
```

Indentation of 1 is to small to emphasize the logical layout of the code. Indentation larger than 4 makes deeply nested code difficult to read and increase the chance that the lines must be split. Choosing between indentation of 2, 3 and 4, 2 and 4 are the more common, and 2 chosen to reduce the chance of splitting code lines.

**Natural form for expression**

Expression should be written as if they are written as comments or spoken out aloud. Conditional expression with negation are always difficult to understand. As an example consider the following code:

```
if (! (block < activeBlock) || !(blockId >= unblocks))
```

The logic becomes much easier to follow if the code is written in the natural form as shown below:

```
if ((block >= activeBlock) || (blockId < unblocks))
```

## Parenthesize to remove ambiguity

Parentheses should always be used as they reduce complexity and clarify things by specifying grouping. It is especially important to use parentheses when different unrelated operators are used in the same expression as the precedence rules are often assumed by the programmers, resulting in logical errors that are very difficult to spot. As an example consider the following statement:

```
if (x & MASK == BITS)
```

This causes problems because == operator has higher precedence than & operator. Hence, MASK and BITS are first compared for equality and then the result, which is 0 or 1, is andded with x. This kind of error will be extremely hard to catch. If, however, parentheses are used, there will be no ambiguity as shown below.

```
if ((x & MASK) == BITS)
```


© Copyright Virtual University of Pakistan

Following is another example of the use of parentheses which makes the code easier to understand and hence easier to maintain.

```
leapYear = year % 4 == 0 && year % 100 != 0 || year % 400 == 0 ;
```

In this case parentheses have not been used and therefore the definition of a leap year is not very clear for the code. The code becomes self explanatory with the help of proper use of parentheses as shown below:

```
leapYear = ((year % 4 == 0) && (year % 100 != 0)) ||
                    (year % 400 == 0);
```

## Breakup complex expressions

Complex expressions should be broken down into multiple statements. An expression is considered to be complex if it uses many operators in a single statement. As an example consider the following statement:

```
*x += (*xp=(2*k < (n-m) ? c[k+1] : d[k--]));
```

This statement liberally uses a number of operators and hence is very difficult to follow and understand. If it is broken down into simple set of statements, the logic becomes easier to follow as shown below:

```
if (2*k < n-m)
        *xp = c[k+1];
else
        *xp = d[k--];
*x = *x + *xp;
```

## Shortcuts and cryptic code

Sometimes the programmers, in their creative excitement, try to write very concise code by using shortcuts and playing certain kinds of tricks. This results in a code which is cryptic in nature and hence is difficult to follow. Maintenance of such code therefore becomes a nightmare. Following are some examples of such code.

1. Let us start with a very simple shortcut, often used by programmers. Assume that we have the following statement.

```
x *= a;
```

For some reason the code was later modified to:


© Copyright Virtual University of Pakistan

```
x *= a + b;
```

This seemingly harmless change is actually a little cryptic and causes confusion. The problem lies with the semantics of this statement. Does it mean x = x*a+b or x = x*(a+b)? The second one is the right answer but is not obvious from the syntax and hence causes problems.

2. Let us now look at a more complex example. What is the following code doing?

```
subkey = subkey >> (bitoff – (bitoff >> 3) << 3));
```

As can be seen, it is pretty hard to understand and therefore difficult to debug in case there is any problem. What this code is actually doing is masking bitoff with octal 7 and then use the result to shift subkey those many time. This can be written as follows:

```
subkey = subkey >> (bitoff & 0x7);
```

It is quite evident that the second piece of code is much follow to read than the first one.

3. The following piece of code is taken from a commercial software:

```
a = a >> 2;
```

It is easy to see that a is shifted right two times. However, the real semantics of this code are hidden – the real intent here is to divide a by 4. No doubt that the code above achieves this objective but it is hard for the reader to understand the intent as to why a is being shifted right twice. It would have been much better had the code been written as follows:

```
a = a/4;
```

4. A piece of code similar to the following can be found in many data structures books and is part of circular implementation of queues using arrays.

```
bool Queue::add(int n)
{
        int k = (rear+1) % MAX_SIZE;
        if (front == k)
                return false;
        else {
                rear = k;
                queue[rear] = n;
                return true;
        }
}
```



© Copyright Virtual University of Pakistan

```
bool Queue::isFull()
{
        if (front == (rear+1 % MAX_SIZE))
                return true;
        else
                return false;
}
```

This code uses the % operator to set the rear pointer to 0 once it has reached MAX_SIZE. This is not obvious immediately. Similarly, the check for queue full is also not trivial.

In an experiment, the students were asked to implement double-ended queue in which pointer could move in both directions. Almost everyone made the mistake of writing something like rear = (rear-1) % MAX_SIZE. This is because the semantics of % operation are not obvious.

It is always much better to state the logic explicitly. Also, counting is also much easier to understand and code as compared to some tricky comparisons (e.g. check for isFull in this case). Application of both these principles resulted in the following code. It is easy to see that this code is easier to understand. It is interesting to note that when another group of students were asked to do implement double-ended queue after showing them this code, almost everyone did it without any problems.

```
bool Queue::add()
{
        if (! isFull() ) {
                rear++;
                if (rear == MAX_SIZE) rear = 0;
                QueueArray[rear] = n;
                size++;
                return true;
        }
        else return false;
}

bool Queue::isFull(int n)
{
        if (size == MAX_SIZE)
                return true;
        else
                return false;
}
```


© Copyright Virtual University of Pakistan

# Lecture No. 31

## Coding Style Guidelines (Continued)

### Switch Statement

In the switch statement, cases should always end with a break. A tricky sequence of fall-through code like the one below causes more trouble than being helpful.

```
switch(c) {
case '-' : sign = -1;
case '+' : c = getchar();
case '.' : break;
default : if (! isdigit(c))
                    return 0;
}
```

This code is cryptic and difficult to read. It is much better to explicitly write what is happening, even at the cost of duplication.

```
switch(c) {
case '-': sign = -1;
                    c = getchar();
                    break;
case '+':    c = getchar();
                    break;
case '.': break;
default:     if (! isdigit(c))
                        return 0;
                    break;
}
```

It would even be better if such code is written using the if statement as shown below.

```
if (c == '-') {
        sign = -1;
        c = getchar();
}
else if (c == '+') {
        c = getchar();
}
else if (c != '.' && !isdigit(c)) {
        return 0;
}
```



© Copyright Virtual University of Pakistan

## Magic Numbers

Consider the following code segment:

```
fac = lim / 20;
if (fac < 1)
        fac = 1;
for (i =0, col = 0; i < 27; i++, j++) {
        col += 3;
        k = 21 – (let[i] /fac);
        star = (let[i] == 0) ? ‘ ’ : ‘*’;
        for (j = k; j < 22; j++)
                draw(j, col, star);
}
draw(23, 1, ‘ ’);
for (i=‘A’; i <= ‘Z’; i++)
        cout << i;
```

Can you tell by reading the code what is meant by the numbers 20, 27, 3, 21, 22, and 23. These are constant that mean something but they do not give any indication of their importance or derivation, making the program hard to understand and modify. To a reader they work like magic and hence are called magic numbers. Any number (even 0 or 1) used in the code is a magic number. It should rather have a name of its own that can be used in the program instead of the number.

The difference would be evident if we look at the code segment below that achieves the same purpose as the code above.

```
enum {
        MINROW = 1,
        MINCOL = 1,
        MAXROW = 24,
        MAXCOL = 80,
        LABELROW = 1,
        NLET = 26,
        HEIGHT = MAXROW –4,
        WIDTH = (MAXCOL-1) / NLET
};

fac = (lim+HEIGHT-1) /HEIGHT;
if (fac < 1)
        fac = 1;
for (i =0; i < NLET; i++) {
        if (let[i] == 0)
                continue;
        for (j = HEIGHT – let[i] / fac; j < HEIGHT; j++)
                draw(j-1 + LABELROW,
                        (i+1)*WIDTH, ‘*’);
}

draw(MAXROW-1, MINCOL+1, ‘ ’);
```


© Copyright Virtual University of Pakistan

```
for (i='A'; i <= 'Z'; i++)
        cout << i;
```

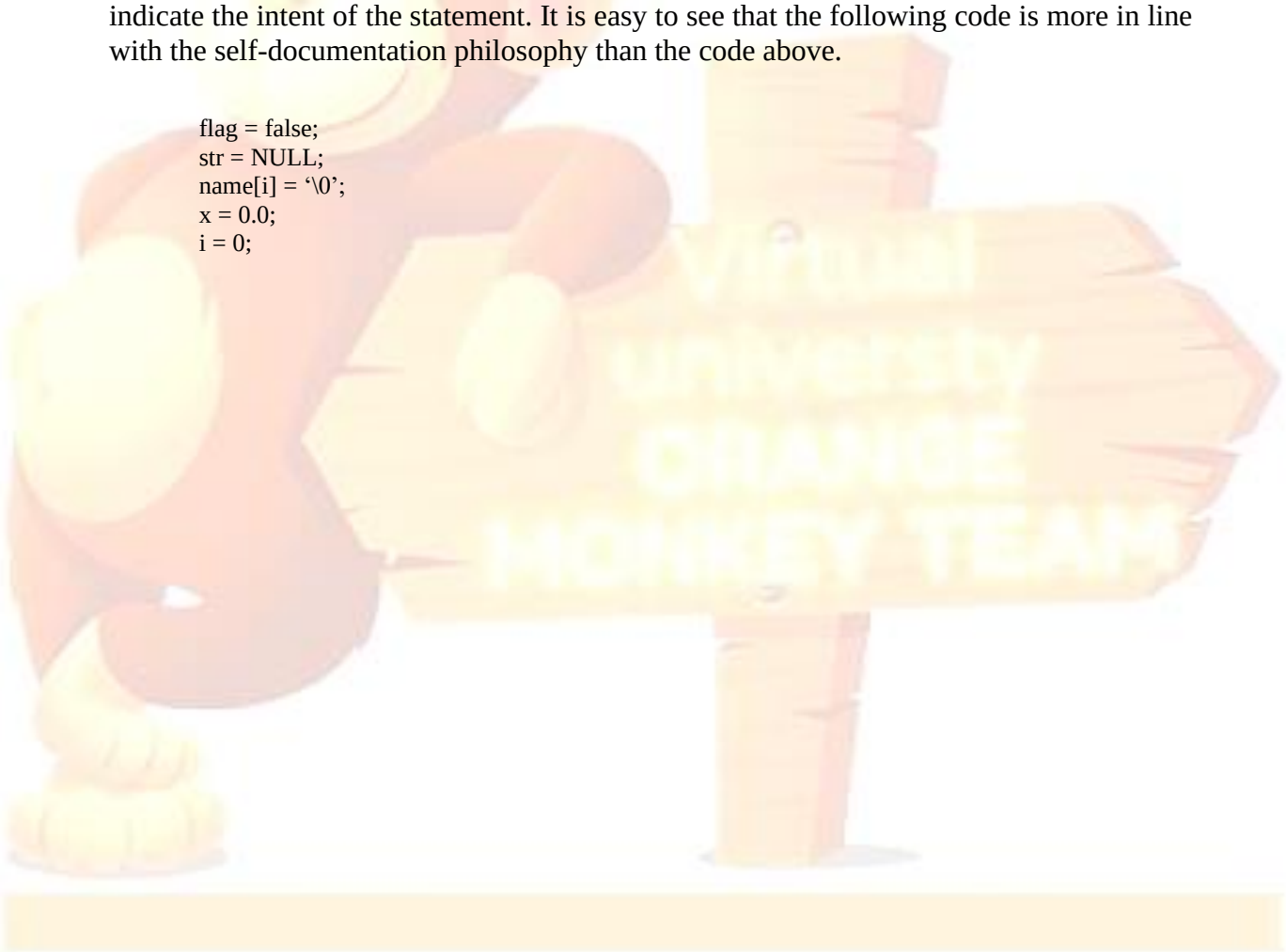
## Use (or abuse) of Zero

The number 0 is the most abused symbol in programs written in C or C++. One can easily find code segment that 0 in a fashion similar to the examples below in almost every C/C++ program.

```
flag = 0;          // flag is boolean
str = 0;           // str is string
name[i] = 0;       // name is char array
x = 0;             // x is floating pt
i = 0;             // i is integer
```

This is a legacy of old style C programming. It is much better to use symbols to explicitly indicate the intent of the statement. It is easy to see that the following code is more in line with the self-documentation philosophy than the code above.

```
flag = false;
str = NULL;
name[i] = '\0';
x = 0.0;
i = 0;
```





© Copyright Virtual University of Pakistan

# Lecture No. 32

## Clarity through modularity

As mentioned earlier, abstraction and encapsulation are two important tools that can help in managing and mastering the complexity of a program. We also discussed that the size of individual functions plays a significant role in making the program easy or difficult to understand. In general, as the function becomes longer in size, it becomes more difficult to understand. Modularity is a tool that can help us in reducing the size of individual functions, making them more readable. As an example, consider the following selection sort function.

```
void selectionSort(int a[], int size)
{
        int i, j;
        int temp;
        int min;

        for (i = 0; i < size-1; i++){
                min = i;
                for (j = i+1; j < size; j++){
                        if (a[j] < a[min])
                                min = j;
                }
                temp = a[i];
                a[i] = a[min];
                a[min] = temp;
        }
}
```

Although it is not very long but we can still improve its readability by breaking it into small functions to perform the logical steps. The modified code is written below:


© Copyright Virtual University of Pakistan

```
void swap(int &x, int &y)
{
        int  temp;
        temp = x;
        x = y;
        y = temp;
}

int minimum(int a[], int from, int to)
{
        int i;
        int min;
        min = a[from];
        for (i = from; i <= to; i++){
                if (a[i] < a[min])
                        min = i;
        }
        return min;
}

void selectionSort(int a[], int size)
{
        int min;
        int i;
        for (i = 0; i < size; i++){
                min = minimum(a, i, size –1);
                swap(a[i], a[min]);
        }
}
```

It is easy to see that the new selectionSort function is much more readable. The logical steps have been abstracted out into the two functions namely, minimum and swap. This code is not only shorter but also as a by product we now have two functions (minimum and swap) that can be reused.

Reusability is one of the prime reasons to make functions but is not the only reason. Modularity is of equal concern (if not more) and a function should be broken into smaller pieces, even if those pieces are not reused. As an example, let us consider the quickSort algorithm below.


© Copyright Virtual University of Pakistan

```
void quickSort(int a[], int left, int right)
{
        int i, j;
        int pivot;
        int temp;
        int mid = (left + right)/2;
        if (left < right){
                i = left - 1;
                j = right + 1;
                pivot = a[mid];
                do {
                        do i++; while (a[i] < pivot);
                        do j--; while (a[i] < pivot);
                        if (i<j){
                                temp = a[i];
                                a[i] = a[j];
                                a[j] = temp;
                        }
                } while (i < j);
                temp = a[left];
                a[left] = a[j];
                a[j] = temp;

                quickSort(a, left, j);
                quickSort(a, j+1, right);
}
```

This is actually a very simple algorithm but students find it very difficult to remember. If is broken in logical steps as shown below, it becomes trivial.

```
void quickSort(int a[], int left, int right)
{
        int p;
        if (left < right){
                p = partition(a, left, right);
                quickSort(a, left, p-1);
                quickSort(a, p+1, right);
        }
}


int partition(int a[], int left, int right)
{
        int i; j;
        int pivot;
        i = left + 1;
        j = right;
        pivot = a[left];
        while(i < right && a[i] < pivot) i++;
        while(j > left && a[j] >= pivot) j++;
        if(i < j)
                swap(a[i], a[j]);
        swap(a[left], a[j]);
        return j;
}
```


© Copyright Virtual University of Pakistan

## Short circuiting || and &&

The logical and operator, &&, and logical or operators, ||, are special due to the C/C++ short circuiting rule, i.e. a || b and a && b are short circuit evaluated. That is, logical expressions are evaluated left to right and evaluation stops as soon as the final truth value can be determined.

Short-circuiting is a very useful tool. It can be used where one boolean expression can be placed first to "guard" a potentially unsafe operation in a second boolean expression. Also, time is saved in evaluation of complex expressions using operators || and &&. However, a number of issues arise if proper attention is not paid.

Let us look at the following code segment taken from a commercially developed software for a large international bank:

```
struct Node {
        int data;
        Node * next;
};


Node *ptr;
...

while (ptr->data < myData && ptr != NULL){
        // do something here
}
```

What's wrong with this code?

The second part of condition, ptr != NULL, is supposed to be the guard. That is, if the value of the pointer is NULL, then the control should not enter the body of the while loop otherwise, it should check whether ptr->data < myData or not and then proceed accordingly. When the guard is misplaced, if the pointer is NULL then the program will crash because it is illegal to access a component of a non-existent object. This code is rewritten as follows. This time the short-circuiting helps in achieving the desired objective which would have been a little difficult to code without such help.

```
while (ptr != NULL && ptr->data < myData){
        // do something here
}
```


© Copyright Virtual University of Pakistan

Operand Evaluation Order and Side Effects

A side effect of a function occurs when the function, besides returning a value, changes either one of its parameters or a variable declared outside the function but is accessible to it. That is, a side effect is caused by an operation that may return an explicit result but it may also modify the values stored in other data objects. Side effects are a major source of programming errors and they make things difficult during maintenance or debugging activities. Many languages do not specify the function evaluation order in a single statement. This combined with side effects causes major problems. As an example, consider the following statement:

```
c = f1(a) + f2(b);
```

The question is, which function (f1 or f2) will be evaluated first as the C/C++ language does not specify the evaluation order and the implementer (compiler writer) is free to choose one order or the other. The question is: does it matter?

To understand this, let's look at the definition of f1 and f2.

```
int f1(int &x)
{
        x = x * 2;
        return x + 1;
}

int f2(int &y)
{
        y = y / 2;
        return y - 1;
}
```

In this case both f1 and f2 have side effects as they both are doing two things - changing the value of the parameter and changing the value at the caller side. Now if we have the following code segment,

```
a = 3;
b = 4;

c = f1(a) + f2(b);
```

then the value of a, b, and c would be as follows:

```
a = 6
b = 2
c = 8
```



© Copyright Virtual University of Pakistan

So far there seem to be any problem. But let us now consider the following statement:

```
c = f1(a) + f2(a);
```

What will be the value of a and c after this statement?

If f1 is evaluated before f2 then we have the following values:

```
a = 3
c = 9 // 7 + 2
```

On the other hand, if f2 is evaluated before f1 then, we get totally different results.

```
a = 2
c = 3 // 3 + 0
```





© Copyright Virtual University of Pakistan

# Lecture No. 33

**Common Coding mistakes**

Following is short list of common mistakes made due to side-effects.

1.
```
array[i++] = i;
```
If i is initially 3, array[3] might be set to 3 or 4.

2.
```
array[i++] = array[i++] = x;
```
Due to side effects, multiple assignments become very dangerous. In this example, a whole depends upon when i is incremented.

3.
“,” is very dangerous as it causes side effects. Let’s look at the following statement:
```
int i, j = 0;
```
Because of the syntax, many people would assume that i is also being initialized to 0, while it is not. Combination of , and = -- is fatal. Look at the following statement:
```
a = b, c = 0;
```
A majority of the programmers would assume that all a, b, and c are being initialized to 0 while only c is initialized and a and b have garbage values in them. This kind of overlook causes major programming errors which are not caught easily and are caused only because there are side effects.

**Guidelines**

If the following guidelines are observed, one can avoid hazards caused by side effects.

1. never use “,” except for declaration
2. if you are initializing a variable at the time of declaration, do not declare another variable in the same statement
3. never use multiple assignments in the same statement
4. Be very careful when you use functions with side effects – functions that change the values of the parameters.
5. Try to avoid functions that change the value of some parameters and return some value at the same time.


© Copyright Virtual University of Pakistan

## Performance

In many cases, performance and maintainability are at odds with one another. When planning for performance, one should always remember the 80/20 rule - you spend 80 percent of your time in 20 percent of the code. That is, we should not try to optimize everything. The proper approach is to profile the program and then identify bottlenecks to be optimized. This is similar to what we do in databases – we usually normalize the database to remove redundancies but then partially de-normalize if there are performance issues.

As an example, consider the following. In this example a function isspam is profiled by calling 10000 times. The results are shown in the following table:

| sec | % | Cum% | cycles | instructions | calls | function |
|---|---|---|---|---|---|---|
| | | | 990000 | 10000 | 00 | strchr |
| | | | 80000 | 6000 | 00 | strncmp |
| | | | 0000 | 0000 | 00 | strstr |
| | | | 5559 | 2213 | 35 | strlen |
| | | | 000 | 000 | | isspam |
| 11 other functions with insignificant performance overhead | | | | | | |

The profiling revealed that most of time was spent in strchr and strncmp and both of these were called from strstr.

When a small set (a couple of functions) of functions which use each other is so overwhelmingly the bottleneck, there are two alternatives:

1. use a better algorithm
2. rewrite the whole set

In this particular case strstr was rewritten and profiled again. It was and found out that although it was much faster but now 99.8% of the time was spent in strstr.

The algorithm was rewritten and restructured again by eliminating strstr, strchr, and strncmp and used memcmp. Now memcmp was much more complex than strstr but it gained efficiency by eliminating a number of loops and the new results are as shown:

| sec | % | Cum% | cycles | instructions | calls | function |
|---|---|---|---|---|---|---|
| | | | 0000 | 90000 | 000 | memcmp |
| | | | 0000 | 0000 | | isspam |
| | | | 4 | 3 | | strlen |

strlen now went from over two million calls to 652.


© Copyright Virtual University of Pakistan

Many details of the execution can be discovered by examining the numbers. The trick is to concentrate on hot spots by first identifying them and then cooling them. As mentioned earlier, most of the time is spent in loops. Therefore we need to concentrate on loops.

As an example, consider the following:

```
for (j = i; j < MAX_FIELD; j++)
   clear(j);
```

This loop clears field before each new input is read. It was observed that it was taking almost 50% of the total time. On further investigation it was found out that MAX_FIELD was 200 but the actual fields that needed to be cleared were 2 or 3 in most cases. The code was subsequently modified as shown below:

```
for (j = i; j < maxField; j++)
   clear(j);
```

This reduced the overall execution time by half.




© Copyright Virtual University of Pakistan

# Lecture No. 34

## Portability

Many applications need to be ported on to many different platforms. As we have seen, it is pretty hard to write error free, efficient, and maintainable software. So, if a major rework is required to port a program written for one environment to another, it will be probably not come at a low cost. So, we ought to find ways and means by which we can port applications to other platforms with minimum effort. The key to this lies in how we write our program. If we are careful during writing code, we can make it portable. On the other hand if we write code without portability in mind, we may end-up with a code that is extremely hard to port to other environment. Following is brief guideline that can help you in writing portable code.

**Stick to the standard**

1. Use ANSI/ISO standard C++
2. Instead of using vendor specific language extensions, use STL as much as possible

**Program in the mainstream**

Although C++ standard does not require function prototypes, one should always write them.

```
double sqrt();              // old style acceptable by ANSI C

double sqrt(double);        // ANSI – the right approach
```

**Size of data types**

Sizes of data types cause major portability issues as they vary from one machine to the other so one should be careful with them.

```
int i, j, k;

…
j = 20000;
k = 30000;

i = j + k;
```


© Copyright Virtual University of Pakistan

```
// works if int is 4 bytes
// what will happen if int is 2 bytes?
```





© Copyright Virtual University of Pakistan

**Order of Evaluation**

As mentioned earlier during the discussion of side effects, order of evaluation varies from one implementation to other. This therefore also causes portability issues. We should therefore follow guidelines mentioned in the side effect discussion.

**Signedness of char**

The language does not specify whether char is signed or unsigned.

```
char c;
        // between 0 and 255 if unsigned
        // -128 to 127 if signed

c = getchar();
if (c == EOF) ??

        // will fail if it is unsigned
```

It should therefore be written as follows:

```
int c;
c = getchar();
if (c == EOF)
```

**Arithmetic or Logical Shift**

The C/C++ language has not specified whether right shift >> is arithmetic or logical. In the arithmetic shift sign bit is copied while the logical shift fills the vacated bits with 0. This obviously reduces portability.

Interestingly, Java has introduced a new operator to handle this issue. >> is used for for arithmetic shift and >>> for logical shift.

**Byte Order and Data Exchange**

The order in which bytes of one word are stored is hardware dependent. For example in Intel architecture the lowest byte is the most significant byte while in Motorola architecture the highest byte of a word is the most significant one. This causes problem when dealing with binary data and we need to be careful while exchanging data between to heterogeneous machines. One should therefore only use text for data exchange. One should also be aware of the internationalization issues and hence should not assume ASCII as well as English.


© Copyright Virtual University of Pakistan

**Alignment**

The C/C++ language does not define the alignment of items within structures, classes, an unions. data may be aligned on word or byte boundaries. For example:

```
struct X {
        char c;
        int i;
};
```

address of i could be 2, 4, or 8 from the beginning of the structure. Therefore, using pointers and then typecasting them to access individual components will cause all sorts of problems.




© Copyright Virtual University of Pakistan

**Bit Fields**

Bit fields allow the packing of data in a structure. This is especially useful when memory or data storage is at a premium. Typical examples:

- Packing several objects into a machine word. *e.g.* 1 bit flags can be compacted -- Symbol tables in compilers.
- Reading external file formats -- non-standard file formats could be read in. *E.g.* 9 bit integers.

C lets us do this in a structure definition by putting :*bit length* after the variable. *i.e.*

```
struct packed_struct {
        unsigned int f1:1;
        unsigned int f2:1;
        unsigned int f3:1;
        unsigned int f4:1;
        unsigned int type:4;
        unsigned int funny_int:9;
} pack;
```

Here the `packed_struct contains 6 members: Four 1 bit` *flags* `f1..f3, a 4 bit type and a 9 bit funny_int.`

C automatically packs the above bit fields as compactly as possible, provided that the maximum length of the field is less than or equal to the integer word length of the computer. If this is not the case then some compilers may allow memory overlap for the fields whilst other would store the next field in the next word.

Bit fields are a convenient way to express many difficult operations. However, bit fields do suffer from a lack of portability between platforms:

- integers may be signed or unsigned
- Many compilers limit the maximum number of bits in the bit field to the size of an `integer` which may be either 16-bit or 32-bit varieties.
- Some bit field members are stored left to right others are stored right to left in memory.
- If bit fields too large, next bit field may be stored consecutively in memory (overlapping the boundary between memory locations) or in the next word of memory.

Bit fields therefore should not be used.


© Copyright Virtual University of Pakistan

# Lecture No. 35

## Exception handling

Exception handling is a powerful technique that separates error-handling code from normal code. It also provides a consistent error handling mechanism. The greatest advantage of exception handling is its ability to handle asynchronous errors.

The idea is to raise some error flag every time something goes wrong. There is a system that is always on the lookout for this error flag. Third, the previous system calls the error handling code if the error flag has been spotted. The raising of the imaginary error flag is simply called *raising* or *throwing* an error. When an error is thrown the overall system responds by *catching* the error. Surrounding a block of error-sensitive code with exception handling is called *trying* to execute a block. The following code segment illustrates the general exception handling mechanism.

```
try {
___...
___...
___throw Exception()
___...
___...
} catch( Exception e )
{
___...
___...
}
```

One of the most powerful features of exception handling is that an error can be thrown over function boundaries. This allows programmers to put the error handling code in one place, such as the *main*-function of your program.


© Copyright Virtual University of Pakistan

## Exceptions and code complexity

A number of invisible execution paths can exist in simple code in a language that allows exceptions. The complexity of a program may increase significantly if there are exceptional paths in it. Consider the following code:

```
String EvaluateSalaryAnadReturnName( Employee e)
{
  if (e.Title() == "CEO" || e.Salary() > 10000)
  {
        cout << e.First() << " " << e.Last() << " is overpaid" << endl;
  }
  return e.First() + " " + e.Last();
}
```

Before moving any further, let's take the following assumptions:

1. Different order of evaluating function parameters are ignored.
2. Failed destructors are ignored
3. Called functions are considered atomic
    a. for example: e.Title() could throw for several reasons but all that matters for this function is whether e.Title() results in an exception or not.
4. To count as different execution paths, an execution path must be made-up of a unique sequence of function calls performed and exited in the same way.

**Question**: How many more execution paths are there?

**Ans**: 23. There are 3 non-exceptional paths and 20 exceptional paths.

**The non-exceptional paths:**

The three non-exceptional paths are enumerated as below:

1. if (e.Title() == "CEO" || e.Salary() > 10000)
    - if e.Title() == "CEO" is true then the second part is not evaluated and e.Salary() will not be called.
    - cout will be performed

2. if e.Title() != "CEO" and e.Salary() > 10000
    - both parts of the condition will be evaluated
    - cout will be performed.

3. if e.Title() != "CEO" and e.Salary() <= 10000
    - both parts of the condition will be evaluated
    - cout will not be performed.


© Copyright Virtual University of Pakistan

**Exceptional Code Paths**

The 20 exceptional code path are listed below.

1. String EvaluateSalaryAnadReturnName( Employee e)

The argument is passed by value, which invokes the copy constructor. This copy operation might throw an exception.

2. if (e.Title() == “CEO” || e.Salary() > 10000)

e.Title() might itself throw, or it might return an object of class type by value, and that copy operation might throw.

3. if (e.Title() == “CEO” || e.Salary() > 10000)

Same as above.

4. if (e.Title() == “CEO” || e.Salary() > 10000)

To match a valid ==() operator, the string literal may need to be converted to a temporary object of class type and that construction of the temporary might throw.

5. if (e.Title() == “CEO” || e.Salary() > 10000)

Same as above.

6. if (e.Title() == “CEO” || e.Salary() > 10000)

operator ==() might throw.

7. if (e.Title() == “CEO” || e.Salary() > 10000)

Same as above.



© Copyright Virtual University of Pakistan

8. if (e.Title() == “CEO” || e.Salary() > 10000)

Same as above.

9-13 cout << e.First() << “ “ << e.Last() << “ is overpaid” << endl;

As per C++ standard, any of the five calls to << operator might throw.

14-15 cout << e.First() << “ “ << e.Last() << “ is overpaid” << endl

similar to 2 and 3.

16-17 return e.First() + “ “ + e.Last();

similar to 14-15.

18-19 return e.First() + “ “ + e.Last();

similar to 6,7, and 8.

20. return e.First() + “ “ + e.Last();

similar to 4.

**Summary:**

- A number of invisible execution paths can exist in simple code in a language that allows exceptions.
- Always be exception-aware. Know what code might emit exceptions.



© Copyright Virtual University of Pakistan

**The Challenge:**
Can we make this code exception safe and exception neutral? That is, rewrite it (if needed) so that it works properly in the presence of an exception and propagates all exceptions to the caller?

**Exception-Safety:**
A function is exception safe if it might throw but do not have any side effects if it does throw and any objects being used, including temporaries, are exception safe and clean-up there resources when destroyed.

**Exception Neutral:**
A function is said to be exception neutral if it propagates all exceptions to the caller.

**Levels of Exception Safety**

- Basic Guarantee: Ensures that temporaries are destroyed properly and there are no memory leaks.
- Strong Guarantee: Ensures basic guarantee as well as there is full-commit or roll-back.
- No-throw Guarantee: Ensure that a function will not throw.

Exception-safety requires either no-throw guarantee or basic and strong guarantee.

**Does the function satisfy basic guarantee?**

Yes. Since the function does not create any objects, in the presence of an exception, it

does not leak any resources.

**Does the function satisfy strong guarantee?**

No. The strong guarantee says that if the function fails because of an exception, the program state must not be change.

This function has two distinct side-effects:

- an overpaid message is emitted to cout.
- A name strings is returned.

As far as the second side-effect is concerned, the function meets the strong guarantee because if an exception occurs the value will never be returned.

As far as the first side-effect is concerned, the function is not exception safe for two reasons:

- if exception is thrown after the first part of the message has been emitted to cout but before the message has been completed (for example if the fourth << operator throws), then a partial message was emitted to cout.
- If the message emitted successfully but an exception occurs in later in the function (for example during the assembly of the return value), then a


© Copyright Virtual University of Pakistan

message was emitted to cout even though the function failed because of an exception. It should be complete commit or complete roll-back.

**Does the function satisfy no-throw guarantee?**

No. This is clearly not true as lots of operations in the function might throw.

**Strong Guarantee**

To meet strong guarantee, either both side-effects are completed or an exception is thrown and neither effect is performed.

```
// First attempt:

String EvaluateSalaryAnadReturnName( Employee e)
{
        String result = e.First() + " " + e.Last();

        if (e.Title() == "CEO" || e.Salary() > 10000)
        {

                String message = result + " is overpaid\n";
                cout << message;

        }
        return result;
}
```

What happens if the function is called as follows:

```
String theName;
theName = evaluateSalarayAndReturnName(someEmplyee);
```

1. string copy constructor is invoked because the result is returned by value.
2. The copy assignment operator is invoked to copy the result into theName
3. If either copy fails then the function has completed its side-effects (since the message was completely emitted and the return value was completely constructed) but the result has been irretrievable lost.



© Copyright Virtual University of Pakistan

Can we do better and perhaps avoid the problem by avoiding the copy?

```
// Second attempt:

String EvaluateSalaryAnadReturnName( Employee e, String &r)
{
        String result = e.First() + " " + e.Last();

        if (e.Title() == "CEO" || e.Salary() > 10000)
        {

                String message = result + " is overpaid\n";
                cout << message;

        }
        r = result;
}
```

Looks better but assignment to r might still fail which leaves us with one side-effect completed and other incomplete.

```
// Third attempt:

auto_ptr<String> EvaluateSalaryAnadReturnName( Employee e)
{
        auto_ptr<String> result = new String(e.First() + " " + e.Last());

        if (e.Title() == "CEO" || e.Salary() > 10000)
        {

                String message = (*result) + " is overpaid\n";
                cout << message;

        }
        return result;();       // rely on transfer of ownership
                                // this can't throw
}
```

We have effectively hidden all the work to construct the second side-effect (the return value), while we ensured that it can be safely returned to the caller using only non-throwing operation after the first side-effect has completed the printing of the message. In this case we know that once the function is complete, the return value will make successfully into the hands of the caller and be correctly cleaned-up in all cases. This is because the aut_ptr semantics guarantee that If the caller accepts the returned value, the act of accepting a copy of the auto_ptr causes the caller to take the ownership and if the caller does not accept the returned value, say by ignoring the


© Copyright Virtual University of Pakistan

return value, the allocated string will automatically be destroyed with proper clean-up.

**Exception Safety and Multiple Side-effects**

It is difficult and some-times impossible to provide strong exception safety when there are two or more side-effects in one function and these side-effects are not related with each other. For example we would not have been able to provide exception safety if there are two output messages, one to cout and the other one to cerr. This is because the two cannot be combined.

When such a situation comes with two or more unrelated side-effects which cannot be combined then the best way to handle such a situation is break it into two separate functions. That way, at least, the caller would know that these are two separate atomic steps.

## Summary

1. Providing the strong exception-safety guarantee often requires you to trade-off performance.
2. If a function has multiple un-related side-effects, it cannot always be made strongly exception safe. If not, it can be done only by splitting the function into several functions, each of whose side-effects can be performed atomically.
3. Not all functions need to be strongly exception-safe. Both the original code and attempt#1 satisfy the basic guarantee. For many clients, attempt # 1 is sufficient and minimizes the opportunity for side-effects to occur in the exceptional situation, without requiring the performance trade-off of attempt #3.


© Copyright Virtual University of Pakistan

# Lecture No. 36

Software Verification and Validation

## Software Testing

To understand the concept of software testing correctly, we need to understand a few related concepts.

### Software verification and validation

Verification and validation are the processes in which we check a product against its specifications and the expectations of the users who will be using it. According to a known software engineering expert Berry Boehm, verification and validation are

**Verification**

- Does the product meet system specifications?
- Have you built the product right?

**Validation**

- Does the product meet user expectations?
- Have you built the right product?

It is possible that a software application may fulfill its specifications but it may deviate from users expectations or their desired behavior. That means, software is verified but not validated. How is it possible? It is possible because during the requirements engineering phase, user needs might not have been captured precisely or the analyst might have missed a major stakeholder in the analysis. Therefore, it is important to verify as well as validate the software product.

## Defect

The second major and a very important concept is Defect. A defect is a variance from a desired product attribute. These attributes may involve system specifications well as user expectation. Anything that may cause customer dissatisfaction, is a defect. Whether these defects are in system specifications or in the software products, it is essential to point these out and fix.

Therefore software defect is that phenomenon in which software deviates from its expected behavior. This is non-compliance from the expected behavior with respect to written specifications or the stakeholder needs.

### Software and Defect

Software and defects go side by side in the software development life cycle. According to a famous saying by Haliburton, Death and taxes are inevitable. According to Kernighan: Death, taxes, and bugs are the only certainties in the life of a programmer. Software and defects cannot be separated, however, it is important to learn how discovering defects at an appropriate stage improves the software quality. Therefore, software application needs to be verified as well as validated for a successful deployment.

### Software Testing

With these concepts, we are in a position to define software testing. Software testing is the process of examining the software product against its requirements. Thus it is a process that involves verification of product with respect to its written requirements and conformance of requirements with user needs. From another perspective, software testing



© Copyright Virtual University of Pakistan

is the process of executing software product on test data and examining its output vis-à-vis the documented behavior.

**Software testing objective**

- The correct approach to testing a scientific theory is not to try to verify it, but to seek to refute the theory. That is to prove that it has errors. (Popper 1965)
- The goal of testing is to expose latent defects in a software system before it is put to use.
- A software tester tries to break the system. The objective is to show the presence of a defect not the absence of it.
- Testing cannot show the absence of a defect. It only increases your confidence in the software.
- This is because exhaustive testing of software is not possible – it is simply too expansive and needs virtually infinite resources.

**Successful Test**

From the following sayings, a successful test can be defined

"If you think your task is to find problems then you will look harder for them than if you think your task is to verify that the program has none" – Myers 1979.

"A test is said to be successful if it discovers an error" – doctor's analogy.

The success of a test depends upon the ability to discover a bug not in the ability to prove that the software does not have one. As, it is impossible to check all the different scenarios of a software application, however, we can apply techniques that can discover potential bugs from the application. Thus a test that helps in discovering a bug is a successful test. In software testing phase, our emphasis is on discovering all the major bugs that can be identified by running certain test scenarios. However it is important to keep in mind that testing activity has certain limitations.

**Limitations of testing**

With the help of the following example, we shall see how difficult it may become to discover a defect from a software application.

## Example (adapted from Backhouse)

This is a function that compares two strings of characters stored in an array for equality. The tester who has to test this function has devised the following test cases in which different combinations of inputs are tried and their expected behavior is documented. From the look of this table, it seems that almost all major combinations of inputs have been documented.

Inputs and Expected Outputs

| A | b | Expected result |
|---|---|---|
| "cat" | "dog" | False |
| "" | "" | True |
| "hen" | "hen" | True |
| "hen" | "heN" | False |
| " " | "" | False |



© Copyright Virtual University of Pakistan

| | | |
|---|---|---|
| “” | “ball” | False |
| “cat” | “” | False |
| “HEN” | “hen” | False |
| “rat” | “door” | False |
| “ ” | “ ” | True |

# Results of testing

The tester runs all the above-mentioned test cases and function returns the same results as expected. But it is still not correct. What can be a problem To analyze the problem, lets look at the code of this string equal routine

# Code of the Function

```
bool isStringsEqual(char a[], char b[])
{
   bool result;
   if (strlen(a) != strlen(b))
      result = false;
   else {
      for (int i =0; i < strlen(a); i++)
         if (a[i] == b[i])
            result = true;
         else result = false;
   }
   return result;
}
```

# Analysis of the code

**It passes all the designated tests but fails for two different strings of same length ending with the same character. For example, “cut” and “rat” would results in true which is not correct.**

**The above-mentioned defect signifies a clear limitation of the testing process in discovering a defect which is not very frequent. However, it should be noted from this example that a tester cannot generate all possible combinations of test cases to test an application as the number of scenarios may become exhaustive.**

## Testing limitations

- In order to prove that a formula or hypothesis is incorrect all you have to do to show only one example in which you prove that the formula or theorem is not working.
- On the other hand, million of examples can be developed to support the hypothesis but this will not prove that it is correct.
- These examples only help you in coming up with a hypothesis but they are not proves by themselves and they only enhance your comfort level in that particular hypothesis or in this particular case, in your piece of software.
- You cannot test a program completely because:
  - the domain of the possible inputs is too large to test.
  - there are too many possible paths through the program to test.
- According to Discrete Mathematics


© Copyright Virtual University of Pakistan

- To prove that a formula or hypothesis is incorrect you have to show only one example.
- To prove that it is correct any numbers of examples are insufficient. You have to give a formal proof of its correctness.

## Test Cases and Test Data

In order to test a software application, it is necessary to generate test cases and test data which is used in the application. Test cases correspond to application functionality such that the tester writes down steps which should be followed to achieve certain functionality. Thus a test case involves

- Input and output specification plus a statement of the function under test.
- Steps to perform the function
- Expected results that the software application produces

However, test data includes inputs that have been devised to test the system.





© Copyright Virtual University of Pakistan

# CS504-Software Engineering-I

# FINAL TERM MCQS

## Prepared by: JUNAID MALIK


# AL-JUNAID TECH INSTITUTE

www.vulmshelp.com









**Language Courses Training Available**

I'm providing paid courses in different languages within 3 Months,Certificate will be awarded after completion.

- HTML
- CSS
- JAVASCRIPT
- BOOTSTRAPS
- JQUERY
- PHP MYSQL
- NODES.JS
- REACT JS

**LMS Handling Services**

**LMS Activities Paid Task**

**Assignments 95% Results**

**Quizes 95% Results**

**GDB 95% Results**

**For CS619 Project Feel Free To Contact With Me**

**Ph# 0304-1659294**

**Email: junaidfazal08@gmail.com**

# AL-JUNAID INSTITUTE GROUP

**ALL answers are verified if found any mistake then Correct ACCORDINGLY**

1. Thin Client Model is one of the configurations of_____Model.
   a. Data-centered
   b. Client Server page130
   c. Layered
   d. Reference
2. STL Stands for_____
   **a. Standard Template Library Page 138**
   b. Standard Type Link
   c. Standard Tempo Line
   d. Standard Type Link
3. The three basic principles that guide maintainability are:
   (a) Clarity
   (b) Enhancement
   (c) Flexibility
   (d) Simplicity
   a. (a),(b) and (c)
   **b. (a),(c) and (d) page 149**
   c. (b),(c) and (d)
   d. All of above
4. Goto statements violate the idea of
   a. object oriented code
   **b. structured code page 164**
   c. control structure
   d. repetition structure
5. A function should not be larger in any case and should not exceed_____in length
   a. Half page
   **b. One page page 150**
   c. Two pages
   d. Three Pages

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

6. Data Centered Architectural Style is also called_____
   a. **Repository model page 129**
   b. Client Server model
   c. Sub System model
   d. Reference model
7. Global variables in C++ should always be referred to by using the
   a. **::operator page 153**
   b. :operator
   c. Without an operator
   d. None of the given
8. There are____layers in OSI reference model.
   a. 5
   b. 6
   c. **7 page 134**
   d. 8
9. Dynamic process model shows the process _____of theSystem.
   a. Components
   b. Objects
   c. **Structure page 129**
   d. Linkage
10. Which of the following shows a commented statement in C++
   a. # Ans = first + second
   b. **// Ans = first + second page 162**
   c. \\ Ans = first + second
   d. /# Ans = first + second
11. _____Architectural model is widely used in Mainframe application.
   a. Client-Server Model
   b. **Repository Model page 130**
   c. Filter Model
   d. Layered Model

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

12. Variables should be initialized where they are_____and they should be declared in the_____scope possible.
    a. defined and smallest
    b. declared and medium
    c. defined and medium
    **d. declared and smallest page 161**

13. Identifier names also play a significant role in enhancing the_____ of a program,
    a. Writ ability
    **b. Readability page 151**
    c. Reliability
    d. All of the given choices

15. MVC stands for_____
    **a. Model View Controller page 140**
    b. Modern View Center
    c. Model View Center
    d. Modern View Controller

16. Which of the following is not a possible server in client server environment?
    a. Database Server
    b. Transaction Server
    c. File Server
    **d. Time Server page 132**

17. In case of using unrelated operators in a single expression,____would be the best choice to prevent the logical errors.
    a. Comments
    b. Indents
    **c. Parenthesis page 166**
    d. Short Cuts

18. In case of header files, construction is to avoid___ errors, the construction should appear in the top of the file (before the file header).
    **a. Compilation page 160**
    b. run time
    c. logical
    d. All of the given

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

19. Which one of these represents the Krutchen's 4+1 architectural view model
    a. **Logical view, Process view. Physical view. Development view. Use case view**
    b. Logical view. Dynamic view. Physical view. Development view. Use case view
    c. Logical view. Process view, Physical view. Development view. Sequence view
    d. Dynamic view, Process view. Physical view, Development view. Use case view
20. Client server is a_____system model.
    a. Integrated
    **b. Distributed page 130**
    c. Heterogeneous
    d. Homogeneous
21. Vertical partitioning is also known as_____
    a. Balancing
    b. Mutating
    c. Parallelizing
    **d. Factoring page 135**
22. "is" prefix should be used for_____variables and methods.
    a. General
    **b. Boolean page 152**
    c. Constant
    d. None of the given
23. Thin client model places a heavy processing load on____
    a. Only server
    b. Only network
    **c. Both server and network page 133**
    d. Neither server not network

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

24. C++ header files should have the extension____.Source files can have the extension_____
    a. .c and .h
    b. .cc and .h
    **c. .h and.cpp page 158**
    d. All of the given
25. Which of following is among ten things, which the basic template of GOF design pattern includes.
    a. IQ
    **b. Context page 142**
    c. Performance
    d. Requirements
26. Vertical partitioning divides the architecture application from a____making perspective.
    **a. Decision page 135**
    b. Design
    c. Conclusion
    d. Move
27. Issues like system performance, availability, scalability, and security are analyzed in_____
    a. Logical View
    **b. Physical View page 127**
    c. Code View
    d. Concurrency View
28. Class variables should never be declared public. Public variables violate whichof the following:
    a. Information hiding
    b. Encapsulation
    **c. Information hiding and Encapsulation page 161**
    d. None of the give
29. Charles Simonyi first discussed Hungarian Notation, He was of_____
    **a. Microsoft page152**
    b. IBM
    c. Dell
    d. Cisco

0304-1659294 www.vulmshelp.com

30. Patterns are devices that allow programs to shareknowledge about their_____
    a. Code
    **b. Design page 137**
    c. Analysis
    d. Implementation
31. Process view in Krutchen's 4+1 architectural view model captures_____
    a. Object model of the design
    **b. Concurrency and synchronization aspects of the design page 122**
    c. Mapping(s) of the software onto the hardware and reflects its distributed aspect
    d. Static organization of the software in its development environment
32. In the N-Tire architecture , the idea is to enhance scalability and_____by disturbing both the date and the application using multiple server machines
    a. Usability
    **b. Performance page 134**
    c. Interoperability
    d. Integrity
34. Names representing methods and functions should be_____and written in mixed case starting with____case
    a. Noun__lower
    **b. Verb___lower page 153**
    c. Noun___upper
    d. Verb___upper
35. Floating point constant should always be written with decimal point and at least
    **a. One decimal page 164**
    b. Two decimal
    c. Three decimal
    d. none of the given
36. Static structural model shows the major system _____
    a. Leaks
    **b. Components page 129**
    c. Activates

0304-1659294 www.vulmshelp.com

37. Objectives Distributing the responsibilities to different subsystems so that we get a software system which is easy to maintain, is called_____ the architecture
    a. Subtracting
    **b. Partitioning page 138**
    c. Cloning
    d. Balancing
38. It ensures that a class only has one instance and provides a global point of access to it
    **a. Singleton pattern page 145**
    b. Observer pattern
    c. Real pattern
    d. None of the given
39. N-tier architecture stems from the struggle to find a___between the fat-client architecture and the thin client architecture
    a. Concurrency
    b. Distribution point
    **c. Middle ground page 134**
    d. Similarity
40. In client server model, the application is modeled as a set of that areprovided by servers.
    a. Request
    b. Protocols
    **c. Service page 130**
    d. Requirements
41. Maintainability and portability issues are discussed in _____
    a. Functional view
    b. Physical view
    **c. Code view Page 126**
    d. Logical view

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

42. In case of a file server, client request selected records from a___and the servertransmits recorded to client over the network
    a. Local memory
    b. Network
    c. Database
    **d. File Page132**
43. Logical view in Krutchen's 4+1 architectural view model captures_____.
    a. Concurrency and synchronization aspects of the design
    b. Mapping(s)of the software onto the hardware and reflect its distributedaspect
    **c. Object model of the design**
    **d. Static organization of the software in its development environment**
44. When programmers use shortcuts and cryptic codes in their program_____ becomes a major problem.
    **a. Maintenance**
    b. Reusability
    c. Availability
    d. Readability
45. Which one is correct?
    a. double total= 0.5;
    b. double total = .5;
    c. double total = .50;
    d. all of the given
46. "Description of communicating objects and classes that are customized to solvea general design problem in a particular context" is called ___.
    **a. Design Pattern**
    b. System Pattern
    c. System Design
    d. Design Method
47. Development view in Krutchen's 4+1 architectural view model captures
    a. Object model of the design
    b. Concurrency and synchronization aspects of the design

0304-1659294

c. Mapping(s) of the software onto the hardware and reflects its distributedaspect

**d. Static organization of the software in its development environment**

48. Client server model tries to_____data and processing.

**a. Distribute**

b. Merge

c. Clone

d. Proceed

49. Which indent size eliminates the chance of code lines splitting?

**a. 2**

b. 4

c. 3

d. 6

50. Which one of these is a correct short cut used by the programmer?

**a. x*=a**

b. x=*a

c. x*a=

d. =a*x

51. _____was the first pure Object Oriented language in which observer patternwas used in implementing its Model View Controller pattern

**a. Smalltalk**

b. PASCAL

c. JAVA

d. C++

52. Identify the TRUE statement

a. Portability effects Security

**b. Size of data types vary from one machine to other**

c. Size of control structures vary from one machine to other

d. None of the given options

53. 1) x = (a +2 > 3)? a : a-1;
2) if((a + 2)> 3)
x =
a;else
x = a -1;

a. Statement (1) is more complex than (2)

AL-JUNAID TECH INSTITUTE 0304-1659294 www.mshelp.com

b. Statement (2) is more complex than (1)
c. Both statements are very complex
d. None of the given options

54. which of the following statements are same in output1) a = a >> 2
2) a = a/4
3) a = a * 2
a. (1) and (3) only
b. (2) and (3) only
**c. (1) and (2) only page 168**
c. All produce the same result

55. Code should not be:
**a. Commented page 130**
b. Indented
c. aligned

56. If (!(block < active Block))is equivalent to:
a. if (block < activeBlock))
b. if ((block == activeBlock))
**c. if ((block >= activeBlock)) page 163**
d. None of the given

57. The form for (;;) should be used for
a. Nested loop
b. **Empty loop page 159**
c. More than 1000 iteration
d. Less than 1000 iterations

58. using proper parenthesis normally makes the code
a. easy to read
**b. easy to understand page 167**
c. less ambigous
d. All of the given option

59. The clients and servers, in a client server environment, are connected through _____.

AL-JUNAID TECH INSTITUTE
www.vulmshelp.com

a. Framework
b. Interface
**c. Middleware page 129**
d. Groupware

60. _____provides a unified interface to a set of intelligence in a sub-system
a. Observer pattern
b. Singleton pattern
**c. Façade pattern page 143**
c. All of the above

61. Complex expressions:
a. Make the code easy to modify
**b. Make the code difficult to modify page 164**
c. Make the code easy to understand
d. Does not affect understandability

62. Bring-in system's a view and define, from the system's perspective, the Software functionally the developers must build.
a. Functional Requirements
b. Non-Functional Requirements
c. User Requirements
d. Business Requirements

63. Which of the following is used for multi-level commenting?
a. // Comment
b. /*Comment*/
c. (Comment)
d. */Comment/*

64. Which of the following is NOT an objective for building an analysis model?
a. Define set of software requirements
b. Describe customer requirements
c. Develop an abbreviated solution for the problem
d. Establish basis for software design

65. Consider the following scenario:
"Student logins the system and checks his/her lecture schedule"
Keeping in mind the use case diagram, and above scenario the one of the post conditions might be:
a. Students should have a login

0304-1659294

b. System should be available
c. Students should log-off after checking schedule
d. Lecture schedule should be displayed date wise

66. OOA is intended to define------, their relationships, and their behavior.
a. Variables
b. Classes
c. Objects
d. Subjects

67. One of the most powerful features of exception handling is that an error can be------over function boundaries.
a. Thrown
b. Called back
c. Caught
d. Sent

68. Inspections can check conformance with a specification but not conformance with the------requirements.
a. Customer's real
b. Developer
c. Tester
d. Manger

69. For inspections, ------are prepared that contain information regarding defects.
a. Tables
b. Checklists
c. Lists
d. Farms

70. The first "bug" was actually a moth, which flew through an open window and into one of the Mark -------'s relays.
a. II
b. I
c. III
d. IV

71. Holistic medicine, concerns itself with the state of the body as a whole, not the-----that is currently attacking it.
a. Target
b. Reason
c. Disease
d. Source

AL-JUNAID TECH INSTITUTE
0304-1639294
www.vulmshelp.com

# AL-JUNAID INSTITUTE GROUP

72. What factor has no precipitation in more sophisticated and complex computer-based systems?
    a. Vast use of personal computers
    b. Vast increases in computer memory and storage capacity.
    c. Greater variety of exotic input/output options.
    d. Profound changes in computer architectures.
73. What types of errors are missed by black-box testing and can be uncovered by white-box testing?
    a. Runtime errors
    b. Logic errors
    c. Performance errors
    d. Input errors
74. When large amount of data is to be shared, repository model is used. This model has been extensively used in the--------based application.
    a. Mainframe
    b. Super Computers
    c. Personal Computers
    d. Real time
75. A (n) ----------is a variance from a desired product attribute.
    a. Error
    b. Exception
    c. Defect
    d. Mistake
76. Performance &usability are example of------requirements.
    a. Business
    b. Functional
    c. Non-Functional
    d. User
77. Camel Case is now the official convention for file names and identifiers in the-------programming language.
    a. C#
    b. C++
    c. Java
    d. Visual Basic. Net
78. Comments should be indented relative to their position in the-------.
    a. Code
    b. Design

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

c. Analysis
d. Requirements

79. One of the guidelines to avoid common mistakes is to never use-------except foe declaration.
    a. ,
    b. :
    c. =
    d. II
80. When planning for performance, one should always remember the-------rule.
    a. 80/20
    b. 70/20
    c. 60/20
    d. 100/20
81. Behavioral class pattern uses-------relationship to distribute behavior between classes.
    a. Composition
    b. Aggregation
    c. Association
    d. Inheritance
82. Software crisis appeared in-------.
    a. Early 50s
    b. Early 60s
    c. Early 70s
    d. Early 80s
83. In C++ or java, named constants are written in all uppercase letters with-----to separate words.
    a. Semicolon;
    b. Underscore_
    c. Colon:
    d. Dot.
84. Indentation larger than------makes deeply nested code difficult to read and increases the chance that the lines must be split.
    a. 2
    b. 4
    c. 3
    d. 1
85. Which phase of software development lifecycle is considered most expensive?

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

a. Requirement Gathering
b. System Design
c. Maintenance
d. Development

86. A software requirement document describes all the-----provided by the system along with the constraints under which it must operate.
a. Conditions
b. Services
c. Tasks
d. Actions

87. ------and functional specification documents are produced in requirement analysis phase.
a. Feasibility report
b. Requirement definition
c. Requirement specification
d. Design definition

88. During the program execution most of the time is spent on-------.
a. Conditional Structure
b. Loops
c. Arrays
d. Functions

89. Many applications need to be ported on to many different-------.
a. Platforms
b. Language
c. Compilers
d. Software

90. There are-------steps involved in identification of structure while deriving object model.
a. Two
b. Three
c. Four
d. Five

91. The technique which is used to separate error-handling code from normal code is called--------.
a. Exceptional handling
b. Function handling
c. Code handling

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

d. Variable handling

92. In order to show the presence of a defect, a-----breaks the system.
    a. Tester
    b. Developer
    c. Requirement engineer
    d. Designer
93. Architectural model proposed by Clements et. al. contains------views.
    a. 3
    b. 4
    c. 5
    d. 6
94. -------server provides set of applications that enable communication among clients using text, images, bulletin boards, video, etc.
    a. File
    b. Database
    c. Transaction
    d. Groupware
95. -----is result of efforts to find a middle ground between the fat client architecture and the thin-client architecture.
    a. Pipe and filter architecture
    b. N-tier architecture
    c. Three tier architecture
    d. Zero install
96. The process view captures the concurrency and-------aspects of the design.
    a. Synchronization
    b. ASynchronization
    c. Mapping
    d. Data
97. -------involve purely black box testing.
    a. Unit testing, Beta testing
    b. Acceptance testing, Interfacing
    c. Beta testing, Acceptance testing
    d. Integration testing, Interface testing
98. ------is the correct formula for calculating Cyclomatic complexity of a program.
    a. V – N +2
    b. E – V + 2

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

c. E – N + 2
d. E + N – 2

99. ---------is not included in test criteria applied in a phase of testing.
a. Functional validity
b. Interface integrity
c. Correctness
d. Programming Logic

100. The nature of software application can be characterized by their information-------.
a. Complexity
b. Content
c. Determinacy
d. Content and determinacy

101. The best way to conduct a requirements validation review is to-------.
a. Examine the system model for errors
b. Have the customer look over the requirements
c. Send them to the design team and see if they have any concerns
d. Use a checklist of questions to examine each requirements

102. Flow charts represents--------.
a. Sequence Activity
b. Random Activity
c. Parallel Activity
d. Shuffle Activity

103. A process which does not take input is called-------.
a. Miracle process
b. Core process
c. Secondary process
d. Zombie process

104. State Transition Diagram is helpful in determining--------.
a. Business understanding
b. Process flow
c. Data store
d. Control flow

105. In a----------, each program module is represented by a rectangular box.
a. Use case diagram
b. Document flow diagram
c. Class diagram

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

d. Data flow diagram

106. Which of the following is not a fundamental structured programming construct?
   a. Recursion
   b. Condition
   c. Repetition
   d. Sequence

107. Which writing style is best regarding identifier role in enhancing the readability of a program?
   a. If (Flag==0)
   b. If (Flag==START_NUMBER)
   c. If (Z==START_NUMBER)
   d. If (Z==0)

108. Exception handling is a powerful technique that separates error-handling code from--------code.
   a. Normal
   b. Abnormal
   c. Single
   d. Complex

109. Static analyzers are software tools for--------processing.
   a. Analysis text
   b. Source text
   c. Design text
   d. Maintenance text

110. Testing is an intellectually demanding activity and has a life cycle------to software development.
   a. Equal
   b. Parallel
   c. Sequential
   d. In contrast

111. --------are used to describe flow of data or control in an application.
   a. Code structures
   b. String matchings
   c. Paths
   d. Flow graphs

112. Writing test cases and generating test data are processes that demand------ building capabilities.

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

a. Code
b. Domain
c. System
d. Scenario

113. Idea of zero install architecture is to develop a system where no installation on the-----is needed.
a. Client side
b. Server side
c. Client & server side
d. Network

114. There are four basic coding structures: sequence, if statement, case statement, and---------.
a. For loop
b. While loop
c. Switch statement
d. Logical operations

115. The pattern movement became very quiet until------when patterns appeared again at OOPSLA conference.
a. 1987
b. 1988
c. 1962
d. 1995

116. In-------coverage scheme, all possible paths of a program from input instruction to the output instruction are tested.
a. Branch coverage
b. Statement coverage
c. Path coverage
d. System coverage

117. Inspections cannot check-------characteristics.
a. Non-Functional
b. Business
c. User
d. Functional

118. Split lines occur when a statement exceed the-----column limit.
a. 80
b. 90
c. 95

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

d. 85

119. STL is a library of reusable components provided by------compilers.
   a. Java
   b. Cobol
   c. Small talk
   d. C++

120. In-------, the responsibility of instantiation of an object is given to some other object.
   a. Behavioral
   b. Structural
   c. Creational
   d. Observer

121. ----------pattern is an example of creational pattern.
   a. Façade
   b. Singleton
   c. Observer
   d. Prototyping

122. ------pattern hides the implementation of the subsystem from clients, making the subsystem easier to use.
   a. Observer
   b. Fecade
   c. Creational
   d. Singleton

123. --------language allows short circuiting.
   a. Pascal
   b. C
   c. C++
   d. ADA

124. Dynamic memory allocation is done from internal memory storage called---.
   a. Cache
   b. Pool
   c. VRam
   d. Boot Memory

125. Context level diagram is also known as-------.
   a. 0-level DFD
   b. 1-level DFD
   c. 2-level DFD

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

d. 3-level DFD

126. Unit testing process is done by--------stakeholder of the software.
   a. Project manger
   b. Customer
   c. Software
   d. Developer

127. The number of paths in a program that contains loops tends to---------.
   a. Finite
   b. Infinity
   c. Limited
   d. Countable

128. UML is among different------which are used for documenting the object oriented design.
   a. Reports
   b. Documents
   c. Notations
   d. SRS

129. White box testing is performed to test the _____ of the program
   a. Logic
   **b. Structure page 198**
   c. Flow
   d. Syntax

130. Some bit field members are stored:
   - left to right
   - right to left
   - in circular array

   a. only (1) is true
   b. Only (ll) is true
   c. Only (lll) is true
   **d. e. Both (l) and (II) are true page 183**

131. In order to write a portable code which of the following guideline will not be helpful:
   a. Stick to the standards
   b. Program in the mainstream
   c. Size of data types
   d. **Using vendor specific language extensions**

132. _____is qualitative benefit of Unit Testing

AL-JUNAID TECH INSTITUTE 0304-1659294 www.aljunaidtechinstitute.com

a. Repeatability
b. **Confidence building page 207**
c. Bounded code
d. Cost effectiveness

133. In order to development life cycle. Instead of using vendor specific language extensions, use _____as much as possible
**a. STL page 179**
b. ANSI
c. ISO
d. CMM

134. In system development life cycle, defects cannot enter into the program during _____ phase.
a. Requirement
b. Design
c. Coding
d. **Implementation page 209**

135. _____and _____ are important short circuiting logical operators.
a. AND b. OR c. NOTd. NOR
a. **a & b pae 173**
b. b&c
c. c&d
d. a & c.

136. _____ testing isolates every part of the program and shows that the individual parts are correct.
a. White Box
b. Black Box
c. **Unit Google**
d. System

137. _____is not a white box testing technique.
a. Statement Coverage
b. Branch Coverage
c. Path Coverage
**d. State Transition Coverage page 202**

138. Chip level testing for hardware is equivalent to _____
a. White Box
b. Black Box
c. **Unit testing page 207**

AL JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

d. System testing

139. For equivalence partitions, we divide the problem in _____ obvious categories.
   a. **Two**
   b. Four
   c. Five
   d. Six

140. Which rewritten form of above line of code is more in line with the self-documentation philosophy than the code above?
   a. x = false
   b. x = NULL
   c. **X = 0.0; page 169**
   d. x = '\0`

141. One of the main reasons to make functions is _____
   a. Reliability
   b. **Reusability page171**
   c. Maintainability
   d. Efficiency

142. 80/20 rule states that:
   a. **you spend 80 percent of your time in 20 percent of the code page 177**
   b. you spend 20 percent of your time in 80 percent of the code
   c. We should try to optimized 80 percent or at least 20 percent of the code
   d. We should try to optimized 20 percent or at least 80 percent of the code

143. _____ identifies which statements in a method or class have been executed.
   a. Method Coverage
   b. **Statement Coverage**
   c. Class Coverage
   d. Branch Coverage

144. In _____ system's internal implementation details are not visible to the tester.
   a. White Box Testing
   b. Gray Box Testing
   c. **Black Box Testing page 198**
   d. Both White Box Testing & Gray Box Testing

0304-1659294 www.vulmshelp.com

145. Verification and validation are the processes in which we check a product against its _____ and the _____ of the users who will be using it

a. specifications arguments
b. statements, expectations
c. **specifications, expectations** **page 192**
d. statements, arguments

146. Objective of _____ is to show that the program does not work

a. Requirements
b. Design
c. Coding
d. **testing** **page 196**

147. The raising of the imaginary error flag is simply called raising or ___ an error

a. Catching
b. Casting
c. **Throwing** **page 184**
d. Repress

148. Be very careful when you use functions with side effects - functions that change the values of the _____

a. Objects
b. Classes
c. Structures
d. **Variables**

**Be very careful when you use functions with side effects – functions that change the values of the parameters. Page 176**

149. The number _____ is the most abused symbol in programs written in C or C++.

a. 1
b. 0 page169
c. 2
d. 3

150. _____ is a creative activity while_____ is a destructive activity.

a. **Development, Testing**
b. Testing, Development
c. Validation, Verification
d. Verification, Validation

151. Testing individual components independent of other components is called:

0304-1659294
www.vulmshelp.com

a. **Unit testing page 189**
b. Module testing
c. Subsystem testing
d. System testing

152. The expressions with logical operators can be evaluated only from_____
a. Right to left
b. **Left to right page173**
c. Top to bottom
d. Bottom to top

153. The complexity of a program may _____if there are exceptional paths in it
a. Decrease
b. **Increase page 185**
c. Remain same
d. Cutback

154. In the switch statement, cases should always end with a _____ statement.
a. Switch
b. Go
c. **Break page 167**
d. Stop

155. Cyclomatic complexity measures the logical complexity____
**a. Quantitatively page 205**
b. Qualitatively
c. Creditably
d. Worthily

156. The size of _____ plays a significant role in making the program easy or difficult to understand
a. **Function page 170**
b. Object
c. data type
d. none of the above

157. _____depicts programming instructions that do not have branching or any control information
a. **Sequence page 200**
b. IF
c. While
d. Case

158. for (i =0, col = 0; i < 27; i++.j++)

In the above line of the code 27 is representing _____.

a. Real Numbers
b. **Magic Numbers page 167**
c. Constant Numbers
d. Positive Numbers

159. The process that involves verification of product with respect to its written requirements is called _____
a. Maintenance
b. Debugging
c. Development
d. **Testing page 192**

160. If an application fulfills its specifications but deviates from users expectations or their desired behavior. This means, software is verified but not _____
a. **Validated page 192**
b. Corrected
c. Checked
d. Traced

161. If a function changes the value of some other accessible data object along with returning its value after execution is called _____

a. Short Circuiting
b. Modularity
c. **Side Effects page 174**
d. Abstraction

162. if (e.Title() == "CEO" || e. Salary() > 10000)
in the above statement if (e.Title()=="CEO") is TRUE then
a. **Second part is not evaluated page 185**
b. Second part is always evaluated
c. Second part must also be TRUE
d. Second part is not evaluated but e.Salary ()is called

163. When an error is thrown the overall system (on the lookout for this error flag) responds by ____the error

a. Ignoring
b. Casting
c. **Catching page 184**
d. Pass over

164. Testing of collection of modules to discover interfacing problems among interacting modules is called

a. Unit testing
b. Module testing
c. **Subsystem testing page 198**
d. System testing

165. Constant values used in the program are called
a. Real Numbers
b. **Magic Numbers page 168**
c. Constant Numbers
d. Positive Numbers

166. Bit fields are a convenient way to express many difficult operations. However, bit fields suffer from one problem
a. Lack of usability
b. Lack of security
c. Lack of performance
d. **Lack of portability page 183**

167. Comma ( ,) is very dangerous because _____
a. Compiler does not recognize this symbol
b. It creates linkage problem
c. **It causes side effects page 176**
d. It does not causes side effects

168. Function testing falls under _____testing technique
a. White Box
b. Black Box
c. **Unit page 207**
d. System

169. The greatest advantage of exception handling is its ability to handle:
a. **Asynchronous errors page 184**
b. Syntax errors
c. Memory errors
d. Control Structure errors

170. After integrating subsystems into a system, and then testing this system as a whole is called
a. Unit testing
b. Component testing
c. Subsystem testing
d. **System testing**

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

171. struct packed_struct{

unsigned int
f1:1; } pack;

Here in "packed_struct":

a. value of f1 = 1
**b. size of f1 = 1 bit page 183**
c. value of f1 should not exceede 1
d. None of given options

172. The C/C++ language has not specified whether _____ is arithmetic or logical.
**a. Right shift >> page 181**
b. Right shift <<
c. &&
d. ||

173. There are_____ basic coding structures.

a. One
b. Two
c. Three
**d. Four page 200**

174. In _____ control can take either of several branches.
a. IF
**b.** While
**c. Case statement page 201**
**d.** Sequence

175. _____ cause major portability issues
a. Loops
b. Bugs in code
**c. Sizes of data types page 179**
d. Conditional Structures

176. A good program must contain _____ infeasible paths.

**a. 0 page 206**

b. 1
c. N
d. Infinite

177. We do not use flow graphs to _____.
a. Show flow of data

0304-1639294 www.vulmshelp.com

b. **Describe decisions**
c. Show Control in an application
d. Describe different coding structures

178. The order in which bytes of one word are stored is _____ dependent
   a. **Hardware page 181**
   b. Software
   c. Language
   d. syntax

179. _____ is the starting point, base document for both testing and the development
   a. Software code
   b. Test case document
   c. Software architecture document
   d. **Functional specification document page196**

180. if a major rework is required to translate a program written for one environment to another, it means code is not or less _____
   a. Secure
   b. Bug Free
   c. platform dependent
   d. **Portable page 179**

181. Which of the following is the correct path for the code:

```
If(a%b==0)      1
C=a+b;          2
Cout<<c;        3
```

   a. 1-2-3-1
   b. 1-3-2
   **c. 1-2-3**
   d. 2-3-1

182. Switch statement is equal to _____ statement.
   a. if
   b. if-else
   c. nested if
   d. **if-else if page 167**

183. Exception handling provides
   a. Portability mechanism

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

b. Code Usability mechanism
c. **Error Handling mechanism page 184**
d. Both Portability & Code Usability mechanism

184. Unit Testing will be done by

a. Testers
b. End Users
c. Customer
d. **Developers page 207**

185. Which type of testing is not concerned with how the inputs are transformed into outputs?

a. White Box
b. Gray Box
c. **Black Box page 198**
d. Both White Box & Gray Box

186. Consider the following statement: int a,b=10;
Which of the following is correct?

a. variable "a" is initialized to 10
b. **Variable "b" is initialized to 10**
c. Both variables "a" and "b" are initialized to 10
d. variables cannot be initialized this way

187. Software _____ is the process of examining the software product against its requirements.

**a. Testing page 192**

b. Debugging
c. Exception handling
d. Fixing

188. The idea behind exception handling is to raise some error flag every time

a. The code compiles
b. The code links

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

c. Memory is allocated
d. **Something goes wrong page 184**

189. Bit fields allow the packing of data in a structure, using Bit fields we can:
a. **Read 9 bit integers page 182**
b. Avoid memory leakages
c. Avoid memory overflow
d. Avoid syntax errors

190. _____ is a tool that can help us in reducing the size of individual functions.
a. Inheritance
b. **Modularity page 170**
c. Association
d. Abstraction

191. _____ and _____ are two important tools that helps in managing the program complexity
a. Composition, Inheritance
b. **Abstraction, Encapsulation page 170**
c. Aggregation, Inheritance
d. Modularity, Composition

192. A path through a program which is never traversed for any input data is called
a. Dependent path
b. Independent path
c. **Infeasible path page 206**
d. Feasible path

193. When a small set of functions (which use each other) is so overwhelmingly the bottleneck, there are two alternatives:
a. **use a better algorithm OR re-write the code page 177**
b. debug the code OR place assertions in code
c. remove the functions OR add more functions
d. changed programming language OR compiler at least

194. **The first step in any OOA process model is to_____**
a. build an object-relationship model
b. define collaborations between objects
c. **elicit customer requirements Page603**
d. select a representation language

# AL-JUNAID INSTITUTE GROUP

195. **A necessary supplement to transform or transaction mapping needed to create a complete architectural design is _____.**
   a. entity relationship diagrams
   b. the data dictionary
   c. **processing narratives for each module Page422**
   d. test cases for each module

196. **All comments should be written in---------**
   a. **English (Page 162)**
   b. French
   c. C++
   d. JAVA

197. **Which test criteria should be applied in a phase of testing?**
   a. functional validity
   b. interface integrity
   c. correctness
   d. **all of the given Page522**

198. **The hardest single part of building a software system is deciding precisely ---------- to build.**
   a. **what (Page 17)**
   b. How
   c. When
   d. Why

199. **In sequence Diagrams objects are organized in a ----------line**
   a. **horizontal (Page 106)**
   b. vertical
   c. horizontal and vertical
   d. none of the above

200. **Every view is potentially Hierarchical in this regard which of the view are Hierarchical?**
   a. Functional View
   b. Development View
   c. C*oncurrency view*
   d. **All of given (Page 125)**

201. **Like analysis models, many different kinds of ----------models are developed**
   a. **Architectural (Page 126)**
   b. System
   c. Design
   d. Logical

AL-JUNAID TECH INSTITUTE
0304-1639294
www.vulmshelp.com

202. **Idea behind zero install architecture is to develop a system where no installation on the ---------- is needed.**
   a. **Client side (Page 130)**
   b. Server Side
   c. Client & Server Side
   d. None of the all

203. **The construction is used to avoid ---------- errors.**
   a. **compilation (Page 157)**
   b. runtime
   c. design time
   d. none of them

204. **Two tests are considered to be equivalent if it is believed that: if one discovers a defect, the other probably will too, and if one does not discover a defect, ----------**
   a. the other probably may
   b. none of them
   c. **the other probably won't either Page 199**
   d. the other probably will

205. **It was lady named ---------- who actually coin the term "bug" for the fiest time.**
   a. Elisay Chistopher
   b. **Admiral Grace Hopper Page 213**
   c. Ana Nicholson
   d. Jane Hopper

206. **Symptoms of logical errors are ----------**
   a. code is misbehaving
   b. Program doesn't crash, but the flow of program takes odd branches through the code.
   c. Results are the opposite
   d. Output looks strange,
   e. **all of the given Page 218**

207. **First-hand accounts of the problem are always useful in Debugging process.**
   a. **True Page 225**
   b. False

208. **architecture elements are further divided into categories which in total are ----------**
   a. 2
   b. **3 Page 122**
   c. 4

0304-1659294 www.vulmshelp.com

d. 5

209. **Establishing responsibilities for objects includes**
   a. Generalization Relationships
   b. Specialization Relationships
   c. all of the above
   d. **identifying Association relationships Page 101**

210. **Source files can have the extension ----------**
   a. .*c++*
   b. .*C*
   c. .cpp
   d. **all of the given Page 155**

211. **Which of the following is not a characteristic of software?**
   a. **Software is tangible**
   b. A change to a piece of code may implicitly affect the functions of the rest of the code.
   c. Software is configurable.
   d. Software does not wear and tear.

212. **Which of the following is a fact finding method?**
   a. Site visits
   b. **Prototyping**
   c. Study of similar systems
   d. All of given

   Seven common fact-finding methods (Sampling, Research, Observation, Questionnaires, Interviews, Prototyping, and Joint Requirements Planning) are introduced as a means to discover requirements.

213. **The statements given below are associated with system development, testing and maintenance. Identify the correct statement from among them:**
   a. Some of the activities in the implementation phase are building individual system components, writing of programs and development of user interfaces.
   b. The post implementation review is performed to see that the computer system is working.
   c. None of Given
   d. System development phase consists of a development phase and an implementation phase.

214. **Quantitative methods for assessing the quality of proposed architectural designs are readily available.**

a. True
b. False

215. **A decision table should be used_____**
a. to document all conditional statements
b. to guide the development of the project management plan
c. only when building an expert system
d. when a complex set of conditions and actions appears in a component

216. **Test cases should be designed long before testing begins.**
a. **True Page 467**
b. False

217. **Which of the following are characteristics of testable software?**
a. observability
b. simplicity
c. stability
d. **all of the given Page 469**

218. **Comparison testing is typically done to test two competing products as part of customer market analysis prior to product release.**
a. True
b. False

219. **By collecting software metrics and making use of existing software reliability models it is possible to develop meaningful guidelines for determining when software testing is done.**
a. True
b. False

220. **Configuration reviews are not needed if regression testing has been rigorously applied during software integration.**
a. True
b. False

221. **A change becomes ---------- because of close presence of data and fucntions**
a. **Localized Page 81**
b. Private
c. Global
d. Accessible

222. **Requirement engineering mainly deals with the ---------- of the system**
a. **definition phase Page 16**
b. development phase
c. maintenance

0304-1659294
www.vulmshelp.com

d. none of the above

223. **40-60% of all defects found in software projects can be traced back to poor ----------**

a. **Requirements Page 17**
b. Design
c. Coding
d. Testing

224. **In the N-Tire Architecture the idea is to enhance scalability and ---------- by distributing both the data and the application using multiple server machines.**

a. **Performance Page 131**
b. Efficiency
c. Usability
d. none of the all

225. **MVC pattern was based on the ---------- pattern.**

a. **Observer Page 140**
b. Structural
c. Behavioral
d. None of them

226. **Classes should be declared in individual header files with the file name matching the ---------- name.**

a. **class Page 155**
b. method
c. object
d. none of the all

227. **Unit testing is roughly equivalent to ---------- testing for hardware in which each chip is tested thoroughly after manufacturing**

a. Circuit level
b. **Chip Level Page 207**
c. component level
d. system level

228. **Software should be tested more like hardware, with Built-in self testing: such that each unit can be tested ----------**

a. Freely
b. **independently Page 207**
c. dependtly
d. completely

229. **A system ---------- is the period in which tremendous pressure is on developers end to fix the problem and make the system running again**

a. Uptime

b. **Downtime Page 214**
c. Currentime
d. Futuretimr

230. **Which one of the given below is not a symptom of memory overrun?**
a. Program crashes quite regularly after a given routine is called, that routine should be examined for a possible overrun condition.
b. If the routine in question does not appear to have any such problem the most likely cause is that another routine, called in the prior sequence, has already trashed variables or memory blocks.
c. Checking the trace log of the called routines leading up to one with the problem will often show up the error.
d. **Compiler warnings. Page 220**

231. **Three tier architecture contains ---------- layers**
a. Presentation
b. Application
c. Database
d. **All of the above Page 131**

232. **Which of the following is a/are tool used in requirement analysis?**
a. Flow Graphs
b. **Data Flow Diagrams**
c. Activity Networks
d. Module Dependency Diagrams

233. **Consider the following comment. "The software which I bought won't run on windows and when it runs I can't use WORD at the same time". Which of the following do you think are violated by the newly bought software?**
a. dependability, interchangeability
b. **platform independence, interoperability**
c. reliability, dependency
d. interoperability, reliability

234. **…… is a diagramming technique used to identify the types of objects in the system and the static relationships that exist among them?**
a. Class Diagram
b. Document flow diagrams
c. Data flow diagrams
d. **Flow charts**

235. **Which of the following interaction style best suit the design of an interface for visually impaired users?**
a. **direct manipulation Page 469**
b. direct manipulation

c. natural language
d. command line

236. **Which of the following testing involve purely black box testing?**
a. unit testing, beta testing
b. acceptance testing, interface testing
c. beta testing, acceptance testing
d. **integration testing, interface testing**

237. **What are the three generic phases of software engineering?**
a. **definition, development, support Page 96**
b. what, how, where
c. programming, debugging, maintenance
d. analysis, design, testing

238. **In the context of requirements analysis, partitioning results in the elaboration of data, function, or behavior.**
a. True
b. False

239. **In refining the DFD during transaction mapping it is unnecessary to create a PSPEC since only the CSPEC is relevant to this type of architectural style.**
a. True
b. False

240. **In transaction mapping the first level factoring results in the_____**
a. creation of a CFD
b. derivation of the control hierarchy
c. distribution of worker modules
d. refinement of the module view

241. **Variable names must be in mixed case starting with upper case.**
a. True
b. **False Page 150**

242. **Class variables should be declared public. This concept violates which of the following.**
a. Information hiding
b. Encapsulation
c. **Information hiding and Encapsulation Page 158**
d. None of given

243. **Real-time applications add a new and potentially difficult element to the testing mix**
a. performance
b. reliability

0304-1639294 www.vulmshelp.com

c. security
d. **time Page 498**

244. **Top-down integration testing has as it's major advantage(s) that**
a. low level modules never need testing
b. major decision points are tested early
c. no stubs need to be written
d. none of the given

245. **is Authorized, assess Performance, get item for UPC, get cashier for number, are the examples of**
a. **Services Page 102**
b. Attributes
c. Links
d. None of the above

246. **the *development* view describes the ----------organization of the software in its development environment,**
a. **Static Page 122**
b. Dynamic
c. Still
d. Static & Dynamic

247. **Hungarian Notation was first discussed by Charles Simonyi of----------**
a. **Microsoft. Page 149**
b. Oracle
c. Apple Macintosh
d. None of the all

248. **The language does not specify whether ---------- is signed or unsigned.**
a. **Char Page 181**
b. integer
c. double
d. constant

249. **A number of invisible execution paths can exist in simple code in a language that allows ----------**
a. **Exceptions Page 185**
b. defects
c. errors
d. all of them

250. **Bugs Fixing is done by which of the teams in Software Development lifecycle?**
a. **Development Team Page 196**
b. Testing Team

0304-1659294
www.vulmshelp.com

c. Analysis & Design Team
d. Process Team

251. **Software Bugs have multiple names, Which one of the below is not the name of Software Bugs ----------**
a. Bugs
b. Defects
c. Errors
d. **Mistakes Page 213**

252. **In the debugging process a stack trace is a very useful tool.**
a. **True Page 226**
b. False

253. **Stakeholders are different people who would be interested in the ----------**
a. **Software Page 24**
b. System
c. Product
d. All of the given

254. **Which of the following is an/are advantage(s) of object oriented analysis & design over structured system analysis & design?**
a. Ease of modeling real world events
b. Reusability
c. Maintainability
d. All of given

255. **The two main documents produced during this phase are Requirement Statement and Requirement Specification. They are also called Requirement Definition and ----------**
a. **Functional Specification Page 25**
b. Mathematical specification
c. System Specification
d. None of the given

256. **A ---------- is a code that explains itself without the need of comments and extraneous documentation**
a. **Self-documenting code Page 147**
b. Self-telling Code
c. Self-Documenting Design
d. None of the above

257. **Bugs that won't "stand still" (almost random) are the ---------- to deal with.**
a. least difficult
b. **most difficult Page 226**

0304-1659294

c. very easy
d. easy

258. **Association is a type of relation**
a. **Weak OOP , 49**
b. Stronger
c. Normal
d. None of above

259. **Bugs that won't (almost random) are the ---------- to deal with.**
a. **stand still Page 226**
b. Dynamic
c. Static
d. None of above

260. **CRUD include following:**
a. Creat
b. Update
c. Read
d. **All of above Page 53**

261. **Project ---------- defines the concept and range of the proposed solution, and limitations identify certain capabilities that the product will not include**
a. **Scope Page 30**
b. Agreement
c. Plan
d. None of the given

262. **The cyclomatic complexity metric provides the designer with information regarding the number of**
a. Cycles in the program
b. Errors in the program
c. **Independent logic paths in the program**
d. Statements in the program

263. **Which of the followings is not a testing type?**
a. Subsystem testing
b. Alpha testing
c. Beta Testing
d. **Gamma Testing Page 198**

264. **Which of the items listed below is not one of the software engineering layers?**
a. Process
b. **Manufacturing**
c. Methods

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

d. Tools

265. **Which piece of code is self-documented?**
   a. if (x==0) // this is the case when we are allocating a new number
   b. if (AllocFlag == 0)
   c. **If (AllocFlag == NEW_NUMBER) Page 148**
   d. None of the given

266. **UML (unified modeling language) analysis modeling focuses on the _____**
   a. Behavioral model and environment model.
   b. **B**ehavioral model and implementation model.
   c. user model and environmental model
   d. **user model and structural model Page 604**

267. **What makes requirements elicitation difficult?**
   a. bounding scope
   b. understanding user needs
   c. requirements volatility
   d. **all of the above Page 285**

268. **In the architecture trade-off analysis method the architectural style should be described using the**
   a. module view
   b. process view
   c. **all of the given Page 403**
   d. data flow view

269. **Which of these are objectives for software testing?**
   a. determine the productivity of programmers
   b. eliminate the need for future program maintenance
   c. eliminate every error prior to release
   d. **uncover software errors Page 467**

270. **In software quality assurance work there is no difference between software verification and software validation**
   a. True
   b. False

271. **Top-down integration testing has as it's major advantage(s) that**
   a. low level modules never need testing
   b. major decision points are tested early
   c. no stubs need to be written
   d. none of the given

272. **Rooko mut jane do depicts one of risks associated with inadequate requriements gathering, the name of that risk is ----------**
   a. **Ambiguous requirements Page 20**
   b. Creeping Requirements
   c. Minimal Specification
   d. None of the above

273. **Interaction Diagrams depict the ---------- Behavior of the system**
   a. Static
   b. **Dynamic Page 106**
   c. Active
   d. None of the above

274. **In Sequence Diagrams events are organized in a----------time line**
   a. Vertical
   b. **Horizontal Page 106**
   c. Vertical & Horizontal
   d. None of the above

275. **object-oriented framework that is a set of cooperative classes that make up reusable _____of a system**
   a. Code
   b. **Design Page 138**
   c. Analysis
   d. none of the all

276. **In order to make a code more portable, Instead of using vendor specific language extensions, use ---------- as much as possible**
   a. **STL Page 179**
   b. ANSI
   c. ISO
   d. CMMI

277. **Performance & Usability are examples of ---------- requirements**
   a. Business
   b. Functioanl
   c. **Non-Functioanl Page 39**
   d. User

278. **According to a survey, when a software application is in the maintenance phase, --------- of its lifecycle cost is attributed towards the defects which are found in the software application after installation**
   a. **20 % Page 214)**
   b. 25 %

0304-1659294
www.vulmshelp.com

c. 30%
d. 35%

279. **Bugs that won't "----------" (almost random) are the most difficult to deal with.**
a. **stand still** **Page 226**
b. dynamic
c. running
d. stand tall

280. **If you are initializing a variable at the time of declaration, do not declare another _____ in the same statement.**
a. **variable** **Page 176**
b. object
c. class
d. None of given

281. **During Requirement Engineering process which of the following is/are document(s) used for fact finding?**
a. Company's employee list
b. Samples of the company databases
c. The company's mission statement and plan
d. **All of given**

282. **Object-oriented domain analysis is concerned with the identification and specification of reusable capabilities within an application domain.**
a. True
b. False

283. Fourth generation techniques
a. Allow software to be developed without any testing.
b. Eliminate the need for costly requirements gathering activities.
c. Can reduce the time required to develop software. **Page 73**
d. Are best used by non-programmers to build small systems

284. Data design actually begins during the creation of the analysis model, not the architectural model.
a. **True** **page397**
b. False

285. The states shown in a state transition diagram do not necessarily correspond to the processes shown in a control flow diagram for the same system.
a. **True** **page397**
b. False

286. The criteria used to assess the quality of an architectural design should be based on system
a. **accessibility and reliability** **page404**

b. data and control
c. functionality
d. implementation details

287. **A useful technique for evaluating the overall complexity of a proposed architecture is to look at the component**
a. number and size of components
b. **flow dependencies and sharing dependencies Page 406**
c. size and cost
d. none of the given

288. The three basic principles that guide maintainability are: simplicity, clarity, and _____.
a. **Generality Page 146**
b. Reliability
c. All of the given choices

289. **In order to make a program self-documented a number of attributes required. Which one is the attribute/s of self-documented program**
a. **All of the given choices Page 147**
b. Size of each function
c. Choice of variable

290. Floating point constants should always be written with decimal point and at least
a. **one decimal Page 161**
b. two decimal
c. three decimal

291. **The code becomes self-explanatory with the help of proper use of parentheses. Select the right one.**
a. leapYear = year % 4 == 0 && year % 100 != 0 || year % 400 == 0;
b. leapYear = ((year % 4 == 0) && (year % 100 != 0) || ((year % 400 == 0));
c. **leapYear = ((year % 4 == 0) && (year % 100 != 0)) || (year % 400 == 0); Page 164**
d. leapYear = (year % 4 == 0) && (year % 100 != 0) || (year % 400 == 0);

292. **Graph-based testing methods can only be used for object-oriented system**
a. True
b. False

293. **A class is a ---------- of objects**
a. Template page85
b. Stereotype
c. Collection
d. None of above

294. A public Interface provides a way for with other Classes.
   a. Communication
   b. Accessibility
   c. Reaching
   d. All of the above

295. **Software architecture defines the high level structure of the software by putting together a number of architectural ---------- in an organized fashion.**
   a. **Elements Page 122**
   b. **Parts**
   c. **Components**
   d. **None of all**

296. **Patterns are devices that allow programs to share knowledge about their ----------.**
   a. **Design Page 137**
   b. Code
   c. Analysis
   d. None of all

297. **Type conversions must always be done ----------**
   a. **Explicitly Page 158**
   b. Implicitly
   c. Simultaneously
   d. None of them

298. **Types that are ----------to one file only can be declared inside that file**
   a. Local page158
   b. Global
   c. Private
   d. General

299. **The use of do while loops should be ----------**
   a. Avoided page159
   b. Encouraged
   c. Practiced
   d. None of them

300. **One of the causes of the portability issues is the order of ---------- varies from one implementation to other.**
   a. **Evaluation page181**
   b. **Numbers**
   c. **Variables**
   d. Symbols

301. **The goal of testing is to expose ---------- defects in a software system before it is put to use.**

0304-1659294 www.vulmshelp.com

a. **Latent** **page193**
b. **Already present**
c. **Current**
d. **Runtime**

302. In the N-tier architecture, the idea is to enhance scalability and ------ by distributing both the data and the application using multiple server machines.
a. Usability
**b. Performance** **page # 134**
c. Interoperability
d. None

303. In case of a file servers, client requests selected records from a ….. and the server transmits records to client over the network.
a. Local memory
b. Network
c. Database
**d. File** **page #132**

304. Alpha testing is:
a. Testing individual components independent of other components
b. Testing a collection of dependent components
c. Validation against user expectations
d. **Acceptance testing for customized projects, in-house testing for products page#201**

305. This code uses the % operator to set the rear pointer to 0 once it has reached
a. Min-Size
**b. Max-Size** **Page 169**
c. Both
d. None

306. Which of these represents the Krutchen's 4+1 architectural view model?
a. Logical view, Process view, Physical view, Development view, Use case view
b. Logical view, Dynamic view, Physical view, Development view, User case view
c. Logical view, Process view, Physical view, Development view, Sequence view
d. Dynamic view, Process view, Physical view, Development view, Use case view

307. Vertical participation divides the architecture application from a making perspective.
a. Decision
b. Design
c. Conclusion
d. Move

308. Anti- Patterns is another concept that corresponds to common in analysis and design.
a. Mistake
b. Issues
c. Problems
d. All of the given

309. block of error-sensitive code with exception handling is called
a. Trying to execute a block

310. Which function throw for several reasons but all that matters for this function is whethere

a. Title ()

311. The best reason for using Independent software test teams is that
**a. Software developers do not need to do any testing**
b. Strangers will test the software mercilessly
c. Testers do not get involved with the project until testing begins
d. The conflicts of interest between developers and testers is reduced

312. _____ is intended to define a many to many relationship between objects so that when
a. One object changes state all its dependents are notified and updated automatically.
**b. Observer Pattern**
c. Facade Pattern
d. Singleton Pattern
e. Joint Pattern

313. Secondary private classes can be declared as _____ and reside in the file of the class
a. Asynchronous Classes
**b. Inner classes PAGE158**
c. Outer classes
d. Synchronous classes

314. The construction should appear at the _____ of the header file.
**a. Top PAGE 160**
b. Bottom
c. Left
d. Right

315. Modularity is a tool that can help us in _____ the size of individual functions.
a. Stabilizing
**b. Reducing PAGE 173**
c. Increasing
d. Strengthening

316. Abstraction and encapsulation are two important tools that can help in managing and mastering the _____ of a program.
a. Usability
**b. Complexity Page173**
c. Understandability
d. Reliability

317. Which of the following is/are NOT one of the umbrella activities?
a. Requirement analysis
b. Architecture design
c. Test case development
**d. All of the given options Page 14**

318. Which one is NOT a type of messages which Sequence Diagrams Depict _____
a. Asynchronous
b. Synchronous
c. Create
**d. Update Page 111**

0304-1659294 www.vulmshelp.com

319. Identifying Whole-Part structures (Aggregations) means, what are my ----------
   **a. Components PAGE 98**
   b. Structures
   c. Modules
   d. Interaction Protocols
320. The design process for identifying the sub-systems making up a system and the Framework for sub-system control and communication is:
   **a. Architectural Design PAGE 118**
   b. Interface Design
   c. Component Design
   d. Data Design
321. A complex System evolves from a
   a. Smaller system
   **b. Simpler system (GOOG)**
   c. Bigger system
   d. Medium system
322. A context diagram _____
   a. Describes the context dependencies of a system
   **b. Is a DFD which gives an overview of the system (Conceptual)**
   c. Is a detailed description of a system
   d. Is not used in drawing a detailed DFD
323. When measure of independence of a module or component is low to the other, How would changes in one component have effect on other component?
   a. No
   b. Low
   **c. High (Conceptual)**
   d. Equal
324. What is meant by the term 'software crisis'?
   a. Ability to deliver software on time with good quality
   b. A situation in which experienced developers leave the company
   **c. Inability of new software to inter-operate with existing software (Conceptual)**
   d. A situation in which large scale software projects took more time and budget than was planned
325. Equivalence _____ help you in designing test cases to test the system effectively and efficiently.
   a. Functions
   **b. Classes PAGE 202**
   c. Objects
   d. Interfaces
326. _____ requirements are often called product features.
   a. Functional
   **b. Business**
   c. User
   d. Non-functional
327. Many compilers limit the maximum number of bits in the bit field to the size of a(n)

AJ TECH INSTITUTE 0301 1659291 www.vulmshelp.com

_____.

**a. Integer PAGE186**

b. Float

c. Character

d. Double

328. In Java, ">>" is used for _____ shift and ">>>" for _____ shift.

**a. Arithmetic, Logical PAGE 184**

b. Mathematical, Logical

c. Incremental, Arithmetic

d. Logical, Arithmetic

329. _____ is only a reference for defining protocols and designing and implementing systems developed by different parties.

a. OSI model

**b. Reference architecture PAGE 137**

c. Layered architecture

d. N-tier architecture

330. Testing activities require destructive instincts in _____ for the purpose of breaking system to discover loopholes into its functionality.

a. Bug Fixers

**b. Tester PAGE 198**

c. Developer

d. Requirement Engineer

331. In presence of _____ bug in a program, the results are the opposite of what is expected.

a. Memory Leak

b. Memory Over-runs

c. Syntax Error

**d. Logical Error Page 221**

332. Which of the following is a non-functional requirement of a website which sells songs?

a. A catalogue of the stock needs to be available for the users to choose from

b. Customer information should be retained to allow future transactions easier

c. Users should be able to choose from a set of different languages for the interface

**d. Time taken to download songs in the catalogue should not irritate users (CONCEPTUAL)**

333. The state transition diagram _____

a. Depicts relationships between data objects

b. Depicts functions that transform the data flow

**c. Indicates how data are transformed by the system (CONCEPTUAL)**

d. Indicates system reactions to external events

334. Comments are not syntax checked

a. TRUE PAGE 165

b. FALSE

335. Modularity is a tool that can help us in increasing the size of individual functions, making them less readable.

a. True

b. False PAGE 173

336. Modules with high cohesion and low coupling can be treated and analysed as
   a. White boxes
   b. black boxes Page 75
   c. grey boxes
   d. none of these
337. While establishing the services for an object, the following fundamental questions should be asked:
   a. Why does the system need this object anyway?
   b. What useful questions can it answer?
   c. What useful action can it perform?
   d. All of the given options.
338. ----------is a role that each actor plays in the system under consideration.
   a. An act
   b. A participant PAGE 96
   c. A function
   d. None of the given
339. Any Engineering approach must be founded on organizational commitment to ---------
   a. Cost
   b. Scheduling
   c. Quality Page 115
   d. Performance
340. Return values in synchronous massages are:
   a. Compulsory
   b. May not used when response is obvious
   c. Not used at all
   d. Represented by solid lines
341. According to Caper Jhones analysis of project activities, coding only has ------- affect part in system development.
   a. 13-14%
   b. 36-40%
   c. 50-60%
   d. 70-80%
342. Following are some statements associated with data flow diagrams. Identify the correct statement from among them.
   a. DFDs are used to model complex interfaces.
   b. DFDs are used to represent only functional processing, data stores and data movements between functions.
   c. DFDs depict only processes which can be decomposed.
   d. DFDs do not show external data sources and external data sinks.
343. A project is considered successful if:
   a. The system was delivered in time and within budget.
   b. The system meets at least some of the customer's requirements.
   c. The system development process has a maximum impact on the ongoing business process.
   d. Minimum time was spent for requirement gathering and designing.

344. A memory leak bug is one in which memory is somehow allocated from either the operating system or an ---------- "pool", but never deallocated when the memory is finished being used
   a. Mixed memory
   b. External memory
   c. Internal Memory Page 219
   d. Mutually exclusive memory

345. Control flow diagrams are:
   a. Needed to model event driven systems
   b. Required for all systems
   c. Used in place of data flow diagrams
   d. Useful for modeling user interfaces

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

**FINALTERM EXAMINATION**
**Spring 2010**
**CS504- Software Engineering - I (Session - 2)**

**Question No: 1 ( Marks: 1 ) - Please choose one**
**The first step in any OOA process model is to_____**

- ► build an object-relationship model
- ► define collaborations between objects
- ► **elicit customer requirements** **(A PRACTITIONER'S APPROACH Page603)**
- ► select a representation language

**Question No: 2 ( Marks: 1 ) - Please choose one**
**A necessary supplement to transform or transaction mapping needed to create a complete architectural design is _____.**

- ► entity relationship diagrams
- ► the data dictionary
- ► **processing narratives for each module** **(A PRACTITIONER'S APPROACH Page422)**
- ► test cases for each module

**Question No: 3 ( Marks: 1 ) - Please choose one**
**What types of errors are missed by black-box testing and can be uncovered by white-box testing?**

- ► behavioral errors
- ► **subtle logic errors** **Click here for detail**
- ► performance errors
- ► input error

**Question No: 4 ( Marks: 1 ) - Please choose one**
**Which test criteria should be applied in a phase of testing?**

- ► functional validity
- ► interface integrity
- ► correctness
- ► **all of the given** **(A PRACTITIONER'S APPROACH Page522)**

**Question No: 5 ( Marks: 1 ) - Please choose one**
**The hardest single part of building a software system is deciding precisely ---------- to build.**

- ► **what (Page 17)**
- ► How
- ► When
- ► Why

**Question No: 6 ( Marks: 1 ) - Please choose one**
**In sequence Diagrams objects are organized in a ----------line**

- ► **horizontal (Page 106)**
- ► vertical
- ► horizontal and vertical
- ► non of the above

**Question No: 7 ( Marks: 1 ) - Please choose one**
**Every view is potentially Hierarchical in this regard which of the view are Hierarchical ?**

- ► Functioanl View
- ► Development View
- ► *Concurrency view*
- ► **All of given (Page 125)**

**Question No: 8 ( Marks: 1 ) - Please choose one**
**Like analysis models, many different kinds of ----------models are developed**

- ► **Architectural (Page 126)**
- ► System
- ► Design
- ► Logical

**Question No: 9 ( Marks: 1 ) - Please choose one**
**Idea behind zero install architecture is to develop a system where no installation on the ---------- is needed.**

- ► **Client side (Page 130)**
- ► Server Side
- ► Client & Server Side
- ► Non of the all

**Question No: 10 ( Marks: 1 ) - Please choose one**
**The construction is used to avoid ---------- errors.**

- ► **compilation (Page 157)**
- ► runtime
- ► design time
- ► non of them

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 11 ( Marks: 1 ) - Please choose one**
**All comments should be written in----------**

- **English (Page 162)**
- French
- C++
- JAVA

**Question No: 12 ( Marks: 1 ) - Please choose one**
**One of the Guidelines to avoid common mistakes is to never use --------- except for declaration**

- **, (Page 176)**
- ;
- =
- ||

**Question No: 13 ( Marks: 1 ) - Please choose one**
**The order in which bytes of one word are stored is ---------- dependent**

- **Hardware (Page 181)**
- Software
- language
- system

**Question No: 14 ( Marks: 1 ) - Please choose one**
**Verification and validation are the processes in which we check a product against its --------- and the ---------- of the users who will be using it**

- **specifications, expectations (Page 192)**
- statements, expectations
- specifications , arguments
- statements, arguments

**Question No: 15 ( Marks: 1 ) - Please choose one**
**Two tests are considered to be equivalent if it is believed that: if one discovers a defect, the other probably will too, and if one does not discover a defect, ----------.**

- the other probably may
- non of them
- **the other probably won't either (Page 199)**
- the other probably will

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 16 ( Marks: 1 ) - Please choose one**
**It was lady named ---------- who actually coin the term “bug" for the fiest time.**

- ► Elisay Chistopher
- ► **Admiral Grace Hopper (Page 213)**
- ► Ana Nicholson
- ► Jane Hopper

**Question No: 17 ( Marks: 1 ) - Please choose one**
**The first “bug” was actually a moth, which flew through an open window and into one of the Mark ----------’s relays.**

- ► **II (Page 213)**
- ► I
- ► III
- ► IV

**Question No: 18 ( Marks: 1 ) - Please choose one**
**Symptoms of logical errors are ----------**

- ► code is misbehaving
- ► The program doesn't crash, but the flow of program takes odd branches through the code.
- ► Results are the opposite
- ► Output looks strange,
- ► **all of the given (Page 218)**

**Question No: 19 ( Marks: 1 ) - Please choose one**
**First hand accounts of the problem are always useful in Debugging process.**

- ► **True (Page 225)**
- ► False

**Question No: 20 ( Marks: 1 ) - Please choose one**
**Software architecture elements are further divided into categories which in total are ----------**

- ► 2
- ► **3 (Page 122)**
- ► 4
- ► 5

**Question No: 21 ( Marks: 1 ) - Please choose one**
**Establishing responsibilities for objects includes**

- ► Generalization Relationships
- ► Specialization Relationships
- ► all of the above
- ► **identifying Association relationships (Page 101)**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 22 ( Marks: 1 ) - Please choose one**
**Source files can have the extension ----------**

- ► *c++* .
- ► .*C*
- ► .cpp
- ► **all of the given (Page 155)**

**Question No: 23 ( Marks: 1 ) - Please choose one**
**Be very careful when you use functions with side effects – functions that change the values of the ----------**

- ► **parameters (Page 176)**
- ► variables
- ► constants
- ► None of given

**Question No: 24 ( Marks: 1 ) - Please choose one**
**Which of the following is not a characteristic of software?**

- ► **Software is tangible**
- ► A change to a piece of code may implicitly affect the functions of the rest of the code.
- ► Software is configurable.
- ► Software does not wear and tear.

**Question No: 25 ( Marks: 1 ) - Please choose one**
**Which of the following is a fact finding method?**

- ► Site visits
- ► **Prototyping**
- ► Study of similar systems
- ► All of given

Seven common fact-finding methods (Sampling, Research, Observation, Questionnaires, Interviews, Prototyping, Joint Requirements Planning) are introduced as a means to discover requirements.

**Question No: 26 ( Marks: 1 ) - Please choose one**
**The statements given below are associated with system development, testing and maintenance. Identify the correct statement from among them:**

► Some of the activities in the implementation phase are building individual system components, writing of programs and development of user interfaces.
► The post implementation review is performed to see that the computer system is working.
► None of Given
► System development phase consists of a development phase and an implementation phase.

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**FINALTERM EXAMINATION**
**Spring 2010**

**Question No: 1 ( Marks: 1 ) - Please choose one**
**Quantitative methods for assessing the quality of proposed architectural designs are readily available.**

- ► True
- ► **False** http://highered.mcgraw-hill.com/sites/0072853182/student_view0/chapter10/multiple_choice_quiz.html

**Question No: 2 ( Marks: 1 ) - Please choose one**
**A decision table should be used_____**

- ► to document all conditional statements
- ► to guide the development of the project management plan
- ► only when building an expert system
- ► **when a complex set of conditions and actions appears in a component** **Click here for detail**

**Question No: 3 ( Marks: 1 ) - Please choose one**
**Test cases should be designed long before testing begins.**

- ► **True** **(A PRACTITIONER'S APPROACH Page 467)**
- ► False

**Question No: 4 ( Marks: 1 ) - Please choose one**
**Which of the following are characteristics of testable software?**

- ► observability
- ► simplicity
- ► stability
- ► **all of the given** **(A PRACTITIONER'S APPROACH Page 469)**

**Question No: 5 ( Marks: 1 ) - Please choose one**
**Comparison testing is typically done to test two competing products as part of customer market analysis prior to product release.**

- ► True
- ► **False** **Click here for detail**

**Question No: 6 ( Marks: 1 ) - Please choose one**
**By collecting software metrics and making use of existing software reliability models it is possible to develop meaningful guidelines for determining when software testing is done.**

- ► **True** **Click here for Detail**
- ► False

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 7 ( Marks: 1 ) - Please choose one**
**Configuration reviews are not needed if regression testing has been rigorously applied during software integration.**

- ► True
- ► **False** **Click here for detail**

**Question No: 8 ( Marks: 1 ) - Please choose one**
**A change becomes ---------- because of close presence of data and fucntions**

- ► **Localized (Page 81)**
- ► Private
- ► Global
- ► Accessible

**Question No: 9 ( Marks: 1 ) - Please choose one**
**Requirement engineering mainly deals with the ---------- of the system**

- ► **definition phase (Page 16)**
- ► development phase
- ► maintenance
- ► non of the above

**Question No: 10 ( Marks: 1 ) - Please choose one**
**40-60% of all defects found in software projects can be traced back to poor ----------**

- ► **Requirements (Page 17)**
- ► Design
- ► Coding
- ► Testing

**Question No: 11 ( Marks: 1 ) - Please choose one**
**In the N-Tire Architecture the idea is to enhance scalability and ---------- by distributing both the data and the application using multiple server machines.**

- ► **Performance (Page 131)**
- ► Efficiency
- ► Usability
- ► non of the all

**Question No: 12 ( Marks: 1 ) - Please choose one**
**MVC pattern was based on the ---------- pattern.**

- ► **Observer (Page 140)**
- ► Structural
- ► Behavioral
- ► Non of them

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 13 ( Marks: 1 ) - Please choose one**
**Classes should be declared in individual header files with the file name matching the ---------- name.**

► **class (Page 155)**
► method
► object
► non of the all

**Question No: 14 ( Marks: 1 ) - Please choose one**
**Unit testing is roughly equivalent to ---------- testing for hardware in which each chip is tested thoroughly after manufacturing**

► Circuit level
► **Chip Level (Page 207)**
► component level
► system level

**Question No: 15 ( Marks: 1 ) - Please choose one**
**Software should be tested more like hardware, with Built-in self testing: such that each unit can be tested ----------**

► freely
► **independently (Page 207)**
► dependtly
► completely

**Question No: 16 ( Marks: 1 ) - Please choose one**
**A system ---------- is the period in which tremendous pressure is on developers end to fix the problem and make the system running again**

► Uptime
► **Downtime (Page 214)**
► Currentime
► futuretimr

**Question No: 17 ( Marks: 1 ) - Please choose one**
**Which one of the given below is not a symptom of memory overrun?**

► Program crashes quite regularly after a given routine is called, that routine should be examined for a possible overrun condition.
► If the routine in question does not appear to have any such problem the most likely cause is that another routine, called in the prior sequence, has already
trashed variables or memory blocks.
► Checking the trace log of the called routines leading up to one with the problem will often show up the error.
► **Compiler warnings. (Page 220)**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 18 ( Marks: 1 ) - Please choose one**
**Software architecture elements are further divided into categories which in total are ----------**

- ► 2
- ► **3 (Page 122) rep**
- ► 4
- ► 5

**Question No: 19 ( Marks: 1 ) - Please choose one**
**Three tier architecture contains ---------- layers**

- ► Presentation
- ► Application
- ► Database
- ► **All of the above (Page 131)**

**Question No: 20 ( Marks: 1 ) - Please choose one**
**1. Variables should be initialized where they are declared and they should be declared in the ---------- scope possible.**

- ► **Smallest (Page 158)**
- ► largest
- ► medium
- ► None of the given

**Question No: 21 ( Marks: 1 ) - Please choose one**
**---------- are two important tools that can help in managing and mastering the complexity of a program.**

- ► **abstraction and encapsulation (Page 170)**
- ► abstraction and Inheritence
- ► abstraction and Polymarhphism
- ► None of given

**Question No: 22 ( Marks: 1 ) - Please choose one**
**Which of the following is a/are tool used in requirement analysis?**

- ► Flow Graphs
- ► **Data Flow Diagrams (not sure)**
- ► Activity Networks
- ► Module Dependency Diagrams

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 23 ( Marks: 1 ) - Please choose one**
**Consider the following comment. "The software which I bought won't run on windows and when it runs I can't use WORD at the same time". Which of the following do you think are violated by the newly bought software?**

- ► dependability, interchangeability
- ► **platform independence, interoperability**
- ► reliability, dependency
- ► interoperability, reliability

**Question No: 24 ( Marks: 1 ) - Please choose one**
**…… is a diagramming technique used to identify the types of objects in the system and the static relationships that exist among them?**

- ► Class Diagram
- ► Document flow diagrams
- ► Data flow diagrams
- ► **Flow charts**

**Question No: 25 ( Marks: 1 ) - Please choose one**
**Which of the following interaction style best suit the design of an interface for visually impaired users?**

- ► **direct manipulation** **(A PRACTITIONER'S APPROACH Page 469)**
- ► direct manipulation
- ► natural language
- ► command line

**Question No: 26 ( Marks: 1 ) - Please choose one**
**Which of the following testing involve purely black box testing?**

- ► unit testing, beta testing
- ► acceptance testing, interface testing
- ► beta testing, acceptance testing
- ► **integration testing, interface testing**

**FINALTERM EXAMINATION**
**Spring 2010**

**Question No: 1 ( Marks: 1 ) - Please choose one**
**What are the three generic phases of software engineering?**

- ► **definition, development, support** **(A PRACTITIONER'S APPROACH Page 96)**
- ► what, how, where
- ► programming, debugging, maintenance
- ► analysis, design, testing

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 2 ( Marks: 1 ) - Please choose one**
**In the context of requirements analysis, partitioning results in the elaboration of data, function, or behavior.**

- **True** **Click here for detail**
- False

**Question No: 3 ( Marks: 1 ) - Please choose one**
**Quantitative methods for assessing the quality of proposed architectural designs are readily available.**

- True
- **False** **Click here for detail**

**Question No: 4 ( Marks: 1 ) - Please choose one**
**In refining the DFD during transaction mapping it is unnecessary to create a PSPEC since only the CSPEC is relevant to this type of architectural style.**

- True
- **False** **click here for detail**

**Question No: 5 ( Marks: 1 ) - Please choose one**
**In transaction mapping the first level factoring results in the_____**

- creation of a CFD
- **derivation of the control hierarchy** **Click here for detail**
- distribution of worker modules
- refinement of the module view

**Question No: 6 ( Marks: 1 ) - Please choose one**
**Variable names must be in mixed case starting with upper case.**

- True
- **False (Page 150)**

**Question No: 7 ( Marks: 1 ) - Please choose one**
**Class variables should be declared public. This concept violates which of the following.**

- Information hiding
- Encapsulation
- **Information hiding and Encapsulation (Page 158)**
- None of given

**Question No: 8 ( Marks: 1 ) - Please choose one**
**Real-time applications add a new and potentially difficult element to the testing mix**

- performance
- reliability
- security
- **time** **(Page 498)**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 9 ( Marks: 1 ) - Please choose one**
**Top-down integration testing has as it's major advantage(s) that**

- ► low level modules never need testing
- ► **major decision points are tested early** **Click here for detail**
- ► no stubs need to be written
- ► none of the given

**Question No: 10 ( Marks: 1 ) - Please choose one**
**Configuration reviews are not needed if regression testing has been rigorously applied during software integration.**

- ► True
- ► **False** **Click here for detail**

**Question No: 11 ( Marks: 1 ) - Please choose one**
**isAuthorized, assess Performance,get item for UPC, get cashier for number, are the examples of**

- ► **Services (Page 102)**
- ► Attributes
- ► Links
- ► Non of the above

**Question No: 12 ( Marks: 1 ) - Please choose one**
**the *development* view describes the ----------organization of the software in its development environment,**

- ► **Static (Page 122)**
- ► Dynamic
- ► Still
- ► Static & Dynamic

**Question No: 13 ( Marks: 1 ) - Please choose one**
**---------- being the first pure Object Oriented language in which observer pattern was used in implementing its Model View Controller**

- ► **Smalltalk (Page 140)**
- ► PASCAL
- ► JAVA
- ► C++

**Question No: 14 ( Marks: 1 ) - Please choose one**
**Hungarian Notation was first discussed by Charles Simonyi of----------**

- ► **Microsoft. (Page 149)**
- ► Oracle
- ► Apple Macintosh
- ► Non of the all

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 15 ( Marks: 1 ) - Please choose one**
**The language does not specify whether ---------- is signed or unsigned.**

- ► **char (Page 181)**
- ► integer
- ► double
- ► constant

**Question No: 16 ( Marks: 1 ) - Please choose one**
**A number of invisible execution paths can exist in simple code in a language that allows ----------**

- ► **exceptions (Page 185)**
- ► defects
- ► errors
- ► all of them

**Question No: 17 ( Marks: 1 ) - Please choose one**
**Bugs Fixing is done by which of the teams in Software Development lifecycle?**

- ► **Development Team (Page 196)**
- ► Testing Team
- ► Analysis & Design Team
- ► Process Team

**Question No: 18 ( Marks: 1 ) - Please choose one**
**Software Bugs have multiple names, Which one of the below is not the name of Software Bugs ----------**
**-**

- ► Bugs
- ► Defects
- ► Errors
- ► **Mistakes (Page 213)**

**Question No: 19 ( Marks: 1 ) - Please choose one**
**Symptoms of logical errors are ----------**

- ► code is misbehaving
- ► The program doesn't crash, but akes odd branches through the code.
- ► Results are the opposite
- ► Output looks strange,
- ► **all of the given (Page 218)**

**Question No: 20 ( Marks: 1 ) - Please choose one**
**First hand accounts of the problem are always useful in Debugging process.**

- ► **True (Page 225)**
- ► False

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 21 ( Marks: 1 ) - Please choose one**
**In the debugging process a stack trace is a very useful tool.**

- **True (Page 226)**
- False

**Question No: 22 ( Marks: 1 ) - Please choose one**
**Establishing responsibilities for objects includes**

- Generalization Relationships
- Specialization Relationships
- all of the above
- **identifying Association relationships (Page 101)**

**Question No: 23 ( Marks: 1 ) - Please choose one**
**Stakeholders are different people who would be interested in the ----------**

- **Software (Page 24)**
- System
- Product
- All of the given

**Question No: 24 ( Marks: 1 ) - Please choose one**
**A ---------- is a code that explains itself without the need of comments and extraneous documentation**

- **Self documenting code (Page 147)**
- Self telling Code
- Self Documenting Design
- None of the given

**Question No: 25 ( Marks: 1 ) - Please choose one**
**1. Variables should be initialized where they are declared and they should be declared in the ---------- scope possible.**

- **Smallest (Page 158)**
- largest
- medium
- None of the given

**Question No: 26 ( Marks: 1 ) - Please choose one**
**Which of the following is an/are advantage(s) of object oriented analysis & design over structured system analysis & design?**

- Ease of modeling real world events
- Reusability
- Maintainability
- **All of given** **Click here for detail**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

## FINAL TERM EXAMINATION SPRING 2010

**Question No: 1 ( Marks: 1 ) - Please choose one**
**The two main documents produced during this phase are Requirement Statement and Requirement Specification. They are also called Requirement Definition and ----------**

- ► **Functional Specification (Page 25)**
- ► Mathematical specification
- ► System Specification
- ► None of the given

**Question No: 2 ( Marks: 1 ) - Please choose one**
**A decision table should be used_____**

- ► To document all conditional statements
- ► To guide the development of the project management plan
- ► Only when building an expert system
- ► **when a complex set of conditions and actions appears in a component** **Click here for detail**

**Question No: 3 ( Marks: 1 ) - Please choose one**
**Which of the following is a/are tool used in requirement analysis?**

- ► Flow Graphs
- ► **Data Flow Diagrams (not sure)**
- ► Activity Networks
- ► Module Dependency Diagrams

**Question No: 4 ( Marks: 1 ) - Please choose one**
**MVC stands for ----------**

- ►**Model View Controller (Page 140)**
- ►Modern View Center
- ►Model View Center
- ►Modern View Controller

**Question No: 5 ( Marks: 1 ) - Please choose one**
**A ---------- is a code that explains itself without the need of comments and extraneous documentation**

- ►**Self documenting code (Page 147)**
- ►Self telling Code
- ►Self Documenting Design
- ►Non of the above

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 6 ( Marks: 1 ) - Please choose one**

**Bugs that won't "stand still" (almost random) are the ---------- to deal with.**

►least difficult

►**most difficult (Page 226)**

►very easy

►easy

**Question No: 7 ( Marks: 1 ) - Please choose one**

**In refining the DFD during transaction mapping it is unnecessary to create a PSPEC since only the CSPEC is relevant to this type of architectural style.**

► True

► **False**

**Question No: 8 ( Marks: 1 ) - Please choose one**

**Association is a type _____of relation**

►**Weak (OOP , 49)**

►Stronger

►Normal

►None of above

**Question No: 9 ( Marks: 1 ) - Please choose one**

**Variable names must be in mixed case starting with upper case.**

► True

► **False (Page 150)**

**Question No: 10 ( Marks: 1 ) - Please choose one**

**Bugs that won't (almost random) are the ---------- to deal with.**

►**stand still (Page 226)**

►Dynamic

►Static

►None of above

**Question No: 11 ( Marks: 1 ) - Please choose one**

**CRUD include following:-**

►Creat

►Update

►Read

►**All of above (Page 53)**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 12 ( Marks: 1 ) - Please choose one**
**Bugs Finding is done by which of the teams in Software Development lifecycle?**

- ► Development Team
- ► **Testing Team (Page 196)**
- ► Analysis & Design Team
- ► Process Team

**Question No: 13 ( Marks: 1 ) - Please choose one**
**Software Bugs have multiple names, which one of the below is not the name of Software Bugs ----------**

- ► Bugs
- ► Defects
- ► Errors
- ► **Mistakes (Page 213)**

**Question No: 14 ( Marks: 1 ) - Please choose one**
**In the debugging process a stack trace is a very useful tool.**

- ► **True (Page 226)**
- ► False

**Question No: 15 ( Marks: 1 ) - Please choose one**
**A ---------- is a code that explains itself without the need of comments and extraneous documentation**

- ► **Self documenting code (Page 147)**
- ► Self telling Code
- ► Self Documenting Design
- ► None of the given

**Question No: 16 ( Marks: 1 ) - Please choose one**
**Project ---------- defines the concept and range of the proposed solution, and limitations identify certain capabilities that the product will not include**

- ► **Scope (Page 30)**
- ► Agreement
- ► Plan
- ► None of the given

**Question No: 17 ( Marks: 1 ) - Please choose one**
**In the N-Tire Architecture the idea is to enhance scalability and ---------- by distributing both the data and the application using multiple server machines.**

- ► **Performance (Page 131)**
- ► Efficiency
- ► Usability
- ► non of the all

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 18 ( Marks: 1 )**

**The cyclomatic complexity metric provides the designer with information regarding the number of**

- ► Cycles in the program
- ► Errors in the program
- ► **Independent logic paths in the program** **Click here for detail**
- ► Statements in the program

**Question No: 19 ( Marks: 1 )**

**Which of the followings is not a testing type?**

- ► Subsystem testing
- ► Alpha testing
- ► Beta Testing
- ► **Gamma Testing (Page 198)**

**Question No: 20 ( Marks: 1 )**

**Which of the items listed below is not one of the software engineering layers?**

- ► Process
- ► **Manufacturing** **Click here for detail**
- ► Methods
- ► Tools

**Question No: 21 ( Marks: 1 )**

**Which piece of code is self-documented?**

- ►if (x==0) // this is the case when we are allocating a new number
- ►if (AllocFlag == 0)
- ►**If (AllocFlag == NEW_NUMBER) (Page 148)**
- ►None of the given

**FINALTERM EXAMINATION**
**Spring 2010**

**Question No: 1 ( Marks: 1 ) - Please choose one**

**UML (unified modeling language) analysis modeling focuses on the _____ .**

- ► behavioral model and environment model.
- ► behavioral model and implementation model.
- ► user model and environmental model
- ► **user model and structural model** **(A PRACTITIONER'S APPROACH Page 604)**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 2 ( Marks: 1 ) - Please choose one**
**What makes requirements elicitation difficult?**

- ► bounding scope
- ► understanding user needs
- ► requirements volatility
- ► **all of the above (A PRACTITIONER'S APPROACH Page 285)**

**Question No: 3 ( Marks: 1 ) - Please choose one**
**In the architecture trade-off analysis method the architectural style should be described using the**

- ► module view
- ► process view
- ► **all of the given (A PRACTITIONER'S APPROACH Page 403)**
- ► data flow view

**Question No: 4 ( Marks: 1 ) - Please choose one**
**In refining the DFD during transaction mapping it is unnecessary to create a PSPEC since only the CSPEC is relevant to this type of architectural style.**

- ► True
- ► **False** click here for detail

**Question No: 5 ( Marks: 1 ) - Please choose one**
**A decision table should be used_____**

- ► to document all conditional statements
- ► to guide the development of the project management plan
- ► only when building an expert system
- ► **when a complex set of conditions and actions appears in a component** Click here for detail

**Question No: 6 ( Marks: 1 ) - Please choose one**
**Variable names must be in mixed case starting with upper case.**

- ► True
- ► **False (Page 150)**

**Question No: 7 ( Marks: 1 ) - Please choose one**
**Which of these are objectives for software testing?**

- ► determine the productivity of programmers
- ► eliminate the need for future program maintenance
- ► eliminate every error prior to release
- ► **uncover software errors (A PRACTITIONER'S APPROACH Page 467)**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 8 ( Marks: 1 ) - Please choose one**
**Comparison testing is typically done to test two competing products as part of customer market analysis prior to product release.**

- ► True
- ► **False** **Click here for detail**

**Question No: 9 ( Marks: 1 ) - Please choose one**
**In software quality assurance work there is no difference between software verification and software validation**

- ► True
- ► **False** **Click here for detail**

**Question No: 10 ( Marks: 1 ) - Please choose one**
**Top-down integration testing has as it's major advantage(s) that**

- ► low level modules never need testing
- ► major decision points are tested early
- ► **no stubs need to be written** **Click here for detail**
- ► none of the given

**Question No: 11 ( Marks: 1 ) - Please choose one**
**Rooko mut jane do depicts one of risks associated with inadequate requriements gathering, the name of that risk is ----------**

- ► **Ambiguous requirements (Page 20)**
- ► Creeping Requirements
- ► Minimal Specification
- ► Non of the above

**Question No: 12 ( Marks: 1 ) - Please choose one**
**Interaction Diagrams depict the ---------- Behavior of the system**

- ► Static
- ► **Dynamic (Page 106)**
- ► Active
- ► Non of the above

**Question No: 13 ( Marks: 1 ) - Please choose one**
**In Sequence Diagrams events are organized in a----------time line**

- ► Vertical
- ► **Horizontal (Page 106)**
- ► Vertical & Horizontal
- ► Non of the above

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 14 ( Marks: 1 ) - Please choose one**
**the development view describes the ----------organization of the software in its development environment,**

- ► **Static (Page 122)**
- ► Dynamic
- ► Still
- ► Static & Dynamic

**Question No: 15 ( Marks: 1 ) - Please choose one**
**Every view is potentially Hierarchical in this regard which of the view are Hierarchical ?**

- ► Functioanl View
- ► Development View
- ► C*oncurrency view*
- ► **All of given (Page 125)**

**Question No: 16 ( Marks: 1 ) - Please choose one**
**object-oriented framework that is a set of cooperative classes that make up reusable ---------- of a system**

- ► Code
- ► **Design (Page 138)**
- ► Analysis
- ► non of the all

**Question No: 17 ( Marks: 1 ) - Please choose one**
**When planning for performance, one should always remember the ---------- rule**

- ► **80/20 (Page 177)**
- ► 20/80
- ► 80/22
- ► 80/90

**Question No: 18 ( Marks: 1 ) - Please choose one**
**In order to make a code more portable, Instead of using vendor specific language extensions, use ---------- as much as possible**

- ► **STL (Page 179)**
- ► ANSI
- ► ISO
- ► CMMI

**Question No: 19 ( Marks: 1 ) - Please choose one**
**Performance & Usability are examples of ---------- requirements**

- ► Business

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

► Functioanl
► **Non-Functioanl (Page 39)**
► User

**Question No: 20 ( Marks: 1 ) - Please choose one**
**Static analyzers are software tools for ---------- processing.**

► analysis text
► **source text (Page 211)**
► design text
► non of the given

**Question No: 21 ( Marks: 1 ) - Please choose one**
**Software Bugs have multiple names, Which one of the belw is not the name of Software Bugs ----------**
► Bugs
► Defects
► Errors
► **Mistakes (Page 213)**

**Question No: 22 ( Marks: 1 ) - Please choose one**
**According to a survey, when a software application is in the maintenance phase, --------- of its lifecycle cost is attributed towards the defects which are found in the software application after installation**

► **20 % (Page 214)**
► 25 %
► 30%
► 35%

**Question No: 23 ( Marks: 1 ) - Please choose one**
**Bugs that won't "----------" (almost random) are the most difficult to deal with.**

► **stand still (Page 226)**
► dynamic
► running
► stand tall

**Question No: 24 ( Marks: 1 ) - Please choose one**
**1. Variables should be initialized where they are declared and they should be declared in the ---------- scope possible.**

► **Smallest (Page 158)**
► largest
► medium
► None of the given

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 25 ( Marks: 1 ) - Please choose one**
**If you are initializing a variable at the time of declaration, do not declare another ---------- in the same statement.**

- ► **variable (Page 176)**
- ► object
- ► class
- ► None of given

**Question No: 26 ( Marks: 1 ) - Please choose one**
**During Requirement Engineering process which of the following is/are document(s) used for fact finding?**

- ► Company's employee list
- ► Samples of the company databases
- ► The company's mission statement and plan
- ► **All of given**

**FINALTERM EXAMINATION**
**Fall 2008**

**Question No: 1 ( Marks: 1 ) - Please choose one**
**Object-oriented domain analysis is concerned with the identification and specification of reusable capabilities within an application domain.**

- ➢ **True** **Click here for detail**
- ➢ False

**Question No: 2 ( Marks: 1 ) - Please choose one**
**Fourth generation techniques**

- ➢ Allow software to be developed without any testing.
- ➢ Eliminate the need for costly requirements gathering activities.
- ➢ **Can reduce the time required to develop software.**

**(A PRACTITIONER'S APPROACH Page 73)**

- ➢ Are best used by non-programmers to build small systems.

**Question No: 3 ( Marks: 1 ) - Please choose one**
**Data design actually begins during the creation of the analysis model, not the architectural model.**

- ➢ **True** **(A PRACTITIONER'S APPROACH Page 397)**
- ➢ False

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 4 ( Marks: 1 ) - Please choose one**
**The states shown in a state transition diagram do not necessarily correspond to the processes shown in a control flow diagram for the same system.**

- **True (A PRACTITIONER'S APPROACH Page 397)**
- False

**Question No: 5 ( Marks: 1 ) - Please choose one**
**The criteria used to assess the quality of an architectural design should be based on system**

- **accessibility and reliability (A PRACTITIONER'S APPROACH Page 404)**
- data and control
- functionality
- implementation details

**Question No: 6 ( Marks: 1 ) - Please choose one**
**A useful technique for evaluating the overall complexity of a proposed architecture is to look at the component**

- number and size of components
- **flow dependencies and sharing dependencies (A PRACTITIONER'S APPROACH Page 406)**
- size and cost
- none of the given

**Question No: 7 ( Marks: 1 ) - Please choose one**
**The three basic principles that guide maintainability are: simplicity, clarity, and_____ .**

- **Generality (Page 146)**
- Reliability
- All of the given choices
- None of the Given choices

**Question No: 8 ( Marks: 1 ) - Please choose one**
**In order to make a program self documented a number of attributes required. Which one is the attribute/s of self documented program**

- **All of the given choices (Page 147)**
- Size of each function
- Choice of variable
- Modularity

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 9 ( Marks: 1 ) - Please choose one**
**Floating point constants should always be written with decimal point and at least**

- **one decimal (Page 161)**
- two decimal
- three decimal
- none of the given

**Question No: 10 ( Marks: 1 ) - Please choose one**
**The code becomes self explanatory with the help of proper use of parentheses. Select the right one.**

- leapYear = year % 4 == 0 && year % 100 != 0 || year % 400 == 0 ;
- leapYear = ((year % 4 == 0) && (year % 100 != 0) || ((year % 400 == 0));
- **leapYear = ((year % 4 == 0) && (year % 100 != 0)) || (year % 400 == 0); (Page 164)**
- leapYear = (year % 4 == 0) && (year % 100 != 0) || (year % 400 == 0);
- none of the given options

**Question No: 11 ( Marks: 1 ) - Please choose one**
**Graph-based testing methods can only be used for object-oriented systems**

- True
- **False** **Click here for detail**

**Question No: 12 ( Marks: 1 ) - Please choose one**
**Configuration reviews are not needed if regression testing has been rigorously applied during software integration.**

- True
- **False** **Click he re for Detail**

**Question No: 13 ( Marks: 1 ) - Please choose one**
**A class is a ---------- of objects**

- **Template (Page 85)**
- Stereotype
- Collection
- Non of the above

**Question No: 14 ( Marks: 1 ) - Please choose one**
**A public Interface provides a way for with other Classes.**

- **Communication**
- Accessibility
- Reaching
- All of the above

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 15 ( Marks: 1 ) - Please choose one**
**Flow Charts represents**

- **sequence Activity (Page 50)**
- random activity
- parallel activity
- non of the above

**Question No: 16 ( Marks: 1 ) - Please choose one**
**Software architecture defines the high level structure of the software by putting together a number of architectural ---------- in an organized fashion.**

- **Elements (Page 122)**
- parts
- components
- non of the all

**Question No: 17 ( Marks: 1 ) - Please choose one**
**Patterns are devices that allow programs to share knowledge about their ----------.**

- **Design (Page 137)**
- Code
- Analysis
- Non of the all

**Question No: 18 ( Marks: 1 ) - Please choose one**
**MVC stands for ----------**

- **Model View Controller (Page 140)**
- Modern View Center
- Model View Center
- Modern View Controller

**Question No: 19 ( Marks: 1 ) - Please choose one**
**A ---------- is a code that explains itself without the need of comments and extraneous documentation**

- **Self documenting code (Page 147)**
- Self telling Code
- Self Documenting Design
- Non of the them

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 20 ( Marks: 1 ) - Please choose one**
**Type conversions must always be done ----------**

- **Explicitly (Page 158)**
- implicitly
- simultaneously
- non of them

**Question No: 21 ( Marks: 1 ) - Please choose one**
**Types that are ----------to one file only can be declared inside that file.**

- **Local (Page 158)**
- Global
- Private
- General

**Question No: 22 ( Marks: 1 ) - Please choose one**
**The use of do while loops should be ----------**

- **Avoided (Page 159)**
- encouraged
- practiced
- non of them

**Question No: 23 ( Marks: 1 ) - Please choose one**
**One of the causes of the portability issues is the order of ---------- varies from one implementation to other.**

- **Evaluation (Page 181)**
- numbers
- variables
- symbols

**Question No: 24 ( Marks: 1 ) - Please choose one**
**---------- is a powerful technique that separates error-handling code from normal code.**

- **Exception handling (Page 184)**
- Code handling
- variable handling
- pointer handling

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 25 ( Marks: 1 ) - Please choose one**
**The goal of testing is to expose ---------- defects in a software system before it is put to use.**

- **Latent (Page 193)**
- already present
- current
- runtime

**Question No: 26 ( Marks: 1 ) - Please choose one**
**Static analyzers are software tools for ---------- processing.**

- analysis text
- **source text (Page 211)**
- design text
- non of the given

**Question No: 27 ( Marks: 1 ) - Please choose one**
**The first bug was actually a moth, which flew through an open window and into one of the Mark --------- 's relays.**

- **II (Page 213)**
- I
- III
- IV

**Question No: 28 ( Marks: 1 ) - Please choose one**
**Which one of the given below is not a symptom of memory overrun?**

- Program crashes quite regularly after a given routine is called, that routine should be examined for a possible overrun condition.
- If the routine in question does not appear to have any such problem the most likely cause is that another routine, called in the prior sequence, has already trashed variables or memory blocks.
- Checking the trace log of the called routines leading up to one with the problem will often show up the error.
- **Compiler warnings. (Page 220)**

**Question No: 29 ( Marks: 1 ) - Please choose one**
**Holistic medicine, concerns itself with the state of the body as a whole, not the ---------- that is currently attacking it.**

- reason
- **disease (Page 224)**
- source
- non of the given

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 30 ( Marks: 1 ) - Please choose one**
**Bugs that won't "stand still" (almost random) are the ---------- to deal with.**

- least difficult
- **most difficult (Page 226)**
- very easy
- easy

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

# Final Term Examination – Spring 2005

**Time Allowed: 150 Minutes**

**Please read the following instructions carefully before attempting any of the questions:**

**1**. Attempt all questions. Marks are written adjacent to each question.

**2**. Do not ask any questions about the contents of this examination from anyone.

***a***. If you think that there is something wrong with any of the questions, attempt it to the best of your understanding.

***b***. If you believe that some essential piece of information is missing, make an appropriate assumption and use it to solve the problem.

***c***. Write all steps, missing steps may lead to deduction of marks.

****WARNING: Please note that Virtual University takes serious note of unfair means. Anyone found involved in cheating will get an `F` grade in this course.**

**Total Marks: 60** **Total Questions: 11**

**Question No. 1** **Marks : 10**

**What is meant by Maintainable Code? Describe it briefly.** **[10 marks]**

**Question No. 2** **Marks : 05**

**Explain how effective modular design is achieved through functional independence of the individual modules?**
**[5 marks]**

**Question No. 3** **Marks : 02**

**Which of the followings can not be used to relate use cases with each other?** **[2 marks]**

- Extends
- Include
- Exclude
- Uses

**Question No. 4** **Marks : 05**

**What are the symptoms of Boolean Bugs?** **[5 marks]**

**Question No. 5** **Marks : 15**

**Following is the prototype of a function to find the square root of a number.**

```
double sqrt (int num);
```

**(a) Identify equivalence partitions to devise black-box test cases for this function.** **[10 marks]**

**(b) Write test cases against each partition identified in part (a).**
**[5 marks]**

**Question No. 6** **Marks : 02**

**Which of the followings is not a testing type?** **[2 marks]**

- Subsystem testing
- Alpha testing
- Beta Testing
- Gamma Testing

**Question No. 7** **Marks : 02**

**Which of the following documents should be prepared before the commencement of a software project?** **[2 marks]**

- Quality plan

- Design specification
- Validation (Test) Plan
- Software requirement specification

**Question No. 8** **Marks : 02**

**During the requirement analysis stage, which of the following requirements should be identified?** **[2 marks]**

- Functional requirements
- Performance requirements
- Software tools requirements
- Personal resources requirements

**Question No. 9** **Marks : 02**

**Which of the followings is design tool?** **[2 marks]**

- Collaboration Diagram
- State Transition Diagram
- Flow Chart
- Data Flow Diagram

**Question No. 10** **Marks : 10**

**Draw the corresponding object model diagram for the following system description.** **[10 marks]**
**(Note: Use the UML Object Model Notation)**

"A bank account is either, a current account, a deposit account, or a mortgage account. All bank accounts can be opened and closed and all bank accounts have a balance. Deposit accounts pay interest. For current accounts it is possible to set up an overdraft facility. All mortgage accounts have a 'term' (the number of years the mortgage runs). A bank customer (who is characterized by name and address) can have one or more accounts."

**Question No. 11** **Marks : 10**

**Which of the followings is design tool?** **[2 marks]**

# Final Term Examination - August 2004

**Time Allowed: 150 Minutes**

## Instructions

Please read the following instructions carefully before attempting any question:

1. The duration of this examination is 150 Mins.
2. You have to attempt all Questions.
3. This examination is closed book, closed neighbors; any one found cheating will get no grade.
4. Do not ask any questions about the contents of this examination from anyone.
    a. If you think that there is something wrong with any of the questions, attempt it to the best of your understanding.
    b. If you believe that some essential piece of information is missing, make an appropriate assumption and use it to solve the problem.
5. You are allowed to use any tool that helps in drawing UML Diagrams like Microsoft Word, Visio etc.

---

**Total Marks: 60** **Total Questions: 6**

**Question No. 1** **Marks : 10**

Write a short note on Façade pattern.

**Question No. 2** **Marks : 9**

Give brief answers to the following:

1. How does software differ from the artifacts produced by other engineering disciplines? (3 Pts)
2. What is meant by the term software reliability? (3 Pts)
3. Describe the principle of information hiding as it applies to software design. (3 Pts)

**Question No. 3** **Marks : 6**

What are the symptoms of pointer errors?

**Question No. 4** **Marks : 15**

Consider the following code:

```
if ((a >= b) && (a >= c)) max = a;
if ((b >= a) && (b >= c)) max = b;
if ((c >= a) && (c >= b)) max = c;
```

(a) Draw the flow graph for this code segment. (5 Pts)
(b) Calculate the cyclomatic complexity for this code (2 Pts)
(c) Identify one infeasible path if any. (1 Pts)
(d) Identify all feasible paths (2 Pts)
(e) Write white-box test cases for all the feasible paths (5 Pts)

**Question No. 5** **Marks : 10**

Write a short note on zero-install client-server configuration.

**Question No. 6** **Marks : 10**

Following is the prototype of a function to find the index of the minimum value in an array.

```
int min(int array[ ], int arraySize);
```

(a) Identify equivalence partitions to devise black-box test cases for this function. (5 Pts)
(b) Write test cases against each partition identified in part (a). (5 Pts)

# CS504_final_spring2006

**(Q 1)**

Following is the prototype of a function to find the square root of a number.

```
double sqrt (int num);
```

**a)** Identify equivalence partitions to devise black-box test cases for this function. (10 Pts)

**b)** Write test cases against each partition identified in part (a). (5 Pts)

**(Q 2)**

For each of the following cases, identify which Design Pattern should be used and why? Discuss briefly.

**a)** There are two objects, "A" and "B". Both of them are interacting with each other in such a way that the state of the object "B" depends on the state of the object "A" and it is the responsibility of "B" to synchronize its state whenever "A" changes.

**b)** A client wants to build Simulation Software, which is used to show customer the preview of interior decoration of rooms. The software presents rooms as object and it can interact with other objects. These objects, like Television, Sofa etc, can change their state. The client has determined that there can be limited number of operations that can be performed on the objects, e.g. a TV can be placed at either of the walls of the rooms etc. The interaction between the objects should be low.

**(Q 3)**

Draw the corresponding object model diagram for the following system description. (Note: Use the UML Object Model Notation)

"An online banking system allows customers to manage their account. Each account has a balance. Customers can deposit to or withdraw from their account. Each deposit or withdrawal is called a *transaction*. A transaction always has a value and a date. At any time a customer can see a statement, which is a listing of each transaction."

**(Q 4)**

definition & example of Equivalence Partitions

**(Q 5)**

What is the importance of Self Documented Code in Software Engineering?

**(Q 6)**

Explain why encapsulation, inheritance, and polymorphism are three important characteristics of object-oriented systems.

**(Q 7)**

List the problems we risk facing if software engineering principles are not applied to software development.

**(Q 8)**

What are the differences between Thin Client and Fat Client architecture?

**(Q 9)**

Following is the list of bugs' symptoms; identify the bug classes in each case.

**a)** System slowdowns.
**b)** The program doesn't crash, but the flow of the program takes odd branches through the code.

**c)** If your program simply locks up, repeatedly displays the same data over and over, or infinitely displays the same message box.

**(Q 9)** **(Marks: 2)**

The component-based development model is

★ Only appropriate for computer hardware design.
★ Not able to support the development of reusable components.
★ Dependent on object technologies for support.
★ Not cost effective by known quantifiable software metrics.

**(Q 10)** **(Marks: 2)**

Use-Cases are scenarios that describe

★ How software is to be used in a given situation.
★ How CASE tool will be used to construct the system.
★ To build plan for a software product.
★ The test cases for a software product.

**(Q 11)** **(Marks: 2)**

The cyclomtic complexity metric provides the designer with information regarding the number of

★ Cycles in the program.
★ Errors in the program.
★ Independent logic paths in the program.
★ Statements in the program.

**(Q 12)** **(Marks: 2)**

In which software development problem solving stage are the results delivered?

★ Status quo
★ Problem definition
★ Technical development
★ Solution integration

**(Q 13)** **(Marks: 2)**

Which of the items listed below is not one of the software engineering layers?

- ★ Process
- ★ Manufacturing
- ★ Methods
- ★ Tools

| | | |
|---|---|---|
| Virtual University | **FINAL EXAMINATION**<br>**SEMESTER FALL 2004**<br>**CS504-Software Engineering** | **Total Marks:60**<br>**Duration:120mins** |

| | |
|---|---|
| **StudentID/LoginID** | |
| **Name** | |
| **PVC Name/Code** | |
| **Date** | **17 Feb, 2005** |

**Maximum Time Allowed: (2 Hour)**

**Please read the following instructions carefully before attempting any of the questions:**

1. Attempt all questions. Marks are written adjacent to each question.
2. Do not copy and paste from handouts. If any thing found copy from handouts, no mark would be awarded for it.
3. Do not ask any questions about the contents of this examination from anyone.

***a***. If you think that there is something wrong with any of the questions, attempt it to the best of your understanding.

***b***. If you believe that some essential piece of information is missing, make an appropriate assumption and use it to solve the problem.

***c***. Write all steps, missing steps may lead to deduction of marks.

****WARNING: Please note that Virtual University takes serious note of unfair means. Anyone found involved in cheating will get an `F` grade in this course.**

For Teacher's use only

| Question | Q1 | Q2 | Q3 | Q4 | Q5 | Q6 | Total |
|---|---|---|---|---|---|---|---|
| Marks | | | | | | | |

**Question No: 1** **Marks: 10**

**What are Design Patterns? Explain them briefly.**

**Question No: 2** **Marks: 15**

**i- Consider the following Code Segment: (10)**

```
Input: sequence of numbers, size, number
Variable: count=0
        Do while (end of sequence)
                If (sequence[i] == prime Number)
                    Count = count++
                 Else
                    Count = count to
                END IF
          END DO
 If (count == size)
          "sequence of number is a prime number sequence"
 if (count is odd)
           a← finalMax(sequence)
           y← a X number
 else
          a← finalMin(sequence)
           y← a X number
  display y
 else
          "sequence of number is not a prime number sequence"
 END IF
      END PROGRAM
```

- **Draw the Flow Graph for the Code Segment.**
- **Find Cyclomatic Complexity of the Flow Graph.**

**(HINT: for the above code segment, ignore any syntax or grammatical errors.)**

**ii- what is Black Box and White Box Testing. Describe briefly. (5)**

**Question No: 3** **Marks: 10**

**Following is the prototype of a function to find the index of the minimum value in an array.**

```
int min(int array[ ], int arraySize);
```

**(a) Identify equivalence partitions to devise black-box test cases for this function. (5 Pts)**
**(b) Write test cases against each partition identified in part (a). (5 Pts)**

**Question No: 4** **Marks: 6**

**What are Boolean Bugs? Describe briefly.What are symptoms of Boolean Bugs?**

**Question No: 5** **Marks: 10**

**What is Software Architecture? What is its importance? Describe briefly.**

| Question No: 6 | Marks: 9 |
|---|---|

**Give brief answers to the following:**

1. **How do Software characteristics differ from Hardware characteristics? (3 Pts)**
2. **Write a short note on Data Flow or Pipes and Filter Architecture Configuration.(3 Pts)**
3. **How do you differentiate between the responsibilities of Developer and Tester? (3 Pts)**

| | FINALTERM EXAMINATION<br>FALL 2006<br>CS504 - SOFTWARE ENGINEERING – I | Marks: 85<br>Time: 120min |
|---|---|---|

StudentID/LoginID: _____

Student Name: _____

Center Name/Code: _____

Exam Date:

---

**Please read the following instructions carefully before attempting any of the questions:**

**1. Attempt all questions. Marks are written adjacent to each question.**

**2. Do not ask any questions about the contents of this examination from anyone.**

***a.*** **If you think that there is something wrong with any of the questions, attempt it to the best of your understanding.**

***b.*** **If you believe that some essential piece of information is missing, make an appropriate assumption and use it to solve the problem.**

***c.*** **Write all steps, missing steps may lead to deduction of marks.**

****WARNING: Please note that Virtual University takes serious note of unfair means. Anyone found involved in cheating will get an `F` grade in this course.**

| For Teacher's use only | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Question | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | Total |
| Marks | | | | | | | | | | | |
| Question | 11 | 12 | 13 | 14 | 15 | | | | | | |
| Marks | | | | | | | | | | | |
| Question | | | | | | | | | | | |
| Marks | | | | | | | | | | | |

Question No: 1 ( Marks: 1 ) - Please choose one

**The linear sequential model of software development is also known as the**

- **Classical life cycle model**
- **Fountain model**
- **Spiral model**
- **Chaos model**

Question No: 2 ( Marks: 1 ) - Please choose one

**Activity Diagram give a pictorial description of**

- **Use case**
- **Objects**
- **State of system**
- **Actors**

Question No: 3 ( Marks: 1 ) - Please choose one

**Public metrics are used**

- **to make strategic changes to the software process.**
- **to make tactical changes during a software project.**
- **to evaluate the performance of software development teams.**
- **to make strategic and tactical changes to the software process**

Question No: 4 ( Marks: 1 ) - Please choose one

**The cyclomatic complexity metric provides the designer with information regarding the number of**

- **Cycles in the program**
- **Errors in the program**
- **Independent logic paths in the program**
- **Statements in the program**

Question No: 5 ( Marks: 1 ) - Please choose one

**The object-behavior model indicates how the system**

- **functions in the operating environment**
- **objects collaborate with one another**
- **responds to external stimuli**
- **responds to internal stimuli**

Question No: 6 ( Marks: 1 ) - Please choose one

**Class responsibilities are defined by**

- **its attributes only**
- **its collaborators**
- **its operations only**
- **both its attributes and operations**

Question No: 7 ( Marks: 1 ) - Please choose one

**Software risk always involves two characteristics**

- **fire fighting and crisis management**
- **known and unknown risks**
- **uncertainty and loss**
- **staffing and budget**

Question No: 8 ( Marks: 1 ) - Please choose one

**The testing technique that requires devising test cases to exercise the internal logic of a software module is called**

- **behavioral testing**
- **black-box testing**
- **grey-box testing**
- **white-box testing**

Question No: 9 ( Marks: 1 ) - Please choose one

**The four layers defined for object-oriented design are the same as design layers used for conventional software design.**

- **True**
- **False**

Question No: 10 ( Marks: 1 ) - Please choose one

**Graph-based testing methods can only be used for object-oriented systems**

▶ **True**

▶ **False**

Question No: 11 ( Marks: 15 )

**Following is the prototype of a function to find the cube root of a number.**

**double function(int a);**

a) **Identify equivalence partitions to devise black-box test cases for this function.**
b) **Write test cases against each partition identified in part (a).**

Question No: 12 ( Marks: 15 )

**Following are given Symptoms of different Bugs. Identify them:**

**1.**

- **System slowdowns**
- **Crashes that occur "randomly" over a long period of time**

**2**

- **Results are the opposite of what is expected.**
- **Output looks strange, but has no obvious symptoms of corruption**

**3.**

- **Unexpected errors in black box testing.**
- **The errors that unexpectedly occur are usually caused by coding errors.**

Question No: 13 ( Marks: 15 )

**Draw a Business Process Diagram according to the following Processes:**

1. **A patient may come to visit In Patient Department (IPD) or output patient department (OPD)**
2. **System determines if he is a company patient or a private patient.**
3. **For a company patient, system verifies him.**
4. **For an OPD patient, system will issue a chit to the patient and inform him about his number and the consultant to whom he has to consult and he will have to wait for his turn.**
5. **After verifying an IPD patient, system will create a visit and allocate him a room or a bed etc. If system cannot allocate this, then it will inform the patient. Otherwise the patient is checked in and his information is maintained in the system.**
6. **System displays information about the expenses of the required service to the patient so that he is informed of his expected expenditure.**
7. **Some advance payment is also received against the required service and this amount is adjusted in the final settlement.**
8. **All this information is supplied to cash office that eventually deals with payments, etc.**
9. **Upon receiving the cash, for OPD patient, a chit will be issued. For IPD patient, an admission form will be filled and this information will be maintained in the system. A receipt will be issued to the patient.**
10. **For credit transaction, corresponding voucher will be prepared.**
11. **So the model depicts process before the start of the treatment.**
12. **A patient may ask to change his service on event of an unsatisfied response from the hospital staff or any other reason. System may cancel his record and pay his amount back.**
13. **Similarly, a doctor may ask a patient to change his status from OPD to IPD.**

Question No: 14 ( Marks: 20 )

**Consider the following code:**

```
sorted = false;
while (!sorted) {              //1
   sorted = true;
   for (i=0; i < N-1; i++) { //2
      if a[i] > a[i+1] {
         swap(a[i], a[i+1]); //3
         sorted = false;
      }                        //4
   }                           //5
}                              //6
```

a) **Draw the flow graph for this code segment. [10]**
b) **Calculate the cyclomatic complexity for this code [5]**
c) **Identify one infeasible path if any. [5]**

Question No: 15 ( Marks: 10 )

**Write Some of the key advantages which make the object-oriented technology significantly attractive than other technologies? Discuss the difference between Object oriented and Function Oriented approach also elaborate the difference using diagram.**

# FINALTERM EXAMINATION
## Spring 2010

**Question No: 27 ( Marks: 2 )**
**What is Software Testing?**
Answer:- (Page 192)
To understand the concept of software testing correctly, we need to understand a few related concepts.

**Question No: 28 ( Marks: 2 )**
**Write unit testing qualitative benefits.**
Answer:- (Page 207)
**Assessment-oriented**: Writing the unit test forces us to deal with design issues - cohesion, coupling.
**Confidence-building**: We know what works at an early stage. Also easier to change when it's easy to retest.

**Question No: 29 ( Marks: 2 )**
**Where Term Compute can be used in Methods?**
Answer:- (Page 152)
The term compute can be used in methods where something is computed.
valueSet.computeAverage(); matrix.computeInverse()

**Question No: 30 ( Marks: 2 )**
**Which of the following is a/are non-functional requirement of an e-learning system?**

**(a) User friendliness**
**(b) Time taken to download study materials through the system should not be too long.**
**(c) On-line assignment submission facility**
**(d) On-line chatting facility**
**(e) Robustness**

Answer:-
(a) User friendliness

**Question No: 31 ( Marks: 3 )**
**Write unit testing principles.**
Answer:-(Page 207)

- In unit testing, developers test their own code units (modules, classes, etc.) during implementation.
- Normal and boundary inputs against expected results are tested.
- Thus unit testing is a great way to test an API.

**Question No: 32 ( Marks: 3 )**
**The use of *do.... while* loops should be avoided. Explain why ?**
Answer:-(Page 159)
The use of *do.... While* loops should be avoided. There are two reasons for this. First is that the construct is superfluous; Any statement that can be written as a *do.... while* loop can equally well be written as a *while* loop or a *for* loop. Complexity is reduced by minimizing the number of constructs being used. The other reason is of readability. A loop with the conditional part at the end is more difficult to read than one with the conditional at the top.

**Question No: 33 ( Marks: 3 )**
**Give 3 Equivalence partitioning guidelines**
Answer:-(Page 199)

- Organize your equivalence classes. Write them in some order, use some template, sequence, or group them based on their similarities or distinctions. These partitions can be hierarchical or organized in any other manner.
- Boundary conditions: determine boundary conditions. For example, adding in an empty linked list, adding after the last element, adding before the first element, etc.
- You should not forget invalid inputs that a user can give to a system. For example, widgets on a GUI, numeric instead of alphabets, etc.

**Question No: 34 ( Marks: 5 )**
**Discus the symptoms and an example of memory overrun bug class.**
Answer:-(Page 220)

## Symptoms

- Program crashes quite regularly after a given routine is called, that routine should be examined for a possible overrun condition.
- If the routine in question does not appear to have any such problem the most likely cause is that another routine, called in the prior sequence, has already trashed variables or memory blocks.
- Checking the trace log of the called routines leading up to one with the problem will often show up the error.

## Example

The array only has 50 slots available in its allocation. What happens at that point is that the function goes past the end of the array and starts to walk on things beyond its control.

```
const kMaxEntries = 50;
int gArray[kMaxEntries];
char szDummyBuffer[256];
```

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

```
int nState = 10;
int ZeroArray (int *pArray)
{
for (inti=0;i<100;++i)
pArray[i] = 0;
}
```

**Question No: 35 ( Marks: 5 )**
**Write at leas 5 General Naming conventions for C++ or Java**
Answer:-(Page 150)
1. Names representing types must be nouns and written in mixed case starting with upper case.
Line, FilePrefix

2. Variable names must be in mixed case starting with lower case.
line, filePrefix

3. Names representing constants must be all uppercase using underscore to separate words.
MAX_ITERATIONS, COLOR_RED

4. Names representing methods and functions should be verbs and written in mixed case starting with lower case.
getName(), computeTotalWidth()

5. Names representing template types in C++ should be a single uppercase letter.
template<class T>
template<class C, class D>

**Question No: 36 ( Marks: 5 )**
**Explain at least 2 code structures.**
Answer:-(Page 201)

## Case:-

In Case statement, control can take either of several branches (as opposed to only two in If statement.) First node represents the switch statement (C/C++) and nodes in middle correspond to all different cases. Program can take one branch and result into the same instruction.

## While

A while loop structure consists of a loop guard instruction through which the iteration in the loop is controlled. The control keeps iterating in the loop as long as the loop guard condition is true. It branches to the last instruction when it becomes false.

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

# FINALTERM EXAMINATION
# Spring 2010
# CS504- Software Engineering - I (Session - 3)

**Question No: 27 ( Marks: 2 )**
**What does this mean" Object Creation and Life Time"?**
Answer:-(Page 87)
From the object creation and life time point of view, when an object is instantiated, all of its parts must also be instantiated at the same time before any useful work can be done and all of its part die with it. While in the case of association, the life time of two associated object is independent of one another. The only limitation is that an object must be alive or has to be instantiated before a message can be sent to it.

**Question No: 28 ( Marks: 2 )**
**How one can avoid hazards caused by side effects while writing code. List the two guidelines.**
Answer:-(Page 176)
never use "," except for declaration
never use multiple assignments in the same statement

**Question No: 29 ( Marks: 2 )**
**What is the greatest advantage of exception handling?**
Answer:-(Page 184)
One of the most powerful features of exception handling is that an error can be thrown over function boundaries. This allows programmers to put the error handling code in one place, such as the *main*-function of your program.

**Question No: 30 ( Marks: 2 )**
**Give 2 Unit Testing Tips.**
Answer:-(Page 208)
- For small projects you can imbed the unit test for a module in the module itself
- For larger projects you should keep the tests in the package directory or a /test subdirectory of the package

**Question No: 31 ( Marks: 3 )**
**Write unit testing quantitative benefits.**
Answer:-(Page 207)
- **Repeatable**: Unit test cases can be repeated to verify that no unintended side effects have occurred due to some modification in the code.
- **Bounded**: Narrow focus simplifies finding and fixing defects.
- **Cheaper**: Find and fix defects early

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 32 ( Marks: 3 )**

**How Comments should be indented relative to their position in the code? Give an example**

Answer:-(Page 162)

Comments should be indented relative to their position in the code.

```
while (true) {          //   NOT: while (true) {
// Do something        //   // Do something
something();           //      something();
}                      //      }
```

**Question No: 33 ( Marks: 3 )**

**Consider the following code fragment.**

**while a**
**{**
**while b**
**c**
**d**
**}**

**If you were to test this code, what would be the test technique to adopt?**

**Question No: 34 ( Marks: 5 )**

**Narrate the manner for the organization of Class and Interface declarations**

Answer:-(Page 157)

Class and Interface declarations should be organized in the following manner:

1. Class/Interface documentation.
2. Class or interface statement.
3. Class (**static**) variables in the order **public**, **protected**, package (no access modifier), **private**.
4. Instance variables in the order **public**, **protected**, package (no access modifier), private.
5. Constructors.
6. Methods (no specific order).

**Question No: 35 ( Marks: 5 )**

**Discus the symptoms and an example of coding error bug class.**

Answer:-(Page 219)

## Symptoms

- Unexpected errors in black box testing.
- The errors that unexpectedly occur are usually caused by coding errors.
- Compiler warnings.
- Coding errors are usually caused by lack of attention to details.

## Example

In the following example, a function accepts an input integer and converts it into a string that contains that integer in its word representation.

```
void convertToString(int InInteger,
char* OutString, int* OutLength)
{
switch(InInteger){
```

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

```
case 1: OutString = "One";OutLength = 3;
break;
case 2: OutString = "Two";OutLength = 3;
break;
case 3: OutString = "Three";OutLength = 5;
break;
case 4: OutString = "Four";OutLength = 4;
break;
case 5: OutString = "Five";OutLength = 4;
break;
case 6: OutString = "Six";OutLength = 3;
break;
case 7: OutString = "Seven";OutLength = 5;
break;
case 8: OutString = "Eight";OutLength = 5;
break;
case 9: OutString = "Nine";OutLength = 4;
break;
}
}
```

**Question No: 36 ( Marks: 5 )**

**Why Code portability is so important? Give out 3 ways / Guide lines to improve the code portability with examples (5+5)**

Answer:-(Page 179)

Many applications need to be ported on to many different platforms. As we have seen, it is pretty hard to write error free, efficient, and maintainable software. So, if a major rework is required to port a program written for one environment to another, it will be probably not come at a low cost. So, we ought to find ways and means by which we can port applications to other platforms with minimum effort. The key to this lies in how we write our program. If we are careful during writing code, we can make it portable. On the other hand if we write code without portability in mind, we may end-up with a code that is extremely hard to port to other environment. Following is brief guideline that can help you in writing portable code.

**Stick to the standard**

1. Use ANSI/ISO standard C++
2. Instead of using vendor specific language extensions, use STL as much as possible

**Program in the mainstream**

Although C++ standard does not require function prototypes, one should always write them.

```
double sqrt(); // old style acceptable by ANSI C
double sqrt(double); // ANSI – the right approach
```

**Size of data types**

Sizes of data types cause major portability issues as they vary from one machine to the other so one should be careful with them.

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

```
int i, j, k;
…
j = 20000;
k = 30000;
i = j + k;
// works if int is 4 bytes
// what will happen if int is 2 bytes?
```

# FINALTERM EXAMINATION
# Spring 2010
# CS504- Software Engineering - I (Session - 4)

**Question No: 28 ( Marks: 2 )**
**What is an Inspection Checklist?**
Answer:-(Page 210)
Checklist of common errors in a program should be developed and used to drive the inspection process. These error checklists are programming language dependent such that the inspector has to analyze major constructs of the programming language and develop checklists to verify code that is written using these checklists.

**Question No: 29 ( Marks: 2 )**
**Give 2 examples of exceptional code pathes.**
Answer:-(Page 186)

- if (e.Title() == "CEO" || e.Salary() > 10000)
  operator ==() might throw.

- cout << e.First() << " " << e.Last() << " is overpaid" << endl;
  As per C++ standard, any of the five calls to << operator might throw.

**Question No: 30 ( Marks: 2 )**
**The following written statement depicts which requirement from the requirement engineering process "Constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards, etc."**

**Question No: 31 ( Marks: 3 )**
**Write unit testing quantitative benefits.**
Answer:- Repeat

**Question No: 32 ( Marks: 3 )**
**Give three general rules for avoiding split lines.**
Answer:-(Page 155)

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

for avoiding split lines don't use these

- Break after a comma.
- Break after an operator.
- Align the new line with the beginning of the expression on the previous line.

**Question No: 33 ( Marks: 3 )**

**Explain about 3 coverage schemes in white box testing.**

Answer:-(Page 202)

**Statement Coverage**: In this scheme, statements of the code are tested for a successful test that checks all the statements lying on the path of a successful scenario.

**Branch Coverage**: In this scheme, all the possible branches of decision structures are tested. Therefore, sequences of statements following a decision are tested.

**Path Coverage**: In path coverage, all possible paths of a program from input instruction to the output instruction are tested. An exhaustive list of test cases is generated and tested against the code.

**Question No: 34 ( Marks: 5 )**

**List five guidelines that can help you in writing portable code.**

Answer:-(Page 179)

**Stick to the standard**

1. Use ANSI/ISO standard C++
2. Instead of using vendor specific language extensions, use STL as much as possible.

**Program in the mainstream**

Although C++ standard does not require function prototypes, one should always write them.

```
double sqrt(); // old style acceptable by ANSI C
double sqrt(double); // ANSI – the right approach
```

**Size of data types**

Sizes of data types cause major portability issues as they vary from one machine to the other so one should be careful with them.

```
int i, j, k;
…
j = 20000;
k = 30000;
i = j + k;
// works if int is 4 bytes
// what will happen if int is 2 bytes?
```

**Order of Evaluation**

As mentioned earlier during the discussion of side effects, order of evaluation varies from one implementation to other. This therefore also causes portability issues. We should therefore follow guidelines mentioned in the side effect discussion.

**Arithmetic or Logical Shift**

The C/C++ language has not specified whether right shift >> is arithmetic or logical. In the arithmetic shift sign

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

bit is copied while the logical shift fills the vacated bits with 0. This obviously reduces portability. Interestingly, Java has introduced a new operator to handle this issue. >> is used for arithmetic shift and >>> for logical shift.

**Question No: 35 ( Marks: 5 )**
**Below is the chunk of code :**

```
result = squareRoot(argument);
assert (abs (result * result – argument) < epsilon);
```

**Write the Contract for square root routine keeping in view unit testing.**

Answer:-(Page 207)

- Pass in a negative argument and ensure that it is rejected
- Pass in an argument of zero to ensure that it is accepted (this is a boundary value)
- Pass in values between zero and the maximum expressible argument and verify that the difference between the square of the result and the original argument is less than some value epsilon.

**Question No: 36 ( Marks: 5 )**
**Parentheses should always be used as they reduce complexity. Explain it with the help of a single example.**

Answer:-(Page 163)

Parentheses should always be used as they reduce complexity and clarify things by specifying grouping. It is especially important to use parentheses when different unrelated operators are used in the same expression as the precedence rules are often assumed by the programmers, resulting in logical errors that are very difficult to spot. As an example consider the following statement:

```
if (x & MASK == BITS)
```

This causes problems because == operator has higher precedence than & operator. Hence, MASK and BITS are first compared for equality and then the result, which is 0 or 1, is added with x. This kind of error will be extremely hard to catch. If, however, parentheses are used, there will be no ambiguity as shown below.

```
if ((x & MASK) == BITS)
```

## FINALTERM EXAMINATION
## Spring 2010

**Question No: 22 ( Marks: 2 )**
**Define Modularity.**

Answer:-(Page 170)

Modularity is a tool that can help us in reducing the size of individual functions, making them more readable.

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Question No: 23 ( Marks: 2 )**

**Differentiate between Architectural Design and System Architecture in one line.**

Answer:- Click here for detail

Architecture faces towards strategy, structure and purpose, towards the abstract while Design faces towards implementation and practice, towards the concrete.

**Question No: 24 ( Marks: 2 )**

**What is meant by Software Debugging?**

Answer:-(Page 214)

Debugging techniques are the only mechanism to reach at the code that is malfunctioning.

**Question No: 26 ( Marks: 3 )**

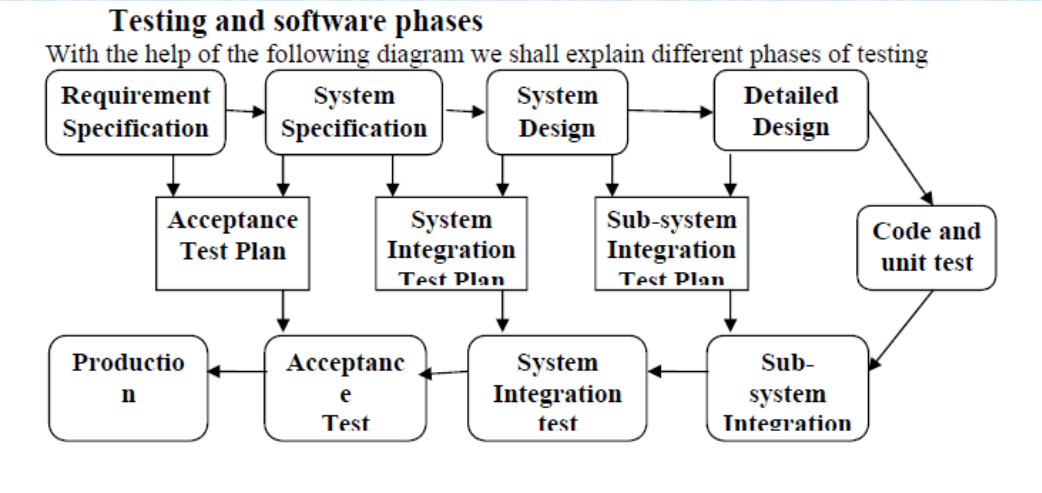
**Why comments are requiring to be indented and how it should be done, explain with one example?**

Answer:- Repeated

**Question No: 27 ( Marks: 3 )**

**What are the different phases of testing? Draw a diagram. (No write up required)**

Answer:-(Page 197)




**Question No: 28 ( Marks: 5 )**

**Why and how complex expressions should be written in multiple short statements, Explain it with example**

Answer:-(Page 164)

Complex expressions should be broken down into multiple statements. An expression is considered to be complex if it uses many operators in a single statement. As an example consider the following statement:

```
*x += (*xp=(2*k < (n-m) ? c[k+1] : d[k--]));
```

This statement liberally uses a number of operators and hence is very difficult to follow and understand. If it is

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

broken down into simple set of statements, the logic becomes easier to follow as shown below:

```
if (2*k < n-m)
*xp = c[k+1];

else
*xp = d[k--];
*x = *x + *xp;
```

**Question No: 29 ( Marks: 5 )**

**What is the Usefulness of Testing?**

Answer:-(Page 197)

Objective of testing is to discover and fix as many errors as possible before the software is put to use. That is before it is shipped to the client and the client runs it for acceptance. In software development organizations, a rift exists between the development and the testing teams. Often developers are found questioning about the significance or even need to have the testing resources in the project teams. Whoever doubts on the usefulness of the testing team should understand what could happen if the application is delivered to client without testing? At the best, the client may ask to fix all the defects (free of cost) he would discover during the acceptance testing. At the worst, probably he would sue the development firm for damages. However, in practice, clients are often seen complaining about the deliverables and a couple of defected deliverables are sufficient for breaking the relations next to the cancellation of contract.

**Question No: 30 ( Marks: 5 )**

**What are the static analyzers, give a check list of the requirements?**

Answer:-(Page 211)

Static analyzers are software tools for source text processing. They parse the program text and try to discover potentially erroneous conditions and bring these to the attention of the verification and validation team.

Checklist for static analysis

| | |
|---|---|
| Data faults | • variable used before initialization<br>• variable declared but never used<br>• variables assigned twice but never used between assignments<br>• possible array bound violations<br>• undeclared variables |
| Control faults | • unreachable code<br>• unconditional branches into loops |
| Input/Output faults | • variable output twice with no intervening assignment |
| Storage Management fault | • unassigned pointers<br>• pointer arithmetic |

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**FINALTERM EXAMINATION**
**Spring 2010**
**CS504- Software Engineering - I (Session - 2)**

**Question No: 27 ( Marks: 2 )**
**What is called self documenting code?**

Answer:-(Page 147)

A self documenting code is a code that explains itself without the need of comments and extraneous documentation, like

- Flow charts,
- UML diagrams,
- Process-flow state

**Question No: 28 ( Marks: 2 )**
**What are the six elements that are present in every computer-based system?**

Answer:- Click here for detail

1. Hardware
2. People/user
3. Data
4. Procedure
5. Connectivity
6. software

**Question No: 29 ( Marks: 2 )**
**Differentiate between Architectural Design and System Architecture in a single line.**

Answer:- Repeated

**Question No: 30 ( Marks: 2 )**
**What are the true statements with respect to equivalence partitioning?**

(a) Input data and output results often fall into different classes where all members of a class are related.
(b) It is a method to partition the requirements into equivalent classes during the requirement analysis process.
(c) Test cases should be chosen to be representative of each equivalence partition.
(d) It is recommended that only boundaries are checked in each partition.
(e) It is recommended that boundaries as well as mid points are checked in each partition.

Answer:-
( A ,b ,c, ) are the true statements with respect to equivalence partitioning

**Question No: 31 ( Marks: 3 )**
**Write unit testing principles.**

Answer:- Repeated

**Question No: 32 ( Marks: 3 )**
**Why Special characters like TAB and page break must be avoided? Explain**

Answer: - (Page 155)

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Special characters like TAB and page break must be avoided. These characters are bound to cause problem for editors, printers, terminal emulators or debuggers when used in a multi-programmer, multi-platform environment

**Question No: 33 ( Marks: 3 )**
**Explain the different phases of Testing with the help of a Diagram. You are required to only draw the diagram.**
Answer:- Repeated

**Question No: 34 ( Marks: 5 )**
**What is the Software testing objective? Also define a successful test.**
Answer: - (Page 193)

## Software testing objective

- The correct approach to testing a scientific theory is not to try to verify it, but to seek to refute the theory. That is to prove that it has errors. (Popper 1965)
- The goal of testing is to expose latent defects in a software system before it is put to use.
- A software tester tries to break the system. The objective is to show the presence of a defect not the absence of it.
- Testing cannot show the absence of a defect. It only increases your confidence in the software.
- This is because exhaustive testing of software is not possible – it is simply too expansive and needs virtually infinite resources.

## Successful Test

From the following sayings, a successful test can be defined "If you think your task is to find problems then you will look harder for them than if you think your task is to verify that the program has none" – Myers 1979.

"A test is said to be successful if it discovers an error" – doctor's analogy.
The success of a test depends upon the ability to discover a bug not in the ability to prove that the software does not have one. As, it is impossible to check all the different scenarios of a software application, however, we can apply techniques that can discover potential bugs from the application. Thus a test that helps in discovering a bug is a successful test. In software testing phase, our emphasis is on discovering all the major bugs that can be identified by running certain test scenarios. However it is important to keep in mind that testing activity has certain limitations.

**Question No: 35 ( Marks: 5 )**
**Discus the symptoms and an example of coding error bug class.**
Answer:- Repeated

**Question No: 36 ( Marks: 5 )**
**Write at leas 5 General Naming conventions for C++ or Java**
Answer:- Repeated

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

# Final Term Papers Spring 2011

**What is unit testing? 2 Numbers**
Answer:- Repeated

**Define Modularity? 3 Numbers**
Answer:- Repeated

**Define Textual Analysis? 3 Numbers**
Answer:- (Page 90)
The oldest techniques to identify objects and their relationships. This technique is called Textual Analysis. It was initially developed by Abbot and then extended by Graham and others. In this technique different parts of speech are identified within the text of the specification and these parts are modeled using different components.

**How one can avoid hazards caused by side effects? 3 Numbers**
Answer:- Repeated

**Describe Loop Errors and symptoms? 5 Numbers**
Answer:- (Page 220)

## Loop Errors

- Loop errors break down into several different subtypes.
- They occur around a loop construct in a program.
- Infinite loops, off-by-one loops, and improperly exited loops.

## Symptoms

- If your program simply locks up, repeatedly displays the same data over and over, or infinitely displays the same message box, you should immediately suspect an infinite loop error.
- Off-by-one loop errors are quite often seen in processes that perform calculations.
- If a hand calculation shows that the total or final sum is incorrect by the last data point, you can quickly surmise that an off-by-one loop error is to blame.
- Likewise, if you were using graphics software and saw all of the points on the screen, but the last two were unconnected, you would suspect an off-by-one error.

**Write Note on Partitioning? 5 Numbers**
Answer:- (Page 135)
Partitioning of architecture is an important concept. What we basically want to do is distribute the responsibilities to different subsystems so that we get a software system which is easy to maintain. Partitioning results in a system that suffers from fewer side effects. This ultimately means that we get a system that is easier to test and extend and hence is easier to maintain.

Partitioning of architecture may be "horizontal" and/or "vertical".

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

In the horizontal partitioning we define separate branches of the module hierarchy for each major function and control modules are used to coordinate communication between functions.

Vertical partitioning divides the application from a decision making perspective. The architecture is partitioned in horizontal layers so that decision making and work are stratified with the decision making modules residing at the top of the hierarchy and worker coming at the bottom. This partitioning is also known as factoring.

# Final Term Papers Spring 2011

**What is testing and also describe successful testing?**
Answer:- Repeated

**Mention two General naming conventions for Java? Same above**
Answer:- Repeated

**Memory and resource leak symptoms 3 marks**
Answer:- (Page 216)

Symptoms

- System slowdowns
- Crashes that occur "randomly" over a long period of time

**Tools and Methods used in software engineering 5**
Answer:- (Page 6 & 11)

- Programming Language
- Programming Language Design
- Software Design Techniques
- Tools
- Testing
- Maintenance
- Development etc.

**Methods:** Methods provide the technical "how-to's" to carryout these tasks. There could be more than one technique to perform a task and different techniques could be used in different situations.
**Tools:** Tools provide automated or semi-automated support for software processes, methods, and quality control.

**Five guidelines using for Code 5**
Answer:- (Page 150)

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

1. Names representing types must be nouns and written in mixed case starting with upper case.
Line, FilePrefix

2. Variable names must be in mixed case starting with lower case. line, filePrefix

3. Names representing constants must be all uppercase using underscore to separate words.
MAX_ITERATIONS, COLOR_RED

4. Names representing methods and functions should be verbs and written in mixed case starting with lower case.
getName(), computeTotalWidth()

5. Names representing template types in C++ should be a single uppercase letter.
template<class C, class D>

# Final Term Papers Spring 2011

**UNIT TESTING?**
Answer:- Repeated

**WHITE BOX TESTING?**
Answer:- (Page 202)
In white box testing we test the structure of the program. In this technique the test cases are written in a manner to cover different possibilities in code.

**OBJECTIVE OF TESTING AND SUCCESSFUL TEST?**
Answer:- Repeated

**LOOP ERROR BUG?**
Answer:- Repeated

**BALANCING ACT IN SOFTWARE ENGINEERING?**
Answer:- (Page 7)
Software Engineering is actually the balancing act. You have to balance many things like cost, user friendliness, Efficiency, Reliability etc. You have to analyze which one is the more important feature for your software is it reliability, efficiency, user friendliness or something else.

**HOW COMMENTS BE INDENTED?**
Answer:- Repeated

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**WHY SPECIAL CHARACTERS TAB AND BREAK PAGE BE AVOIDED?**
Answer:- Repeated

**THREE NON EXCEPTIONAL PATHS?**
Answer:- (Page 185)

- if (e.Title() == "CEO" || e.Salary() > 10000)
  · if e.Title() == "CEO" is true then the second part is not evaluated and e.Salary() will not be called.
  · cout will be performed
- if e.Title() != "CEO" and e.Salary() > 10000
  · both parts of the condition will be evaluated
  · cout will be performed.
- if e.Title() != "CEO" and e.Salary() <= 10000
  · both parts of the condition will be evaluated
  · cout will not be performed.

**INCLUDE FILES AND INCLUDE STATEMENT?**
Answer:- (Page 157)
1. Header files must include a construction that prevents multiple inclusions. The convention is an all uppercase construction of the module name, the file name and the h
Suffix.

```
#ifndef MOD_FILENAME_H
#define MOD_FILENAME_H
:
#endif
```

The construction is to avoid compilation errors. The construction should appear in the top of the file (before the file header) so file parsing is aborted immediately and compilation time is reduced.

# Final Term Papers Spring 2011

**How one can avoid hazards caused by side effects?**
Answer:- Repeated

**Define debugging?**
Answer:- Repeated

**Define testing?**
Answer:- Repeated

**Software development processes?**
Answer:- (Page 8)
Software development is a process of balancing among different characteristics of software described in the previous section. And it is an art to come up with such a good balance and that art can be learned from experience.

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Inspection pre-conditions?**
Answer:- (Page 210)
A precise specification must be available before inspections. Team members must be familiar with the organization standards. In addition to it, syntactically correct code must be available to the inspectors. Inspectors should prepare a checklist that can help them during the inspection process.

**How software engineer can work on domains if he gets the technical work..?**
Answer:- (Page 5)
"All aspects of software production' Software engineering is not just concerned with the technical processes of software development but also with activities such as software project management and with the development of tools, methods and theories to support software production".
These definitions make it clear that Software Engineering is not just about writing code.

# Final Term Papers Spring 2011

**i. describe two Unit Testing Tips**
Answer:- Repeated

**ii. Equivalence Classes**
Answer:- (Page 199)
## Equivalence Classes or Equivalence Partitioning
Two tests are considered to be equivalent if it is believed that:
if one discovers a defect, the other probably will too, and
If one does not discover a defect, the other probably won't either.
Equivalence classes help you in designing test cases to test the system effectively and efficiently. One should have reasons to believe that the test cases are equivalent.

**iii. Where the term compute method use.**
Answer:- Repeated

**3 Question of 3 marks**
**i. Describe three coverage schemes related to white box testing**
Answer:- Repeated

**ii. Memory over-runs and their symptoms**
Answer:- (Page 220)
A memory overrun occurs when you use memory that does not belong to you. (**Symptoms repeated**)

**iii. The use of do.... while loops should be avoided, why is that?**
Answer:- Repeated

**3 Question of 5 marks**
**i. write a note on Usefulness of testing**
Answer:- Repeated

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**ii. Identifier names also play a significant role**
Answer:- (Page 148)
Identifier names also play a significant role in enhancing the readability of a program. The names should be chosen in order to make them meaningful to the reader. In order to understand the concept, let us look at the following statement.

if (x==0) // this is the case when we are allocating a new number

In this particular case, the meanings of the condition in the if-statement are not clear and we had to write a comment to explain it. This can be improved if, instead of using x, we use a more meaningful name. Our new code becomes:

if (AllocFlag == 0)

The situation has improved a little bit but the semantics of the condition are still not very clear as the meaning of 0 is not very clear. Now consider the following statement:

If (AllocFlag == NEW_NUMBER)

We have improved the quality of the code by replacing the number 0 with a named constant NEW_NUMBER. Now, the semantics are clear and do not need any extra comments, hence this piece of code is self-documenting.

**iii. Good clues, Easy Bugs explain this term.**
Answer:- (Page 226)
**Good clues, Easy Bugs**
Get A Stack Trace
In the debugging process a stack trace is a very useful tool.
Following stack trace information may help in debugging process.

- Source line numbers in stack trace is the single, most useful piece of debugging information.
- After that, values of arguments are important
  - Are the values improbable (zero, very large, negative, character strings with non-alphabetic characters?
- Debuggers can be used to display values of local or global variables.
  - These give additional information about what went wrong.

# Final Term Papers Spring 2011

**Q1 Define modularity**
Answer:- Repeated

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

**Q2 Define Unit testing**
Answer:- Repeated

**Q3 What is the syntax used for naming objects in a sequence diagrams?**
Answer:- (Page 107)
The syntax used for naming objects in a sequence diagram is as follows:

- syntax: [instanceName][:className]
- Name classes consistently with your class diagram (same classes).
- Include instance names when objects are referred to in messages or when several objects of the same type exist in the diagram.

**Q4 Define these terms: Branch Coverage, Statement Coverage.**
Answer:- (Page 202)
**Branch Coverage**: In this scheme, all the possible branches of decision structures are tested. Therefore, sequences of statements following a decision are tested.

**Statement Coverage**: In this scheme, statements of the code are tested for a successful test that checks all the statements lying on the path of a successful scenario.

**Q5 Bit fields do suffer from a lack of portability between platforms. Why?**

Bit fields are a convenient way to express many difficult operations. However, bit fields
do suffer from a lack of portability between platforms:
Answer:- (Page 183)

- integers may be signed or unsigned
- Many compilers limit the maximum number of bits in the bit field to the size of an integer which may be either 16-bit or 32-bit varieties.
- Some bit field members are stored left to right others are stored right to left in memory.
- If bit fields too large, next bit field may be stored consecutively in memory (overlapping the boundary between memory locations) or in the next word of memory.

**Q6 Consider the following Use Case diagram: Identify the system actors in given use case diagram.**

**Q7 Why Special characters like TAB and page break must be avoided? Explain**
Answer:- Repeated

**Q8 Write down at least two (3) guideline that can avoid hazards caused by side effects.**
Answer:- Repeated

**Q9 Discuss five points, how a "variable "is efficiently used in a program?**
Answer:- Repeated

**Q10 Differentiate between Black box testing and white box testing.**
Answer:- (Page 198)

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

- In this type of testing, a component or system is treated as a black box and it is tested for the required behavior. This type of testing is not concerned with how the inputs are transformed into outputs.
- As opposed to black box testing, in structural or white box testing we look inside the system and evaluate what it consists of and how is it implemented.

**Q 11What do we mean by ambiguous requirements. Explain with the help of a example.**

Answer:- (Page 198)

Ambiguity means that two different readers of the same document interpret the requirement differently. Ambiguity arises from the use of natural language. Because of the imprecise nature of the language, different readers interpret the statements differently. As an example, consider the following Urdu Phrase: ***"Rooko mut jane doo".*** Now, depending upon where a reader places the comma in this statement, two different readers may interpret it in totally different manner. If a comma is palced after ***"Rooko"***, the sentence will become ***"Rooko, mut jane doo",*** meaning ***"don't let him go".*** On the other hand if the comma id placed after ***"mut",*** the sentence will become ***"Rooko mut, jane doo",*** meaning ***"let him go".*** Ambiguous requirements therefore result in misunderstandings and mismatched expectations, resulting in a wasted time and effort and an undesirable product.

**Q12 You have learnt about Static analyzers, What are static analyzers? Give a checklist for static analysis.**

Answer:- Repeated

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari